Functorial Type Theory: Univalent Foundations for Mathematics

The emdash contributors

expanded development edition / version 0.4.0-dev / 2026-08-05

Expanded development edition

This is a working edition of *Functorial Type Theory: Univalent Foundations for Mathematics*. The WalkingEnd/Nat encode-decode argument remains its mathematical centre. Around it, a second spiral develops cut elimination, category theory, weighted universal constructions, directed duality, and a categorical-kernel-first formal presentation. A third follows Yoneda's probes through presheaves, sieves, sites, and sheafification into constructive algebraic geometry, taking the invertibility sieve $D_R(f)$ as prior to any representing open. Chapter details, notation, and cross-references may still change. The active implementation remains authoritative whenever prose and code disagree.

> **Formal status — research boundary.** The title names the intended foundational programme. It does not assert that this edition already implements a finished proof assistant, every weak omega-categorical coherence, or a universal computational univalence principle.

Preface

Type theory is often introduced through terms and substitution. Category theory is often introduced through objects and arrows. Functorial type theory starts from the conviction that these are not two unrelated beginnings. Substitution has action; action has coherence; and a useful formal language should retain that structure rather than recover it after the fact.

The resulting theory has two kinds of motion. Equality supplies groupoidal paths: they can be reversed, transported along, and compared through equivalence and univalence. Categories supply directed arrows: an arrow may have no inverse, and that failure is mathematical information. Functors act on both objects and higher arrows. Cat-valued families turn dependent substitution into directed reindexing. Transformations and their higher components express the coherence of those actions.

This edition is organized around a single calculation because a foundation is best learned while it is doing something. Consider an opaque category generated, in the higher-inductive sense selected by emdash, by a base object and one directed endomorphism. Its endomorphisms look like

$$\mathrm{id}, \quad \ell, \quad \ell^2, \quad \ell^3, \quad \ldots.$$

The calculation proves that this list is exhaustive at the level of the underlying carrier:

$$\mathrm{Hom}_W(*, *) \simeq \mathbb{N}.$$

The proof is not obtained by defining the hom to be a datatype of words. Instead it constructs a Cat-valued code, transports zero forward along a directed arrow, builds a contextual decoder, and produces a directed normalization cell from every based arrow toward its coded power. Only then does one-dimensionality turn that cell into equality. This order—directed structure first, equality at the boundary—is the small example in which the larger programme becomes visible.

The argument deliberately echoes the calculation $\Omega(S^1) \simeq \mathbb{Z}$ in the *Homotopy Type Theory* book. The echo is not an identification. A loop in an identity type is invertible, while the walking endomorphism is not. The circle needs positive and negative powers; the directed object has only natural powers. The circle's code uses univalence to turn successor on the integers into a path in a universe; the directed code sends its generator directly to a successor functor, which need not be an equivalence. What fails to transfer is as instructive as what does.

The title's phrase "univalent foundations" therefore describes a layer, not a device forced into every proof. Emdash contains checked equality, equivalence, groupoid-univalence, restricted truncated-universe univalence, and equality-valued omega-equivalence interfaces. The WalkingEnd calculation also shows why a univalent foundation for directed mathematics must permit actions that are not equivalences.

The exposition follows a spiral. The prologue states the central theorem with minimal prerequisites. Chapters 1–7 then develop the judgments, categories, families, logic, equivalence, induction, directed higher induction, and categorical height needed to understand the proof. Chapter 8 returns to the calculation in full.

The later chapters move outward from that proof in a second spiral. Chapter 9 organizes functorial computation as a calculus of cuts. Chapters 10–15 develop categories, functors, adjunctions, Yoneda, duality, structure identity, and saturation. Chapters 16–17 treat weighted limits and colimits before returning to directed geometry through join.

A third spiral begins in Chapters 18–24. It turns Yoneda's field of probes toward local-to-global geometry: presheaves organize changing views, higher sieves retain categories of witnesses, and ordinary sieves record stable local questions. Sites select which sieves cover, while matching families state descent as one restriction-of-Hom problem. Direct cover completion then freely adjoins coherent solutions and assembles the Cat-valued sheaf reflector from its whole Hom universal property. Commutative algebra then supplies set-carrier rings, finite unit-ideal certificates, free extension interfaces, and localizations characterized by contractible factor spaces. The recurring geometric example is the sieve $D_U(s)$ of every probe along which a section becomes invertible. A localization may represent this question on affine points, but the sieve is meaningful before representability is known. The affine functor of points then turns selected localizations into basic charts, multiplication into pointwise intersection, and finite unit-ideal certificates into the generated big Zariski topology, while keeping structure-sheaf and localization-locality assumptions explicit. Starting from one supplied global ringed object, a covering sieve, and two constructively generating affine realizations, the spiral then reaches a binary site-relative scheme presentation. Whole slice restrictions and a selected actual chart intersection inherit their maps from the single global structure presheaf; atlas-first gluing and comparison with classical or functorial qcqs schemes remain visible boundaries. On that actual intersection, polynomial and localization universality construct the Laurent coordinate changes of two supplied affine-line charts. The spiral ends with an assumption-explicit projective-line presentation: the overlap calculation is checked, while construction of the global object, graded `Proj`, general projective space, and non-affineness remain visible boundaries.

Appendix G then states how the mathematical surface, checked categorical kernel, bounded TypeScript elaborator through explicit Core, and external models fit together, with the Lambdapi kernel remaining the mathematical authority.

The book is evidence-aware without being a source-code catalogue. Checked claims name their evidence in compact notes. Free mathematical development is welcome, but it is named as such and states what an emdash implementation would require. This lets the prose reach beyond the current library without blurring the line between a plausible design and a theorem already accepted by the kernel.

> **Formal status — mathematical development.** This preface states the expository and research programme. Each theorem-like claim in the chapters carries its own status and, when checked, a machine-verified evidence link.

How To Read This Book

The shortest route is the prologue, followed by the compact prerequisite review and proof in Chapter 8. The foundational route reads Chapters 1–7 first and then returns to the same calculation with every interface available. The second spiral begins with the cut calculus in Chapter 9, develops ordinary and native category theory through Chapter 15, and culminates in weighted universals, duality, and join in Chapters 16–17. The local-to-global spiral begins with presheaves and sieves in Chapter 18, then selects covers and formulates descent in Chapter 19, and constructs Cat-valued sheafification by direct cover completion in Chapter 20. Chapter 21 adds the representation-free commutative algebra that turns invertibility into computational affine charts, and Chapter 22 constructs the functor-of-points bridge from $D(f)$ and localization to the generated big Zariski site. Chapter 23 begins with a supplied global ringed object, recognizes two constructively generating regions as affine, imposes topology-local ring behaviour on the actual slice, and derives a selected chart intersection from the global structure presheaf. Chapter 24 constructs Laurent coordinate changes on that literal overlap, packages the resulting supplied projective-line capability, and separates it from the still-unconstructed graded `Proj` route. The contents and glossary/index provide stable anchor-based navigation.

Five reading paths make the dependencies explicit:

Reader	Main path	Consult when needed
type theorist	Prologue; Chapters 1, 3–8, 10, and 15	Chapters 2 and 9 for directed action; Appendix G for the formal presentation
category theorist	Prologue; Chapters 2, 5, and 8–24	Chapters 1, 3, 4, and 7 for equality, propositions, univalence, and height
algebraic geometer	Chapters 13, 16, and 18–24	Chapters 2, 3, 5, 6, and 12 for the directed, logical, inductive, universal, and adjoint foundations
implementer	Chapters 1, 2, 6, 8, and 9; Appendices A, B, E, F, and G	the theorem chapters whose evidence route is being inspected
external reviewer	Chapters 2.6, 8, and 9; then the integrated reviewer, live or local	Appendices A, B, F, and G for notation, evidence, status, and architecture

These are paths through one dependency graph, not separate foundations. In particular, the category-theory route still uses equality-local reasoning, and the type-theory route still needs directed functor action.

For the executable-review path, open the integrated reviewer or run `./scripts/pnpmw run reviewer:dev` from the repository root. The wholly client-side workbench offers editable examples across the four binder modes. It lets the reader inspect explicit Core, inferred and expected classifiers, structural lowering, computation, and source-located failures; the same page runs the three-part research report and opens this book. Its text notation is a bounded executable subset. The mathematical notation used throughout the book is intentionally broader and should not be read as a complete parser grammar.

Composition is written in categorical order:

$$g \circ f : x \longrightarrow z$$

means "first f, then g." Functor action is written $F[x]$ on objects and $F[f]$ on arrows. The path category $\mathsf{Path}(A)$ retains equality-local, hence groupoidal, structure inside the directed calculus. Symbols such as W, $*$, ℓ, Code, encode, and power are mathematical abbreviations; the notation appendix maps them to active Lambdapi names.

Evidence Status

Every theorem-like assertion has one of four evidence statuses:

- **Checked.** An active declaration and a regression or reviewer example establish the stated interface.
- **Formal consequence.** The assertion follows from named checked interfaces, but the library does not yet package the result under the stated name.
- **Mathematical development.** The theory is developed in ordinary mathematics with explicit prerequisites and a plausible future emdash owner.
- **Research boundary.** A construction is conjectural, underspecified, or blocked on named infrastructure.

A typical note looks like this:

> **Formal status — checked.** Evidence `WE-HOM-NAT-CARRIER`.

The evidence identifier resolves through `book/evidence.json` to declarations and executable checks. It is traceability metadata, not a replacement for the proof in the prose.

The active Lambdapi sources outrank the book. The current implementation and safe-development procedure are described in the repository's current-status report; the canonical-syntax report owns the mathematical notation, which is broader than the reviewed executable text subset. Dated reports preserve design history but do not silently revive retired interfaces.

Passages structurally or conceptually adapted from the *Homotopy Type Theory* book, Zeuner's constructive algebraic geometry, and Pédrot's computational sheafification work are versioned and section-mapped in `book/references/third-party-sources.json`. The relevant licenses and kinds of adaptation are recorded there, and emdash's mathematical changes are stated rather than hidden behind a change of symbols.

Contents

Front matter

- Preface
- How To Read This Book

Main text

- Prologue: The Natural Number Hidden In A Directed Loop
- 1. Judgments, Universes, And Computation
- 2. Categories, Functors, And Directed Families
- 3. Logic, Propositions, And Sets
- 4. Equivalence And Univalence
- 5. Induction, Arrow Induction, And Universal Properties
- 6. Directed Higher Inductive Types
- 7. Truncation And Categorical Height
- 8. Synthetic Directed Homotopy Theory
- 9. Transfors And The Calculus Of Cuts
- 10. Categories, Precategories, And Categorical Identity
- 11. Functors, Transfors, And Functor Categories
- 12. Adjunctions And Equivalences
- 13. Yoneda, Representability, And Profunctors
- 14. Strictness, Dagger Structure, And Duality
- 15. Structure Identity And Saturation
- 16. Weighted Universal Constructions
- 17. Weighted Colimits, Duality, And Join
- 18. Presheaves And Sieves
- 19. Sites, Covers, And Descent
- 20. Sheafification By Cover Completion
- 21. Commutative Algebra By Universal Property
- 22. Affine Geometry And The Sieve $D(f)$
- 23. Schemes From Covering Charts
- 24. The Projective Line And The Boundary Of Construction

Appendices

- Appendix A. Notation

- Appendix B. Emdash Evidence
- Appendix C. From The Circle To The Walking Endomorphism
- Appendix D. Glossary And Concept Index
- Appendix E. Computation And Normalization
- Appendix F. Implementation Status And Research Directions
- Appendix G. Formal Presentation Of Functorial Type Theory

Back matter

- Bibliography
- Credits And Third-Party Attribution
- License For The Book

Prologue: The Natural Number Hidden In A Directed Loop

Imagine a world with one distinguished place and one permitted step that returns to it. Call the place `*` and the step `ell`. We may stand still, take the step once, take it twice, and continue:

$$\mathrm{id}_*, \qquad \ell, \qquad \ell \circ \ell, \qquad \ell \circ \ell \circ \ell, \qquad \ldots.$$

If this is the *free* directed situation, every journey from `*` back to `*` should have a unique length. That sentence sounds obvious only because it quietly identifies two descriptions:

1. the syntactic description of a journey as a finite word in one generator;
2. the universal object characterized by how maps out of it behave.

Functorial type theory keeps them apart. The walking endomorphism `W` is presented opaquely by a category, a base, a directed loop, explicit one-dimensionality, and a contextual elimination principle. Its object type, hom-categories, identities, and composition do not reduce to a word datatype.

> **Formal status — checked.** Evidence `WE-SIGNATURE` and `WE-ONE-DIMENSIONAL`. One-dimensionality says that each hom-category is discrete; it neither makes `W` a discrete category nor makes `ell` invertible.

There is also a concrete category `BNat`. It has one object, natural numbers as endomorphisms, zero as identity, and addition as composition. The recursor for `W` gives a functor

$$W \longrightarrow BNat$$

sending `*` to the sole object and `ell` to one. This is the expected arithmetic model. It is not a definition of `W`, and a functor into a model cannot by itself prove that two arrows of `W` with the same image are equal.

> **Formal status — checked.** Evidence `WE-BNAT-MODEL`. `BNat` is separate model evidence. A reverse functor and a categorical equivalence have not been packaged.

The central theorem supplies the missing exhaustiveness statement by a different route.

> **Theorem preview.** There is an equivalence of underlying carriers
>
> $$> \mathrm{Hom}_W(*,*) \simeq \mathbb{N}. >$$
>
> **Formal status — checked.** Evidence `WE-HOM-NAT-CARRIER`. The current package is a `TypeEquiv`, together with a native equality-valued facade. A monoid or hom-category equivalence is not being asserted here.

Measuring An Abstract Arrow

How can an opaque arrow be assigned a number? We construct a Cat-valued family

$$\mathsf{Code} : W \longrightarrow \mathsf{Cat}$$

whose fibre at `*` is the path category of natural numbers. The generator acts by the successor functor. For any endpoint `x` and any based arrow $p : * \to x$, define

$$\mathsf{encode}_x(p) \;=\; \mathsf{Code}[p](0).$$

This is directed transport in its plainest form: a functor acts on an object. When `p` begins with the generator, its code begins with a successor. There is no predecessor action because none was requested by the signature.

To go back, natural numbers produce powers:

$$\ell^0 = \mathrm{id}_*, \qquad \ell^{n+1} = \ell \circ \ell^n.$$

At first this decoder exists only over the base. The decisive move is to generalize it over every endpoint. The based representable family

$$\mathsf{Rep}_*(x) = \mathrm{Hom}_W(*, x)$$

acts on a base arrow by postcomposition. A displayed transformation

$$\mathsf{decode}^d : \mathsf{Code} \Longrightarrow \mathsf{Rep}_*$$

must therefore reconcile successor in `Code` with postcomposition by `ell` in the representable. Its coherence winds forward through the natural levels:

$$\ell \circ \ell^n \longrightarrow \ell^{n+1}.$$

This family of higher arrows is the **directed spiral**.

Normalize Before Comparing

Applying the displayed decoder's arrow action to $p : * \to x$ at zero produces

$$\nu_p : p \longrightarrow \mathsf{decode}_x(\mathsf{encode}_x(p))$$

inside the hom-category `Hom_W(*,x)`. This is not initially an equation. It is a directed 2-cell whose orientation records a normalization process.

> **Formal status — checked.** Evidence `WE-NORMALIZATION-CELL`. The normalization cell is obtained from the contextual displayed eliminator; it is not postulated as a word-reduction axiom.

Only now is one-dimensionality used. The hom-category is discrete, so a cell between parallel based arrows determines equality:

$$p = \mathsf{decode}_x(\mathsf{encode}_x(p)).$$

At the base, `decode` computes to `power`. Reversing the displayed equality gives

$$\ell^{\mathsf{encode}(p)} = p.$$

The other composite is arithmetic. The generator-prefix equation and natural number induction give

$$\mathsf{encode}(\ell^n) = n.$$

These two equations package the carrier equivalence.

> **Formal status — checked.** Evidence `WE-NORMALIZATION-PATH`, `WE-POWER-ENCODE`, and `WE-ENCODE-POWER`. Direction is retained until the first of these equalities is extracted.

Why The Answer Is Not The Integers

For the circle, a loop is an equality path and therefore has an inverse. Its powers are indexed by integers. Here `ell` is a directed arrow. Nothing in the category laws manufactures a reverse arrow, and the calculation detects that absence.

Encoding sends `ell` to one and the identity to zero. Consequently the generator is not the identity. If an arrow `r` were a right inverse, then encoding $\ell \circ r = \mathrm{id}$ would force a successor to equal zero. Hence no right inverse exists, and in particular `ell` carries no native omega-equivalence evidence.

> **Formal status — checked.** Evidence `WE-LOOP-NOT-IDENTITY`, `WE-LOOP-NO-RIGHT-INVERSE`, and `WE-LOOP-NONINVERTIBLE`.

The missing negative integers are therefore not an incomplete case of the proof. They are the numerical trace of directionality.

The Road To The Proof

The next seven chapters unpack the interfaces used above.

- Chapter 1 separates definitional computation, proof-time comparison, and equality evidence.
- Chapter 2 introduces iterated hom-categories, functors, transfors, and directed Cat-valued families.
- Chapter 3 develops constructive logic, propositions, and sets.
- Chapter 4 distinguishes equality, ordinary equivalence, categorical isomorphism, and univalence.
- Chapter 5 moves from ordinary induction to arrow induction and contextual universal properties.
- Chapter 6 states the selected directed higher-inductive signature and its contextual eliminator.
- Chapter 7 explains categorical height and why a directed cell in a discrete hom-category yields equality.

- Chapter 8 returns to every step of the construction, including the spiral, the two inverse laws, and the noninvertibility consequences.

The second spiral asks what the same computational discipline contributes to category theory. Chapter 9 isolates its cut calculus; Chapters 10–15 develop categorical identity, functor categories, adjunctions, Yoneda, duality, and saturation; and Chapters 16–17 organize limits, colimits, and join by representability and opposite duality. Appendix G collects the formal rule schemas and states the metatheoretic boundary.

The third spiral asks how local questions become geometry. Chapters 18–20 pass from presheaves and sieves through sites and descent to direct Cat-valued sheafification. Chapters 21–24 develop universal-property algebra and affine localization, place the invertibility sieve before any representing open, and assemble site-relative scheme and supplied projective-line presentations without concealing their hypotheses.

The larger aim is not merely to calculate one hom. It is to show how a type theory can let groupoidal equality and noninvertible arrows coexist, interact, and compute—without quietly turning one into the other.

1. Judgments, Universes, And Computation

A foundation does not begin by placing mathematical objects in a previously given container. It begins by saying which expressions may be formed, which expressions inhabit which classifiers, and which calculations count without further proof. The elementary judgments have the familiar shape

$$A\ \mathsf{classifier}, \qquad a : A, \qquad a \equiv b : A.$$

The last display uses $\equiv$ only as the glyph for *judgmental* or definitional equality. It is not an inhabitant of an identity type. This distinction will matter throughout the book: computation may make two expressions the same input to every later rule, while a path is mathematical data that can itself be transported, inverted, and compared.

Emdash represents its small type-like classifiers by a universe called `Grpd`. A classifier `A:Grpd` decodes to the ambient type of its elements, written mathematically simply as `a:A` and represented in the kernel by `a:tau(A)`. This two-level presentation is an implementation device. Unless a decoding boundary is at issue, we suppress `tau` and reason in ordinary type-theoretic notation.

Appendix G gives the full formal reading of this convention. It separates external contexts and typing judgments from internal classifiers, then asks of each major construction for its formation, introduction, elimination, computation, uniqueness, and functorial action. The last two clauses will often be independent: a beta rule can be checked even when a general uniqueness theorem remains open, and a pointwise construction is incomplete until its arrow and higher-cell action is known.

1.1 Contexts And Families

A judgment rarely stands alone. It is made in a context:

$$x : A, \quad y : B(x), \quad z : C(x, y) \ \vdash\ t : D(x, y, z).$$

Each declaration may depend on those before it. Substitution replaces a variable by a term of the right classifier and acts through every expression that follows. In ordinary dependent type theory this action is often left implicit in notation. Functorial type theory will progressively expose it: first as path action, then as functor action on directed arrows, and finally as coherent action on higher cells.

An ordinary function classifier is written

$$A \longrightarrow B,$$

and a dependent function classifier as

$$\prod_{x:A} B(x).$$

An element `f` of the second classifier assigns `f(x):B(x)`. Lambda abstraction introduces such an element, and application to an argument computes by beta reduction. The nondependent function classifier is the constant-family case.

A dependent pair classifier is

$$\sum_{x:A} B(x).$$

An element is a pair `(x,u)` with `u:B(x)`. Equality of dependent pairs is not merely a pair of unrelated equalities: it consists of a base path together with a fibre path lying over that base path. Emdash exposes this observation explicitly. For dependent functions it exposes pointwise path observation and a converse function-extensionality constructor. Thus the basic Sigma and Pi layers already carry the path structure later used in equivalence and truncation arguments.

> **Formal status — checked.** Evidence `TT-SIGMA-PI-PATHS` covers dependent-pair projections and path views together with the checked `happly` / `funext` equivalence for dependent functions. This is an implemented equality interface, not a claim that every future categorical section category is merely a pointwise Pi type.

1.2 Three Ways Expressions Agree

The word “equal” hides three operationally different relations in the current development.

First, a **runtime rewrite** chooses a normal form. For example, applying the Nat eliminator to zero returns its zero branch, while applying it to a successor returns the successor branch with the recursive result. Such a rule is part of computation: later expressions literally see the reduced term.

Second, a **proof-time comparison** helps elaboration recognize two typed presentations without orienting either as the other's runtime normal form. This is useful when a mathematical construction has two stable categorical views—for example, a displayed-family facade and the corresponding ordinary functor-category view. Proof-time comparison is intentionally narrower than an equation available to the mathematician.

Third, an **identity term**

$$p : x =_A y$$

is internal evidence. It may be an argument to a function, the index of a dependent family, or the object of a higher identity type. Its existence does not by itself make `x` and `y` judgmentally interchangeable.

These layers cooperate, but they should not be conflated. A rewrite is chosen because a form is intended to compute. A proof-time comparison is chosen because two rigid presentations should elaborate together. A path is chosen because equality is part of the mathematics. Much of emdash's engineering is the discipline of putting a law at the right one of these layers.

As a small example, Nat addition is defined by recursion in its left input:

$$0 + n \equiv n, \qquad \mathsf{succ}(m) + n \equiv \mathsf{succ}(m + n).$$

Associativity is an identity term proved by Nat induction; it is not installed as a global reassociation rewrite. The distinction keeps computation predictable while leaving the algebra available propositionally.

> **Formal status — mathematical development.** The three-layer terminology is the book's explanation of the active kernel discipline. Exact rewrite and unification ownership remains an implementation matter governed by the current SOP; the mathematical chapters rely only on the checked interfaces cited in their evidence notes.

1.3 Elementary Inductive Classifiers

The initial object language contains classifiers for Empty, Unit, Bool, and Nat. Their constructors have the expected readings:

Empty	no constructor
Unit	tt
Bool	$\mathsf{false}, \mathsf{true}$
$\mathbb{N}$	$0, \mathsf{succ}(n).$

An inductive classifier is characterized computationally by how dependent data are defined from its constructors. For Nat, a motive `P:Nat -> Grpd`, a base value `z:P(0)`, and a step

$$s : \prod_{n:\mathbb{N}} P(n) \longrightarrow P(n+1)$$

determine

$$\mathsf{ind}_{\mathbb{N}}(P, z, s, n) : P(n)$$

with the zero and successor computations. Recursion is the constant-motive special case; induction retains the dependence on the number being analyzed. The power function in Chapter 8 is exactly such a Nat elimination, with a hom carrier as its motive.

The equality classifiers of visible Unit, Bool, and Nat constructors also have bounded observational computations. In particular, zero cannot equal a visible successor, and equality of two visible successors reduces to equality of their predecessors. These classifier computations do not erase a generic reflexivity proof into the unique constructor of Unit, nor do they establish a global normalization or canonicity theorem for every open Nat term.

> **Formal status — checked.** Evidence `TT-ELEMENTARY-INDUCTION` covers the decoded elementary classifiers and their dependent eliminators. The currently selected reusable Nat library additionally supplies addition, associativity, successor action, and sethood. A retired experimental bin-

> ary coproduct is not part of this edition's active elementary API.

1.4 Identity And Path Induction

For `x,y:A`, the classifier `x=y` records identity paths. Reflexivity gives

$$\mathsf{refl}_x : x = x.$$

The fundamental elimination rule says that a property of paths can be proved by treating the reflexive path. In the right-based form used by emdash, fix `y:A`, take a motive

$$P : \prod_{x:A} (x = y) \longrightarrow \mathsf{Grpd},$$

and provide `u:P(y,refl_y)`. Then every `p:x=y` receives an element of `P(x,p)`, and the construction computes to `u` on literal reflexivity.

Path symmetry, transitivity, and action follow from this eliminator:

$$p^{-1} : y = x,$$
$$q \cdot p : x = z,$$
$$\mathsf{ap}_f(p) : f(x) = f(y).$$

For a dependent function `f:prod_(x:A) B(x)`, ordinary `ap` is not well typed because its two outputs lie in different fibres. Dependent path action instead gives a path over `p`, written

$$\mathsf{apd}_f(p) : f(x) =^B_p f(y).$$

This is the first appearance of a recurring idea: motion in the base changes the classifier in which the endpoint lives. In the equality-local fragment, the motion is invertible and is handled by path induction. In a directed family over a category, Chapter 2 will replace it with an explicit functor between fibres.

> **Formal status — checked.** Evidence `TT-EQUALITY-INDUCTION` covers reflexivity, right-based dependent equality induction, `ap`, and `apd`. Emdash does not assume equality reflection: possessing a term `p:x=y` does not turn an arbitrary expression containing `x` into the same runtime term containing `y`.

1.5 Propositions As Classifiers

Under the broad propositions-as-types reading, to prove a statement is to construct an inhabitant of its classifier. Conjunction is represented by a pair, implication by a function, universal quantification by a dependent function, and existential data by a dependent pair. Negation of `A` is the function classifier

$$\neg A := A \longrightarrow \mathsf{Empty}.$$

This reading is constructive: an inhabitant of an existential Sigma type contains a witness, and an inhabitant of an implication is an operation on evidence. It does not make every classifier proof-irrelevant. A Boolean, for example, carries the information of which constructor was chosen. Chapter 3 will isolate the narrower classifiers whose inhabitants are all equal and will call those *proposition-valued*.

No classical principle is needed for the WalkingEnd calculation. The negative results there are constructive functions: an alleged equality or inverse is sent to the empty classifier by inspecting its Nat code.

1.6 From Terms To Actions

The material so far is groupoidal. Identity paths are reversible, and an ordinary function acts on them through `ap`. That is enough for homotopy type theory's slogan that functions behave functorially on paths. Functorial type theory retains this layer but adds a second one: a function between object carriers is not automatically the whole meaning of a directed functor. A directed functor must act on arrows, on arrows between arrows, and so on.

The transition is therefore not from “types” to an unrelated category theory. It is from implicit action generated by identity induction to explicit action over possibly noninvertible arrows. The next chapter makes that action part of the primary syntax.

2. Categories, Functors, And Directed Families

An ordinary type family varies with a term and transports along equality. A directed family varies with an object and acts along arrows that need not be equalities. This small change reorganizes the entire dependent calculus. The action is no longer reconstructed only from path induction: it is presented as a functor and is therefore available at every cell dimension.

The central example already suggests the shape. `Code` is not merely an assignment of a category to the base object of `W`. It must send the generator to the successor functor. The based representable is not merely the collection of hom carriers. It must act on an arrow by postcomposition. The decoder is not merely a family of functions. It must compare these two actions coherently.

For classical category-theoretic background, see Mac Lane. For broader context on directed type theory, see GPT 5.6 Codex. The present calculus makes its own computational and formal choices.

2.1 Categories With Iterated Homs

For a category `A`, write

$$\mathrm{Obj}(A)$$

for its classifier of objects. For `x,y:Obj(A)`, emdash does not take `Hom_A(x,y)` to be only a set or a type of arrows. It is another category. Its objects are 1-arrows `f:x -> y`; its arrows are 2-cells between such arrows; iterating `Hom` reveals 3-cells and beyond.

Schematically,

$$\begin{aligned} f &: x \longrightarrow y, \\ \alpha &: f \Longrightarrow g, \\ \Gamma &: \alpha \Rrightarrow \beta, \\ &\vdots \end{aligned}$$

are read by repeatedly taking the object classifier of a hom-category. This is the strict/lax omega-oriented *shape* of the implementation. It should not be mistaken for a completed semantics of every weak omega-category: the available composition, action, and coherence interfaces are explicit and grow as consumers require them.

Every object has an identity arrow, and composable arrows have a composite

$$x \xrightarrow{f} y \xrightarrow{g} z \qquad \rightsquigarrow \qquad g \circ f : x \longrightarrow z.$$

We always read `g o f` as first `f`, then `g`. Identity, composition, and their higher actions are global operations of the category calculus. Some laws are selected as runtime reductions; others are exposed as typed propositional or proof-time comparisons. The mathematical reading is the familiar category law, while the implementation distinguishes which direction is a useful normal form.

> **Formal status — checked.** Evidence `CAT-ITERATED-HOMS` covers the category, object, and iterated-hom interface. The phrase "omega-oriented" describes this unbounded hom iteration; it does not claim a finished theory of arbitrary weak coherence.

2.2 The Equality-Local Category

Every groupoid classifier `A` determines a path category

$$\mathsf{Path}(A).$$

Its objects are elements of `A`, and its hom from `x` to `y` is the path category of the identity classifier `x=y`. Its identity is reflexivity and its composition is path concatenation. Since paths have symmetry, this is the groupoidal fragment of the categorical universe.

An ordinary function `f:A -> B` lifts to an iterable functor

$$\mathsf{Path}(f) : \mathsf{Path}(A) \longrightarrow \mathsf{Path}(B).$$

On objects it is `f`; on a path it is `ap_f`; on higher paths it continues by the generic functor calculus. This is a precise bridge between the type-theoretic material of Chapter 1 and the directed material here.

It is important that the bridge is an inclusion of a special case, not a collapse. A directed category `C` may contain an arrow `x -> y` without any path `x=y`. Conversely, an object path can be mapped to an arrow through the equality-local core inclusion. Later equivalence interfaces state when that inclusion captures all arrows.

> **Formal status — checked.** Evidence `CAT-PATH-CATEGORY` covers the recursive path category and the functor induced by an ordinary function. No rule identifies a general directed hom-category with an equality type.

2.3 Functors Act At Every Dimension

A functor `F:A -> B` has an object action

$$x \longmapsto F[x]$$

and, for every pair `x,y`, a hom functor

$$F_1[x, y] : \mathrm{Hom}_A(x, y) \longrightarrow \mathrm{Hom}_B(F[x], F[y]).$$

Evaluating the hom functor at an arrow `f` gives the familiar `F[f]`. Because the result before evaluation is itself a functor, the same construction already acts on 2-cells between `f` and `g`, then on higher cells. The notation suppresses this projection ladder, but the structure does not stop at the first arrow.

Functoriality has the expected equations

$$F[\mathrm{id}_x] = \mathrm{id}_{F[x]}, \qquad F[g \circ f] = F[g] \circ F[f].$$

In the active calculus these are owned globally. A named construction that is already a functor does not receive a private copy of the identity and composition laws. This is more than code reuse: it expresses the foundational thesis that substitution-like operations should be internalized until their action and coherence come from the generic functor structure.

> **Formal status — checked.** Evidence `CAT-FUNCTOR-CALCULUS` covers object action, full hom action, capped arrow action, and the generic identity and composition surfaces.

2.4 Transfors And Naturality

Given functors `F,G:A -> B`, the next hom-category is written

$$\mathsf{Transf}(F, G).$$

An object `eta:F => G` has a component at every object,

$$\eta[x] : F[x] \longrightarrow G[x].$$

Its action over an arrow `f:x -> y` expresses naturality. In a 1-categorical shadow, this is the familiar square comparing

$$G[f] \circ \eta[x] \qquad \text{and} \qquad \eta[y] \circ F[f].$$

In the iterated-hom presentation, naturality is not an external proposition attached after the components are chosen. It is part of the higher action of the transfor object. Further homs between transfors provide higher transfors and their components.

Vertical composition is computed componentwise, and identity transfors have identity components. Whiskering and functor action provide horizontal interaction. Chapter 8.2 uses precisely this infrastructure when the two compositions on 2-endomorphisms meet in the Eckmann–Hilton calculation.

> **Formal status — checked.** Evidence `CAT-TRANSFOR-CALCULUS` covers transfors, their point components, and the next naturality projection. The book uses “transfor” for the dimension-uniform notion and “transformation” when discussing its ordinary categorical shadow.

2.5 The Directed Universe And Its Families

The category of categories has

$$\mathrm{Obj}(\mathsf{Cat}) = \mathsf{Cat}, \qquad \mathrm{Hom}_{\mathsf{Cat}}(A, B) = \mathsf{Functor}(A, B).$$

Thus an arrow in the categorical universe is a functor. A Cat-valued directed family over `K` is consequently a functor

$$E : K \longrightarrow \mathsf{Cat}.$$

We write its fibre over `k` as `E[k]`. A base arrow `p:k -> l` induces the reindexing functor

$$E[p] : E[k] \longrightarrow E[l].$$

This is covariant directed transport. It can carry an object `u:E[k]` forward to `E[p](u)` even when `p` has no inverse. Ordinary path transport is recovered when `K` is a path category, but directed transport is strictly more general.

A morphism between families `E,D:K -> Cat` is a displayed or fibrewise functor

$$\Phi : E \Longrightarrow D.$$

It has a functor `Phi[k]:E[k] -> D[k]` in each fibre. Over a base arrow `p:k -> l`, its coherent comparison has the directed form

$$D[p](\Phi[k](u)) \longrightarrow \Phi[l](E[p](u)).$$

The arrow points from "map, then transport in `D`" toward "transport in `E`, then map." It need not be an identity and need not be invertible. This is the first elementary appearance of laxity in the dependent calculus. Strict/cartesian constructions may make the comparison compute to an identity, but that is additional structure or focused computation, not the default meaning of a family morphism.

> **Formal status — checked.** Evidence `CAT-DIRECTED-FAMILIES` covers Cat-valued families, fibres, arrow reindexing, fibre functors, and the canonical displayed comparison cell. A general directed family is not required to send its base arrows to equivalences.

2.6 Totals And Sections

A family can be read in two complementary ways. Its **total category** gathers all fibre objects together:

$$\sum_{k:K} E[k].$$

An object is a pair `(k,u)` with `u:E[k]`. An arrow

$$(k, u) \longrightarrow (l, v)$$

contains a base arrow `p:k -> l` and a fibre arrow

$$\alpha : E[p](u) \longrightarrow v$$

in `E[l]`. The canonical transport arrow is the special case

$$(p, \mathrm{id}_{E[p](u)}) : (k, u) \longrightarrow (l, E[p](u)).$$

This description makes directed dependence concrete. Transport is an actual arrow in the total category, and its direction is inherited from the base.

The **section category** is the dependent product

$$\prod_{k:K} E[k].$$

A section `s` assigns an object `s[k]:E[k]` and, for every base arrow, a coherent comparison

$$s[p] : E[p](s[k]) \longrightarrow s[l].$$

An arrow between sections is not an arbitrary pointwise family of fibre arrows. Its components must be natural over the base. This distinction disappears over a trivial base but is essential over a directed one.

For a constant family with fibre `A`, the total is the product category `K x A`, while the section category is the functor category `Functor(K,A)`. These are useful checks on the definitions: a section of a constant family is precisely a functorial choice varying over `K`.

> **Formal status — checked.** Evidence `CAT-SIGMA-PI` covers the Sigma total, its canonical transport arrow, the Pi section facade, and section evaluation. Some broader Sigma/Pi adjunctions and generic section-total packaging remain future work; they are not prerequisites for Chapter 8.

2.6.1 Fibrewise Contexts

Totals make genuine dependence visible, but not every pair of declarations over a base depends on one another. Let $B, C : K \vdash \mathsf{Cat}$ be two families over the same category. Their fibrewise product is the transparent family

$$P(B, C)[k] := B[k] \times C[k], \qquad P(B, C)[p] := B[p] \times C[p].$$

The second formula uses the same base arrow p in both components. It is the action required by a context of independent siblings

$$k : K, \qquad b : B[k], \qquad c : C[k].$$

This differs from a genuinely dependent context

$$k : K, \qquad a : A[k], \qquad b : B[k, a].$$

In the first context, b and c may be projected, paired, exchanged, or contracted while the common prefix k is fixed. In the second, exchanging a and b without transporting the classifier of b is generally ill-typed.

The fixed-base structural maps are

$$\begin{aligned} \mathsf{projL}_d(B, C) &: P(B, C) \Longrightarrow B, \\ \mathsf{projR}_d(B, C) &: P(B, C) \Longrightarrow C, \\ \mathsf{pair}_d(\Phi, \Psi) &: E \Longrightarrow P(B, C), \end{aligned}$$

where $\Phi : E \Longrightarrow B$ and $\Psi : E \Longrightarrow C$. They satisfy the displayed product cuts

$$\mathsf{projL}_d \circ \mathsf{pair}_d(\Phi, \Psi) \rightsquigarrow \Phi, \qquad \mathsf{projR}_d \circ \mathsf{pair}_d(\Phi, \Psi) \rightsquigarrow \Psi.$$

Their fibre functors, base-arrow actions, and selected internalized higher cells compute componentwise. Exchange and contraction are consequently derived maps:

$$\begin{aligned} \mathsf{swap}_d(B, C) &:= \mathsf{pair}_d(\mathsf{projR}_d, \mathsf{projL}_d), \\ \mathsf{diag}_d(B) &:= \mathsf{pair}_d(\mathsf{id}_d, \mathsf{id}_d). \end{aligned}$$

The family $P(B, C)$ is built from ordinary categorical product functoriality; it is not a new primitive family former. Reindexing a grouped context is presented canonically by reindexing its two families and rebuilding their fibrewise product.

> **Formal status — checked.** Evidence `CAT-FIBREWISE-CONTEXT` covers the transparent fibrewise family, its displayed projections and pairing, componentwise object, arrow, and selected higher action, the two product cuts, and the derived swap and diagonal. It does not license exchange across a genuine dependency edge.

2.6.2 Base Change And Evaluation

A functor $F : A \vdash K$ can change the base of a family $D : K \vdash \mathsf{Cat}$. The pulled-back family F^*D has fibre $(F^*D)[a] = D[F[a]]$. Its total category carries a canonical functor

$$\mathsf{Tot}(F, D) : \sum_{a:A} (F^*D)[a] \longrightarrow \sum_{k:K} D[k]$$

with computations

$$(a, u) \longmapsto (F[a], u), \qquad (p, \alpha) \longmapsto (F[p], \alpha).$$

The base component is acted on by F, while the fibre component is retained. This is the asymmetric Grothendieck totalization of family reindexing. It is not a claim that arbitrary functors between total categories admit a computational pullback.

The same fibrewise structure supports coherent evaluation. Fix an ordinary category A and a family $B : K \vdash \mathsf{Cat}$, and write

$$S(A, B)[k] := \mathrm{Functor}(A, B[k]).$$

Then

$$\mathsf{Eval}_d(B) : P\big(S(A, B), \mathsf{Const}_K(A)\big) \Longrightarrow B$$

projects in each fibre to ordinary functor evaluation:

$$\mathsf{Eval}_d(B)[k] = \mathsf{Eval}(A, B[k]).$$

Weakening is supplied by the displayed terminal map

$$\mathsf{Terminal}_d(E) : E \Longrightarrow \mathsf{Const}_K(1).$$

Composing it with a constant section yields a fixed argument. Pairing that argument with a varying functor and applying Eval_d therefore interprets application without asking the user to supply a separate naturality square. Base-arrow and higher action are inherited from the internal functorial calculus.

> **Formal status — checked.** Evidence `CAT-BASE-CHANGE-TOTALIZATION` covers the object and arrow action of asymmetric pullback totalization. Evidence `CAT-DISPLAYED-EVALUATION` covers constant-domain displayed evaluation, terminal weakening, and their retained generic base and higher action. Arbitrary mixed-domain evaluation remains outside this result.

2.7 Representables And Variance

Fix `x:K`. The covariant fixed-source representable family is

$$\mathsf{Rep}_x[k] := \mathrm{Hom}_K(x, k).$$

For `p:k -> l`, it acts by postcomposition:

$$\mathsf{Rep}_x[p](q) = p \circ q.$$

This is exactly the action used in the WalkingEnd decoder: a based arrow `q:* -> k` is carried along `p` to `p o q:* -> l`.

The source variable is contravariant. Given `r:x -> y`, precomposition defines a family morphism

$$\mathsf{Rep}_y \Longrightarrow \mathsf{Rep}_x, \qquad q \longmapsto q \circ r.$$

Keeping these two variances distinct is not pedantry. Postcomposition changes the target of an arrow; precomposition changes its source. Emdash gives them separate runtime owners and compares alternate presentations only where the types justify it.

> **Formal status — checked.** Evidence `CAT-REPRESENTABLE` covers the fixed-source representable and its contravariant source action. Generic functoriality owns its action in the varying target.

2.8 The Interfaces Needed By The Main Proof

The Chapter 8 calculation uses only a compact part of this chapter:

1. `Code:W -> Cat` is a directed family, so every based arrow `p:* -> x` acts by a functor `Code[p]`.
2. The target family is the representable `Rep_*`, whose base-arrow action is postcomposition.
3. Natural powers form a functor from `Path(Nat)` to the based hom-category, not merely a function on objects.
4. A contextual decoder is a family morphism from `Code` to `Rep_*`; its base-arrow comparison produces the normalization cell.
5. The ordinary and higher functor laws supply identity, composite, and cell action without WalkingEnd-specific copies.

The proof therefore depends on category theory at exactly the point where a carrier-only argument would lose direction. The next chapters add the logical, equivalence, induction, and height interfaces needed to turn that directed construction into a numerical classification.

3. Logic, Propositions, And Sets

The propositions-as-types reading begins with a useful ambiguity. Any classifier may be read as a proposition: to prove it is to construct an inhabitant. But not every classifier behaves like a truth value. An inhabitant of Bool tells us which Boolean was chosen; a natural number contains still more data. Logic becomes precise only after we distinguish *having evidence* from *having no relevant choice of evidence*.

This distinction is particularly important in a directed foundation. A classifier may carry higher identity information even when the surrounding category also carries noninvertible arrows. Proposition and set evidence control the groupoidal identity layer; they do not erase directed structure.

3.1 Propositions As Types, Broadly Read

The basic logical readings are constructive:

statement	classifier
P and Q	$P \times Q$
P implies Q	$P \longrightarrow Q$
for every $x : A,\ P(x)$	$\prod_{x:A} P(x)$
there is $x : A$ with $P(x)$	$\sum_{x:A} P(x)$
falsehood	$\mathsf{Empty}.$

Under this reading, a proof of an existential statement contains a witness and its evidence. A proof of an implication is an operation that transforms evidence. Negation is

$$\neg P := P \longrightarrow \mathsf{Empty}.$$

This is enough to state the negative theorems of Chapter 8. To prove that the walking loop is not the identity, for instance, one assumes an equality and constructs an inhabitant of Empty by mapping it to an impossible Nat equality.

What this reading does not supply is automatic proof irrelevance. If `P` has two inhabitants, the bare fact that both prove a statement says nothing yet about whether they are equal. That is a separate property of `P`.

3.2 Contractibility And Proposition-Valuedness

A classifier `A` is **contractible** when it has a chosen centre and a path from that centre to every point:

$$\mathsf{isContr}(A) := \sum_{a_0 : A} \prod_{a : A} (a_0 = a).$$

Contractibility contains both existence and uniqueness, with uniqueness understood homotopically. The centre is data, but the total classifier of contractibility evidence is itself proposition-valued: any two such packages are equal.

A classifier is **proposition-valued** when its identity classifiers are contractible. In the familiar shorthand, any two of its inhabitants are equal, and there is no further choice among the paths witnessing that fact. It may be inhabited, like Unit, or uninhabited, like Empty. If it is inhabited, it is contractible.

The adjective "proposition-valued" is deliberate. We still use every classifier as a proposition in the broad Curry–Howard reading. The additional property says that inhabitation carries no mathematical information beyond truth.

> **Formal status — checked.** Evidence `LOGIC-TRUNCATION-PREDICATE` covers `IsContr` and the recursive truncation predicate whose `-1` and `0` specializations are proposition- and set-valuedness.

3.3 Sets As A Homotopical Property

A classifier `A` is **set-valued** when every identity classifier `x=y` is proposition-valued. Equivalently, any two parallel paths in `A` are equal. This is the type-theoretic notion of a set: it concerns the height of identity information, not membership in a global cumulative set universe.

The hierarchy begins

$$\begin{aligned} -2 &: \text{contractible classifiers},\\ -1 &: \text{proposition-valued classifiers},\\ 0 &: \text{set-valued classifiers},\\ 1 &: \text{groupoid-valued classifiers},\\ &\vdots \end{aligned}$$

and each successor level asks that every identity classifier lie one level lower. Chapter 7 develops this recursion uniformly. For now, sethood provides the exact uniqueness principle needed for arithmetic codes.

Unit is proposition-valued because it is contractible. Empty is proposition-valued by elimination: from an alleged first inhabitant one may derive the required path data. Nat is set-valued by nested induction on its visible constructors. Equality of zero with a successor reduces to Empty; equality of two successors reduces to predecessor equality; the remaining cases reduce to the proposition evidence for Unit or Empty.

> **Formal status — checked.** Evidence `LOGIC-NAT-SETHOOD` supplies explicit terms witnessing that Unit and Empty are proposition-valued and Nat is set-valued. Nat sethood is not inferred merely from a rewrite table; it is an inhabitant of the internal `IsSetGrpd(Nat)` classifier.

3.4 Evidence Is Not Erasure

It is tempting to read "being a property" as permission for the kernel to erase every proof to one constant. That is not the policy here. Emdash retains evidence terms and proves that the classifier containing them is proposition-valued.

For every level `n` and classifier `A`, the classifier

$$\mathsf{isTrunc}_n(A)$$

of evidence that `A` lies at level `n` is proposition-valued. Hence two truncation witnesses are equal, but their constructors and projections remain available to computation. This matters for packaged universes: a package can retain both a carrier and its truncation evidence while still having equality controlled by the carrier.

The same style recurs for equality-valued equivalence evidence. Rather than declaring inverse-law proofs irrelevant by fiat, the theory proves that their classifier is a proposition. Retained evidence supports computation and transport; proposition-valuedness supports uniqueness and truncation.

> **Formal status — checked.** Evidence `LOGIC-TRUNCATION-EVIDENCE-PROP` establishes property-valuedness at every recursive truncation level. It introduces no proof-erasure rewrite.

3.5 Constructive Reasoning

The foundational rules used in this book are constructive. To prove `P or Q` one must select a side in whatever sum-like interface is in scope; to prove an existential one must provide a witness; to refute a claim one must map its evidence to Empty. The law of excluded middle and double negation elimination are not silently used as global inference rules.

This does not forbid classical mathematics. A classical principle can be studied as additional structure, restricted to a suitable proposition-valued universe, or derived in a context where decidability data are available. The point is bookkeeping: a proof should reveal when such data enter.

The WalkingEnd theorem needs none. Its code is computed by functor action, its inverse laws by contextual elimination and Nat induction, and its noninvertibility by successor/zero discrimination. The argument is therefore valid in the constructive core presented here.

> **Formal status — mathematical development.** This section states the logical discipline of the book. It does not claim a complete internal library of all connectives, propositional truncations, choice principles, or classical axioms.

3.6 Two Uses Of Sethood In The Main Calculation

Sethood enters Chapter 8 in two distinct ways.

First, Nat sethood controls equality in the code fibre `Path(Nat)`. It ensures that higher coherence between arithmetic equalities is unique at the needed level. This lets the power construction lift equality of naturals without introducing uncontrolled higher data.

Second, the based hom carrier is proved set-valued after the calculation. One proof comes directly from the one-dimensional signature of `W`; another transports Nat sethood backward across the carrier equivalence. Agreement of the conclusions does not collapse the proofs: each exposes a different reason for sethood.

This is typical of univalent foundations. A property may be reached through dimension, through equivalence, or through a direct induction. Since the classifier of evidence is proposition-valued, the resulting witnesses agree, yet their constructions remain mathematically informative.

4. Equivalence And Univalence

Univalence is often summarized by the phrase “equivalent things may be identified.” In a directed setting, that sentence must be typed with care. Equivalent *classifiers*, isomorphic objects of a 1-category, recursively invertible arrows, and arbitrary functors are different notions. The theory becomes clearer, not more cumbersome, when each receives its own interface.

The WalkingEnd example is the test. Its generator is a perfectly good arrow, and `Code` sends it to a perfectly good functor, but neither is an equivalence. At the same time, the final comparison between its endomorphism carrier and Nat is a type equivalence. Univalent foundations must support both statements without forcing one into the shape of the other.

4.1 Maps With Contractible Fibres

For a function `f:A -> B` and `b:B`, its homotopy fibre is

$$\mathsf{fib}_f(b) := \sum_{a:A} (f(a) = b).$$

The function is an equivalence when every such fibre is contractible. A `TypeEquiv(A,B)` packages `f` with this evidence. The centre of the fibre over `b` selects an inverse value `f^{-1}(b)`, and its path component gives

$$f(f^{-1}(b)) = b.$$

Contractibility of the fibre also yields the other law

$$f^{-1}(f(a)) = a.$$

This definition has two advantages. It treats being an equivalence as a property of a fixed map, and it supplies a canonical selected inverse from the contractible-fibre evidence. Conversely, explicit inverse data with both homotopies can be adjusted and converted into contractible-fibre evidence. That is the route used by the WalkingEnd carrier comparison: `encode` and `power` are built first, their two round trips are proved, and then the equivalence package is formed.

> **Formal status — checked.** Evidence `EQUIV-TYPE` covers homotopy fibres, the equivalence-map predicate, `TypeEquiv`, its selected inverse, and both inverse paths.

4.2 Algebra Of Type Equivalences

Equivalences have reflexivity, symmetry, and composition:

$$\begin{aligned} \mathsf{id}_A &: A \simeq A, \\ e^{-1} &: B \simeq A, \\ e_{BC} \circ e_{AB} &: A \simeq C. \end{aligned}$$

Composition is written in categorical order; the forward map of the last line sends `a` first through `e_AB` and then through `e_BC`. The inverse laws are constructed from explicit quasi-inverse data and the path algebra of Chapter 1.

Not every mathematically true equation between equivalence packages is chosen as a runtime eta rule. What computes is the public interface needed by consumers: the selected forward map of reflexivity, symmetry, and composition, together with designated projections of inverse data. Remaining coherence is propositional. This is the same separation between computation and evidence that we saw for Nat associativity.

> **Formal status — checked.** Evidence `EQUIV-TYPE-ALGEBRA` covers reflexivity, symmetry, composition, and their selected forward-map computation.

4.3 Univalence In The Groupoid Universe

Every universe path `p:A=B` induces a type equivalence by transporting elements:

$$\mathsf{idtoequiv}(p) : A \simeq B.$$

Univalence supplies the converse in a coherent way. In the groupoid universe, emdash has an operational decoder

$$\mathsf{ua}(e) : A = B$$

and named round trips

$$\mathsf{ua}(\mathsf{idtoequiv}(p)) = p, \qquad \mathsf{idtoequiv}(\mathsf{ua}(e)) = e.$$

Transport along the decoded path agrees propositionally with the forward map of `e`. The qualification “propositionally” is important. A single operational decoder owns the comparison; the kernel does not turn every universe transport expression into arbitrary equivalence application by a broad rewrite.

This gives the groupoid/type layer its univalent reading: identity in the universe and equivalence of decoded classifiers carry the same information through the named interface. It does not say that a noninvertible arrow in an arbitrary category is an identity.

> **Formal status — checked.** Evidence `UNIV-GROUPOID` covers the decoder-oriented groupoid-univalence equivalence, both round trips, and the selected transport agreement.

4.4 Universes With Retained Truncation Evidence

For each truncation level `n`, the theory also packages a classifier together with evidence that it is `n`-truncated:

$$\mathsf{TruncU}_n := \sum_{A:\mathsf{Grpd}} \mathsf{isTrunc}_n(A).$$

The evidence is retained, not erased. Nevertheless, because truncation evidence is proposition-valued, equality of packages is controlled by equality of their carriers. Combining that fact with groupoid univalence gives the restricted comparison

$$(A, h_A) = (B, h_B) \simeq A \simeq B.$$

The familiar universes of propositions, sets, and groupoids are the `-1`, `0`, and `1` cases of this package. This is a concrete example of structure identity: once a field is proposition-valued, identifying the carrier supplies the dependent identification of that field.

> **Formal status — checked.** Evidence `UNIV-TRUNCATED` covers the carrier-equivalence view of paths between packaged truncated classifiers and its named encode/decode laws. It is a restricted univalence theorem, not a truncation reflector.

4.5 Three Categorical Notions Of Invertibility

At the categorical layer we distinguish three interfaces.

An ordinary isomorphism between objects `x,y:C` consists of a forward arrow, one inverse arrow, and equality-valued left and right inverse laws:

$$f : x \to y, \qquad g : y \to x, \qquad g \circ f = \mathrm{id}_x, \qquad f \circ g = \mathrm{id}_y.$$

For a fixed arrow `f`, the native classifier `OmegaEquivAlong(f)` carries recursively equality-valued inverse evidence. Its selected left and right inverse arrows may be presented separately, with their laws living in the next hom-categories. Finally, `OmegaEquiv(C,x,y)` packages a chosen forward arrow together with such evidence.

The distinction allows the interface to remain iterable. Inverse laws at one dimension are equality data in a hom-category, where the same machinery can act again. An ordinary isomorphism has a checked one-way lift into the native omega-equivalence package, but the notions are not collapsed definitionally.

> **Formal status — checked.** Evidence `EQUIV-OMEGA` covers the fixed-arrow and packaged equality-valued interfaces and the transparent package built from an object path. Evidence `EQUIV-ORDINARY-ISO-LIFT` covers the one-way lift from ordinary isomorphism evidence.

For an object path `p:x=y`, path induction constructs a forward arrow, an inverse, and both cancellation laws. Thus equality can be *reified* as a computational equivalence package. A bare classifier-level cast may permit an identity term to be used at the stable equivalence facade, but it does not magically give that term the projections of a package; observable computation uses the explicit construction.

The converse direction has a more delicate status. The current native surface supports selected equality/equivalence casts and rigid universe boundaries, but a full general equivalence between arbitrary object equality and ordinary 1-categorical isomorphism evidence is not a compatibility prerequisite and is not claimed here.

> **Formal status — research boundary.** Evidence `UNIV-FULL-OBJECT-ISO` records the intentionally absent two-sided ordinary-isomorphism/object-equality theorem. Its absence does not weaken the checked path-to-equivalence construction used elsewhere in the book.

4.6 Equivalence Evidence Is A Property

For a fixed arrow `f`, two complete choices of equality-valued inverse evidence are equal. The proof does not erase the choices. It shows that the left- and right-inverse fibres are contractible, using ordinary associativity, unit laws, and the supplied inverse equations, and then transfers that contractibility to the native record.

Consequently,

$$\mathsf{OmegaEquivAlong}(f)$$

is proposition-valued for every category and fixed arrow, without assuming that the category has finite height or locally set-valued homs. This is the categorical analogue of the fact that "is an equivalence" should be a property of a map even though an explicit quasi-inverse package may contain choices.

> **Formal status — checked.** Evidence `EQUIV-EVIDENCE-PROP` is the dimension-independent property theorem for the native fixed-arrow evidence.

4.7 Hom Action And Groupoidal Sources

Equivalence evidence must itself act functorially if the foundation is to be usable at higher dimension. The one-way derived hom-action layer sends fixed-arrow omega-equivalence evidence through the next hom action of an ordinary functor. In particular, if a category is coherently groupoidal, its core inclusion is an omega-equivalence and each directed arrow can be related to an object path through the selected homwise inverse.

For a directed family `D:C -> Cat`, coherent groupoidality of `C` then implies that transport along a base arrow is an equivalence:

$$D[f] : D[x] \simeq D[y].$$

This recovers the familiar groupoidal behavior of path-indexed families as a special case. It also states the boundary sharply: without groupoidality, `D[f]` remains only a functor.

> **Formal status — checked.** Evidence `EQUIV-HOM-ACTION` covers the one-way next-hom action and its specialization to equivalence of fibre transport over a coherently groupoidal source.

4.8 Why Successor Needs No Univalence In Chapter 8

The circle's universal cover sends an invertible loop to successor on the integers. To define it as a universe-valued family, one first recognizes successor as an equivalence and then uses univalence to obtain a universe path.

The WalkingEnd code has a different type:

$$\mathsf{Code} : W \longrightarrow \mathsf{Cat}.$$

Its generator is a directed arrow in `W`, so its image need only be a functor. Natural-number successor qualifies directly even though it misses zero and has no inverse. Forcing univalence into this step would change the mathematics by demanding reversibility where the signature deliberately has none.

The carrier equivalence at the end of the proof is nevertheless genuinely a type equivalence, and the surrounding universes retain their checked univalence interfaces. The lesson is not that univalence disappears. It is that univalence governs equivalence and identity, while directed functoriality also governs noninvertible action.

5. Induction, Arrow Induction, And Universal Properties

Induction is often introduced as a collection of unrelated proof rules: one for natural numbers, another for equality, and still others for higher inductive types. A more durable view is that an inductively presented object comes with a way to extend compatible data from its generators. Recursion extends nondependent data; induction extends data in a family; contextual elimination lets both the source and target vary. The increasing amount of context is not bureaucratic overhead. It is what records naturality.

In a functorial setting, this extension principle should itself preserve arrows and higher cells. That demand turns a familiar logical rule into a categorical construction. This chapter begins with ordinary induction, then develops the outgoing-arrow category and the arrow-induction interface used throughout emdash. The result is a bridge between equality induction and the contextual eliminator of the walking endomorphism.

5.1 Data On Generators

For the natural numbers, dependent induction has the familiar shape. Given a family `P(n)`, a base element

$$z : P(0),$$

and a step operation

$$s_n : P(n) \longrightarrow P(n+1),$$

there is a section

$$\mathsf{ind}_{\mathbb{N}}(z, s) : \prod_{n:\mathbb{N}} P(n)$$

whose observations at zero and successor compute to the supplied data. If `P` is constant, the same interface is recursion. Thus recursion is not a second principle: it is dependent elimination with no meaningful dependency.

The same pattern appears for a category presented by an object and an endomorphism. To map it into a category `C`, one supplies an object of `C` and an endomorphism of that object. To define a dependent section, one supplies an object in the fibre over the generating object and a lift over the generating arrow. To compare two varying families, one supplies a functor between their base fibres and a coherent cell over the generator. Chapters 6 and 8 will instantiate precisely these three levels.

This perspective separates two questions:

1. what data an eliminator accepts; and

2. how its observations compute at the constructors.

Both matter. Existence without computation cannot drive a normalization proof; computation without a coherent extension does not define an action on the whole object.

In the rule schema of Appendix G.4, generator data are the introduction clauses, the induced section or functor is elimination, and the constructor observations are computation. Uniqueness or initiality is a separate universal clause, while coherent action on arrows and higher cells is the specifically functorial clause. We will report all five independently rather than treating the word "induction" as evidence for whichever clauses have not yet been supplied.

5.2 Equality Induction As The Local Case

For a type `A`, right-based equality induction says that, with the right endpoint `y` fixed, a family

$$C(x, p) \qquad (x : A,\ p : x = y)$$

is determined by its value at `(y,refl_y)`. In one conventional orientation,

$$\frac{d : C(y, \mathsf{refl}_y)}{\mathsf{J}(d, x, p) : C(x, p)}.$$

The reflexive observation computes to `d`. Ordinary action on paths and dependent action on paths are specializations of this principle. They explain why a function respects equality and how a section over a family produces a dependent path.

> **Formal status — checked.** Evidence `TT-EQUALITY-INDUCTION`. The equality-local layer provides reflexivity, right-based dependent induction, ordinary path action, and dependent path action.

Equality induction is already functorial when it is read inside the path category `Path(A)`: objects are elements of `A`, arrows are equalities, and every arrow is invertible. But this is a special, groupoidal base. A directed category contains arrows that need not have inverses, so we want an induction interface whose statement does not presuppose that a given arrow is an equality.

5.3 The Category Of Outgoing Arrows

Fix a category `Z` and an object `x`. The category of arrows leaving `x` is the total category of the represented family

$$\mathsf{PathOut}_Z(x) := \sum_{y:\, Z} \mathrm{Hom}_Z(x, y).$$

Its objects are pairs `(y,p)` with $p : x \to y$. It has a distinguished reflexive object

$$\mathsf{reflout}_x := (x, \mathrm{id}_x).$$

For every outgoing arrow $p : x \to y$, functorial action of the representable family supplies a canonical total arrow

$$\rho_{x,y,p} : (x, \mathrm{id}_x) \longrightarrow (y, p) \quad \text{in } \mathsf{PathOut}_Z(x).$$

Its base component is `p`; its fibre endpoint reduces by the unit law $p \circ \mathrm{id}_x = p$. The direction of `rho` is significant. It starts at the reflexive arrow and reaches `p`. Nothing in the construction produces a reverse arrow, and no invertibility premise is used.

> **Formal status — checked.** Evidence `IND-PATHOUT`. `PathOut_cat` is the Sigma total of the fixed-source representable, `pathout_refl_obj` is its distinguished object, and `pathout_refl_arrow` is the canonical Sigma transport arrow `rho`.

The name "PathOut" emphasizes an analogy, not an identification. When `Z=Path(A)`, its objects really are equality paths out of `x`. For a general `Z`, they are directed arrows. The same categorical shape therefore hosts both ordinary path induction and a genuinely directed extension principle.

The source object also varies contravariantly. An arrow $r : x \to y$ induces a functor

$$\mathsf{PathOut}_Z(y) \longrightarrow \mathsf{PathOut}_Z(x), \qquad (z, q) \longmapsto (z, q \circ r).$$

Precomposition changes which object the arrows leave, while postcomposition changes where they arrive. Keeping these variances separate will prevent a common mistake in Chapter 8: the based representable there varies covariantly in its endpoint, not contravariantly in its source.

5.4 Fixed-Source Arrow Induction

Let

$$E : \mathsf{PathOut}_Z(x) \longrightarrow \mathsf{Cat}$$

be a directed family. An element

$$u \in E[\mathsf{reflout}_x]$$

can be transported along every canonical arrow `rho`. This produces a section

$$\mathsf{path\_ind}(x, E, u) : \prod_{q:\, \mathsf{PathOut}_Z(x)} E[q],$$

whose readable value is

$$\mathsf{path\_ind}(x, E, u)(y, p) = E[\rho_{x,y,p}](u).$$

This formula is arrow induction: data at the identity arrow extends along every outgoing arrow. Because `E` is Cat-valued, the result is a section with coherent action, rather than merely a function that chooses one object in each fibre.

> **Formal status — checked.** Evidence `IND-ARROW`. The fixed-source section is `path_ind_sec`; `PathInd_func` packages its action in the motive and base datum, and `PathInd_transfd` internalizes the theorem while the source object varies.

There are three layers here, and it helps not to conflate them:

- `path_ind_sec` is the readable theorem at a fixed source;
- `PathInd_func` says that this theorem acts functorially on motives and their data;
- `PathInd_transfd` records coherence as the source object moves.

The third layer uses the contravariant source action of `PathOut` and the corresponding pullback of motives. Its naturality is internal to the displayed transformation. One need not append an external family of hand-written commuting squares.

5.5 Composition As A Diagnostic

An abstract eliminator earns its keep by recovering an operation we already understand. Fix `x` and give `(y,p)` the represented motive

$$E(y,p) := \big(\mathrm{Rep}_Z(y) \longrightarrow \mathrm{Rep}_Z(x)\big).$$

At the reflexive arrow, the identity transformation supplies the base datum. Arrow induction then extends it to every $p : x \to y$. Evaluating the result at $q : y \to z$ gives

$$q \circ p : x \longrightarrow z.$$

Thus ordinary categorical composition appears as transport from the identity case. The resulting operation is still packaged functorially, so its action on higher cells remains available to later constructions.

> **Formal status — checked.** Evidence `IND-COMPOSITION`. `CompMotive_catd` is the represented motive, `path_comp_sec` is the induced section, and `path_comp_func` exposes composition as the object action of a functor.

This benchmark is more informative than a proof that some carrier-level function exists. It checks the direction of `rho`, the variance of the representable, and the agreement of the generic Sigma and Pi constructions. If any of those are reversed, the result has the wrong endpoints before one even asks for associativity.

5.6 Returning To Literal Equality

Set `Z=Path(A)` and let `D` be a directed family over that path category. There are now two ways to move `u` along an equality `p:x=y`:

1. use the functorial action of `D` on the arrow `p`; or

2. use primitive equality induction with a function-valued motive.

They need not be definitionally the same expression. The active calculus proves that they are propositionally equal:

$$\mathsf{transport}_D^{\mathrm{structured}}(p, u) = \mathsf{transport}_D^{\mathsf{J}}(p, u).$$

> **Formal status — checked.** Evidence `IND-PATH-COMPARISON`. `path_cat_structured_transport_agrees_ind_eqr` compares the existing directed-family action with primitive right-based equality induction. It does not introduce a second equality eliminator or turn the comparison into a global rewrite.

This is the precise relation between the equality-local and directed views. Functorial transport extends beyond equality paths; on a literal path category, it agrees with `J` by a theorem. The groupoidal case is therefore recovered without making all directed arrows groupoidal.

5.7 Contextual Elimination

Fixed-source arrow induction varies a motive over outgoing arrows. A contextual eliminator goes one step further: it compares two families over an inductively presented base. Schematically, suppose $R, D : K \to \mathsf{Cat}$. At each base object it should provide a functor

$$T_k : R[k] \longrightarrow D[k],$$

and for each base arrow $f : k \to k'$ a directed comparison

$$D[f] \circ T_k \Longrightarrow T_{k'} \circ R[f].$$

This is exactly the data of a displayed functor in the lax direction selected by the calculus. If `R` is terminal, a displayed functor becomes a section of `D`. If both `R` and `D` are constant, it becomes an ordinary functor out of `K`. Section induction and recursion are therefore special cases of contextual elimination.

For an inductively presented `K`, the desired eliminator should build the whole displayed functor from compatible data at the constructors. The WalkingEnd interface of Chapter 6 realizes this pattern for one point and one directed loop. Chapter 8 then uses the genuinely contextual form: its source is the code family and its target is a representable family, so neither side can be discarded as mere bookkeeping.

5.8 Universal Properties: What Is And Is Not Packaged

Induction principles often admit a universal-property formulation. For an ordinary inductive type, one expects the canonical algebra to be initial, or homotopy-initial when maps carry higher equality. For a directed higher inductive category, one similarly expects a category of algebras, structured functors, and coherent transfors in which the presented object is suitably initial.

That formulation is mathematically attractive because it gathers recursion, induction, uniqueness, and higher coherence into one statement. It is also more demanding than writing a selected eliminator. One must define the algebra category at every relevant cell dimension, identify its notion of equivalence, and prove contractibility or an equivalent universal mapping property.

> **Formal status — research boundary.** Evidence `IND-GENERAL-INITIALITY`. The current calculus packages Nat and equality induction, generic `PathOut` arrow induction, and the selected WalkingEnd eliminator. It does not yet prove a general equivalence between such interfaces and homotopy-initial categorical algebras.

We will therefore use “walking” in its presentation-theoretic sense and use the eliminator that is actually checked. Full functor-category initiality is a strengthening target, not an implicit premise of the computation.

5.9 The Interface Needed For Encode–Decode

The next two chapters isolate the additional ingredients required by the walking-endomorphism calculation:

- Chapter 6 supplies a contextual eliminator whose generator datum is a coherent directed transformation;
- Chapter 7 supplies one-dimensionality, allowing a directed 2-cell between parallel based arrows to be read as equality.

The roles are deliberately ordered. Elimination constructs directed data. Truncation may later show that some of that data is unique or equality-like. Reversing the order would erase the very directionality the example is meant to expose.

6. Directed Higher Inductive Types

An ordinary inductive type is specified by point constructors. A higher inductive type may also be specified by path constructors and constructors between paths. In a directed foundation, the corresponding presentation may contain objects, directed arrows, and higher directed cells. The change is small in notation and large in meaning: an arrow constructor does not carry an inverse unless inverse data are separately supplied.

This chapter develops the one directed higher-inductive interface that the current emdash calculus actually implements. It is enough to support the central calculation of Chapter 8 and to reveal what a reusable directed-HIT schema would have to provide. We will not promote this single example into a general theorem about all cell presentations.

6.1 Constructors With Direction

At a schematic level, a directed higher-inductive category may have:

- **object constructors**, such as `b:Obj(W)`;
- **arrow constructors**, such as $\ell : b \to b$;
- **cell constructors**, identifying or comparing composites of arrows;
- **dimension conditions**, controlling which higher cells remain nontrivial.

The eliminator must mirror these constructors. A nondependent recursor into a category `C` asks for an interpretation of every object, arrow, and cell constructor in `C`. A dependent eliminator into a family `D` asks for lifts in the appropriate fibres. A contextual eliminator between two families asks for fibre functors together with comparison cells. In each case, constructor computations say that the induced map observes the supplied data at the constructors.

For a groupoidal path constructor, one may freely use inverse paths in later reasoning. For a directed arrow constructor, that move is unavailable. If a presentation contains only

$$\ell : b \longrightarrow b,$$

then it generates forward composites $\mathrm{id}_b, \ell, \ell \circ \ell, \ldots$; no negative power has been introduced. This is the fundamental distinction between the walking endomorphism and the circle.

The formal ledger for such a signature is therefore longer than a constructor list. It records formation, introductions and their boundaries, the chosen eliminator, constructor beta rules, higher action and coherence, dimension data, and the current uniqueness status. The general template and the exact WalkingEnd instance are collected in Appendix G.6; this chapter develops the same clauses in mathematical order.

6.2 The Selected Walking Signature

Write `W` for the category `WalkingEnd`, with constructors

$$* : \mathrm{Obj}(W), \qquad \ell : \mathrm{Hom}_W(*, *).$$

The category is opaque. Its object classifier, hom-categories, identities, and composition are not defined by reducing them to a Nat model. Alongside the constructors, the signature retains evidence that `W` is one-dimensional: every hom-category is discrete.

> **Formal status — checked.** Evidence `WE-SIGNATURE`. `WalkingEnd_cat`, `walking_base`, and `walking_loop` are opaque declarations. `walking_end_is_one_cat` is explicit signature evidence; it does not unfold the category into a concrete presentation.

One-dimensionality is a height condition, not an invertibility condition. It says that parallel 2-cells in each hom are equality-like and that there are no nontrivial higher directed layers there. It does not say that every 1-cell of `W` has a reverse. Chapter 7 will make the distinction precise.

Why keep `W` opaque? Because the theorem in Chapter 8 is then a consequence of its constructors, eliminator, and dimension evidence. If its based hom were defined to be Nat, the calculation would merely restate a definition. Opacity turns the comparison with Nat into a normalization theorem.

6.3 The Contextual Algebra

Let

$$R, D : W \longrightarrow \mathsf{Cat}$$

be directed Cat-valued families. At the base, suppose we are given a functor

$$u : R[*] \longrightarrow D[*].$$

There are two ways to move an element of `R[*]` once around the generator and then apply `u`:

$$D[\ell] \circ u, \qquad u \circ R[\ell].$$

The selected lax orientation asks for a transformation

$$\sigma : D[\ell] \circ u \Longrightarrow u \circ R[\ell].$$

These data form the contextual algebra of the walking endomorphism. Its eliminator produces a displayed functor

$$\mathsf{ind}^d(R, D, u, \sigma) : R \Longrightarrow D$$

over the whole of `W`.

> **Formal status — checked.** Evidence `WE-CONTEXTUAL-ELIMINATOR`. The owner is `walking_end_ind_funcd`. Its base component computes to `u`, and its displayed action on the literal generator computes to the component of `sigma` at the selected source object.

The direction of `sigma` deserves attention. It is not an equality asserting strict commutation, nor is it automatically reversible. It is a directed cell comparing the two composites. This is the right amount of coherence for the displayed-functor interface used by emdash: composition with base action commutes laxly in the selected direction, and higher naturality continues through the generic transfor calculus.

The word "contextual" means that the eliminator retains `R`, the context from which inputs are drawn. Removing `R` too early would reduce `u` to a single object and `sigma` to a single lift. That specialization is useful, but it cannot construct Chapter 8's decoder from one nonconstant family to another.

6.4 Sections And Recursors Are Special Cases

Take `R` to be the constant terminal family. A functor $R[*] \to D[*]$ is determined by an object `d` of the base fibre. The generator coherence reduces to an arrow

$$\bar{\ell} : D[\ell](d) \longrightarrow d.$$

Contextual elimination then yields a dependent section

$$\mathsf{ind}(D, d, \bar{\ell}) : \prod_{x:W} D[x].$$

Take `D` constant as well, at a category `C`. Its transport is the identity, so the data become an object `c:Obj(C)` and an endomorphism $f : c \to c$. The resulting section is an ordinary functor

$$\mathsf{rec}(C, c, f) : W \longrightarrow C$$

with the expected observations

$$\mathsf{rec}(*) = c, \qquad \mathsf{rec}[\ell] = f.$$

> **Formal status — checked.** Evidence `DHIT-DERIVED-ELIMINATORS`. `walking_end_ind_sec` and `walking_end_rec_func` are transparent specializations of the contextual eliminator, with checked base and loop observations.

This dependency is architecturally important. There is one semantic elimination principle, not three unrelated black boxes. The derived views retain the generic functor and transfor action, which is why a recursor-built object such as `Code` can later be acted on at higher cells.

6.5 What "Computes" Means

A constructor computation can be visible in several ways:

1. a runtime reduction may expose the supplied constructor datum;
2. a typed equality may state that two stable observations agree;
3. a proof-time comparison may join expressions without selecting either as the runtime normal form.

For the walking eliminator, the contextual base projection and the displayed generator cell are the constructor-specific runtime owners. Narrow projections expose the same data through the derived section and recursor views. Transparent equality theorems record those observations at their readable types.

This distinction prevents a tempting but dangerous simplification. One should not add a new constructor-specific rewrite merely because a theorem is mathematically true. Generic functoriality already owns preservation of identities, composition, and ordinary naturality. The HIT-specific rules own only what observing the literal constructors must reveal.

The book will normally write a constructor equation with `=`. A formal status note determines whether that equation is runtime computation, propositional equality, or a mathematical development. This keeps the main line readable without blurring the operational contract.

6.6 From A Pointwise Step To A Coherent Spiral

Chapter 8 requires more than the ordinary recursor. Its code family and based representable family are both nonconstant:

$$\mathsf{Code}, \mathsf{Rep}_{*} : W \longrightarrow \mathsf{Cat}.$$

At the base, natural-number powers give a carrier function

$$n \longmapsto \ell^{n}.$$

Because the source is the path category of Nat, equality action lifts this function to a functor

$$\mathsf{power} : \mathsf{Path}(\mathbb{N}) \longrightarrow \mathrm{Hom}_{W}(*, *).$$

The contextual algebra still needs a transformation

$$\mathsf{Rep}_{*}[\ell] \circ \mathsf{power} \Longrightarrow \mathsf{power} \circ \mathsf{Code}[\ell].$$

Pointwise, the endpoints are $\ell \circ \ell^{n}$ and ℓ^{n+1}. Nat recursion makes them propositionally equal, but a family of equalities is not by itself the required transformation into a directed hom-category. The higher action and endpoint comparisons must be assembled coherently.

Emdash does this through a restricted equality-local core inclusion and a `PathLift` construction. Informally, `PathLift` turns a function whose source carries equality paths into a functor with directed target. The non-strict spiral inserts the necessary comparison before the lifted step; ordinary

path functoriality discharges the other side. Its readable component has the forward direction

$$\ell \circ \ell^n \longrightarrow \ell^{n+1}.$$

> **Formal status — checked.** Evidence `WE-SPIRAL`. The selected `walking_power_spiral` is the explicit restricted-core-inclusion construction, and `walking_power_spiral_cell` exposes the directed component shown above.

The spiral is a small but representative lesson in functorial type theory. Carrier equality supplies useful input, yet the consumer asks for a coherent cell with functorial higher action. The bridge between the two must be an explicit construction; it cannot be hidden by treating every directed hom as an identity type.

6.7 The Derived Decoder Interface

Substituting

$$R = \mathsf{Code}, \qquad D = \mathsf{Rep}_*, \qquad u = \mathsf{power}, \qquad \sigma = \mathsf{spiral}$$

into contextual elimination yields

$$\mathsf{decode}^d : \mathsf{Code} \Longrightarrow \mathsf{Rep}_*.$$

Its fibre component at `x` is a functor from codes over `x` to based arrows ending at `x`; at the base it computes to the power functor. Naturality of this single displayed functor is what later produces a directed normalization cell for every opaque based arrow.

This is stronger than defining only a function `Nat -> Hom_W(*,*)`. The latter supplies candidate normal forms but contains no explanation of why an arbitrary arrow reaches its candidate. The contextual eliminator supplies exactly that missing action.

6.8 Toward A General Directed-HIT Schema

The walking interface suggests a reusable architecture. A general schema would need at least:

- a language of object, arrow, and higher-cell boundaries;
- generated recursion, dependent elimination, and contextual elimination;
- functorial action at every exposed cell level;
- a policy distinguishing computational constructor rules from propositional coherences;
- subject-reduction and overlap checks for the generated rules;
- algebra and morphism categories suitable for universal-property theorems;
- optional dimension or truncation constructors whose consequences are separately controlled.

Pushouts, quotients, directed intervals, and cell complexes would test different parts of such a design. None follows merely by changing the name of the walking generator.

> **Formal status — research boundary.** Evidence `DHIT-GENERAL-SCHEMA`. The active code implements the selected opaque WalkingEnd signature and its eliminator interfaces. It does not yet implement a generic signature compiler or arbitrary directed higher-inductive categories.

The selected example is nevertheless substantial enough to guide that future work. It forces object action, arrow action, a genuine lax coherence cell, higher functorial lifting, constructor computation, and an interaction with finite categorical height. The next chapter develops the last item before we use all of them together.

7. Truncation And Categorical Height

An induction principle tells us how to construct data. A truncation principle tells us how much distinction that data can retain. These ideas meet in the walking-endomorphism proof: contextual elimination first constructs a directed 2-cell, and one-dimensionality then shows that the parallel 1-arrows it connects are equal.

There are two related height measures in emdash. Recursive groupoidal truncation measures the identity types of a classifier. Recursive categorical dimension measures the hom-categories of a directed category. They agree in important boundary cases, but they are not the same definition. Keeping them separate lets us use equality locally without declaring all directed arrows invertible.

7.1 Recursive Truncation

The truncation levels begin at `-2` and continue by successor. For a classifier `A`, define

$$\mathsf{isTrunc}_{-2}(A) := \mathsf{isContr}(A),$$
$$\mathsf{isTrunc}_{n+1}(A) := \prod_{x,y:A} \mathsf{isTrunc}_n(x = y).$$

Thus:

- a `(-2)`-truncated classifier is contractible;
- a `(-1)`-truncated classifier is a proposition;
- a `0`-truncated classifier is a set;
- a `1`-truncated classifier has set-valued identity types;
- each further level permits one more layer of identity structure.

The recursive equation is the important part. To prove that `A` is `(n+1)`-truncated, one proves that every identity classifier `x=y` is `n`-truncated. Truncation arguments can therefore descend through identity types until they reach contractibility.

> **Formal status — checked.** Evidence `LOGIC-TRUNCATION-PREDICATE`. `IsTruncGrpd` implements this recursion, with `IsPropGrpd` and `IsSetGrpd` as the proposition and set abbreviations.

This is a property of an existing classifier. Supplying `h:isTrunc_n(A)` does not replace `A` by a quotient, erase its points, or create a new type. It certifies that higher identity distinctions already collapse at the stated level.

7.2 Truncation Evidence Is Itself A Property

There may be many expressions witnessing that `A` is `n`-truncated. Mathematically, the choice should not matter. The evidence classifier itself is proposition-valued:

$$\mathsf{isProp}\big(\mathsf{isTrunc}_n(A)\big).$$

Consequently any two truncation witnesses are equal. Packages may retain the witness for later use without turning it into observable structure.

> **Formal status — checked.** Evidence `LOGIC-TRUNCATION-EVIDENCE-PROP`. The theorem is uniform in the truncation level and the classifier.

There is a useful distinction between **retaining** evidence and **erasing** it. Emdash generally retains evidence in Sigma-like packages so a consumer can project it. Proposition-valuedness then proves that two choices are equal when equality is needed. This is constructive proof irrelevance, not an instruction to remove the field from the data representation.

7.3 Closure Principles

The recursive definition supports the standard structural operations.

First, truncation is monotone:

$$\mathsf{isTrunc}_n(A) \longrightarrow \mathsf{isTrunc}_{n+1}(A).$$

A type with no distinctions above level `n` also has none above the weaker level `n+1`.

Second, dependent functions preserve a pointwise truncation bound. If every `B(x)` is `n`-truncated, then

$$\prod_{x:A} B(x)$$

is `n`-truncated. At successor levels the proof uses the equivalence between equality of functions and pointwise equality.

Third, dependent sums preserve a common bound. If `A` and every `B(x)` are `n`-truncated, then

$$\sum_{x:A} B(x)$$

is `n`-truncated. The successor proof analyzes equality of dependent pairs as a base path together with a path over it.

> **Formal status — checked.** Evidence `TRUNC-CLOSURE`. The active operations are `is_trunc_grpd_succ`, `is_trunc_pi`, and `is_trunc_sigma`, each with recursive base and successor behavior.

Truncation is also invariant under equivalence and closed under retracts. For the latter, suppose `Y` is a retract of `X`: there are maps

$$Y \xrightarrow{s} X \xrightarrow{r} Y$$

with $r \circ s$ equal to the identity on `Y`. Any truncation bound on `X` descends to `Y`.

> **Formal status — checked.** Evidence `TRUNC-RETRACT`. `is_trunc_retract` works uniformly at every recursive truncation level from explicit retraction data.

These closure results are not merely a catalogue. Pi closure makes proposition evidence stable under universal quantification; Sigma closure controls total spaces and evidence-retaining universes; retract closure lets a normalization or equivalence argument transfer height to a less explicit carrier.

7.4 Universes Of Truncated Classifiers

For each level `n`, the package

$$\mathsf{TruncGrpdU}(n) := \sum_{A:\mathsf{Grpd}} \mathsf{isTrunc}_n(A)$$

retains a classifier and its truncation evidence. Since the evidence is a proposition, equality of packages is governed by the carriers rather than by an arbitrary choice of proof.

The selected truncated-universe univalence theorem identifies package equality with carrier equivalence:

$$(X = Y) \simeq \mathsf{TypeEquiv}(\mathsf{carrier}(X), \mathsf{carrier}(Y)).$$

Both directions and their round trips are named. This is a useful, precise univalent universe: it ranges over classifiers already equipped with one fixed truncation bound.

> **Formal status — checked.** Evidence `UNIV-TRUNCATED`. The package is `TruncGrpdU`; `trunc_grpd_univalence_type_equiv` supplies the carrier equivalence between package identity and `TypeEquiv`.

The adjective “restricted” matters. This theorem does not by itself identify objects in an arbitrary directed category with ordinary categorical isomorphisms. Nor does it construct a truncation of an arbitrary input. It is a univalence theorem for an evidence-retaining subuniverse of the groupoidal classifier universe.

7.5 Finite Directed Dimension

Truncation follows identity types. Directed dimension follows hom-categories. The nonnegative dimension codes are generated by

$$0_{\mathsf{cat}} \qquad \text{and} \qquad \mathsf{succ}_{\mathsf{cat}}(n).$$

Their classifier is recursive:

$$\begin{aligned}\mathsf{isNCat}(0, C) &:= \mathsf{isDiscreteCat}(C),\\ \mathsf{isNCat}(n+1, C) &:= \prod_{x,y:\mathrm{Obj}(C)} \mathsf{isNCat}\big(n, \mathrm{Hom}_C(x,y)\big).\end{aligned}$$

A zero-dimensional category is discrete. A one-dimensional category may have nonidentity directed 1-arrows, but each of its hom-categories is discrete. A two-dimensional category may have nontrivial 2-cells, while the next homs are discrete, and so on.

There is a corresponding object-truncation level:

$$\begin{aligned}\mathsf{catLevel}(0) &= 0,\\ \mathsf{catLevel}(n+1) &= \mathsf{catLevel}(n) + 1.\end{aligned}$$

If `C` has categorical dimension `n`, then its object classifier is truncated at `catLevel(n)`. In particular, objects of a discrete category form a set, and objects of a one-dimensional category form a 1-truncated classifier.

> **Formal status — checked.** Evidence `CAT-DIMENSION`. `IsNCat` owns the homwise recursion, `cat_dim_trunc_level` computes the corresponding groupoidal level, and `ncat_obj_trunc` proves the object-truncation consequence.

This bridge does not collapse the two notions. `IsNCat` constrains all iterated directed homs; object truncation records only equality structure on the object classifier. Two categories can have equally truncated object classifiers while differing radically in their directed arrows.

7.6 One-Dimensionality Of The Walking Endomorphism

The WalkingEnd signature contains

$$\mathsf{isNCat}(1, W).$$

Unfolding the successor clause gives, for every `x,y:Obj(W)`,

$$\mathsf{isDiscreteCat}\big(\mathrm{Hom}_W(x,y)\big).$$

In particular the based hom

$$H_x := \mathrm{Hom}_W(*, x)$$

is discrete. Given two based arrows $p, q : * \to x$, a directed 2-cell

$$\alpha : p \longrightarrow q \quad \text{in } H_x$$

can therefore be converted to an equality `p=q` of objects of the hom-category.

> **Formal status — checked.** Evidence `WE-ONE-DIMENSIONAL`. `walking_end_hom_discrete` specializes the dimension witness, and `walking_end_based_cell_to_path` converts a based 2-cell to equality.

This operation is local to the next hom. It does **not** produce an arrow $q \to p$, an inverse for `p`, or an inverse for `ell` in `W`. Equality between the *objects of a discrete hom-category* and invertibility of those objects as *arrows of the ambient category* are different statements.

7.7 The Exact Height Step In Encode–Decode

Chapter 8 constructs, for every based arrow $p : * \to x$, a directed normalization cell

$$\nu_p : p \longrightarrow \mathsf{decode}_x(\mathsf{encode}_x(p))$$

inside `H_x`. Only then does hom-discreteness give

$$p = \mathsf{decode}_x(\mathsf{encode}_x(p)).$$

This is the sole step in the hard inverse where one-dimensionality is used. It converts already-constructed directed information into equality; it does not help construct the information.

The other inverse,

$$\mathsf{encode}_*(\ell^n) = n,$$

is proved independently by Nat induction using the zero and successor computations. It does not require hom-discreteness. Nat sethood plays two nearby but distinct roles: it makes the concrete `BNat` hom-category discrete, and it can be transported backward along the final carrier equivalence to give a second proof that the based-endomorphism carrier is a set. Neither role should be substituted for the directed normalization step.

This separation is a model for later proofs:

1. build a cell using functorial or contextual action;
2. invoke a dimension hypothesis at the exact hom level where the cell lives;
3. extract equality only if the target theorem needs it.

The intermediate cell can carry an orientation or support further composition even when its equality shadow cannot.

7.8 Properties Are Not Reflectors

A general truncation operation would assign to every classifier `A` a new classifier `||A||_n`, together with a universal map and an elimination principle into `n`-truncated targets. In a directed setting one may also ask for categorical truncations that collapse cells above a chosen dimension while preserving lower directed structure.

The active layer does not provide either general reflector. It provides predicates on existing classifiers, closure theorems, evidence-retaining universes, and consequences of finite categorical dimension. Those tools are enough for the WalkingEnd calculation because its one-dimensionality is signature data rather than something that must be freely imposed afterward.

> **Formal status — research boundary.** Evidence `TRUNC-REFLECTOR`. A general truncation HIT or directed categorical reflector remains future work and must come with its own universal and computational properties.

We now have every prerequisite for the main proof: equality-local action, functors and directed families, contextual elimination, equivalence packages, recursive truncation, and homwise categorical height. The next chapter puts them together without identifying direction with invertibility.

8. Synthetic Directed Homotopy Theory

Homotopy type theory studies spaces by calculating identity types and loop spaces internally. A directed foundation admits another kind of calculation: we may calculate a hom whose arrows are not assumed invertible. The first example is the walking endomorphism.

Write `W` for `WalkingEnd`, `*` for its base, and $\ell : * \to *$ for its generating arrow. Composition is in categorical order, so $g \circ f$ means first `f` and then `g`. For any object `x` of `W`, abbreviate the based hom-category by

$$H_x \;:=\; \mathrm{Hom}_W(*, x).$$

An object of `H_x` is a directed arrow $* \to x$. An arrow of `H_x` is a directed 2-cell between two such arrows. The signature states that `W` is one-dimensional, which means precisely that every `H_x` is discrete. We will exploit this discreteness only after a directed 2-cell has been constructed.

The proof uses six interfaces:

- `Path(A)`, the equality-local path category of a type `A`;
- functor action `F[f]` on arrows and its higher action on cells;
- a directed Cat-valued family $E : K \to \mathsf{Cat}$ and its fibre `E[k]`;
- a displayed functor between two such families;
- natural-number induction;
- a carrier equivalence `TypeEquiv(A,B)`, consisting of maps and inverse laws.

Chapters 1–7 develop these notions. This chapter uses them as a compact working language and returns to implementation names only in formal-status notes.

The proof also follows the formal rule ledger of Appendix G.4. The WalkingEnd constructors and contextual eliminator supply formation, introduction, elimination, and beta computation; the code family and spiral supply the missing action and coherence; one-dimensionality is used only afterward. Full initiality is the uniqueness clause and remains separate from the checked encode–decode calculation.

8.1 The Based Endomorphisms Of The Walking Endomorphism

The target statement is:

> **Theorem 8.1 (the walking-endomorphism calculation).** There is an equivalence of underlying carriers
>
> $$\mathrm{Obj}(H_*) = \mathrm{Hom}_W(*, *) \;\simeq\; \mathbb{N}.$$

> **Formal status — checked.** Evidence `WE-HOM-NAT-CARRIER`. The active package is `walking_hom_nat_type_equiv`, with an additional equality-valued omega-equivalence facade at the groupoid/type level.

The qualification "underlying carriers" matters. The theorem does not definitionally replace `H_*` by a concrete Nat category. Nor does the current package include a monoid isomorphism, a reverse functor from the concrete model `BNat`, an equivalence of hom-categories, or the full initiality of `W`. We will identify exactly which stronger statements follow on paper and which still require formal infrastructure.

8.1.1 Getting Started

The obvious endomorphisms are the natural powers of the generator:

$$\begin{aligned} \ell^0 &:= \mathrm{id}_*, \\ \ell^{n+1} &:= \ell \circ \ell^n. \end{aligned}$$

The recursion prefixes one copy of `ell` at each successor. Thus ℓ^2 is $\ell \circ \ell$ and no inverse power is present. The definition is an ordinary Nat eliminator whose motive is the object carrier of `H_*`.

> **Formal status — checked.** Evidence `WE-POWER`. The object map is `walking_power`, and `walking_power_func` : $\mathsf{Path}(\mathbb{N}) \to H_*$ supplies its equality-local higher action.

To prove that these powers exhaust the endomorphisms, we seek a measurement

$$\mathsf{encode}_* : \mathrm{Obj}(H_*) \longrightarrow \mathbb{N}$$

such that

$$\mathsf{encode}_*(\ell^n) = n \quad \text{and} \quad \ell^{\mathsf{encode}_*(p)} = p.$$

The first equation is approachable by Nat induction. The second quantifies over an arbitrary opaque arrow $p : * \to *$. We cannot inspect `p` as a word because no word datatype is installed as the hom of `W`.

There is a second, subtler obstruction. An induction principle for a based path normally becomes useful only after the other endpoint has been generalized. Fixing both endpoints too early hides the varying family on which induction acts. The same phenomenon appears here in directed form: the calculation must be stated not only for `H_*` but for every based hom `H_x`.

We therefore look for a family of codes `Code[x]` and maps

$$\begin{aligned} \mathsf{encode}_x &: \mathrm{Obj}(H_x) \longrightarrow \mathrm{Obj}(\mathsf{Code}[x]), \\ \mathsf{decode}_x &: \mathrm{Obj}(\mathsf{Code}[x]) \longrightarrow \mathrm{Obj}(H_x), \end{aligned}$$

natural in the directed endpoint `x`. The base instance will then be the desired Nat comparison.

The encoder is the easy half once `Code` is known: start at zero and let the arrow act,

$$\mathsf{encode}_x(p) := \mathsf{Code}[p](0).$$

> **Formal status — checked.** Evidence `WE-ENCODE`. `walking_encode` is defined for every endpoint and every based arrow, not only for endomorphisms at the base.

At the base, the computations we want are

$$\mathsf{encode}_*(\mathrm{id}_*) = 0$$

and

$$\mathsf{encode}_*(\ell \circ p) = \mathsf{succ}(\mathsf{encode}_*(p)).$$

The first is the identity action of a functor. The second will follow from the generator computation of `Code` together with generic functoriality on a composite. No WalkingEnd-specific composition rewrite is needed.

> **Formal status — checked.** Evidence `WE-ENCODE-PREFIX`. The prefix equation is propositional; its owner specializes ordinary functor action on composition and the literal generator computation.

8.1.2 The Free-Monoid Model

Before constructing `Code`, it helps to exhibit the arithmetic object we expect `W` to resemble. Define a category `BNat` by

$$\mathrm{Obj}(BNat) = \mathbf{1}, \qquad \mathrm{Hom}_{BNat}(\bullet, \bullet) = \mathsf{Path}(\mathbb{N}).$$

Its identity is zero. With our composition convention,

$$m \circ n := m + n,$$

where addition recurses in its left input:

$$0 + n = n, \qquad (m + 1) + n = (m + n) + 1.$$

The underlying objects of the sole hom-category are therefore the natural numbers. Its higher arrows are equality paths between naturals. Since `Nat` is a set, that hom-category is discrete, so `BNat` satisfies the same one-dimensionality contract as `W`.

The generator in `BNat` is `1`. Applying the ordinary recursor of `W` produces

$$J : W \longrightarrow BNat, \qquad J(*) = \bullet, \qquad J[\ell] = 1.$$

Functoriality forces `J` to send a displayed composite of generators to the corresponding sum. Thus `BNat` demonstrates that the walking signature has the expected one-object Nat-monoid interpretation.

> **Formal status — checked.** Evidence `WE-BNAT-MODEL`. The identity and recursive composition of `BNat`, their propositional agreement with Nat addition, its one-dimensionality, and `walking_bnat_model_func` are checked.

This model does **not** settle Theorem 8.1. A functor `J` gives a number to every endomorphism, but it need not reflect equality: two unknown arrows might map to the same number. Nor does a map *out of* an opaque inductive object supply a map back. The model is valuable precisely because it remains separate:

- it checks that the signature can be interpreted without collapsing zero and one;
- it fixes the intended orientation of identity, composition, and generator;
- it predicts the normal forms;
- it does not place those normal forms into the definition of `W`.

This is the categorical analogue of testing a presentation in a familiar model before proving its universal consequences. The exhaustiveness proof must come from the eliminator and dimension evidence of `W` itself.

> **Formal status — research boundary.** Evidence `WE-FULL-CATEGORICAL-COMPARISON` records what is absent: a reverse $\mathsf{BNat} \to W$ functor, a packaged categorical equivalence, and full functor-category initiality require reusable monoid-action-to-functor and functor-extensionality infrastructure.

8.1.3 The Directed Cover In Functorial Type Theory

We now define the family that measures arrows. The ordinary recursor for `W` says that a functor out of the walking object is determined by a target object and one endomorphism of that object. Take the target category to be `Cat`, choose `Path(Nat)` as the object, and choose the successor functor as its endomorphism. The result is

$$\mathsf{Code} : W \longrightarrow \mathsf{Cat}$$

with constructor computations

$$\mathsf{Code}[*] = \mathsf{Path}(\mathbb{N}), \qquad \mathsf{Code}[\ell] = \mathsf{Succ}.$$

> **Formal status — checked.** Evidence `WE-CODE`. Both the base fibre and literal generator action are checked recursor observations.

There is an important contrast with the circle. A universe-valued family over the circle must send its loop to an equality of types. Univalence can produce such an equality from successor on the integers because integer successor is an equivalence. Natural-number successor is not an equivalence: it

misses zero. The directed recursor asks only for an endofunctor, so it accepts successor directly. Univalence remains part of the surrounding foundation, but it is neither needed nor appropriate to turn this action into a reversible path.

At the base fibre, picture

$$0 \longrightarrow 1 \longrightarrow 2 \longrightarrow 3 \longrightarrow \cdots$$

as levels of a helix over the directed loop. One traversal moves upward by one level. There is a boundary at zero and no downward motion. The picture is a guide to the action of the family; it is not a claim that a topological covering space or a contractible total category has been constructed.

For any based arrow $p : * \to x$, functor action gives

$$\mathsf{Code}[p] : \mathsf{Code}[*] \longrightarrow \mathsf{Code}[x].$$

Evaluating at zero defines `encode_x(p)`. At `x=*` the result is a natural number. For a generator-prefixed arrow, strict functoriality factors the action:

$$\begin{aligned} \mathsf{encode}_*(\ell \circ p) &= \mathsf{Code}[\ell \circ p](0) \\ &= \mathsf{Code}[\ell](\mathsf{Code}[p](0)) \\ &= \mathsf{succ}(\mathsf{encode}_*(p)). \end{aligned}$$

The target of the decoder is the based representable family

$$\mathsf{Rep}_* : W \longrightarrow \mathsf{Cat}, \qquad \mathsf{Rep}_*[x] = H_x.$$

For $f : x \to y$, its action is postcomposition:

$$\mathsf{Rep}_*[f](q) = f \circ q.$$

Thus a decoder varying over `x` should be a displayed functor

$$\mathsf{decode}^d : \mathsf{Code} \Longrightarrow \mathsf{Rep}_*.$$

At the base, its object map ought to send `n` to `ell^n`. But an object map alone is too little. It must act on equality paths between naturals, and it must be coherent with the base generator.

The pointwise power function lifts to a functor

$$\mathsf{power} : \mathsf{Path}(\mathbb{N}) \longrightarrow H_*.$$

The lift uses equality action and inclusion into the directed hom-category. This retains the higher action needed by the contextual eliminator rather than truncating power to a bare function.

The generator coherence has the form

$$\mathsf{Rep}_*[\ell] \circ \mathsf{power} \Longrightarrow \mathsf{power} \circ \mathsf{Code}[\ell].$$

At a natural number `n`, its readable component is

$$\sigma_n : \ell \circ \ell^n \longrightarrow \ell^{n+1}$$

inside `H_*`. The endpoints express the same recursive power equation, but the contextual eliminator requires a coherent transformation, not merely a family of object equalities. Emdash constructs it by lifting the equality between the two step functions through the restricted equality-local core and adding the endpoint adjustments demanded by the ambient directed hom. This is the **spiral**.

> **Formal status — checked.** Evidence `WE-SPIRAL`. The selected spiral is the explicit-core-inclusion two-factor construction; its readable component has the direction shown above.

The contextual elimination principle for `W` may be read as follows. Given directed families $R, D : W \to \mathsf{Cat}$, a base functor

$$u : R[*] \longrightarrow D[*],$$

and a transformation

$$D[\ell] \circ u \Longrightarrow u \circ R[\ell],$$

it produces a displayed functor $R \Rightarrow D$. Substituting `R=Code`, `D=Rep_*`, `u=power`, and `sigma` equal to the spiral yields the desired contextual decoder.

> **Formal status — checked.** Evidence `WE-CONTEXTUAL-ELIMINATOR` and `WE-CONTEXTUAL-DECODER`. The displayed decoder is `walking_directed_decode_funcd`, and its base fibre computes to the power functor.

This construction is why the generalization over all endpoints is not a stylistic flourish. The decoder is coherent because it is one displayed functor over the whole opaque object, not a collection of unrelated functions defined only at the base.

8.1.4 The Encode-Decode Proof

We now follow the dependency order of the construction. The order matters: compressing the argument into two carrier functions would hide its directed content.

Step 1: powers with higher action

Nat induction defines `power(n)=ell^n` on objects. Equality action lifts this function to

$$\mathsf{power} : \mathsf{Path}(\mathbb{N}) \longrightarrow H_*.$$

At zero and successor,

$$\mathsf{power}(0) = \mathrm{id}_*, \qquad \mathsf{power}(n+1) = \ell \circ \mathsf{power}(n).$$

The functorial lift is essential: the next step needs to transport equality of naturals to a 2-cell between powers.

Step 2: the spiral

Postcomposition by `ell` after power and power after successor are two functors from `Path(Nat)` to `H_*`. The recursive power equation gives equality of their underlying object functions. The equality-local lift, restricted core inclusion, and directed endpoint adjustments turn this into

$$\sigma : \mathsf{Rep}_*[\ell] \circ \mathsf{power} \Longrightarrow \mathsf{power} \circ \mathsf{Code}[\ell].$$

Its component `sigma_n` points from generator-prefix composition toward the successor power. This is the coherence algebra consumed by the HIT eliminator.

Step 3: the contextual decoder

Contextual elimination now supplies

$$\mathsf{decode}^d : \mathsf{Code} \Longrightarrow \mathsf{Rep}_*.$$

Projecting to a fibre gives, for every `x`,

$$\mathsf{decode}^d[x] : \mathsf{Code}[x] \longrightarrow H_x.$$

Write `decode_x(c)` for its object action. At the base this functor is judgmentally the power functor, so

$$\mathsf{decode}_*(n) = \ell^n.$$

Step 4: the directed normalization cell

Let $p : * \to x$ be arbitrary. A displayed functor does more than provide fibrewise maps: it compares transport in its source and target families along every base arrow. Apply this comparison to `p` and to zero in `Code[*]`.

On the source side, the representable action gives

$$\mathsf{Rep}_*[p](\mathsf{power}(0)) = p \circ \mathrm{id}_* = p.$$

On the target side, Code action gives `Code[p](0)=encode_x(p)`, then the fibre decoder gives `decode_x(encode_x(p))`. The displayed comparison is therefore a directed 2-cell

$$\nu_p : p \longrightarrow \mathsf{decode}_x(\mathsf{encode}_x(p))$$

in `H_x`.

> **Formal status — checked.** Evidence `WE-NORMALIZATION-CELL`. The term is the displayed hom-action of the contextual decoder at `p` and zero. Its source reduces through representable postcomposition, `power(0)`, and the right unit.

This is the conceptual climax of the proof. It says that an unknown arrow can move, by a directed higher cell, toward the canonical power selected by its code. We have not yet said that the two arrows are equal.

Step 5: equality from categorical height

The one-dimensionality witness for `W` makes `H_x` discrete. In a discrete category, a hom between two objects can be converted to equality of those objects. Applying this operation to `nu_p` gives

$$\bar{\nu}_p : p = \mathsf{decode}_x(\mathsf{encode}_x(p)).$$

> **Formal status — checked.** Evidence `WE-NORMALIZATION-PATH`. The implementation explicitly constructs `walking_directed_normalization_cell` before applying hom-discreteness in `walking_directed_normalization_path`.

This final conversion forgets the orientation of normalization, but it does not make `p` or `ell` invertible. It uses the absence of nontrivial 2-dimensional variation in a hom-category; it says nothing about inverses for its objects as 1-arrows of `W`.

Step 6: the hard inverse

Specialize to `x=*`. Because the base decoder computes to power, `bar(nu)_p` becomes

$$p = \ell^{\mathsf{encode}_*(p)}.$$

The inverse law for the desired equivalence is conventionally oriented the other way, so we take symmetry:

$$\ell^{\mathsf{encode}_*(p)} = p.$$

> **Formal status — checked.** Evidence `WE-POWER-ENCODE`. This is the difficult carrier inverse, and it is derived from directed normalization rather than from induction on an exposed word.

In the circle calculation, the analogous fixed-loop problem is repaired by generalizing the endpoint and using path induction. Here the endpoint is also generalized, but the eliminator supplies a directed displayed action. The normalization cell is the directed replacement for the equality that path induction would have produced immediately in a groupoidal setting.

Step 7: the easy inverse

For `n:Nat`, prove

$$\mathsf{encode}_*(\ell^n) = n$$

by Nat induction.

At zero, power is the identity and a functor sends identity to identity, so acting on zero returns zero. At a successor,

$$\begin{aligned}\mathsf{encode}_*(\ell^{n+1}) &= \mathsf{encode}_*(\ell \circ \ell^n) \\ &= \mathsf{succ}(\mathsf{encode}_*(\ell^n)) \\ &= \mathsf{succ}(n).\end{aligned}$$

The middle equality is the generator-prefix encoding formula; the last is the induction hypothesis acted on by successor. No negative case is required.

> **Formal status — checked.** Evidence `WE-ENCODE-POWER`. The theorem `walking_encode_power_roundtrip` is native Nat induction over the checked prefix equation.

Step 8: package the equivalence

The forward function is `encode_*` and the inverse is `power`. The previous two steps provide both quasi-inverse laws, so they determine

$$\mathrm{Hom}_W(*, *) \simeq \mathbb{N}.$$

The encoder is also packaged as a functor

$$H_* \longrightarrow \mathsf{Path}(\mathbb{N}),$$

obtained by taking the hom-action of `Code` and evaluating at zero. This functor acts on 2-cells between endomorphisms. Its ordinary functor laws should not be confused with preservation of the *horizontal* monoid composition of the endomorphisms themselves.

> **Formal status — checked.** Evidence `WE-STRUCTURED-ENCODER` and `WE-HOM-NAT-CARRIER`. The structured encoder and carrier equivalence are active; a structured reverse functor and a monoid package are not.

The proof can now be summarized without erasing its architecture:

$$\begin{array}{c} p \\ \downarrow\ \nu_p \\ \mathsf{decode}(\mathsf{encode}(p)) \end{array} \qquad \Longrightarrow_{\text{discreteness}} \qquad p = \ell^{\mathsf{encode}(p)},$$

followed by the independent Nat-inductive calculation `encode(ell^n)=n`.

8.1.5 Consequences And The Missing Negative Integers

The equivalence has immediate structural consequences, but the most illuminating ones concern what the walking generator cannot do.

The based hom is a set

There are two checked proofs that the underlying carrier `Hom_W(*,*)` is a set.

1. **By dimension.** One-dimensionality makes `H_*` discrete, and discreteness includes sethood of its object carrier.
2. **By comparison.** Natural numbers form a set, and truncation is invariant under the carrier equivalence.

> **Formal status — checked.** Evidence `WE-HOM-SETHOOD`. The two proofs are separately named, so the dimensional and equivalence-based explanations remain visible.

The generator is not the identity

The prefix computation gives

$$\mathsf{encode}_*(\ell) = 1, \qquad \mathsf{encode}_*(\mathrm{id}_*) = 0.$$

If `ell=id_*`, functorial action of `encode` on that equality would yield `1=0`, whose Nat equality classifier is empty.

> **Formal status — checked.** Evidence `WE-LOOP-NOT-IDENTITY`.

The generator has no right inverse

Suppose $r : * \to *$ and

$$\ell \circ r = \mathrm{id}_*.$$

Encoding the left side and using the prefix formula gives `succ(encode(r))`; encoding the right side gives zero. Again a successor cannot equal zero. Therefore no such `r` exists.

> **Formal status — checked.** Evidence `WE-LOOP-NO-RIGHT-INVERSE`. The statement is specifically the absence of a right inverse in the displayed composition orientation; no stronger cancellation theorem is being silently imported.

Native omega-equivalence evidence for an arrow contains, among its data, a right inverse and its equality law. The preceding result therefore rules out such evidence for `ell`.

> **Formal status — checked.** Evidence `WE-LOOP-NONINVERTIBLE`.

Composition and addition

The carrier theorem strongly suggests the monoid formula

$$\mathsf{encode}_*(q \circ p) = \mathsf{encode}_*(q) + \mathsf{encode}_*(p).$$

Its orientation agrees with `BNat`: `q` is the outer arrow and its code is the left input of addition. A paper proof follows from the checked interfaces. First use Nat induction and associativity to show

$$\ell^{m+n} = \ell^m \circ \ell^n.$$

Then replace `q` and `p` by their normalized powers and use `encode(power(k))=k`. What is absent is not the mathematical argument but its selected library package and its interaction with a future reverse functor.

> **Formal status — formal consequence.** Evidence `WE-COMPOSITION-ADDITION`. The current kernel does not expose a named monoid-isomorphism object for this statement.

Why naturals replace integers

For the circle, every loop is an equality path and hence can be reversed. Powers extend from naturals to integers, the code action has both successor and predecessor, and the result is group-valued.

For `W`, the generator is a directed arrow with no supplied inverse. The code action moves

$$0 \mapsto 1 \mapsto 2 \mapsto \cdots$$

and cannot move below zero. Natural numbers record the free monoid generated by forward motion. The negative integers are missing for the same reason the right inverse is missing: direction has not been group-completed.

A future comparison with the circle should therefore proceed by an explicit group-completion construction, not by renaming `Nat` to `Int` or declaring `ell` invertible.

> **Formal status — research boundary.** Evidence `WE-GROUP-COMPLETION`. No `BInt`/circle group completion or comparison functor is active.

The calculation has reached its intended boundary. It proves that an opaque directed generator has exactly the expected natural powers, and it proves noninvertibility rather than assuming it. Stronger categorical universal properties are the next layer, not hidden premises of this one.

8.2 Higher Groupoidal Shadows

The surrounding calculus is directed, but equality-local and groupoidal phenomena remain inside it. The first neighboring example is the Eckmann–Hilton argument.

Let `B` be a category and `x` an object. The 2-endomorphisms of the identity 1-arrow form the carrier

$$2\mathrm{End}_B(x) := \mathrm{Hom}_{\mathrm{Hom}_B(x,x)}(\mathrm{id}_x, \mathrm{id}_x).$$

There are apparently two ways to combine elements. Vertical composition is ordinary composition in the hom-category `Hom_B(x,x)`. Horizontal composition is obtained by whiskering/postcomposition at the identity 1-arrow. They share the identity 2-cell as a unit, and the ordinary functoriality of whiskering supplies interchange. The classical Eckmann–Hilton calculation then makes the operations agree and commute:

$$\beta \cdot \alpha = \alpha \cdot \beta.$$

> **Formal status — checked.** Evidence `EH-COMMUTATIVITY`. The active term `EH_comm` derives commutativity from the two compositions, shared units, and interchange in the iterated-hom representation.

This result does not undo the directed character of `W`. It lives one dimension higher at the identity arrow, where the relevant comparison is equality-local. The coexistence is the point: a directed theory can contain groupoidal shadows at controlled boundaries without declaring all of its arrows reversible.

9. Transfors And The Calculus Of Cuts

Composition is indispensable, but a syntax made only of nested composites quickly forgets why a particular reassociation matters. Functorial type theory therefore gives important cuts a name and a computational owner. A functor acts on an arrow; postcomposition and precomposition act on represented homs; a transfor acts off the diagonal; and a universal construction eliminates a map through the object it represents. Each operation controls its own reassociation.

This point of view is inspired by the categorical proof theory of Došen's *Cut Elimination in Categories*. We use that work only as a conceptual reference. The presentation below is newly written for emdash, whose iterated homs and directed families require a different formal architecture.

The gain is not merely prettier notation. A selected cut can compute while retaining the functor or transfor that acts on the next hom. An unrestricted associativity rewrite would erase that information, create competing normal forms, or loop by repeatedly changing brackets. Controlled cut elimination instead answers three questions at once: what is being eliminated, where its normal form lives, and which higher action survives.

9.1 Four Levels Of Cuts

We shall use four levels throughout the rest of the book.

1. An **arrow cut** composes one chosen arrow on the left or right. Its owners are the lower-star postcomposition and upper-star precomposition actions.
2. A **family cut** composes next to an arrow that varies naturally. Its owner is the off-diagonal action of a transfor.
3. A **structural cut** eliminates data introduced by a product, dependent total, curry, or related type former. Its owner is the corresponding projection or eliminator.
4. A **universal cut** factors through a chosen representing object. Its owner is an adjunction, representability comparison, or another explicit universal-property interface.

These levels are not four unrelated collections of rewrite rules. They form a progression. Arrow cuts explain local composition; family cuts explain naturality; structural cuts explain computation for categorical type formers; and universal cuts explain why constructions such as adjoints, Yoneda maps, and weighted limits compute.

We also keep four equality modes distinct. A runtime reduction selects a normal form. A proof-time comparison helps Lambdapi elaborate two intended presentations without making either one compute to the other. A propositional equality is an internal witness. A mathematical equality in free-form theory states the intended theorem while naming the interface still needed to check it. The symbol $\rightsquigarrow$ below is reserved for an actual selected runtime reduction; an ordinary equality sign does not silently make that claim.

In the terminology of Appendix G.4, each named cut is an elimination followed by a computation rule. Its formation and introduction data determine which composite is well typed; its full functor or transfor owner supplies higher action; and any eta or uniqueness principle is stated separately. This is why a pointwise naturality equation cannot replace `tapp1_func`, and why a universal comparison needs both beta and eta rather than one attractive factorization formula.

9.2 Arrow Cuts

Composition is written $g \circ f$: first f, then g. The two star actions record which side of a represented hom is moving.

If $g : w \to x$ and $u : x \to y$, then

$$u_*(g) := u \circ g : w \to y$$

is **postcomposition** by u. If $u : x \to y$ and $h : y \to z$, then

$$u^*(h) := h \circ u : x \to z$$

is **precomposition** by u. Lower star is covariant in the moving target; upper star is contravariant in the moving source. They are different operations, not typographic variants.

The implemented forms are slightly more general. For $K : A \to B$ and $p : x \to y$ in A, postcomposition uses $K[p]$ on a hom ending at Kx, while precomposition uses $K[p]$ on a hom beginning at Ky. Retaining K is what lets the action iterate on higher cells.

9.2.1 Example 1: Postcomposition Accumulates

Let

$$g : w \to x, \qquad u : x \to y, \qquad v : y \to z.$$

Two consecutive lower-star cuts reduce to one:

$$v_*(u_*(g)) \rightsquigarrow (v \circ u)_*(g).$$

Both sides are arrows $w \to z$. The selected normal form retains a single postcomposition action whose moving arrow is $v \circ u$; it does not expand to a raw threefold composite. The generic owner is `hom_postcomp_fapp0`, and the displayed equality mode is runtime reduction. Before capping at g, `hom_postcomp_func` remains a functor between hom-categories, so its action on 2-cells between possible values of g is still available.

The functor-indexed version says the same thing. If $p : x \to y$ and $q : y \to z$ in A, then consecutive action by $K[p]$ and $K[q]$ accumulates under the single arrow $q \circ p$. Ordinary functoriality belongs to the generic `fapp*` calculus; no constructor receives a private composition law.

9.2.2 Example 2: Precomposition Reverses The Action Order

Let

$$u : w \to x, \qquad v : x \to y, \qquad h : y \to z.$$

The corresponding upper-star reduction is

$$u^*(v^*(h)) \rightsquigarrow (v \circ u)^*(h).$$

Both sides are arrows $w \to z$. The selected normal form is one precomposition action along $v \circ u$. The generic owner is `hom_precomp_along_fapp0`, and this is again runtime reduction. Its uncapped form `hom_precomp_along_func` remains a functor on the represented hom, with the next hom-action intact.

The order is worth reading twice. Starting with h, we first precompose by v and then by u, but the accumulated base arrow is $v \circ u$. Thus

$$u^* \circ v^* = (v \circ u)^*.$$

That reversal is contravariance, not a special exception to associativity.

> **Formal status — checked.** Evidence `CAT-HOM-CUTS`. The full and capped lower-star and upper-star actions have identity, consecutive-action, and adjacent raw-cut computations. Their ordinary-composition readings remain proof-time comparisons where selecting a second runtime normal form would be harmful.

9.2.3 Why There Is No Global Associativity Rewrite

The ordinary associator is available as a proof-time comparison and as propositional evidence. It is not installed as a pair of unrestricted runtime rules. Such rules would either orient every composite toward a normal form that ignores semantic owners or permit both bracketings and loop.

Star accumulation is narrower. It reassociates exactly when one side is a represented-hom action, and its result retains that action as a stable head. The same policy will govern the next three levels: eliminate the cut at the construction that understands it.

9.3 Family Cuts

Let $F, G : A \to B$ and let $\eta : F \Rightarrow G$. A point component $\eta_x : Fx \to Gx$ is only the diagonal of a more useful operation. For every $x, y : A$ there is an off-diagonal functor

$$\eta_{x,y} : \mathrm{Hom}_A(x, y) \longrightarrow \mathrm{Hom}_B(Fx, Gy).$$

For $f : x \to y$, write its value as $\eta[f] : Fx \to Gy$. Setting $f = \mathrm{id}_x$ recovers η_x; retaining the whole functor retains its action on 2-cells between arrows f.

> **Formal status — checked.** Evidence `TRANSF-POINT-OFFDIAGONAL`. `tapp0_fapp0` observes the point component, while `tapp1_func` and `tapp1_fapp0` expose the iterable and capped off-diagonal actions.

9.3.1 Example 3: The Two Naturality Cuts

Take arrows

$$h : w \to x, \qquad f : x \to y, \qquad g : y \to z.$$

There are two neighboring cuts, one on each side of the varying arrow:

$$G[g] \circ \eta[f] \rightsquigarrow \eta[g \circ f],$$
$$\eta[f] \circ F[h] \rightsquigarrow \eta[f \circ h].$$

The first source and target are $Fx \to Gz$; the second are $Fw \to Gy$. In each case the selected normal form is one off-diagonal action on the composite source arrow. The generic owner is `tapp1_fapp0`, and both displayed equalities are runtime reductions. At the uncapped level `tapp1_func` remains a functor between hom-categories, so a 2-cell between f and f' is carried to a 2-cell between $\eta[f]$ and $\eta[f']$ after the cut has normalized.

> **Formal status — checked.** Evidence `TRANSF-STRICT-NATURALITY`. Both full-functor and capped-arrow forms are owned by the generic `tapp*` calculus. Constructor-specific copies of ordinary naturality are neither needed nor desired.

The familiar naturality square is the identity-boundary instance. For $f : x \to y$, the expressions $G[f] \circ \eta_x$ and $\eta_y \circ F[f]$ both normalize through the common interior $\eta[f]$. Naturality is therefore not an equality proof added after defining a family of point components. It is computation exposed by the family action itself.

Identity and vertical composition follow the same architecture. The identity transfor and a vertical composite live in the transformation category, so their component projections use the generic identity and composition calculus. Higher transfors arise by iterating the next hom rather than by inventing a separate coherence language at every dimension.

9.4 Structural Cuts

A structural cut applies an eliminator to data whose introduction form is known. Product projections are the simplest case, but they already reveal an important distinction between the general categorical theory and the currently checked category-of-categories instance.

9.4.1 Example 4: The Product/Projection Benchmark In A General Category

Let K be a category equipped with chosen binary products. Take objects

$$A_0, A_1, B_0, B_1, C : \mathrm{Obj}(K)$$

and arrows

$$h : A_0 \to A_1, \qquad k : B_0 \to B_1, \qquad g : A_1 \to C.$$

Write $h \times k : A_0 \times B_0 \to A_1 \times B_1$ for the induced product arrow, and distinguish the two projections by

$$\pi_1^1 : A_1 \times B_1 \to A_1, \qquad \pi_1^0 : A_0 \times B_0 \to A_0.$$

Upper-star precomposition in K gives

$$(\pi_1^1)^*(g) : A_1 \times B_1 \to C.$$

The Došen-style product cut is the equation

$$(\pi_1^1)^*(g) \circ (h \times k) = (\pi_1^0)^*(g \circ h),$$

with both sides in $\mathrm{Hom}_K(A_0 \times B_0, C)$. Its intended computational orientation is left to right. The calculation combines two controlled steps:

$$\pi_1^1 \circ (h \times k) = h \circ \pi_1^0$$

by product projection, followed by upper-star accumulation. The arrow k disappears because the first projection observes only the first component.

The source is a composite out of $A_0 \times B_0$ and the target is a single upper-star cut with the same source and codomain. The proposed normal form is $(\pi_1^0)^*(g \circ h)$. Its future generic owners should be the upper-star action together with chosen product projections and the bifunctorial action of the product structure in K. The equality mode here is mathematical development: emdash does not yet package binary products of objects in an arbitrary ambient category with this universal computation. Such an owner should retain the next-hom actions of the projection and product-arrow operations, rather than stop at a 1-categorical equation.

> **Formal status — mathematical development.** The general theorem assumes a chosen binary-product interface internal to an arbitrary category K, including product arrows, projection beta, and iterable higher action. The active product package instead supplies binary products of categories, which gives the specialization $K = \mathsf{Cat}$ described next.

9.4.2 The Checked Cat-Specialized Legs

Set $K = \mathsf{Cat}$. Then A_i, B_i, C are categories and h, k, g are functors. The active product-valued-functor representation exposes projection by a Sigma observation. Its checked structural reduction is

$$\mathsf{fst}(h \times k) \rightsquigarrow h \circ \pi_1^0.$$

The checked, owner-aligned upper-star cut is

$$(\pi_1^0)^*(h^*(g)) \rightsquigarrow (h \circ \pi_1^0)^*(g),$$

and $h^*(g)$ is proof-time comparable with the readable composite $g \circ h$. Both owner-aligned sides retain the upper-star precomposition head and therefore its higher action on transfors between possible functors g.

The literal Cat instance of the general equation is not currently one runtime reduction. A focused typed audit found both sides well formed, checked the two owner-aligned legs, and found that neither runtime conversion nor typed reflexivity joins the raw composite $\pi_1^1 \circ (h \times k)$ directly to the selected observation $\mathsf{fst}(h \times k)$. Packaging that narrow projection/composition comparison would suffice. Installing a broad product eta rewrite would be a much stronger and unsafe response.

> **Formal status — formal consequence.** Evidence `CUT-PRODUCT-PROJECTION`. The Cat-specialized equation follows from controlled associativity, upper-star composition, and the checked product projection. The diagnostic suite checks the owner-aligned reduction. The literal raw-composite bridge is not packaged, so the textbook display is not labeled a checked kernel reduction.

9.4.3 Example 5: Product Beta Is Elimination After Introduction

In a category K with chosen products, arrows $p : X \to A$ and $q : X \to B$ have a pairing $\langle p, q \rangle : X \to A \times B$. The characteristic structural cuts are

$$\pi_1 \circ \langle p, q \rangle = p, \qquad \pi_2 \circ \langle p, q \rangle = q.$$

The source and target of the first equation lie in $\mathrm{Hom}_K(X, A)$, and those of the second lie in $\mathrm{Hom}_K(X, B)$. Their proposed normal forms are p and q. For arbitrary K, the owners and higher action belong to the same future chosen-product interface as Example 4, so these equations are mathematical development rather than claims about the present kernel.

The active category-of-categories specialization is nevertheless concrete. For arrows $p : a \to a'$ in A and $q : b \to b'$ in B, the pair (p, q) is an arrow $(a, b) \to (a', b')$ in the product category. The projection functors compute:

$$\pi_1[p, q] \rightsquigarrow p, \qquad \pi_2[p, q] \rightsquigarrow q.$$

Here the selected normal forms are the component arrows themselves. The specialized owners are the capped hom-actions of `Product_projL_func` and `Product_projR_func`, and the equality mode is runtime reduction. Before capping, each full hom-action remains a projection functor from a product of hom-categories, so higher component cells remain available.

> **Formal status — checked.** Evidence `CAT-PRODUCT-CALCULUS`. Product categories, product maps, projection functors, and their object and hom projections have focused componentwise computations. This is evidence for the desired general architecture, not an assertion that arbitrary

> object-level products in every K are already implemented.

9.4.4 Fibred Structural Cuts

The same introduction/elimination pattern appears in a dependent categorical context. Let $B, C : K \vdash \mathsf{Cat}$ be independent families over one base, and let $P(B, C)$ be their fibrewise product. For displayed functors

$$\Phi : E \Longrightarrow B, \qquad \Psi : E \Longrightarrow C,$$

pairing introduces a map $\mathsf{pair}_d(\Phi, \Psi) : E \Longrightarrow P(B, C)$, while the two displayed projections eliminate it. Their structural cuts are whole displayed-functor reductions:

$$\mathsf{projL}_d \circ \mathsf{pair}_d(\Phi, \Psi) \rightsquigarrow \Phi, \qquad \mathsf{projR}_d \circ \mathsf{pair}_d(\Phi, \Psi) \rightsquigarrow \Psi.$$

These equations say more than pointwise product beta. At an object $k : K$, pairing is the ordinary product pairing in the fibre. Over a base arrow $p : k \to l$, its action is the pair of the two displayed actions over the same p. Its canonical internalized cell at a fibre object u is likewise componentwise:

$$\mathsf{cell}\big(\mathsf{pair}_d(\Phi, \Psi), p, u\big) = \big(\mathsf{cell}(\Phi, p, u), \mathsf{cell}(\Psi, p, u)\big).$$

Thus the elimination cuts remain valid while object action, base-arrow action, and the selected next-cell observation stay internally functorial. This is the structural calculus of independent siblings $k : K, b : B[k], c : C[k]$; it does not exchange a variable with another whose classifier depends on it.

> **Formal status — checked.** Evidence `CAT-FIBREWISE-CONTEXT` covers the fixed-base displayed projections and pairing, both whole projection-after-pairing reductions, and their componentwise fibre, base-arrow, internalized-cell, and selected higher observations. The arbitrary-K chosen-object-product interface of Examples 4 and 5 remains a separate mathematical development.

9.5 Universal Cuts

A universal property turns a family of maps into a chosen object together with inverse ways of introducing and eliminating a factorization. Its computation laws are higher-level cut elimination. Adjunction triangles are the first example; representability, co-Yoneda, and weighted limits continue the same line.

9.5.1 Example 6: The Two Adjunction Triangles

Let $F : R \to L$ be left adjoint to $G : L \to R$, with unit $\eta : \mathrm{id}_R \Rightarrow GF$ and counit $\varepsilon : FG \Rightarrow \mathrm{id}_L$.

For $g : X \to X'$ in R and $f : FX' \to Y$ in L, the left triangle cut is

$$\varepsilon[f] \circ F[\eta[g]] \rightsquigarrow f \circ F[g].$$

Both sides have source FX and target Y. The selected normal form removes the adjacent unit-counit detour while preserving the ordinary functor action $F[g]$.

Dually, for $f : X \to GY'$ in R and $g : Y' \to Y$ in L, the right triangle cut is

$$G[\varepsilon[g]] \circ \eta[f] \rightsquigarrow G[g] \circ f.$$

Both sides have source X and target GY. The selected normal form again removes the universal detour and retains the functorial image of the boundary arrow.

The owners are not arbitrary transformations with the same types. They are the stable unit and counit observations of the indexed `Adjunction` witness, and both equations are runtime reductions. Their components use the off-diagonal `tapp1` action, so action on higher cells in f and g remains part of the surrounding functorial calculus.

> **Formal status — checked.** Evidence `ADJ-TRIANGLE-CUTS`. `unit_adj_transf` and `counit_adj_transf` are the selected observations that trigger the two reductions. Independently named unit-shaped and counit-shaped transfors do not acquire these computations by type alone.

9.5.2 Example 7: The Shaped Co-Yoneda Beta Cut

For a profunctor $P : A \rightsquigarrow B$, the right and left unit maps have the forms

$$P \otimes_B U_B \Longrightarrow P, \qquad U_A \otimes_A P \Longrightarrow P.$$

If p is a shaped element and id is the matching identity-shaped hom element, their component cuts reduce as

$$\varepsilon^R_P(p \otimes \mathrm{id}) \rightsquigarrow p, \qquad \varepsilon^L_P(\mathrm{id} \otimes p) \rightsquigarrow p.$$

The source is a shaped element of the corresponding tensor and the target is the original shaped element of P. The normal form is p; the owners are the two `Prof_coyoneda_*` transformations; and the equality mode is runtime reduction on the selected shaped cells. Naturality-fusion remains available for a profunctor map $P \to P'$, so this beta law is not merely a capped set-level equation.

> **Formal status — checked.** Evidence `PROF-COYONEDA`. Chapter 13 develops the representable and profunctor context required to read this calculation as the computational core of Yoneda.

9.5.3 Example 8: Weighted-Limit Beta And Eta

A computational weighted-limit witness is a comparison between the weighted cone profunctor and the representable hom profunctor at the proposed limit. After reindexing along a probe $M : I \to B$, it supplies operations

$$\mathsf{push}(r) : R \Longrightarrow \mathrm{Hom}(M, L), \qquad \mathsf{pull}(s) : R \Longrightarrow \mathrm{Cone}_W(M, F).$$

Their universal cuts reduce in both directions:

$$\mathsf{pull}(\mathsf{push}(r)) \rightsquigarrow r, \qquad \mathsf{push}(\mathsf{pull}(s)) \rightsquigarrow s.$$

The source and target are profunctor maps of the displayed kinds, and the normal forms are the original maps r and s. The generic owners are `prof_comparison_push` and `prof_comparison_pull`; the weighted names merely specialize their types. The equality mode is runtime beta/eta reduction. The comparison can still be reindexed, symmetrized, and composed, so the higher universal structure is retained instead of being collapsed to a chosen set-level bijection.

> **Formal status — checked.** Evidence `PROF-COMPARISON-BETA-ETA`. Chapter 16 turns this general comparison calculus into the weighted-limit interface.

9.5.4 Example 9: A Right Adjoint Preserves The Weighted Limit

Suppose $L_a : A \to B$ is left adjoint to $R_a : B \to A$, and a comparison certifies $L : J' \to B$ as the W-weighted limit of $F : J \to B$. Three universal cuts compose: move a cone across the adjunction, use the given representation, and move the representing hom back. The output is a comparison certifying $R_a L$ as the weighted limit of $R_a F$.

The source is the supplied weighted-limit comparison together with the adjunction; the target is the transported comparison in A. The selected normal form is the composite of the two adjunction-mate comparisons and the reindexed limit comparison. Its owner is `right_adjoint_preserves_weighted_limit_cov_comp`. This is a checked construction, not a rewrite asserting that arbitrary limit syntax commutes with every right adjoint. The resulting comparison retains its push/pull beta-eta action on every probe and profunctor map.

> **Formal status — checked.** Evidence `WEIGHTED-LIMIT-PRESERVATION`. The theorem is developed in Chapter 16 after weights, cones, and mate comparison have been introduced.

9.5.5 Example 10: The Weighted-Colimit Dual

A W-weighted colimit in B is represented by the corresponding weighted limit in B^{op}. Applying the preceding theorem to the opposite adjunction gives the dual statement: a left adjoint preserves the selected weighted colimit.

The source is a weighted-colimit comparison and an adjunction; the target is the transported comparison after the left adjoint. The selected normal form is the opposite of the right-adjoint preservation composite, with double opposites and reversed composition reduced by the generic duality owners. The construction is checked under `left_adjoint_preserves_weighted_colimit_con`. Its output is still a full comparison with push/pull behavior after passing to opposites, not merely an equality of chosen objects.

> **Formal status — checked.** Evidence `WEIGHTED-COLIMIT-PRESERVATION`. Chapter 17 develops the variance and opposite-category calculation in full.

9.6 What Cut Elimination Preserves

The ten examples have a common shape:

$$\text{introduction or action} \quad + \quad \text{matching elimination} \quad \longrightarrow \quad \text{one semantic owner.}$$

Their normal forms are deliberately not all raw composites. Lower star keeps postcomposition visible; upper star keeps precomposition visible; `tapp1` keeps the varying arrow family visible; product projection keeps a structural component visible; and a universal comparison keeps its inverse operations visible. This is what makes the calculus iterable.

In particular, a capped equation is not the whole meaning of a construction. The full `hom_postcomp_func`, `hom_precomp_along_func`, and `tapp1_func` operations act on the next hom. Product projection has a full hom-functor. Profunctor comparison can be reindexed and composed. A good reduction removes the cut without discarding the object that higher-dimensional consumers need.

9.7 Directed Family Transfors And Lax Comparison

Strict cut elimination should not be confused with strictness of every directed comparison. Let $E, D : K \to \mathsf{Cat}$ be directed families and let

$$\Phi : E \Longrightarrow D$$

be a natural family morphism. At each $k : K$ it has a fibre functor $\Phi_k : E[k] \to D[k]$. A transfor $\epsilon : \Phi \Rightarrow \Psi$ has a fibrewise transfor and point components at objects of each fibre.

> **Formal status — checked.** Evidence `TRANSFD-FIBRE-COMPONENTS`. The stable displayed presentations are `Transfd_cat` and `Transfd`; `Fibre_transf` and `Fibre_transf_app` expose their fibre and object projections.

When the base moves along $p : x \to y$, the two relevant transports do not have to be equal. Starting with $u : E[x]$, the internal displayed hom action gives a directed cell

$$\chi^{\Phi}_{p,u} : D[p](\Phi_x u) \longrightarrow \Phi_y(E[p]u)$$

in $D[y]$. This is the displayed laxity cell. It has a direction and need not be invertible. The generic functor and transfor cuts around it compute strictly, but the comparison itself remains mathematical data.

> **Formal status — checked.** Evidence `FUNCTORD-DISPLAYED-LAXITY`. The endpoint functors are `functord_transport_lhs_func` and `functord_transport_rhs_func`; the active component is `fdapp1_int_cell`.

For a fibre arrow $\alpha : E[p](u) \to v$, the capped displayed action has the readable form

$$\Phi_y[\alpha] \circ \chi^{\Phi}_{p,u} : D[p](\Phi_x u) \longrightarrow \Phi_y v.$$

The direct internal-hom projection is the runtime owner. Expanding it into this composite on every use would discard the stable higher action and give the comparison a second owner.

9.8 Total Categories And The WalkingEnd Decoder

The family morphism induces a functor on total categories,

$$\Sigma\Phi : \sum_{k:K} E[k] \longrightarrow \sum_{k:K} D[k].$$

An arrow in the source has the form $(p, \alpha) : (x, u) \to (y, v)$, where $\alpha : E[p](u) \to v$. Its image is

$$(p, \Phi^d[p, u, v, \alpha]).$$

Thus the base arrow is retained and the fibre arrow is exactly the capped displayed hom action. The laxity cell is the special case in which $v = E[p](u)$ and α is the identity.

> **Formal status — checked.** Evidence `FUNCTORD-SIGMA-ACTION`. `sigma_map_func` owns the total functor, whose arrow projection uses the capped internal displayed action rather than reconstructing a comparison composite.

For the contextual WalkingEnd decoder

$$\mathsf{decode}^d : \mathsf{Code} \Longrightarrow \mathsf{Rep}_*,$$

the generator comparison is the spiral. Evaluated at a base arrow $p : * \to x$ and zero, its endpoints reduce to

$$p \qquad \text{and} \qquad \mathsf{decode}_x(\mathsf{encode}_x(p)).$$

The generic laxity component is therefore the directed normalization cell of Chapter 8. Replacing it by a commuting equality at the outset would erase the content of normalization. Equality is extracted only later, using discreteness of the target hom-category.

9.9 The Packaging Boundary

The calculus now has a continuous line from local arrow composition to weighted universal properties. Its discipline is equally important in the negative direction.

- The arbitrary-K product benchmark needs a general chosen-product interface. Its Cat instance still awaits one narrow packaged projection/composition comparison; neither gap justifies broad product eta.

- A displayed family morphism has the component-level laxity cell and its full internal projection ladder, but no duplicate whole-square facade.
- General ends, coends, and arbitrary Kan extensions require universal interfaces stronger than the selected profunctor operations.
- Runtime conversion, proof-time comparison, internal equality, and equivalence remain different judgments.

> **Formal status — research boundary.** Evidence `FUNCTORD-WHOLE-LAXITY`. A future whole-transfor comparison between $D[p] \circ \Phi_x$ and $\Phi_y \circ E[p]$ should be derived from the internal displayed action, project coherently through higher homs, and serve a concrete consumer. It must not duplicate the component semantics.

Cut elimination is therefore not a feature catalogue. It is the organizing principle by which functorial type theory decides what should compute, what should merely compare, and what higher structure must remain visible after a calculation is complete.

10. Categories, Precategories, And Categorical Identity

Chapter 2 introduced the categorical language needed for the WalkingEnd calculation. We now return to the word *category* with a different question: when should equality of objects agree with categorical sameness? The answer depends on which categorical layer is under discussion. This second pass is a spiral over Chapters 2, 4, and 7, not a replacement for their definitions.

The distinction is foundational. Ordinary univalent category theory begins with set-valued homs and asks for an identity-to-isomorphism map to be an equivalence. Emdash begins with iterated hom-categories, where arrows may have directed higher arrows between them. The ordinary theory is therefore an important specialization of functorial type theory, but it cannot serve as the definition of the ambient native category.

10.1 A Translation Table, Not A Collapse

The following table fixes the translation used throughout the remainder of the book.

Layer	Characteristic data	Equality or height condition	Role here
Native `Cat`	objects and iterated `Hom_cat` classifiers	no set-valued-hom premise in the definition	ambient directed higher language
Equality-local `Path(A)`	elements of A and identity evidence between them	every arrow is induced by equality	groupoidal specialization used for ordinary type theory
Finite `IsNCat` evidence, including `OneCat`	a native category plus a recursive hom-height witness	one-dimensionality makes each next hom discrete	bridge to ordinary category-shaped examples
HoTT precategory	a type of objects and set-valued homs	equality between parallel arrows is proposition-valued	ordinary 1-categorical specialization
HoTT category	a HoTT precategory for which object identity agrees with isomorphism	`idtoiso` is an equivalence	univalent 1-category notion, not the definition of native `Cat`
HoTT strict category	a HoTT precategory whose object type is a set	object identity is proposition-valued, without assuming `idtoiso` is an equivalence	a qualified truncation notion, unrelated to runtime strictness

Three warnings belong beside the table.

First, *set-valued hom* means a truncation condition, not that a hom must be presented by a classical set outside type theory. Second, *strict category* in the HoTT sense refers to the truncation level of the object type; it says nothing about which emdash expressions reduce at runtime. Third, a HoTT strict category need not be a univalent category: object identity may be proposition-valued while nontrivial isomorphisms remain. Finally, a `OneCat` witness controls the height of a native category but does not by itself identify object equality with isomorphism.

> **Formal status — mathematical development.** The table is a translation discipline, not an implemented equivalence of packages. A generic construction identifying native finite-height categories with HoTT precategories would have to choose the ordinary hom-set presentation and prove compatibility with native composition and object equality.

10.2 The Ordinary Precategory Specialization

An ordinary HoTT precategory $\mathcal{A}$ consists of:

- a type $\mathrm{Obj}(\mathcal{A})$ of objects;
- for each a, b, a set $\mathrm{Hom}_{\mathcal{A}}(a, b)$;
- identity arrows id_a;
- composition $g \circ f$, typed so that $f : a \to b$ and $g : b \to c$;
- left-unit, right-unit, and associativity equalities.

Because each hom is a set, equality between parallel arrows is a proposition. The category laws therefore do not require an additional tower of coherence data. This is precisely the simplifying hypothesis that ceases to be available in a genuinely higher native category: there, the equality or cell between two composites may itself have nontrivial higher structure.

The emdash specialization replaces an external record of hom sets by a native category C equipped with one-dimensional evidence. For every pair x, y, the witness

$$\mathrm{IsNCat}(1, C)$$

makes the next hom-category $\mathrm{Hom}_C(x, y)$ discrete. Its objects are the ordinary arrows; the discreteness evidence controls their higher equality. The same witness also implies the corresponding truncation bound on $\mathrm{Obj}(C)$, but it still says nothing about whether every ordinary isomorphism comes from object identity.

> **Formal status — checked.** Evidence `CAT-ITERATED-HOMS` and `CAT-DIMENSION`. Native `Cat` has category-valued homs, `IsNCat` recurses through those homs, and finite-dimensional evidence yields the predicted truncation evidence for the object classifier.

This gives a reliable rule of reading: use ordinary precategory reasoning when the hom-height assumptions make it valid, but keep the native iterated owner visible whenever a construction must act on the next cell.

10.3 Isomorphism And Identity-To-Isomorphism

For objects a, b of an ordinary precategory, an isomorphism $a \cong b$ consists of arrows

$$f : a \longrightarrow b, \qquad g : b \longrightarrow a$$

with $g \circ f = \mathrm{id}_a$ and $f \circ g = \mathrm{id}_b$. Since the homs are sets, an inverse to a fixed f is unique when it exists. Consequently the type of isomorphism evidence between two fixed objects is itself a set.

Object identity always produces an isomorphism. Given $p : a = b$, identity induction defines

$$\mathrm{idtoiso}_{\mathcal{A}}(p) : a \cong b$$

by sending reflexivity to the identity isomorphism. This map exists before assuming univalence. The additional assertion that $\mathcal{A}$ is a *category* in the HoTT sense is that

$$\mathrm{idtoiso}_{\mathcal{A}} : (a = b) \longrightarrow (a \cong b)$$

is an equivalence for every a, b.

> **Formal status — mathematical development.** Evidence `UCAT-IDENTITY-ISOMORPHISM`. This is the ordinary univalent 1-categorical theorem adapted from the HoTT category spine. It is not a theorem about every native `Cat`.

Univalence changes how one reasons about structure. A property or construction invariant under isomorphism can be transported along object identity because isomorphism and identity carry the same information. It also implies that the object type of an ordinary category is a 1-type: its identity types are equivalent to isomorphism types, and those are sets.

The effect on arrows is concrete. If $p : a = a'$, $q : b = b'$, and $f : a \to b$, then transporting f across both endpoint identities agrees with

$$\mathrm{idtoiso}(q) \circ f \circ \mathrm{idtoiso}(p)^{-1} : a' \longrightarrow b'.$$

Thus endpoint transport is a lower-star cut at the target and an upper-star cut at the source. This ordinary formula anticipates the represented-hom actions used throughout the native calculus.

The implication should not be reversed carelessly. Having a 1-type of objects does not make a precategory univalent, just as having discrete homs does not force isomorphic objects to be equal.

10.4 Five Nearby Notions Of Sameness

The native theory places several useful notions next to one another. Their proximity is a feature, but it is not an identification.

Notion	Typical expression	What it compares
object identity	$p : x = y$ in $\mathrm{Obj}(C)$	two objects in the underlying equality layer
path-generated arrow	$\mathrm{path\_to\_hom}(p) : x \to y$	the directed arrow induced by object identity
ordinary isomorphism	$\mathrm{IsoEvidence}_C(x, y)$	inverse arrows with equality-valued cancellation
carrier equivalence	$A \simeq B$ via `TypeEquiv`	two decoded groupoidal carriers
native omega-equivalence	$\mathrm{OmegaEquiv}_C(x, y)$	a selected arrow with recursively usable inverse evidence

There are checked maps between some adjacent entries. Object identity yields a path-generated arrow and a transparent native omega-equivalence package. Ordinary isomorphism evidence also yields native omega-equivalence evidence. A `TypeEquiv`, however, compares carriers rather than two objects of an arbitrary category, and it must not be substituted for categorical equivalence without an explicit construction.

> **Formal status — checked.** Evidence `EQUIV-OMEGA` and `EQUIV-TYPE`. The active interfaces expose object-path and carrier-equivalence constructions separately; neither declaration asserts a blanket identification of all five rows.

This ladder explains why the book qualifies the word *equivalence*. A theorem about carrier equivalence, ordinary categorical equivalence, or recursive omega-equivalence has different inputs and different higher consequences.

10.5 Equality-Local Categories

For a groupoidal classifier A, the native category

$$\mathrm{Path}(A)$$

has the elements of A as objects and identity evidence $x = y$ as the hom from x to y. Reflexivity supplies identities, and ordinary functions act functorially on paths. This is the native home of the familiar principle that equality behaves like a groupoid.

> **Formal status — checked.** Evidence `CAT-PATH-CATEGORY`. `Path_cat`, `Path_cat_func`, and `path_map_func` retain the equality-local category and its iterable functor action.

The ordinary and native readings coincide only under the appropriate height condition. If A is a 1-type, each identity type $x = y$ is a set, so $\mathrm{Path}(A)$ can be read as an ordinary HoTT precategory. For a general A, its identity types may have higher identity types, and the native iterated-hom presentation retains them rather than truncating them away.

This example also separates groupoidality from directed categorical univalence. Every arrow in $\mathrm{Path}(A)$ comes from identity by construction. An arbitrary native category may contain noninvertible arrows, and its identity layer cannot be recovered merely by declaring those arrows to be paths.

10.6 Preorders, Posets, And Skeletal Reasoning

An ordinary preorder can be regarded as a precategory whose homs are mere propositions: there is at most one arrow from a to b. Isomorphism then means $a \leq b$ and $b \leq a$. The preorder is a univalent category exactly when this mutual reachability agrees with identity—the type-theoretic form of antisymmetry. Thus posets are the proposition-valued-hom examples of the general identity-to-isomorphism principle.

This example is useful because it exposes a frequent confusion. A skeletal presentation says that isomorphic objects are literally or propositionally the same in a chosen presentation. Univalence says that the canonical map from identity to isomorphism is an equivalence, so the identification is stable under dependent reasoning. The latter is an internal principle, not merely a convention for selecting representatives.

The category of small sets is the other basic example. Its arrows are functions, and its identity-to-isomorphism map is the restriction of the identity-to-equivalence map for types. Universe univalence therefore makes it a univalent category. Categories of groups, rings, and similar structures follow when equivalence of carriers preserving the structure is shown to agree with identity of the structured object. Chapter 15 develops that structure-identity pattern.

> **Formal status — mathematical development.** The preorder/poset example is ordinary univalent 1-category theory, as are the category-of-sets and structured-object examples. The active code has proposition and set truncation evidence plus selected universe-univalence results, but no generic packaged preorder or structured-category construction is claimed here.

10.7 The Checked Ordinary-Isomorphism Lift

The active calculus supports one direction of the native comparison with no finite-height premise. Given ordinary isomorphism evidence $i : x \cong y$, its forward arrow and inverse laws define

$$\mathrm{iso\_evidence\_omega\_equiv}(i) : \mathrm{OmegaEquiv}_C(x, y).$$

The construction keeps the forward arrow selected by i and packages its inverse evidence in the recursive native facade. It is a structural lift, not a decoder identifying isomorphism evidence with object identity.

> **Formal status — checked.** Evidence `EQUIV-ORDINARY-ISO-LIFT`. The owners are `iso_evidence_omega_along` and `iso_evidence_omega_equiv`; diagnostics check their projections and reflexive behavior.

This is the chapter's central checked theorem. It says that ordinary inverse data is strong enough to enter the native equivalence calculus. It does not say that native equivalence, ordinary isomorphism, and object identity are interchangeable in every category.

10.8 The Native Univalence Boundary

A full categorical identity theorem for the ambient directed theory would need to answer at least four questions.

1. Which equivalence classifier is compared with object identity?
2. Does the comparison retain a selected directed arrow?
3. How does it act on every next hom-category?
4. Under what height, saturation, or groupoidality hypotheses are the two directions inverse?

The active code has selected univalence theorems for groupoidal and truncated universes, object-path-to-equivalence constructions, finite-height truncation, and the one-way ordinary-isomorphism lift. It does not package a general equivalence

$$(x = y) \simeq \mathrm{IsoEvidence}_C(x, y)$$

for arbitrary native C, nor a general saturation theorem that freely adds such identities.

> **Formal status — research boundary.** Evidence `UNIV-FULL-OBJECT-ISO`. A future owner must compare object identity and the selected categorical equivalence notion while remaining coherent with iterated hom action. Chapter 15 returns to this problem through the structure identity principle and Rezk-style saturation.

The title *univalent foundations* should therefore be read layer by layer. Univalence is the disciplined alignment of identity with the appropriate equivalence, not a license to erase the distinctions that make the directed higher theory computational.

11. Functors, Transfors, And Functor Categories

Chapter 9 studied transfors as a calculus of cuts. We now make their categorical organization explicit. Functors are objects of a functor category, transfors are arrows between those objects, and higher transfors are obtained by iterating the same hom construction.

This is another spiral rather than a repetition. Ordinary 1-category theory starts from object functions, hom functions, and pointwise naturality squares. The native calculus packages all of these as iterable categorical action. A point component is the identity-boundary case of an off-diagonal action, and a naturality square is the visible boundary of a computation.

11.1 Functors Act At Every Retained Dimension

For ordinary precategories $\mathcal{A}$ and $\mathcal{B}$, a functor $F : \mathcal{A} \to \mathcal{B}$ consists of an object function and functions

$$F_{a,b} : \mathrm{Hom}_{\mathcal{A}}(a, b) \longrightarrow \mathrm{Hom}_{\mathcal{B}}(Fa, Fb)$$

that preserve identities and composition. In the set-valued-hom specialization, these are ordinary functions between hom sets.

A native emdash functor retains more structure. Its action on a pair of objects is itself a functor

$$F_{x,y} : \mathrm{Hom}_{A}(x, y) \longrightarrow \mathrm{Hom}_{B}(Fx, Fy).$$

Evaluating that functor at an arrow $f : x \to y$ gives $F[f]$. Keeping the full hom functor visible also keeps its action on 2-cells between arrows, and then on higher cells by iteration.

> **Formal status — checked.** Evidence `CAT-FUNCTOR-CALCULUS`. `fapp0` is object action, `fapp1_func` is the full next-hom action, and `fapp1_fapp0` is its value at one arrow. Identity and composition reductions belong to these generic owners.

The preservation law is oriented as cut elimination:

$$F[g] \circ F[f] \rightsquigarrow F[g \circ f].$$

This is not a theorem copied onto each constructor. It is computation of the global functor-action interface. A specialized construction should expose its own semantic projections, while ordinary functoriality remains owned here.

11.2 From Natural Transformations To Transfors

For ordinary functors $F, G : \mathcal{A} \to \mathcal{B}$, a natural transformation $\eta : F \Rightarrow G$ is usually presented by arrows

$$\eta_x : Fx \longrightarrow Gx$$

such that every $f : x \to y$ satisfies

$$G[f] \circ \eta_x = \eta_y \circ F[f].$$

The native transfor retains the common interior of this square. For every pair x, y it supplies an off-diagonal functor

$$\eta_{x,y} : \mathrm{Hom}_A(x, y) \longrightarrow \mathrm{Hom}_B(Fx, Gy).$$

We write $\eta[f] : Fx \to Gy$ for its value at f. The point component is recovered at the identity:

$$\eta[\mathrm{id}_x] \rightsquigarrow \eta_x.$$

> **Formal status — checked.** Evidence `TRANSF-POINT-OFFDIAGONAL`. The point projection is `tapp0_fapp0`; `tapp1_func` and `tapp1_fapp0` expose the full and capped off-diagonal actions.

This presentation does not add exotic data to an ordinary natural transformation. In the one-dimensional specialization, naturality determines the off-diagonal value in either familiar way:

$$\eta[f] = G[f] \circ \eta_x = \eta_y \circ F[f].$$

In the native higher setting, however, retaining $\eta_{x,y}$ as a functor also retains its action on cells between possible f's. Point components alone would hide that action and force it to be reconstructed later.

11.3 Naturality Is A Pair Of Family Cuts

Take composable arrows

$$h : w \to x, \qquad f : x \to y, \qquad g : y \to z.$$

The two strict naturality computations are

$$G[g] \circ \eta[f] \rightsquigarrow \eta[g \circ f],$$
$$\eta[f] \circ F[h] \rightsquigarrow \eta[f \circ h].$$

Setting f to an identity makes the usual naturality square reappear. Both boundary composites normalize through the same off-diagonal interior $\eta[f]$. Thus naturality is not merely a proposition verified after a family of components has been assembled; it is the way the family action absorbs neighboring cuts.

> **Formal status — checked.** Evidence `TRANSF-STRICT-NATURALITY`. Both capped equations and their uncapped hom-functor forms are runtime reductions of the global `tapp1*` calculus. The full forms retain action on the next cells.

This is the chapter's central checked theorem. It explains why the calculus uses a transfor rather than a bare dependent function of point components: the transfor is the computational natural family.

11.4 The Functor Category

For native categories A and B, the category

$$[A, B] := \mathrm{Functor\_cat}(A, B)$$

has functors $A \to B$ as objects. Its hom-category between F and G is

$$\mathrm{Transf\_cat}(F, G).$$

An identity arrow in $[A, B]$ is the identity transfor. Composition in $[A, B]$ is vertical composition of transfors. Iterating the hom of $\mathrm{Transf\_cat}(F, G)$ yields modifications and higher transfors without changing the ambient notion of category.

> **Formal status — checked.** Evidence `CAT-TRANSFOR-CALCULUS`. `Functor_cat` and `Transf_cat` are active native categories, and `Hom_cat(Functor_cat(A,B),F,G)` reduces to the corresponding transfor category.

In the ordinary set-valued-hom specialization, the same construction gives a precategory of functors and natural transformations. Equality between natural transformations is pointwise because the codomain homs are sets. A natural transformation is a natural isomorphism exactly when each component is an isomorphism.

The native formulation is intentionally stronger at the interface. It does not force all higher transfor structure to be proposition-valued merely because the first components resemble ordinary natural transformations.

11.5 Whiskering And Horizontal Composition

Functor composition acts on transfors in two one-sided ways. If $\eta : F \Rightarrow G$ and $K : B \to C$, post-whiskering gives

$$K\eta : KF \Rightarrow KG.$$

If $H : X \to A$, pre-whiskering gives

$$\eta H : FH \Rightarrow GH.$$

These are the transfor-level actions of the same postcomposition and precomposition functors studied in Chapter 9. They are not independent definitions of naturality.

Now take

$$\alpha : F \Rightarrow G : A \to B, \qquad \beta : H \Rightarrow K : B \to C.$$

Their horizontal composite can be read at an object a in either of the ordinary forms

$$\beta_{Ga} \circ H[\alpha_a], \qquad K[\alpha_a] \circ \beta_{Fa}.$$

Naturality of β identifies the two. The native calculus packages the pair (α, β) under the product-composition action, so its off-diagonal value first transports the A-arrow through α and then through β. This gives a single iterable owner rather than two competing component formulas.

> **Formal status — checked.** Evidence `TRANSF-HORIZONTAL-CALCULUS`. The generic owner is `comp_prod_fapp1_fapp0`; diagnostics check its point, full off-diagonal, and capped off-diagonal projections. The ordinary two-formula equality is its 1-categorical reading.

11.6 Interchange And Controlled Coherence

Vertical and horizontal composition satisfy interchange. Schematically, for a composable four-cell grid,

$$(\beta_2 \circ \beta_1) * (\alpha_2 \circ \alpha_1) = (\beta_2 * \alpha_2) \circ (\beta_1 * \alpha_1).$$

In an ordinary functor precategory this is an equality of natural transformations, proved componentwise using associativity and naturality. In the native calculus, the corresponding computation is organized by the generic product-composition action and the off-diagonal vertical-composite folds. A representable four-cell instance is exposed as propositional interchange evidence.

The equality mode matters. Functor composition has associativity and unit comparisons, and ordinary category theory packages their familiar pentagon and triangle coherences. Emdash does not install unrestricted reassociation in both directions as runtime computation. Instead:

- a semantic owner absorbs a neighboring cut when it has a selected normal form;
- proof-time comparison is used when two presentations should elaborate together but neither should replace the other globally;
- propositional equality records a theorem without changing runtime normal forms;
- higher transfors retain coherence data that is not truncated away.

This is the categorical version of controlling associativity. Parentheses may be suppressed in mathematical prose when the intended composite is unambiguous, but the formal presentation must still select an owner and an equality mode.

11.7 The Ordinary Functor-Category Theorem

Let $\mathcal{A}$ be an ordinary precategory and $\mathcal{B}$ an ordinary univalent category. Then the functor precategory $[\mathcal{A}, \mathcal{B}]$ is a univalent category. The reason is pointwise but not merely syntactic:

1. an identity $F = G$ gives a natural isomorphism by identity induction;
2. a natural isomorphism gives pointwise isomorphisms $Fx \cong Gx$;
3. univalence of $\mathcal{B}$ turns those into pointwise identities;
4. function extensionality assembles equality of object functions;
5. the functor laws and naturality data are propositions at this height, so the assembled equality determines the whole functor.

The same analysis shows that identity between ordinary functors agrees with natural isomorphism.

> **Formal status — mathematical development.** Evidence `UCAT-FUNCTOR-CATEGORY`. This is the HoTT 1-categorical theorem under set-valued-hom and univalent-codomain hypotheses. The active native `Functor_cat` supplies the categorical object, but this general identity-to-natural-isomorphism equivalence is not a checked emdash theorem.

11.8 The Native Functor-Category Boundary

A native analogue cannot be obtained by deleting the word *set* from the ordinary proof. One must choose the relevant sameness of functors—object identity, ordinary isomorphism in `Functor_cat`, pointwise native omega-equivalence, or a higher adjoint equivalence—and then prove that the comparison respects:

- point components;
- off-diagonal arrow action;
- cells between source arrows;
- vertical and horizontal composition;
- every further retained hom level.

Pointwise object formulas are therefore necessary but insufficient. The active code has the functor category, the full transfor calculus, and selected object-path and ordinary-isomorphism lifts. It does not yet combine them into a general univalence theorem for native functor categories.

> **Formal status — research boundary.** The missing owner is a native category-univalence package stable under `Functor_cat` and coherent with `tapp1_func` at every retained dimension. Chapter 15 discusses the saturation problem that such a theorem would have to solve.

The computational lesson survives independently of that boundary. Functors and transfors already form a native higher category, and their naturality is already internal computation. Univalence would identify the right notion of sameness in this existing structure.

12. Adjunctions And Equivalences

An adjunction is the first universal construction in which the cut calculus becomes a theorem about a chosen relationship between two functors. It can be presented by a unit and counit, by a natural equivalence of homs, or by representability. These presentations explain one another, but they should not be collapsed before their hypotheses and equality modes have been stated.

Equivalence requires the same care. Carrier equivalence, isomorphism of two objects, equivalence of ordinary categories, and native omega-equivalence answer different questions. This chapter builds the adjunction interface first, then uses it to organize those notions.

12.1 Indexed Adjunction Data

Let

$$F : A \longrightarrow B, \qquad G : B \longrightarrow A.$$

An adjunction $F \dashv G$ has a unit and counit

$$\eta : \mathrm{id}_A \Rightarrow GF, \qquad \varepsilon : FG \Rightarrow \mathrm{id}_B,$$

subject to the two triangle identities. In ordinary notation these say

$$(\varepsilon F) \circ (F\eta) = \mathrm{id}_F, \qquad (G\varepsilon) \circ (\eta G) = \mathrm{id}_G.$$

The active `Adjunction` classifier keeps F and G as indices. Its `unit_adj_transf` and `counit_adj_transf` observations are stable computational heads. This matters: triangle computation is attached to the selected adjunction witness, not to every pair of transformations having the same displayed types.

The direct-TypeScript authoring layer can package already declared F, G, η, and ε—or a counit and whole natural hom transpose—as an indexed witness with proof-time agreements. It expands into ordinary logical-framework declarations: no new adjunction notion and no runtime alias. Appendix G.5 places this convenience at its precise trust boundary.

12.2 The Triangle Cuts

The checked equations are stronger than the diagonal component formulas. Take

$$g : X \to X', \qquad f : FX' \to Y.$$

The left triangle cut is

$$\varepsilon[f] \circ F[\eta[g]] \rightsquigarrow f \circ F[g].$$

Both sides are arrows $FX \to Y$. The unit moves g into the GF boundary, the functor F acts on that off-diagonal component, and the counit removes the resulting FG detour.

Dually, for

$$f : X \to GY', \qquad g : Y' \to Y,$$

the right triangle cut is

$$G[\varepsilon[g]] \circ \eta[f] \rightsquigarrow G[g] \circ f.$$

Both sides are arrows $X \to GY$. Setting f and g to suitable identities recovers the familiar pointwise triangle identities. Retaining arbitrary f and g exhibits the naturality and higher action consumed by the cut.

> **Formal status — checked.** Evidence `ADJ-TRIANGLE-CUTS`. The indexed owner is `Adjunction`; `unit_adj_transf` and `counit_adj_transf` expose the rigid observations that trigger both runtime reductions.

These are the chapter's central checked computations. They illustrate the general policy from Chapter 9: a universal detour reduces only at the universal construction that owns it.

12.3 Transposing Arrows

The ordinary hom formulation of the same adjunction is a natural equivalence

$$\Phi_{a,b} : \mathrm{Hom}_B(Fa, b) \simeq \mathrm{Hom}_A(a, Gb).$$

Starting from the unit and counit, its two directions are

$$\Phi_{a,b}(u) = G[u] \circ \eta_a,$$
$$\Phi^{-1}_{a,b}(v) = \varepsilon_b \circ F[v].$$

The triangle identities cancel the introduced unit-counit pairs, while naturality makes the construction contravariant in a and covariant in b. This is why transposition is more than a family of bijections: it is a comparison of represented hom functors.

In the active interface, let $M : I \to A$ and $H : K \to B$ be arbitrary probes. The reindexed comparison has the profunctor form

$$\mathrm{Hom}_B(FM, H) \simeq \mathrm{Hom}_A(M, GH).$$

The endpoints I and K remain variable, so naturality in both arguments is part of the comparison. Maps into either side can be pushed across the adjunction and pulled back, with beta and eta computation supplied by the generic comparison owner.

> **Formal status — checked.** Evidence `ADJ-HOM-PROF-COMPARISON`. `Adjunction_hom_prof_comparison` is the binary representable comparison, and `Adjunction_hom_prof_comparison_along` reindexes it along arbitrary probes. Its push/pull beta and eta laws are inherited from `ProfComparison`; no second adjunction-specific cancellation calculus is added.

The component formulas above are the mathematical reading of this package. The stable runtime owner is the profunctor comparison, not a global rewrite that expands every mate into a unit/counit composite.

12.4 Adjoints As Representability

Fix $F : A \to B$ and an object $b : B$. The contravariant hom functor

$$a \longmapsto \mathrm{Hom}_B(Fa, b)$$

is represented by an object Gb exactly when there is a natural equivalence

$$\mathrm{Hom}_B(F-, b) \simeq \mathrm{Hom}_A(-, Gb).$$

If such representing objects are chosen coherently as b varies, they form a functor $G : B \to A$, and the representations assemble into $F \dashv G$. Conversely, an adjunction supplies these representations by its hom comparison.

This characterization explains the direction of the terminology. A *right adjoint* assigns representing objects to the functors $\mathrm{Hom}_B(F-, b)$; a *left adjoint* is the functor whose outgoing homs are being represented.

> **Formal status — mathematical development.** Evidence `UCAT-ADJOINT-REPRESENTABILITY`. The equivalence between ordinary adjunction data and coherent pointwise representability belongs to the univalent 1-category development. The active code checks the forward adjunction-to-profunctor comparison and has chosen representation packages, but it does not reconstruct a general adjunction from arbitrary local representations.

Chapter 13 will supply the Yoneda theorem that makes representing objects unique in the correct sense. Chapter 16 will then define weighted limits by the same pattern.

12.5 Uniqueness Of Adjoints

Suppose $F : A \to B$ has two right adjoints G and G'. Their hom representations give, for every b,

$$\mathrm{Hom}_A(-, Gb) \simeq \mathrm{Hom}_B(F-, b) \simeq \mathrm{Hom}_A(-, G'b).$$

Yoneda turns this into a canonical isomorphism $Gb \cong G'b$, natural in b. If A is an ordinary univalent category, those isomorphisms determine identity of the right-adjoint functors. Hence the type asserting that F has a right adjoint is a mere proposition under the appropriate univalence hypothesis.

> **Formal status — mathematical development.** Evidence `UCAT-ADJOINT-UNIQUENESS`. This is the ordinary HoTT theorem. A native version must choose among identity, natural isomorphism, adjoint equivalence, and omega-equivalence while retaining higher coherence.

Uniqueness therefore does not mean that all chosen units and counits are judgmentally the same. It means that the space of choices has the claimed truncation once categorical identity has been aligned with the relevant equivalence.

12.6 A Ladder Of Equivalence Notions

The word *equivalence* will be qualified according to the following table.

Notion	Data or property	What it controls
carrier equivalence	a `TypeEquiv` between decoded classifiers	elements and identity structure of two carriers
ordinary object isomorphism	inverse arrows $x \rightleftarrows y$ inside one category	categorical sameness of two objects at the 1-cell level
isomorphism of precategories	a functor that is fully faithful and whose object map is a carrier equivalence	strict invertibility of the whole ordinary presentation
categorical equivalence	a functor with a quasi-inverse promoted to coherent adjoint-equivalence data	sameness up to natural isomorphism
weak equivalence	fully faithful and merely essentially surjective	property-level ordinary categorical equivalence criterion
native omega-equivalence	a selected arrow with recursively usable inverse evidence	equivalence inside an arbitrary native iterated category

These rows interact only through explicit theorems. A carrier equivalence between object types need not preserve homs. An ordinary isomorphism compares objects in one category, not two entire categories. A categorical equivalence uses functors and natural isomorphisms. Native omega-equivalence can be applied to objects of `Cat_cat`, but its recursive equality-valued interface is not definitionally the HoTT package of fully faithful and essentially surjective data.

12.7 Full Faithfulness And Essential Surjectivity

For an ordinary functor $F : \mathcal{A} \to \mathcal{B}$:

- F is **fully faithful** if every hom map from $\mathrm{Hom}_{\mathcal{A}}(a, a')$ to $\mathrm{Hom}_{\mathcal{B}}(Fa, Fa')$ is an equivalence;

- F is **split essentially surjective** if each $b : \mathcal{B}$ comes with a chosen $a : \mathcal{A}$ and a chosen isomorphism $Fa \cong b$;
- F is **essentially surjective** if the existence of such a and such an isomorphism is merely asserted.

For ordinary precategories, an adjoint equivalence yields full faithfulness and split essential surjectivity, and those chosen data reconstruct an adjoint equivalence. Replacing split existence by mere existence gives the weaker property traditionally called a weak equivalence.

When both sides are univalent categories, full faithfulness makes the type of possible preimages of an object a proposition. Essential surjectivity can then be upgraded to a coherent choice, so weak equivalence and categorical equivalence agree. The same univalence also turns an equivalence into an isomorphism of the underlying precategory presentations.

> **Formal status — mathematical development.** Evidence `UCAT-EQUIVALENCE-CRITERIA`. These are the ordinary HoTT 1-categorical equivalence theorems. No native fully-faithful or essentially-surjective package with coherent higher action is claimed active.

The distinction between split and mere essential surjectivity is constructive, not bureaucratic. An adjoint equivalence needs data that can be applied; a mere existence statement deliberately hides its witness. Univalence supplies the uniqueness needed to recover that data in the ordinary categorical case.

12.8 Adjointification

Sometimes one begins with a functor F, a proposed inverse G, and natural isomorphisms

$$GF \cong \mathrm{id}_A, \qquad FG \cong \mathrm{id}_B$$

whose chosen unit and counit do not yet satisfy the triangle equations. In ordinary category theory, one of the two isomorphisms can be adjusted so that the triangles hold. This **adjointification** turns equivalence data into an adjoint equivalence.

The triangles are therefore coherent normalization data, not an arbitrary extra burden. They make mate transposition inverse in a controlled way and permit universal cuts to reduce without choosing a fresh proof at every use.

> **Formal status — mathematical development.** Ordinary adjointification is part of the 1-categorical theory. The active `Adjunction` relation starts with a selected triangle-computing witness; it does not expose a generic constructor that adjusts arbitrary quasi-inverse transfors.

12.9 Identity Of Ordinary Categories

There is one further univalent step. For ordinary precategories, identity of the complete precategory structures corresponds to isomorphism of precategories: a fully faithful functor whose object map is a carrier equivalence. When the precategories are univalent categories, categorical equivalence and

such isomorphism agree. Consequently identity of ordinary categories corresponds to equivalence of categories.

Schematically,

$$(\mathcal{A} = \mathcal{B}) \simeq (\mathcal{A} \cong \mathcal{B}) \simeq (\mathcal{A} \simeq_{\mathrm{cat}} \mathcal{B}),$$

with the second comparison restricted to univalent categories. Since the type of functors has the expected 1-categorical truncation, the type of ordinary categories is a 2-type.

> **Formal status — mathematical development.** Evidence `UCAT-CATEGORY-IDENTITY`. This is the ordinary HoTT result, not a checked identity-to-equivalence theorem for the native universe `Cat_cat`. Chapter 15 places it in the broader structure-identity and saturation programme.

12.10 What The Active Equivalence Layer Proves

At the object level, the active code packages carrier equivalence and native omega-equivalence separately. It also lifts ordinary isomorphism evidence to native omega-equivalence evidence.

> **Formal status — checked.** Evidence `EQUIV-ORDINARY-ISO-LIFT`. The lift is one-way and retains the ordinary forward arrow. It does not prove the full-faithful/essentially-surjective characterization of functors.

At the functor level, an adjunction has checked triangle cuts and a checked binary representable comparison. These facts are sufficient for the later weighted-limit preservation theorem. They are not yet a complete native theory of categorical equivalence.

12.11 The Native Higher Boundary

A higher fully-faithful functor should compare whole hom-categories, then the homs between their arrows, and so on. A higher essential-surjectivity condition should specify which equivalence witnesses inhabit its fibres and whether their evidence is propositional. Turning those conditions into a native adjoint equivalence would require:

- iterable hom-equivalence packages;
- a coherent object-level essential-surjectivity interface;
- natural unit and counit transfors with full off-diagonal action;
- triangle coherence at every retained level;
- a chosen relationship with object identity or saturation.

> **Formal status — research boundary.** The missing result is a native fully-faithful-plus-essentially-surjective characterization compatible with `OmegaEquiv`, `Functor_cat`, and higher transfors. Ordinary HoTT equivalence theorems are used only in their stated 1-categorical specialization.

The next chapter begins where the ordinary and native stories already meet: representable homs. Yoneda explains why their maps are controlled by elements; the checked co-Yoneda cut then gives that explanation a directed profunctorial computation.

13. Yoneda, Representability, And Profunctors

Representability turns the arrows out of, or into, one object into a universal coordinate system. Yoneda says that a natural map out of such a coordinate system is determined by its value at an identity. Adjunction says that one hom coordinate system is represented by another object. Weighted limits will repeat the same argument for a more elaborate functor.

The ordinary Yoneda lemma is therefore the conceptual center of this part of the book. We first develop its univalent 1-categorical form, then identify the native representable families already used by arrow induction, and finally pass to Cat-valued profunctors. The chapter's central *checked* computation is a shaped co-Yoneda cut: tensor an element with the appropriate identity-shaped hom, apply the co-Yoneda map, and recover the original element.

The full ordinary Yoneda lemma below is mathematical development adapted from the HoTT Book. The active artifact checks native representables and the shaped Cat-valued co-Yoneda beta/fusion equations. It does not yet package a general Cat-valued Yoneda equivalence or a semantic coend. Kelly supplies enriched background for these patterns, while Bénabou is a classical reference for their bicategorical organization.

13.1 Representables And Variance

Let $\mathcal{A}$ be an ordinary precategory. A presheaf is a functor

$$P : \mathcal{A}^{\mathrm{op}} \longrightarrow \mathsf{Set}.$$

For $a : \mathcal{A}$, the contravariant representable presheaf is

$$y(a) := \mathrm{Hom}_{\mathcal{A}}(-, a).$$

An arrow $u : a \to b$ induces a natural transformation $y(u) : y(a) \Rightarrow y(b)$ by postcomposition:

$$y(u)_x(f) = u \circ f = u_*(f).$$

This defines the Yoneda embedding

$$y : \mathcal{A} \longrightarrow [\mathcal{A}^{\mathrm{op}}, \mathsf{Set}].$$

The active formal development also lifts this picture from sets to Cat-valued presheaves. Its Yoneda presheaf evaluates to the represented hom, and restriction reuses the existing pullback of directed families. Gathering all arrows into a produces the restriction-oriented total

$$\mathrm{Into}^{-}_{\mathcal{A}}(a) := \sum_{x : \mathcal{A}^{\mathrm{op}}} \mathrm{Hom}_{\mathcal{A}}(x, a)$$

whose opposite is the conventional slice $\mathcal{A}/a$. A Cat-valued family on this total is a higher sieve; requiring its values to be subterminal gives an ordinary sieve. Chapter 18 begins the local-to-global spiral by developing these constructions as mathematics, separating witness-bearing higher sieves from proposition-valued membership, and explaining why sieve pullback is the natural language of a changing probe.

This bridge does not strengthen the Yoneda theorem claimed in the present chapter. A general Cat-valued Yoneda equivalence and full-faithfulness theorem remain separate from the checked representable action and shaped co-Yoneda calculation below.

There is a covariant mirror

$$y^a := \mathrm{Hom}_{\mathcal{A}}(a, -) : \mathcal{A} \longrightarrow \mathsf{Set}.$$

It is contravariant in the represented object a: an arrow $u : a \to b$ induces precomposition

$$u^* : \mathrm{Hom}_{\mathcal{A}}(b, -) \Longrightarrow \mathrm{Hom}_{\mathcal{A}}(a, -).$$

The lower-star and upper-star actions from Chapter 9 are therefore the two variance legs of representability.

13.2 The Ordinary Yoneda Equivalence

For a presheaf $P : \mathcal{A}^{\mathrm{op}} \to \mathsf{Set}$, the Yoneda equivalence is

$$\mathrm{Nat}(y(a), P) \simeq P(a).$$

Its forward map evaluates a natural transformation at the identity:

$$\mathrm{encode}(\alpha) := \alpha_a(\mathrm{id}_a).$$

For $p : P(a)$, the reverse map defines a natural transformation by

$$\mathrm{decode}(p)_x(f) := P[f](p), \qquad f : x \to a.$$

The first composite is immediate from preservation of identity:

$$\mathrm{encode}(\mathrm{decode}(p)) = P[\mathrm{id}_a](p) = p.$$

For the other composite, naturality of α at $f : x \to a$ gives

$$P[f]\big(\alpha_a(\mathrm{id}_a)\big) = \alpha_x(f).$$

Thus $\mathrm{decode}(\mathrm{encode}(\alpha)) = \alpha$ by extensionality. The proof has the same shape as Chapter 8: choose a code by evaluation, decode it by functorial transport, and calculate both composites. Here the distinguished constructor is the identity arrow of a representable.

The covariant form is equally important. For $Q : \mathcal{A} \to \mathsf{Set}$,

$$\mathrm{Nat}\big(\mathrm{Hom}_{\mathcal{A}}(a, -), Q\big) \simeq Q(a),$$

and the inverse sends $q : Q(a)$ to the family $f : a \to x \mapsto Q[f](q)$.

> **Formal status — mathematical development.** Evidence `UCAT-YONEDA`. These are the ordinary set-valued Yoneda equivalences. They are stated under the HoTT precategory assumptions and are not labeled as the active full Cat-valued theorem.

13.3 Full Faithfulness And Uniqueness Of Representation

Apply Yoneda with $P = y(b)$. One obtains

$$\mathrm{Nat}(y(a), y(b)) \simeq \mathrm{Hom}_{\mathcal{A}}(a, b).$$

Under this equivalence, an arrow $u : a \to b$ corresponds to postcomposition by u. Hence the Yoneda embedding is fully faithful: it recovers every hom from the natural maps between representables.

More generally, a representation of P consists of an object a and a natural isomorphism $y(a) \cong P$. If a and b both represent P, full faithfulness produces a canonical isomorphism $a \cong b$. When $\mathcal{A}$ is a univalent category, this isomorphism corresponds to object identity, and representability becomes a property rather than an uncontrolled choice of presentation.

> **Formal status — mathematical development.** This full-faithfulness and uniqueness result is part of evidence `UCAT-YONEDA`. The conclusion uses ordinary category univalence exactly where an isomorphism of representing objects is turned into identity.

13.4 Native Representable Families

For a native category Z and an object x, emdash has the Cat-valued fixed-source representable

$$\mathrm{Rep}_Z(x)[y] := \mathrm{Hom}_Z(x, y).$$

Its target action is postcomposition. Its dependence on the represented source is contravariant: for $p : x \to y$, the transport

$$\mathrm{Rep}_Z(y) \longrightarrow \mathrm{Rep}_Z(x)$$

acts by

$$q \longmapsto q \circ p = p^*(q).$$

The full operation is a functor between directed families, so it retains action on cells between possible q's.

> **Formal status — checked.** Evidence `CAT-REPRESENTABLE`. `Rep_catd` owns the fixed-source family and `Rep_transport_func` owns its upper-star source action.

The outgoing-arrow category from Chapter 5 is the total category

$$\mathrm{PathOut}_Z(x) = \sum_{y:Z} \mathrm{Hom}_Z(x, y).$$

Its distinguished object (x, id_x) and its canonical arrow to every (y, p) are built from the representable family. Arrow induction extends data from that reflexive outgoing arrow.

This is Yoneda-shaped, but it is not literally the ordinary Yoneda lemma. Yoneda classifies natural transformations from a representable; the selected arrow-induction theorem eliminates a dependent motive over the total category of outgoing arrows. Both begin at an identity arrow because identities are the universal points of representables.

13.5 From One Endpoint To Two

Opposites and products make the two variances explicit. In the ordinary 1-categorical setting, currying and uncurrying compare functors

$$\mathcal{A} \times \mathcal{B} \longrightarrow \mathcal{C}$$

with functors

$$\mathcal{A} \longrightarrow [\mathcal{B}, \mathcal{C}].$$

Applying this organization to composition shows that hom is functorial in both endpoints, contravariantly in its source and covariantly in its target.

The two representable variances combine in the hom bifunctor

$$\mathrm{Hom}_X(-,-) : X^{\mathrm{op}} \times X \longrightarrow \mathsf{Cat}.$$

Reindexing its two endpoints along functors $F : A \to X$ and $G : B \to X$ gives

$$\mathrm{Hom}_X(F-, G-) : A^{\mathrm{op}} \times B \longrightarrow \mathsf{Cat}.$$

This two-variable view is the bridge from Yoneda to profunctors. It makes contravariance and covariance simultaneous, lets endpoint functors vary, and turns hom transposition for an adjunction into a comparison of profunctors.

The price of the richer codomain is that ordinary elementwise proofs no longer automatically supply all higher naturality. The active development therefore selects shaped elements and explicit comparison maps before claiming a general coend-based composition.

13.6 Cat-Valued Profunctors

For categories `A` and `B`, a Cat-valued profunctor from `A` to `B` is a functorial family

$$P : A^{\mathrm{op}} \times B \longrightarrow \mathsf{Cat}.$$

We write `P:A prof B` informally and write its fibre at `(a,b)` as `P(a,b)`. The opposite on `A` records the variance:

- an arrow $p : a' \to a$ acts contravariantly in the first endpoint;
- an arrow $q : b \to b'$ acts covariantly in the second endpoint.

Profunctors with fixed endpoints form a category. A vertical map

$$r : P \Longrightarrow Q$$

is a natural family morphism over `A^op times B`. Its identity, vertical composition, components, and higher action are inherited from the directed-family and transfor calculus of Chapters 2 and 9.

> **Formal status — checked.** Evidence `PROF-CATEGORY`. `Prof_base(A,B)=A^op times B`; `Prof_cat` is the fixed-endpoint category and `ProfMap` its vertical-map classifier.

This definition introduces no new primitive notion of naturality. A profunctor is a familiar Cat-valued functor on a product base, viewed through notation that makes its two variances visible.

13.7 The Unit Hom Profunctor

Every category `X` has a canonical profunctor

$$U_X : X^{\mathrm{op}} \times X \longrightarrow \mathsf{Cat}, \qquad U_X(x, y) := \mathrm{Hom}_X(x, y).$$

On an arrow pair

$$p : x' \to x, \qquad q : y \to y',$$

it acts by cutting on both sides:

$$h \longmapsto q \circ h \circ p.$$

The opposite variance is not decorative: it is exactly what makes the first composition type-correct. Identity and composite action follow from the generic hom bifunctor.

More generally, given functors

$$F : A \longrightarrow X, \qquad G : B \longrightarrow X,$$

reindex the unit to obtain the binary representable

$$\mathrm{Hom}_X(F-, G-) : A \rightsquigarrow B$$

with fibres

$$\mathrm{Hom}_X(Fa, Gb)$$

and action

$$h \longmapsto G(q) \circ h \circ F(p).$$

> **Formal status — checked.** Evidence `PROF-REPRESENTABLE`. `Unit_prof` owns the hom profunctor, and `Hom_prof_along(F,G)` is its endpoint-reindexed presentation with the two-sided `Hom_fapp0` action.

Two specializations are useful. The **companion** of $F : A \to B$ is represented covariantly by `F`; the **conjoint** is represented contravariantly by `F`. These are not assumed to be inverse profunctors. They are the two variance choices obtained from the same ambient hom.

13.8 Reindexing Endpoints

Let `P:A prof B`, $F : A' \to A$, and $G : B' \to B$. Endpoint reindexing is

$$(F, G)^* P : A' \rightsquigarrow B', \qquad ((F, G)^* P)(a', b') := P(Fa', Gb').$$

Semantically this is pullback along

$$F^{\mathrm{op}} \times G : (A')^{\mathrm{op}} \times B' \longrightarrow A^{\mathrm{op}} \times B.$$

The operation acts on vertical maps, is neutral at identity endpoint functors, and accumulates nested reindexings by composing both endpoints. For representables, it gives the expected equation

$$(F', G')^* \,\mathrm{Hom}_X(F-, G-) = \mathrm{Hom}_X((F \circ F')-, (G \circ G')-).$$

> **Formal status — checked.** Evidence `PROF-REINDEXING`. `Prof_reindex` is the stable object owner and `Prof_reindex_func` provides its vertical functorial action.

The separation between a profunctor and its reindexed view is analogous to the separation between WalkingEnd and `BNat`. A readable model or pullback presentation does not become a definitional replacement for the object it explains.

13.9 Representability As A Chosen Comparison

A profunctor `P:B prof J` is right-represented by a functor $L : J \to B$ when it is isomorphic, in the fixed-endpoint profunctor category, to the conjoint of `L`:

$$P \cong \mathrm{Hom}_B(-, L-).$$

A representation package retains both the chosen functor and its isomorphism evidence:

$$\sum_{L:J\to B} \mathsf{IsoEvidence}\big(P, \mathrm{Hom}_B(-, L-)\big).$$

> **Formal status — checked.** Evidence `PROF-REPRESENTATION-PACKAGE`. The classifiers are `IsRepresentedBy_iso` and `Representation_iso`; the current comparison is ordinary isomorphism evidence in `Prof_cat`.

This is a useful universal-property interface even before uniqueness of the representing object is packaged. A weighted limit, for example, can be presented as a chosen representation of a cone profunctor. The computational strength of that use depends on how much of the comparison is exposed by the consumer.

13.10 Cells With Moving Endpoints

Vertical maps keep endpoints fixed. Equipment-style cells allow endpoint functors to move as well. Given

$$R' : A' \rightsquigarrow B', \quad R : A \rightsquigarrow B, \quad F : A' \to A, \quad G : B' \to B,$$

a cell over `F,G` is a natural family morphism

$$c : R' \Longrightarrow (F, G)^* R.$$

If the source profunctor is the unit `U_I`, such a cell is a shaped element of `R` with endpoint functors $I \to A$ and $I \to B$. The narrow application operation sends a shaped element of `R'` through `c` to the corresponding shaped element of `R`.

> **Formal status — checked.** Evidence `PROF-SHAPED-CELLS`. `Prof_transf_cat` is the endpoint-changing cell category, `Prof_hom` is its unit-source specialization, and `Prof_cell_apply` is the selected application operation.

The operation is intentionally narrower than arbitrary horizontal composition of equipment cells. It supplies the consumer needed by the co-Yoneda calculation without pretending that all bicategorical coherence is already present.

13.11 Tensor As A Selected Composite

For

$$P : A \rightsquigarrow B, \qquad Q : B \rightsquigarrow X,$$

write

$$P \otimes_B Q : A \rightsquigarrow X$$

for their selected profunctor tensor. In ordinary enriched category theory one expects a coend-like formula

$$(P \otimes_B Q)(a, x) \simeq \int^{b:B} P(a, b) \times Q(b, x).$$

This formula is the intended mathematical reading, not a definition unfolded by the active kernel. `Prof_tensor` is opaque. What is exposed is the behavior required by current consumers:

- reindexing in the outer endpoints distributes through the tensor;
- the middle category stays fixed;
- vertical maps in both factors induce a vertical map of tensors;
- compatible shaped elements compose to a shaped tensor element.

> **Formal status — checked.** Evidence `PROF-TENSOR`. The fixed-middle object is `Prof_tensor`; `Prof_tensor_func` owns fixed-endpoint bifunctoriality and `Prof_tensor_hom_hom` composes shaped elements.

Opacity is mathematically honest here. A genuine coend construction would need a quotient or universal colimit with its own eliminator and computation rules. The selected tensor can support tested cut-elimination interfaces without serving as evidence that this general construction has already been built.

13.12 The Shaped Co-Yoneda Cut

The unit profunctor should behave as a unit for tensor. The active interface supplies natural maps in both orientations:

$$\varepsilon^R_P : P \otimes_B U_B \Longrightarrow P,$$
$$\varepsilon^L_P : U_A \otimes_A P \Longrightarrow P.$$

They are components of transformations natural in `P`, rather than a collection of unrelated vertical maps.

Now let `p` be a shaped element of `P` with middle shape $M : I \to B$. The unit profunctor has a canonical shaped element given by the identity transformation on `M`. Tensor the two elements and apply the right co-Yoneda map. The cut eliminates:

$$\varepsilon^R_P\big(p \otimes \mathrm{id}_M\big) = p.$$

Dually, for an `A`-shaped endpoint `F`,

$$\varepsilon^{L}_{P}\big(\mathrm{id}_{F} \otimes p\big) = p.$$

> **Formal status — checked.** Evidence `PROF-COYONEDA`. `Prof_coyoneda_cov_transf` and `Prof_coyoneda_con_transf` are the two natural transformations; their component maps have the shaped-element beta equations above.

These equations are the chapter's central theorem. They express Yoneda-style evaluation as computation: insert an identity-shaped hom, form the composite, and the co-Yoneda counit removes the cut.

The transformations also fuse with an arbitrary vertical map $r : P \to P'$. Applying the naturality component of co-Yoneda to $p \otimes \mathrm{id}$ reduces to applying `r` directly to `p`:

$$\varepsilon_{P'}\big((r \otimes \mathrm{id})(p \otimes \mathrm{id})\big) = r(p),$$

with the analogous left-unit equation. This is not a separate law attached to every `r`; it is the off-diagonal action of the co-Yoneda transfor from Chapter 9.

13.13 Adjunctions As Representable Hom Comparisons

Chapter 12 characterized a right adjoint $G : B \to A$ to $F : A \to B$ by representations

$$\mathrm{Hom}_{B}(F-, b) \simeq \mathrm{Hom}_{A}(-, Gb).$$

Yoneda explains both the representing object and its uniqueness. The profunctor formulation adds simultaneous naturality in the source probe and in b. For arbitrary $M : I \to A$ and $H : K \to B$, the active comparison is

$$\mathrm{Hom}_{B}(FM, H) \simeq \mathrm{Hom}_{A}(M, GH).$$

> **Formal status — checked.** Evidence `ADJ-HOM-PROF-COMPARISON`. The adjunction supplies this reindexable `ProfComparison`; its push and pull operations have the generic comparison beta/eta laws.

Conversely, an ordinary pointwise family of such representations reconstructs a right adjoint only after the representing objects and comparisons have been made coherent in b. The active representation package records a chosen representing functor and ordinary isomorphism evidence in the profunctor category, but no general reverse constructor from that package to `Adjunction` is asserted.

> **Formal status — mathematical development.** The ordinary reverse representability theorem is part of `UCAT-ADJOINT-REPRESENTABILITY`. Evidence `PROF-REPRESENTATION-PACKAGE`, checked in Section 13.9, supplies the native chosen representation interface on which a future reverse construction could be based.

This is the bridge to weighted universals. A weighted cone construction will produce another profunctor; a weighted limit is the functor that represents it. The same comparison beta/eta laws will eliminate the corresponding universal cuts.

13.14 The Full Cat-Valued Yoneda Boundary

The computations capture an essential Yoneda idea: represented hom data can be inserted as an identity-shaped cut and then eliminated. The existing right-representable embedding, endpoint reindexing, shaped cells, and co-Yoneda maps give a credible computational core for a broader theorem.

A full Cat-valued Yoneda theorem would package more. One expects a functor from `A` into an appropriate presheaf or profunctor category, a precise mapping-category equivalence, naturality in the represented object and the presheaf, and a proof that the embedding is fully faithful at every retained cell level. Those data have not been gathered into one active theorem.

> **Formal status — research boundary.** Evidence `YONEDA-FULLY-FAITHFUL`. The intended future owner should be built from the existing representable functor, shaped-cell, and co-Yoneda interfaces and should expose a reusable equivalence rather than add isolated projection rules.

The checked beta equations remain valuable without that package. They are local computational theorems, not merely motivational analogies.

13.15 The Coend And Bicategorical Boundary

To promote the tensor to a general profunctor composition, one would need:

- a coend or coinserter construction for Cat-valued data;
- its dinatural elimination and computation principles;
- associativity and left/right unit comparisons;
- coherent higher cells for those comparisons;
- horizontal composition of arbitrary endpoint-changing cells;
- compatibility with opposites, reindexing, and internal homs.

Only selected parts of this list are active. Fixed-endpoint tensor action, outer reindexing, shaped composition, and both co-Yoneda counits have checked owners. They do not automatically assemble into a bicategory.

> **Formal status — research boundary.** Evidence `PROF-GENERAL-COEND`. A future implementation should give the opaque tensor a universal semantics or relate it to a separately constructed coend; it should not infer general coherence from the current endpoint beta rules alone.

This staged boundary mirrors the rest of the book. We expose enough functorial structure to compute a central example, retain the higher action that example consumes, and state the missing universal package explicitly.

14. Strictness, Dagger Structure, And Duality

The word *strict* enters category theory by several doors. A HoTT strict category has a set of objects. An emdash naturality cut is strict when it selects a runtime normal form. A rewrite is strict in yet another sense when it is judgmental computation rather than internal equality. None of these conditions implies either of the others.

The neighboring notion of a dagger illustrates why this vocabulary matters. A dagger is a *chosen* reversal internal to one category, whereas the opposite construction reverses any category from the outside. The distinction is the same one that has guided the book throughout: a generic operation, a selected structure, and a computational presentation have different owners.

This chapter adapts the strict- and dagger-category discussions of the HoTT Book to the ordinary 1-categorical specialization, then states the additional coherence a native directed version would need. Its central checked theorem is opposite duality. The dagger theory is mathematical development with an explicit implementation boundary.

14.1 A Qualified Strictness Vocabulary

The following terms remain separate throughout the book.

Qualified notion	Meaning	Does not imply
HoTT strict category	an ordinary precategory whose object type is a set	category univalence
HoTT category	an ordinary precategory for which `idtoiso` is an equivalence	that the object type is a set
gaunt category	a HoTT category that is also strict	runtime strictness
native `IsNCat(n,C)`	recursive finite height of the hom-categories	object identity agrees with isomorphism
strict naturality cut	a selected `tapp1` composite reduces to one off-diagonal action	all coherence is judgmental
runtime strictness	an oriented kernel reduction chooses a normal form	object truncation or invertibility
dagger category	identity agrees with *unitary* isomorphism	identity agrees with every isomorphism

In particular, the HoTT phrase *strict category* begins with a **precategory**, not with a univalent category. Chapter 10's translation table uses this definition. A strict precategory may still have nontrivial automorphisms that cannot come from its proposition-valued object identity.

14.2 Strict Categories In Ordinary Univalent Foundations

Let $\mathcal{A}$ be an ordinary precategory. It is *strict* when $\mathrm{Obj}(\mathcal{A})$ is a set:

$$\prod_{x,y:\mathrm{Obj}(\mathcal{A})} \mathrm{isProp}(x = y).$$

This condition says that two proofs of equality between objects agree. It does not say that every isomorphism comes from equality. For example, the one-object category associated to a nontrivial group is strict: its object type is the unit type. It is not a HoTT category, since its nonidentity group elements are automorphisms while the unique object has only the reflexive identity path.

A poset on a set-valued carrier gives the contrasting example. Its homs are propositions and antisymmetry identifies mutual reachability with object identity, so it is both strict and univalent. More generally, a HoTT category is strict exactly when it is *gaunt*: its isomorphisms contain no additional object sameness beyond proposition-valued identity.

Strict categories therefore support a stricter package-level notion of sameness than categorical equivalence. Equality of their presentations agrees with isomorphism of the corresponding precategory structures, whereas an equivalence can still change a presentation by choosing merely equivalent objects. This can be useful, but it is not the default notion of sameness in univalent category theory.

> **Formal status — mathematical development.** Evidence `UCAT-STRICT-CATEGORY`. The definition and the gauntness comparison belong to ordinary univalent 1-category theory. They are not a definition of native `Cat` or a theorem about arbitrary finite-dimensional native categories.

14.3 Three Native Notions That Strict Categories Do Not Control

Finite directed height is the first separate notion. A witness

$$\mathrm{IsNCat}(n, C)$$

recurses through native hom-categories. At dimension one it says that every hom-category is discrete and entails that the object classifier is a 1-type. HoTT strictness instead asks directly that the object classifier be a set. Neither condition supplies a native identity-to-isomorphism theorem.

The second notion is strict naturality. For an ordinary transfor $\eta : F \Rightarrow G$, the generic off-diagonal action has the two reductions

$$G[g] \circ \eta[f] \rightsquigarrow \eta[g \circ f],$$
$$\eta[f] \circ F[h] \rightsquigarrow \eta[f \circ h].$$

The phrase *strict transfor* in this book describes this selected two-sided cut behavior; it is not a new classifier of categories. A displayed lax comparison can instead retain a directed naturality cell without forcing it to equality.

The third notion is runtime strictness itself. The arrow $t \rightsquigarrow u$ records a chosen normal form in the Lambdapi theory. An internal equality $t = u$, a proof-time comparison, and a free-form mathematical equation have different force. A category can have proposition-valued object identity while none of its interesting operations compute, or support rich directed higher cells while selected naturality cuts do compute.

> **Formal status — checked.** Evidence `CAT-DIMENSION` and `TRANSF-STRICT-NATURALITY`. The active theory separately checks recursive dimension/object truncation and the two ordinary `tapp1` naturality reductions. No checked theorem identifies these interfaces.

14.4 Opposite Duality Computes

For every native category C, the opposite category has the same objects and reversed homs:

$$\mathrm{Hom}_{C^{\mathrm{op}}}(x, y) = \mathrm{Hom}_C(y, x).$$

Identity arrows are unchanged, while the factors of composition reverse. The active operation is involutive not only on categories but through the iterable functor and transfor layers:

$$(C^{\mathrm{op}})^{\mathrm{op}} \rightsquigarrow C,$$
$$(F^{\mathrm{op}})^{\mathrm{op}} \rightsquigarrow F,$$
$$(\alpha^{\mathrm{op}})^{\mathrm{op}} \rightsquigarrow \alpha.$$

Opposite reverses vertical composition of transfors. It also turns an adjunction

$$F \dashv G$$

into

$$G^{\mathrm{op}} \dashv F^{\mathrm{op}}.$$

The new unit is the opposite of the old counit, and the new counit is the opposite of the old unit. Applying opposite twice reduces to the original adjunction package. This is a computational duality, not a silent convention that suppresses variance.

> **Formal status — checked.** Evidence `OP-DUALITY`. The principal owners are `Op_cat`, `Op_func`, `Op_transf`, and `Op_adjunction`. Their involution and variance-reversal rules are the checked basis for the weighted-colimit duality in Chapter 17.

14.5 Dagger Structure Is Chosen Self-Duality

An ordinary †-precategory is a precategory $\mathcal{A}$ equipped with an operation

$$(-)^{\dagger} : \mathrm{Hom}_{\mathcal{A}}(x, y) \longrightarrow \mathrm{Hom}_{\mathcal{A}}(y, x)$$

satisfying

$$(\mathrm{id}_x)^\dagger = \mathrm{id}_x,$$
$$(g \circ f)^\dagger = f^\dagger \circ g^\dagger,$$
$$(f^\dagger)^\dagger = f.$$

Equivalently, it is an identity-on-objects functor-like map

$$D : \mathcal{A}^{\mathrm{op}} \longrightarrow \mathcal{A}$$

with a chosen involution law. The word *chosen* is essential. The opposite construction gives $\mathcal{A}^{\mathrm{op}}$ for every $\mathcal{A}$; it does not give a functor from that opposite back to $\mathcal{A}$, much less one that fixes objects and squares to the identity.

14.6 Unitary Arrows And Dagger Univalence

An arrow $f : x \to y$ is *unitary* when its dagger is its two-sided inverse:

$$f^\dagger \circ f = \mathrm{id}_x, \qquad f \circ f^\dagger = \mathrm{id}_y.$$

Every unitary arrow is an isomorphism, but an isomorphism need not be unitary. Object identity always produces a unitary isomorphism by identity induction, so there is a canonical map

$$\mathrm{idtoUnitary}_{\mathcal{A}} : (x = y) \longrightarrow (x \cong_\dagger y).$$

A $\dagger$-category is a $\dagger$-precategory for which this map is an equivalence. Its selected notion of object sameness is unitary isomorphism, not arbitrary isomorphism.

Two examples separate the notions. In a groupoid, define $f^\dagger = f^{-1}$; then every arrow is unitary. For finite-dimensional inner product spaces with arbitrary linear maps, the dagger is the adjoint linear map. The unitary isomorphisms are the isometries, while many invertible linear maps are not unitary. Thus this $\dagger$-category need not be a HoTT category under ordinary isomorphism.

> **Formal status — mathematical development.** Evidence `UCAT-DAGGER-CATEGORY`. This is the ordinary set-valued-hom theory: dagger laws are equalities, unitarity is a property, and dagger univalence compares object identity with unitary isomorphism.

14.7 Opposite, Duality, And Dagger Compared

The three reversal notions have different input data.

Construction	Data supplied	Result
opposite	any category C	a category C^{op} with arrows reversed
categorical duality	a chosen equivalence $C^{\mathrm{op}} \simeq D$	a comparison between two categories
dagger	a chosen identity-on-objects involutive map $C^{\mathrm{op}} \to C$	a unitary notion internal to C

A native dagger would have to act at every retained hom dimension. At minimum it would require:

1. a functor $D : C^{\mathrm{op}} \to C$;
2. identity-on-objects data, with an explicit decision about whether it computes or is witnessed coherently;
3. an involution comparison between $D \circ D^{\mathrm{op}}$ and id_C;
4. compatibility of that comparison with off-diagonal and higher action;
5. a unitary-arrow classifier whose evidence is stable under identity, composition, and the next hom action;
6. a qualified identity-to-unitary-equivalence interface.

The ordinary equations above are a plausible strict specialization of this design, not a license to erase the higher coherence. In particular, `Op_cat` supplies only the first half of the ambient reversal and cannot serve as a native dagger by itself.

> **Formal status — research boundary.** Evidence `NATIVE-DAGGER-INTERFACE`. No dagger/unitary owner is active. Side task `FTTX-S12` remains a specification target, and this chapter does not add a prose-only kernel name or infer dagger structure from opposite duality.

14.8 Duality As A Proof Method

Opposite duality becomes useful when a theorem has already exposed its variance. A right adjoint preserves weighted limits because a representable comparison can be transported through its hom adjunction. Passing to opposites exchanges:

$$\begin{aligned} \text{right adjoint} &\longleftrightarrow \text{left adjoint}, \\ \text{weighted limit} &\longleftrightarrow \text{weighted colimit}, \\ \text{unit} &\longleftrightarrow \text{counit}, \\ \text{upper-star source action} &\longleftrightarrow \text{lower-star target action}. \end{aligned}$$

Chapter 17 will use these checked opposite owners to derive the colimit preservation theorem rather than repeat the limit proof with reversed arrows. A dagger could internalize a particular self-dual instance of such an argument, but the general theorem needs only opposite duality. This is why the book includes dagger structure for conceptual completeness without making it the foundation of categorical duality.

15. Structure Identity And Saturation

Univalence is most useful when it propagates from bare carriers to the structures mathematicians put on them. The *structure identity principle* says that structure-preserving equivalence is the identity of structured objects. *Saturation* asks the complementary question: if a categorical presentation does not yet have the desired identity-to-equivalence property, can it be completed universally into one that does?

The ordinary 1-categorical answers are the structure identity theorem and Rezk completion. They also reveal the architecture a directed version would need. Evidence must be a property when it is intended to be inessential; transport of structure must act coherently; the selected equivalence notion must be stated; and a completion is characterized by its mapping property, not merely by a newly constructed carrier.

This chapter adapts Sections 9.8 and 9.9 of the HoTT Book. The ordinary theorems are mathematical development. Emdash already checks several local ingredients, but a general native structure identity principle and Rezk completion remain research boundaries.

15.1 Truncation And Saturation Answer Different Questions

The following distinctions are essential.

Condition	What it controls	What it does not supply
object truncation	the height of $x = y$	a comparison with categorical equivalence
finite `IsNCat` evidence	recursive height of hom-categories	category univalence
ordinary category saturation	$(x = y) \simeq (x \cong y)$	proposition-valued object identity
dagger saturation	$(x = y) \simeq (x \cong_{\dagger} y)$	identity with every ordinary isomorphism
prospective native saturation	identity agrees with one specified higher equivalence classifier	a canonical choice of that classifier

A strict category can therefore be unsaturated, while a saturated category can have a non-set-valued object type. Likewise, the finite-height theorem used for WalkingEnd bounds identity types but does not turn an arbitrary ordinary isomorphism into object identity.

15.2 A Notion Of Structure Over A Carrier Category

Let $\mathcal{X}$ be an ordinary precategory. A notion of structure (P, H) over $\mathcal{X}$ consists of:

1. a type $P(x)$ of structures on each object x;
2. for $f : x \to y$, $\alpha : P(x)$, and $\beta : P(y)$, a proposition $H_{\alpha,\beta}(f)$ saying that f preserves the structures;

3. evidence that identity arrows preserve structure;
4. evidence that composites of structure-preserving arrows preserve structure.

For structures $\alpha, \beta : P(x)$ on the same carrier, define

$$\alpha \leq_x \beta \equiv H_{\alpha,\beta}(\mathrm{id}_x).$$

Identity and composition make this a preorder. The notion of structure is called *standard* when this preorder is antisymmetric in every fibre. In particular, each $P(x)$ is then a set.

The associated precategory $\mathsf{Str}_{P,H}(\mathcal{X})$ has objects

$$(x, \alpha) : \sum_{x:\mathrm{Obj}(\mathcal{X})} P(x)$$

and arrows

$$(x, \alpha) \longrightarrow (y, \beta) \quad :\equiv \quad \sum_{f:x\to y} H_{\alpha,\beta}(f).$$

Because H is proposition-valued, it adds a preservation condition without adding competing arrow data. Identities and composition come from $\mathcal{X}$ and the two closure laws.

15.3 The Ordinary Structure Identity Theorem

> **Structure identity principle.** If $\mathcal{X}$ is an ordinary univalent category and (P, H) is a standard notion of structure over it, then $\mathsf{Str}_{P,H}(\mathcal{X})$ is an ordinary univalent category.

The proof isolates the two uses of uniqueness. An identity

$$(x, \alpha) = (y, \beta)$$

is a carrier identity $p : x = y$ together with

$$\mathrm{transport}_P(p, \alpha) = \beta.$$

Since the structure fibres are sets, the second clause is a proposition. An isomorphism of structured objects is a carrier isomorphism $f : x \cong y$ together with preservation by f and by f^{-1}; those clauses are also propositions.

Univalence of $\mathcal{X}$ converts f into a carrier identity p. By identity induction, it remains to consider $p \equiv \mathrm{refl}_x$. The two preservation clauses then say

$$\alpha \leq_x \beta \qquad \text{and} \qquad \beta \leq_x \alpha.$$

Antisymmetry gives $\alpha = \beta$. This constructs the inverse to `idtoiso` for structured objects; proposition-valuedness supplies the remaining coherence.

> **Formal status — mathematical development.** Evidence `UCAT-STRUCTURE-IDENTITY`. The theorem is the ordinary HoTT structure identity principle. It depends on set-valued homs, proposition-valued preservation, base-category univalence, and fibrewise antisymmetry.

15.4 Examples And The Role Of Standardness

Functor structure is a representative example. Begin with object functions $F_0 : \mathrm{Obj}(\mathcal{A}) \to \mathrm{Obj}(\mathcal{B})$. The additional structure assigns arrow actions preserving identities and composition. A pointwise family of arrows is a homomorphism exactly when it is natural. If $\mathcal{B}$ is univalent, the structure identity theorem recovers the functor-category result from Chapter 11: natural isomorphism agrees with identity of functors.

Ordinary algebraic and relational structures fit the same scheme. A signature specifies operations and relations on a carrier set, while H says that a function preserves them. Function extensionality and proposition extensionality make the structure fibres sets; mutual preservation by the identity function forces equality of structures. Thus isomorphic groups, rings, ordered sets, and similar standard structures become identical in their univalent categories.

Standardness is not cosmetic. If two distinct structures on the same carrier admit identity-carrier homomorphisms in both directions, structured isomorphism contains less discriminating information than structure identity. The theorem correctly refuses to identify them until antisymmetry or an appropriate higher replacement has been supplied.

15.5 Checked Native Footholds

The active theory contains four ingredients that a native structure identity theorem should reuse.

First, truncation evidence is proposition-valued at every implemented level. Adding an `IsTruncGrpd` field to a carrier package therefore does not add a second independent notion of identity between witnesses.

Second, for a fixed arrow, native equality-valued omega-equivalence evidence is proposition-valued. The chosen arrow remains data, but two proofs that the same arrow is an omega-equivalence agree.

Third, the packaged universes of truncated classifiers have a local structure-identity theorem: package identity is equivalent to `TypeEquiv` of the retained carriers. This is the closest checked example of univalence propagating through an evidence field.

Fourth, ordinary isomorphism evidence maps one way into native omega-equivalence evidence, and finite `IsNCat` evidence yields the expected object-truncation bound. These maps organize nearby notions without asserting the missing reverse object-identity comparison.

> **Formal status — checked.** Evidence `LOGIC-TRUNCATION-EVIDENCE-PROP`, `EQUIV-EVIDENCE-PROP`, `UNIV-TRUNCATED`, `EQUIV-ORDINARY-ISO-LIFT`, and `CAT-DIMENSION`. These are local evidence-property, package-univalence, comparison, and truncation theorems. Their conjunction is not a generic structure identity principle.

15.6 A Plausible Native Structure-Identity Interface

The ordinary (P, H) schema suppresses the higher cells of preservation evidence. A directed native version should expose them. One plausible architecture begins with:

- a native carrier category K;
- a directed family $S : K \to \mathsf{Cat}$ of structures;
- for each base arrow $f : x \to y$ and structures $\alpha : S(x)$, $\beta : S(y)$, a category $H_f(\alpha, \beta)$ of structure-preserving lifts;
- identity, composition, and higher action for those lifts;
- a selected classifier $\mathrm{StructuredEquiv}((x, \alpha), (y, \beta))$.

The total category of structures is Sigma-shaped, but a general $H_f(\alpha, \beta)$ may carry more information than the canonical transport arrow of a bare directed family. A *standardness* condition must say exactly when that extra information is property-like and when it is genuinely higher structure.

The prospective comparison is

$$\mathrm{idtoStructuredEquiv} : \big((x, \alpha) = (y, \beta)\big) \longrightarrow \mathrm{StructuredEquiv}\big((x, \alpha), (y, \beta)\big).$$

A native SIP would prove this to be an equivalence under qualified base univalence and standardness hypotheses, while retaining its off-diagonal and next-hom action. The target might use ordinary isomorphism, adjoint equivalence, or native omega-equivalence; the theorem cannot be stated honestly until that choice is part of the signature.

> **Formal status — research boundary.** Evidence `NATIVE-STRUCTURE-IDENTITY`. The active `Catd`, `Sigma_cat`, `Hom_catd`, evidence-property, and equivalence interfaces are plausible ingredients, but there is no generic structure signature, structured-equivalence classifier, or identity theorem. Side task `FTTX-S9` records this prospective owner.

15.7 Weak Equivalences And The Universal Property Of Completion

Return to ordinary precategories. A functor

$$I : \mathcal{A} \longrightarrow \widehat{\mathcal{A}}$$

is a *weak equivalence* when it is fully faithful and essentially surjective, where essential surjectivity is merely inhabited rather than split by a chosen inverse on objects. A Rezk completion of $\mathcal{A}$ consists of such an I with $\widehat{\mathcal{A}}$ an ordinary univalent category.

The construction is characterized by what saturated targets see. For every ordinary univalent category $\mathcal{C}$, precomposition induces

$$I^* : \mathcal{C}^{\widehat{\mathcal{A}}} \longrightarrow \mathcal{C}^{\mathcal{A}}, \qquad G \longmapsto G \circ I,$$

and I^* is an isomorphism of ordinary precategories. Equivalently, every functor $\mathcal{A} \to \mathcal{C}$ extends essentially uniquely across I, and every natural transformation between extensions is determined by its restriction.

The word *universal* belongs here, not merely in the statement that $\widehat{\mathcal{A}}$ is saturated. Any two completions with this mapping property are uniquely equivalent in the appropriate functor category.

15.8 Why Saturated Targets See Weak Equivalences As Equivalences

The proof of the mapping property is a lesson in constructive uniqueness. If I is essentially surjective, a natural transformation out of $\widehat{\mathcal{A}}$ is determined by its components on the image of I; this makes precomposition faithful. Fullness of I, together with essential surjectivity, reconstructs the missing components and proves naturality, so precomposition is fully faithful.

To extend a functor on objects, one knows only that each object of $\widehat{\mathcal{A}}$ is isomorphic to something in the image. Choosing an arbitrary representative would require choice. Instead one describes the candidate image and its comparison data as a contractible type. The crucial step uses univalence of $\mathcal{C}$: uniqueness up to unique isomorphism becomes uniqueness by identity, which is enough to define a function. This proves essential surjectivity of I^* and hence the mapping property.

In short, saturated targets turn weak equivalences into equivalences of functor categories.

The converse detects saturation. An ordinary precategory $\mathcal{C}$ is univalent exactly when every weak equivalence $H : \mathcal{A} \to \mathcal{B}$ makes

$$H^* : \mathcal{C}^{\mathcal{B}} \longrightarrow \mathcal{C}^{\mathcal{A}}$$

an isomorphism. For the reverse implication, apply the assumption to a Rezk completion $I : \mathcal{C} \to \hat{\mathcal{C}}$. Precomposition then constructs an inverse to I, making $\mathcal{C}$ isomorphic to the univalent category $\hat{\mathcal{C}}$ and hence univalent itself.

This characterization also explains why a weak equivalence need not be an isomorphism between unsaturated presentations. It becomes invertible to every target precisely when object isomorphism in that target can be absorbed as identity.

15.9 The Yoneda-Image Completion

The first construction uses Chapter 13. Let

$$\mathsf{PSh}(\mathcal{A}) := \mathsf{Set}^{\mathcal{A}^{\mathrm{op}}}.$$

Define $\widehat{\mathcal{A}}$ to be the full subcategory whose objects are presheaves P for which there merely exists an $a : \mathcal{A}$ and an isomorphism

$$y(a) \cong P.$$

The ambient presheaf category is univalent, and the condition of being merely representable is a proposition, so the full subcategory is univalent. The Yoneda embedding

$$y : \mathcal{A} \longrightarrow \widehat{\mathcal{A}}$$

is fully faithful by Yoneda and essentially surjective by the definition of the image. It is therefore a Rezk completion.

This proof is short because representability has already packaged the necessary universal coordinates. Its cost is universe size: the presheaf category may live in a larger universe than $\mathcal{A}$.

15.10 The Higher-Inductive Completion

A second construction stays closer to the original universe by freely adding the missing object identities. Its object type $\mathrm{Obj}(\widehat{\mathcal{A}})$ is generated by:

- $i(a)$ for every object $a : \mathcal{A}$;
- a path $j(e) : i(a) = i(b)$ for every isomorphism $e : a \cong b$;
- coherences $j(\mathrm{id}_a) = \mathrm{refl}_{i(a)}$ and $j(g \circ f) = j(f) \cdot j(g)$;
- 1-truncation, so parallel 2-paths agree.

The hom family is then defined by double induction on the new object type, starting with

$$\mathrm{Hom}_{\widehat{\mathcal{A}}}(i(a), i(b)) := \mathrm{Hom}_{\mathcal{A}}(a, b).$$

Transport along $j(e)$ is postcomposition or precomposition by e and its inverse. The identity and composition laws of $\mathcal{A}$ discharge the HIT coherences, after which identities and composition on the new hom family are defined by induction.

To show saturation, use encode-decode:

$$\begin{aligned} &\mathrm{encode}_{x,y} : (x = y) \longrightarrow (x \cong y), \quad \mathrm{encode} = \mathrm{idtoiso}, \\ &\mathrm{decode}_{x,y} : (x \cong y) \longrightarrow (x = y), \quad \mathrm{decode}(e) = j(e) \text{ on generators}. \end{aligned}$$

Induction over the HIT and its paths proves both composites. The identity and composition constructors are precisely the cases needed for the code family of isomorphisms to respect reflexivity and path concatenation. Finally, $I(a) := i(a)$ is fully faithful by construction and essentially surjective by HIT induction.

> **Formal status — mathematical development.** Evidence `UCAT-REZK-COMPLETION`. The ordinary theorem states that every precategory has a univalent Rezk completion, constructed either as the Yoneda image or by the 1-truncated HIT above, and that weak equivalences into univalent targets have the stated functor-category mapping property.

15.11 The Encode-Decode Analogy With WalkingEnd

The HIT proof deliberately echoes Chapter 8, but the two constructions solve different problems.

Feature	WalkingEnd	Rezk-completion HIT
generators	one object and one directed loop	old objects and paths for old isomorphisms
code	natural powers of the loop	isomorphisms between completed objects
decode	contextual directed normalization	a path constructor $j(e)$
invertibility	the loop is checked noninvertible	every added generator is a path and hence invertible
purpose	compute the free directed endomorphism monoid	saturate object identity

Both proofs define a code family, construct encode and decode, and calculate two composites by the relevant eliminator. That is an analogy of proof architecture. WalkingEnd is **not** a Rezk completion: its defining loop is not an isomorphism, its hom corresponds to natural numbers, and its universal problem is directed generation rather than saturation.

> **Formal status — checked comparison.** Evidence `WE-LOOP-NONINVERTIBLE` verifies the decisive negative fact on the WalkingEnd side. The Rezk construction in the other column remains the ordinary mathematical development above.

15.12 Specification Of A Native Rezk Completion

A native completion cannot be obtained by replacing the word “isomorphism” with “equivalence” and leaving the rest implicit. A prospective interface must specify:

1. a classifier $\mathsf{Eqv}_C(x, y)$ of the selected categorical sameness;
2. a coherent map $(x = y) \to \mathsf{Eqv}_C(x, y)$;
3. the saturation predicate asserting that this map is an equivalence;
4. a category $\mathrm{Rezk}(C)$ and unit functor $\eta_C : C \to \mathrm{Rezk}(C)$;

5. local full faithfulness at every retained hom dimension and essential surjectivity relative to Eqv;
6. for every saturated D, an equivalence between the iterable functor categories out of $\mathrm{Rezk}(C)$ and out of C;
7. naturality of that mapping equivalence in D, together with its transformation and higher-cell action.

Ordinary isomorphism, adjoint equivalence, and native omega-equivalence are different candidates for Eqv. Chapter 10 supplies maps among some of them, but no general theorem chooses one as object identity in every native category. The completion's higher universal property must therefore be designed together with its saturation predicate.

> **Formal status — research boundary.** Evidence `NATIVE-REZK-COMPLETION`. No native saturation predicate, completion object, unit weak equivalence, or iterable mapping property is active. Constructing a carrier without these laws would not discharge `FTTX-S9`.

15.13 Identity Principles And Universal Constructions

The two halves of the chapter meet in a useful division of labor. A structure identity theorem proves that a well-behaved construction *preserves* saturation: structured objects over a saturated carrier form another saturated category. Rezk completion *creates* saturation from a presentation that lacks it. Yoneda supplies one completion because representables turn objects into universal coordinates; the HIT supplies another because encode-decode computes the freely generated identity structure.

Chapters 16 and 17 now return to universal constructions internal to a fixed categorical world. Their limits, colimits, and joins do not require a native Rezk completion to be stated, but the distinction between presentation and saturated identity will remain visible whenever uniqueness is upgraded from equivalence to equality.

16. Weighted Universal Constructions

A limit is often introduced by drawing a cone and asking for a universal vertex. That picture is indispensable, but it hides the operation that makes the vertex universal: for every probe object, the category of cones must be represented by a hom. Weighted limits expose this operation directly.

The resulting account is not a catalogue of products, equalizers, ends, and Kan extensions. It is one theorem with several specializations:

$$\text{weight and diagram} \longrightarrow \text{cone classifier} \longrightarrow \text{representing comparison.}$$

The same comparison calculus that eliminated cuts in Chapter 9 then supplies the universal beta and eta laws. Adjunction mates transport the comparison, and therefore every right adjoint preserves every selected weighted limit for which the comparison has been supplied.

Our mathematical conventions follow the Cat-enriched viewpoint of Kelly. The active artifact deliberately implements a narrower computational interface: tensor and its two residuals are symbolic objects with checked vertical beta/eta operations, not constructions from general ends and coends. This distinction lets the theorem compute without claiming semantic infrastructure that has not yet been built.

16.1 Weights Are Parameterized Shapes

Let

$$F : J \longrightarrow B$$

be a diagram. A parameterized covariant weight has the form

$$W : J' \rightsquigarrow J, \qquad W : (J')^{\mathrm{op}} \times J \longrightarrow \mathsf{Cat}.$$

For each $j' : J'$, the partial functor $W(j', -)$ is a Cat-valued weight on J. The first endpoint is contravariant, so an arrow $j'_0 \to j'_1$ acts from the weight at j'_1 to the weight at j'_0. This is exactly the variance needed for the resulting representing objects to assemble into a functor

$$L : J' \longrightarrow B.$$

The familiar unparameterized definition is the special case $J' = \mathbf{1}$. Retaining J' is important: choosing one limit object for each parameter is not enough. The comparison below must also make those choices functorial in the parameter and iterable at the next hom level.

In the ordinary set-enriched case, a weight is a functor $W : J \to \mathsf{Set}$. The set $W(j)$ describes how many copies, positions, or generalized inputs the object Fj contributes. In the Cat-valued case those positions themselves have arrows, so a weighted cone carries coherence along both J and the internal categories $W(j', j)$.

16.2 The Profunctor Of Weighted Cones

For a probe object $b : B$ and a parameter $j' : J'$, the expected category of weighted cones is

$$\mathrm{Cone}_W(b, F; j') \simeq [J, \mathsf{Cat}]\big(W(j', -), \mathrm{Hom}_B(b, F-)\big).$$

An object of this category is a natural family of arrows from b into the diagram, indexed by the weight. Its arrows are the corresponding higher transformations. Varying b acts by precomposition, while varying j' acts through the contravariant endpoint of W. Thus the cone construction has the profunctorial type

$$\mathrm{Cone}_W(F) : B \rightsquigarrow J'.$$

The active definition does not unfold the functor-category expression above. It uses the covariant profunctor implication:

$$\mathrm{Cone}_W(F) := \mathrm{ProfImply}_{\mathrm{cov}}\big(\mathrm{Hom}_B(-, F-), W\big).$$

The literal owner is `WeightedCone_prof`, defined through `Prof_imply_cov`. This is a useful separation. The mathematical formula says what weighted cones mean in a semantic model with the necessary ends; the implication is the stable computational owner used by the present theory.

16.3 Tensor And The Two Residuals

Suppose

$$P : A \rightsquigarrow B, \qquad Q : B \rightsquigarrow X, \qquad O : A \rightsquigarrow X.$$

Tensor makes the middle endpoint into a cut:

$$P \otimes_B Q : A \rightsquigarrow X.$$

There are two ways to solve a mapping problem against this composite. Holding Q fixed gives a right residual with type $A \rightsquigarrow B$; holding P fixed gives a left residual with type $B \rightsquigarrow X$. The active interfaces are characterized by inverse operations

$$\begin{aligned}\mathrm{ProfMap}\big(P, \mathrm{ProfImply}_{\mathrm{cov}}(O, Q)\big) &\simeq \mathrm{ProfMap}(P \otimes_B Q, O),\\ \mathrm{ProfMap}\big(Q, \mathrm{ProfImply}_{\mathrm{con}}(P, O)\big) &\simeq \mathrm{ProfMap}(P \otimes_B Q, O).\end{aligned}$$

Evaluation introduces the tensor cut; lambda abstraction removes it. In both orientations the selected composites reduce:

$$\begin{aligned}\lambda(\mathrm{eval}(t)) &\rightsquigarrow t,\\ \mathrm{eval}(\lambda(u)) &\rightsquigarrow u.\end{aligned}$$

These are universal-property beta and eta laws in the same sense as the upper-star and `tapp1` reductions of Chapter 9. They choose a stable owner for associativity rather than globally reassociating arbitrary composites.

> **Formal status — checked.** Evidence `PROF-CLOSED-CALCULUS`. `Prof_imply_cov` and `Prof_imply_con` are the two opaque residual objects. `Prof_eval_cov_map`, `Prof_lambda_cov_map`, `Prof_eval_con_map`, and `Prof_lambda_con_map` expose the checked fixed-endpoint beta/eta calculus. This status does not assert an end formula for either residual.

The weighted cone is now forced rather than guessed. For every $P : B \rightsquigarrow J'$,

$$\mathrm{ProfMap}\big(P, \mathrm{Cone}_W(F)\big) \simeq \mathrm{ProfMap}\big(P \otimes_{J'} W, \mathrm{Hom}_B(-, F-)\big).$$

A map on the right is precisely a P-shaped family of W-weighted cone data. The residual packages all such maps into one profunctor. In this form, a weight is not an annotation on a limit symbol: it is the profunctor cut that the cone classifier abstracts.

The tensor itself remains an opaque fixed-middle composite. Its current functorial action and shaped-element constructor are checked, but a semantic coend, associator, unitors as equivalences, and the coherence of a full profunctor bicategory are not consequences of those interfaces.

> **Formal status — checked interface and research boundary.** Evidence `PROF-TENSOR` covers the selected tensor object, outer-endpoint reindexing, vertical bifunctoriality, and shaped tensor elements. Evidence `PROF-GENERAL-COEND` records what is missing. The displayed residual mapping laws are checked only through the fixed-endpoint eval/lambda operations above.

16.4 A Weighted Limit Is A Representation

A W-weighted limit of F is a functor

$$L : J' \longrightarrow B$$

together with a representation of the cone profunctor:

$$\Phi : \mathrm{Cone}_W(F) \simeq \mathrm{Hom}_B(-, L-).$$

At a pair (b, j'), this says

$$\mathrm{Cone}_W(b, F; j') \simeq \mathrm{Hom}_B(b, Lj').$$

The direction of the hom is worth checking. A limit receives a cone *from* the probe b, so maps from b to the limit classify cones from b. A colimit will reverse this direction in Chapter 17.

There are two active representation classifiers. The ordinary classifier `IsWeightedLimit_cov_iso` asks for isomorphism evidence in the profunctor category. The computational classifier `IsWeightedLimit_cov_comp` asks for a `ProfComparison`. The latter retains selected forward and inverse maps whose beta and eta laws compute on every incoming profunctor map.

> **Formal status — checked.** Evidence `WEIGHTED-LIMIT-REPRESENTABILITY`. `WeightedCone_prof` constructs the cone profunctor, while `IsWeightedLimit_cov_iso` and `IsWeightedLimit_cov_comp` give the ordinary and computational representation interfaces. The checked claim is the interface and its reductions, not existence of a weighted limit for every diagram.

This is the universal-property row of the rule schema in Appendix G.4. Formation fixes F, W, and the proposed representing functor L; introduction supplies a comparison certificate; push and pull are eliminations; their inverse reductions are beta and eta; and reindexing in the probe is the action clause. Existence and univalent uniqueness are additional principles, not hidden fields of the checked classifier.

This formulation also separates *being* a limit from *choosing* one. A theorem may accept a comparison for a particular F, W, and L without postulating a global limit operator. In a univalent specialization, representability can later make the choice unique in the appropriate sense; the preservation theorem below does not need that stronger uniqueness package.

16.5 Universal Introduction And Elimination

Let

$$M : I \longrightarrow B$$

be a shaped family of probe objects, and let

$$R : I \rightsquigarrow J'$$

index a family of test data. Reindexing Φ along M gives inverse operations

$$\begin{aligned} \mathsf{push} &: \mathrm{ProfMap}\big(R, \mathrm{Cone}_W(M, F)\big) \longrightarrow \mathrm{ProfMap}\big(R, \mathrm{Hom}_B(M, L)\big), \\ \mathsf{pull} &: \mathrm{ProfMap}\big(R, \mathrm{Hom}_B(M, L)\big) \longrightarrow \mathrm{ProfMap}\big(R, \mathrm{Cone}_W(M, F)\big). \end{aligned}$$

Here

$$\mathrm{Cone}_W(M, F) := \mathrm{ProfImply}_{\mathrm{cov}}\big(\mathrm{Hom}_B(M-, F-), W\big).$$

The operation `push` eliminates a supplied cone through the universal comparison and obtains its mediating map into L. The operation `pull` introduces a cone by composing a map into L with the universal cone. Their cuts reduce in both directions:

$$\mathsf{pull}(\mathsf{push}(r)) \rightsquigarrow r,$$
$$\mathsf{push}(\mathsf{pull}(s)) \rightsquigarrow s.$$

> **Formal status — checked.** Evidence `PROF-COMPARISON-BETA-ETA`. The weighted operations `weighted_limit_cov_push` and `weighted_limit_cov_pull` are typed specializations of the generic `prof_comparison_push` and `prof_comparison_pull` owners. The beta/eta rules belong to the generic comparison, so no new cancellation rule is attached to each kind of limit.

The role of R is easy to underestimate. Taking a single element would test only one cone. Allowing an arbitrary incoming profunctor map says that the universal operation is stable under families, endpoint action, and the next categorical layer retained by the fixed-endpoint profunctor category. This is the functorial type-theoretic replacement for an unstructured bijection of sets of cones.

16.6 Conical Limits As The Terminal-Weight Specialization

Set $J' = \mathbf{1}$ and take the terminal weight

$$\mathbf{1}_J : \mathbf{1} \rightsquigarrow J.$$

Its only fibre is the terminal category. In the usual semantic interpretation,

$$\begin{aligned}\mathrm{Cone}_{\mathbf{1}_J}(b, F; *) &\simeq [J, \mathsf{Cat}]\big(\mathbf{1}, \mathrm{Hom}_B(b, F-)\big) \\ &\simeq \mathrm{Cone}(b, F).\end{aligned}$$

A functor $\ell : \mathbf{1} \to B$ selects a vertex. The weighted representation becomes the familiar conical universal property

$$\mathrm{Cone}(b, F) \simeq \mathrm{Hom}_B(b, \ell).$$

Products, terminal objects, pullbacks, and equalizers arise by choosing their usual indexing categories J. The weighted formulation does not require a new universal-property mechanism for each of them.

At the active-interface level, the substitution is exact:

$$\mathrm{IsWeightedLimit}\big(F, \mathrm{TerminalProf}(\mathbf{1}, J), \ell\big)$$

is a well-formed instance of `IsWeightedLimit_cov_comp`. This variance fact is permanent regression evidence. What is not checked is the semantic identification of the opaque profunctor implication with the displayed functor category of ordinary cones.

> **Formal status — formal consequence.** Evidence `WEIGHTED-LIMIT-SPECIALIZATIONS`. `Terminal_prof`, `IsWeightedLimit_cov_comp`, and the focused classifier diagnostics establish the terminal-weight instance and its preservation corollary. Calling its fibres the usual cone categories is mathematical development contingent on the end semantics described below.

16.7 Right Kan Extensions As Conjoint-Weighted Limits

Let

$$K : J \longrightarrow J'$$

and retain the diagram $F : J \to B$. The conjoint of K is the profunctor

$$K^* : J' \rightsquigarrow J, \qquad K^*(j', j) = \mathrm{Hom}_{J'}(j', Kj).$$

It has exactly the variance of a limit weight. Define a selected right Kan-extension comparison along K to be the weighted-limit comparison

$$\mathrm{IsWeightedLimit}\big(F, K^*, \mathrm{Ran}_K\, F\big).$$

When the residual has its expected end semantics, the fibrewise formula is

$$\mathrm{Hom}_B\big(b, (\mathrm{Ran}_K\, F)(j')\big) \simeq \mathrm{Nat}\left(\mathrm{Hom}_{J'}(j', K-), \mathrm{Hom}_B(b, F-)\right).$$

This is the standard pointwise right Kan-extension formula. It says that maps into the value at j' are cones whose shape is the representable weight out of j'.

The use of a conjoint is not a mnemonic guess. `Conjoint_prof K` has type $J' \rightsquigarrow J$, so the expression

$$\mathrm{IsWeightedLimit}\big(F, \mathrm{Conjoint}(K), R\big)$$

is a well-formed active classifier for every $R : J' \to B$. The focused variance audit checks precisely this substitution.

The following two claims must nevertheless remain separate:

1. **Interface specialization:** conjoint weight yields an instance of the selected weighted comparison. This is a formal consequence of the active types.
2. **Semantic identification:** that instance agrees with the ordinary natural-transformation or end definition of pointwise right Kan extension. This requires a semantic end owner and coherence for the relevant Cat-valued transformation category.

> **Formal status — mathematical development.** Evidence `WEIGHTED-END-KAN-SEMANTICS`. The standard conical and pointwise Kan-extension formulas are mathematically part of the weighted theory. `Prof_imply_cov` currently packages their computational residual but does not unfold to a general end.

16.8 Adjunction Mates Transport Representables

Let

$$S : A \longrightarrow B, \qquad R : B \longrightarrow A, \qquad S \dashv R.$$

For arbitrary shaped functors $M : I \to A$ and $D : J \to B$, the adjunction supplies the representable comparison

$$\mathrm{Hom}_B(SM, D) \simeq \mathrm{Hom}_A(M, RD).$$

It is natural in both endpoints and therefore lives as a `ProfComparison`, not merely as a family of unrelated pointwise equivalences. Passing this comparison through the fixed-weight implication transports whole cone profunctors.

> **Formal status — checked.** Evidence `ADJ-HOM-PROF-COMPARISON`. `Adjunction_hom_prof_comparison_along` is the reindexed mate comparison. Its inverse directions and their cancellation are inherited from the generic profunctor-comparison calculus.

In classical category theory, a mate is often introduced by a formula using the unit and counit. Computationally, the mate is better understood as a change of representable coordinates. The triangle reductions of Chapter 12 are exactly the beta/eta laws ensuring that moving across the adjunction and back removes the cut.

16.9 The Preservation Theorem

Assume that

$$\ell : J' \longrightarrow B$$

is supplied with a computational comparison certifying it as the W-weighted limit of $D : J \to B$. We prove that

$$R\ell : J' \longrightarrow A$$

is the W-weighted limit of $RD : J \to A$.

Test the proposed representation at an arbitrary $M : I \to A$. There are three comparisons:

$$\begin{aligned} \mathrm{Cone}_W(M, RD) &\simeq \mathrm{Cone}_W(SM, D) && \text{by the inverse mate under the residual,} \\ &\simeq \mathrm{Hom}_B(SM, \ell) && \text{by the supplied limit comparison,} \\ &\simeq \mathrm{Hom}_A(M, R\ell) && \text{by the mate at the candidate limit.} \end{aligned}$$

The middle comparison is the original representation reindexed along S. The first and third are the same hom adjunction used in opposite directions and at different diagrams. Composing them yields

$$\mathrm{Cone}_W(RD) \simeq \mathrm{Hom}_A(-, R\ell-),$$

which is the required weighted-limit comparison.

The implementation mirrors this proof without inserting a special rewrite that says "right adjoints preserve limits." Its three factors are:

1. `right_adjoint_weighted_limit_comp_step1`, the inverse mate mapped through the fixed-weight implication;
2. `right_adjoint_weighted_limit_comp_step2`, the supplied comparison reindexed along the left adjoint;
3. `right_adjoint_weighted_limit_comp_step3`, the mate at ℓ.

`right_adjoint_preserves_weighted_limit_cov_comp` composes these certificates. Since the result is again a `ProfComparison`, all universal push/pull beta and eta behavior survives.

> **Theorem 16.1 — Right adjoints preserve selected weighted limits.** Given a computational W-weighted-limit comparison for D and an adjunction $S \dashv R$, the active construction returns a computational W-weighted-limit comparison for RD represented by $R\ell$.
>
> **Formal status — checked.** Evidence `WEIGHTED-LIMIT-PRESERVATION`. The theorem is conditional on a supplied comparison. It neither asserts that every weighted limit exists nor that a right adjoint creates one.

This proof is the promised continuation of cut elimination. The first mate changes coordinates, the given universal comparison eliminates the central cone cut, and the final mate changes coordinates back. Associativity is controlled by composition of comparison certificates; it is not delegated to a global reassociation rule.

16.10 Ordinary-Limit And Kan-Extension Corollaries

The theorem is uniform in W, so the two audited specializations immediately give interface-level corollaries.

For the terminal weight:

> If ℓ carries the selected conical-limit comparison for D, then $R\ell$ carries the corresponding comparison for RD.

For the conjoint of $K : J \to J'$:

> If $\mathrm{Ran}_K\, D$ carries the selected right-Kan comparison, then $R\,\mathrm{Ran}_K\, D$ carries the corresponding comparison for $\mathrm{Ran}_K(RD)$.

These are not new preservation algorithms. They are the same `right_adjoint_preserves_weighted_limit_cov_comp` term with $W = \mathrm{TerminalProf}(\mathbf{1}, J)$ or $W = \mathrm{Conjoint}(K)$. The formal consequence is therefore stronger than a slogan and weaker

than the unimplemented semantic end theorem: the classifier, transported comparison, and beta/eta interface are active; the identification with every conventional presentation of limits or Kan extensions is not.

16.11 Ends, Coends, And Dependent Universal Constructions

A semantic completion of this chapter would construct the residual by an end, for example

$$\mathrm{Cone}_W(b, F; j') \simeq \int_{j:J} [W(j', j), \mathrm{Hom}_B(b, Fj)]\,,$$

and tensor by a coend

$$(P \otimes_B Q)(a, x) \simeq \int^{b:B} P(a, b) \times Q(b, x).$$

Such constructions must provide more than object formulas. They need introduction and elimination principles, action on base arrows and transfors, beta/eta or comparison laws, associativity and unit coherence, and a validation route through the shaped co-Yoneda theorem. Until those owners exist, the formulas are semantic specifications for the opaque interfaces.

Dependent category theory asks for a further chain. For a base functor $f : X \to Y$, one expects suitable change-of-base functors and adjunctions

$$\Sigma_f \dashv f^* \dashv \Pi_f,$$

with Beck--Chevalley and higher naturality conditions. The current Sigma and Pi categories of directed families are important footholds, but no general dependent adjunction package connects them to arbitrary base change. A pointwise object formula would be insufficient: the construction must retain base-arrow action, off-diagonal transfor action, and iteration into the next hom.

> **Formal status — research boundary.** Evidence `DEPENDENT-ADJUNCTIONS`. General end/coend owners, pointwise Kan packages, and the dependent adjunction chain remain separate formal projects. The active weighted comparison is the design constraint those projects should refine, not a claim that they are already present.

Chapter 17 now applies the safest of all extensions: opposite duality. It turns the checked limit theorem into a colimit theorem, after which the join shows how terminal-weight cross-arrow data can itself be internalized as a directed categorical shape.

17. Weighted Colimits, Duality, And Join

The preceding chapter classified cones by homs *into* a representing object. Colimits classify cocones by homs *out of* one. Every variance reverses: companions replace conjoints, left adjoints replace right adjoints, and the universal arrows point away from the colimit.

Opposite duality makes this reversal a proof method rather than an invitation to duplicate the theory. The active weighted-colimit classifier is defined through the weighted-limit classifier in opposite categories. Its preservation theorem is therefore the right-adjoint theorem of Chapter 16 applied once to the opposite adjunction.

The second half of the chapter gives this duality a directed geometry. The join $A \star B$ contains a left part, a right part, and an internally natural family of arrows directed from left to right. Its recursor has the same input shape as the collage of a terminal profunctor. That observation connects joins to weighted cone data while leaving the stronger collage semantics explicitly open.

17.1 Weighted Cocones

Let

$$F : J \longrightarrow B$$

be a diagram and let

$$W : J \rightsquigarrow J'$$

be a contravariant weight, meaning a Cat-valued profunctor

$$W : J^{\mathrm{op}} \times J' \longrightarrow \mathsf{Cat}.$$

For fixed $j' : J'$, the weight $W(-, j')$ is contravariant in J. At a candidate target $b : B$, its expected cocone category is

$$\mathrm{Cocone}_W(F; j', b) \simeq [J^{\mathrm{op}}, \mathsf{Cat}]\big(W(-, j'), \mathrm{Hom}_B(F-, b)\big).$$

A W-weighted colimit is a functor

$$C : J' \longrightarrow B$$

with a representation

$$\mathrm{Cocone}_W(F; j', b) \simeq \mathrm{Hom}_B(Cj', b)$$

natural in j' and b. Compare this with the limit equation

$$\mathrm{Cone}_W(b, F; j') \simeq \mathrm{Hom}_B(b, Lj').$$

The same words “represented by” occur in both formulas, but the representable hom has changed side.

17.2 One Universal Owner Through Opposite Categories

For

$$F : J \to B, \qquad W : J \nrightarrow J', \qquad C : J' \to B,$$

the active definition is

$$\mathrm{IsWeightedColimit}_B(F, W, C) := \mathrm{IsWeightedLimit}_{B^{\mathrm{op}}}\big(F^{\mathrm{op}}, W^{\mathrm{op}}, C^{\mathrm{op}}\big).$$

Reversing the profunctor endpoints gives

$$W^{\mathrm{op}} : (J')^{\mathrm{op}} \nrightarrow J^{\mathrm{op}},$$

which is exactly the variance expected by the limit classifier in the opposite categories. The representing equation there is

$$\mathrm{Hom}_{B^{\mathrm{op}}}(b, Cj') = \mathrm{Hom}_B(Cj', b),$$

so it recovers the desired cocone orientation.

The owner `WeightedColimit_con` is transparent to `IsWeightedLimit_cov_comp` after applying `Op_func` and `Op_prof`. The conversion operations between a limit witness and the corresponding opposite colimit witness are identity-like wrappers after double-opposite and product-swap computation. Colimit beta/eta therefore comes from the same profunctor comparison used for limits.

> **Formal status — checked.** Evidence `OP-DUALITY`. The involutive category, functor, transfor, profunctor, and adjunction operations justify the variance reversal. No independent colimit cancellation calculus is introduced.

17.3 Conical Colimits And Left Kan Extensions

The terminal-weight specialization now uses the opposite orientation:

$$\mathbf{1}^J : J \nrightarrow \mathbf{1}.$$

For a vertex $c : \mathbf{1} \to B$, the active classifier

$$\mathrm{IsWeightedColimit}\big(F, \mathrm{TerminalProf}(J, \mathbf{1}), c\big)$$

is well formed. Under the standard semantic interpretation its fibres become ordinary cocone categories, and the representation reads

$$\mathrm{Cocone}(F, b) \simeq \mathrm{Hom}_B(c, b).$$

Thus initial objects, coproducts, pushouts, and coequalizers are terminal weight specializations on their usual indexing categories.

For a functor

$$K : J \longrightarrow J',$$

the companion has the required colimit variance:

$$K_* : J \rightsquigarrow J', \qquad K_*(j, j') = \mathrm{Hom}_{J'}(Kj, j').$$

A selected left Kan-extension comparison is therefore

$$\mathrm{IsWeightedColimit}\big(F, K_*, \mathrm{Lan}_K\, F\big).$$

With the expected coend or cocone semantics, it gives the pointwise formula

$$\mathrm{Hom}_B\big((\mathrm{Lan}_K\, F)(j'), b\big) \simeq \mathrm{Nat}\left(\mathrm{Hom}_{J'}(K-, j'), \mathrm{Hom}_B(F-, b)\right).$$

The focused variance audit checks both active substitutions: `Terminal_prof J Terminal_cat` and `Companion_prof K` inhabit the weight slot of `WeightedColimit_con`. As in Chapter 16, this does not by itself identify the opaque opposite residual with a semantic category of cocones.

> **Formal status — formal consequence.** Evidence `WEIGHTED-COLIMIT-SPECIALIZATIONS`. The terminal-weight and companion classifiers, together with their preservation instances, follow from the active types and the opposite definition. Their standard conical and pointwise-left-Kan interpretations remain mathematical development under `WEIGHTED-END-KAN-SEMANTICS`.

The companion/conjoint distinction is now visible in one table:

Construction along $K : J \to J'$	**Weight**	**Fibre**
right Kan extension	conjoint $K^* : J' \rightsquigarrow J$	$\mathrm{Hom}_{J'}(j', Kj)$
left Kan extension	companion $K_* : J \rightsquigarrow J'$	$\mathrm{Hom}_{J'}(Kj, j')$

Suppressing the profunctor endpoints would make these formulas look deceptively interchangeable. The endpoints are the type-level variance audit.

17.4 Left Adjoints Preserve Weighted Colimits

Let

$$S : A \longrightarrow B, \qquad R : B \longrightarrow A, \qquad S \dashv R,$$

and suppose

$$C : J' \longrightarrow A$$

carries a comparison certifying it as the W-weighted colimit of $F : J \to A$. Passing to opposites turns the data into:

1. a weighted-limit comparison for C^{op} and F^{op} in A^{op};
2. the opposite adjunction $R^{\mathrm{op}} \dashv S^{\mathrm{op}}$;
3. a right adjoint $S^{\mathrm{op}} : A^{\mathrm{op}} \to B^{\mathrm{op}}$.

Theorem 16.1 therefore supplies a weighted-limit comparison represented by

$$S^{\mathrm{op}} C^{\mathrm{op}} = (SC)^{\mathrm{op}}.$$

Turning the result back around gives a W-weighted-colimit comparison for SF represented by SC.

At the level of cocones, the three-comparison proof is the familiar chain

$$\begin{aligned} \mathrm{Cocone}_W(SF; b) &\simeq \mathrm{Cocone}_W(F; Rb), \\ &\simeq \mathrm{Hom}_A(C, Rb), \\ &\simeq \mathrm{Hom}_B(SC, b). \end{aligned}$$

The first and last steps are adjunction mates, and the middle step is the given colimit representation. The implementation obtains this chain by calling the right-adjoint theorem on `Op_adjunction`; it does not repeat the three steps under new primitive names.

> **Theorem 17.1 — Left adjoints preserve selected weighted colimits.** Given a computational W-weighted-colimit comparison for F and an adjunction $S \dashv R$, the active construction returns a computational comparison for SF represented by SC.
>
> **Formal status — checked.** Evidence `WEIGHTED-COLIMIT-PRESERVATION`. The owner `left_adjoint_preserves_weighted_colimit_con` applies `right_adjoint_preserves_weighted_limit_cov_comp` to the opposite adjunction. The theorem is conditional on the supplied colimit comparison.

Consequently, the terminal and companion specializations yield the familiar interface-level corollaries: left adjoints preserve selected conical colimits and selected left Kan extensions. No new proof is hidden behind either phrase.

17.5 A Variance Ledger

The dual theorem can be remembered without reversing formulas in one's head:

Feature	Weighted limit	Weighted colimit
weight	$J' \rightsquigarrow J$	$J \rightsquigarrow J'$
universal data	arrows $b \to Fj$	arrows $Fj \to b$
represented hom	$\mathrm{Hom}_B(b, Lj')$	$\mathrm{Hom}_B(Cj', b)$
Kan weight	conjoint	companion
preserving adjoint	right	left
proof	direct three-comparison chain	the same chain in opposites

This ledger is more than notation. It identifies which endpoint acts by upper-star precomposition and which acts by lower-star postcomposition. The opposite construction exchanges those actions while preserving their computational owners.

17.6 The Directed Join Signature

For native categories A and B, the primitive join is a category

$$A \star B$$

with inclusion functors

$$\iota_A : A \longrightarrow A \star B, \qquad \iota_B : B \longrightarrow A \star B.$$

Its characteristic constructor is not an equality between the two parts. It is an internally natural family of arrows from the left part to the right part:

$$\chi : \mathbf{1}_{A,B} \Longrightarrow \mathrm{Hom}_{A\star B}(\iota_A -, \iota_B -),$$

where

$$\mathbf{1}_{A,B} : A \rightsquigarrow B$$

is the terminal profunctor. At objects, χ supplies

$$\chi_{a,b} : \iota_A(a) \longrightarrow \iota_B(b).$$

Because χ is a profunctor cell, naturality in both a and b is part of one internal datum. It is not an externally quantified family followed by a separately asserted equation. For shaped functors $a : I \to A$ and $b : I \to B$, `join_cross_hom` evaluates the same cell to the corresponding shaped cross arrow and retains its higher action.

There is no reverse constructor $\iota_B(b) \to \iota_A(a)$. The join is directed even when A and B individually happen to be groupoidal.

17.7 Recursion And Its Three Beta Observations

To define a functor

$$H : A \star B \longrightarrow E,$$

the active nondependent recursor accepts:

1. a functor $F : A \to E$;
2. a functor $G : B \to E$;
3. an internally natural cross cell

$$\gamma : \mathbf{1}_{A,B} \Longrightarrow \mathrm{Hom}_E(F-, G-).$$

It returns `join_elim_func F G gamma`. The two restrictions compute:

$$H \circ \iota_A \rightsquigarrow F,$$
$$H \circ \iota_B \rightsquigarrow G.$$

The selected observation of the image of the universal cross cell also computes:

$$H(\chi) \rightsquigarrow \gamma.$$

The literal third owner is `join_elim_cross_transf`. It records the cross-cell beta rule without adding a broad equation for arbitrary functor application to primitive join syntax.

> **Formal status — checked.** Evidence `JOIN-RECURSOR`. The active owners are `Join_cat`, `join_fst_func`, `join_snd_func`, `join_cross_transf`, `join_cross_hom`, `join_elim_func`, and `join_elim_cross_transf`. The checked interface is a nondependent recursor with its three observations; it is not yet a uniqueness theorem for all functors out of the join.

The recursor repeats the book's computational pattern. The inclusions and cross cell introduce the join; restriction and the selected cross observation eliminate it; matching introduction/elimination cuts reduce to the supplied data.

17.8 Ordinary Cones And Cocones As Join Diagrams

The join makes the orientation of conical universal data concrete. A functor

$$\mathbf{1} \star J \longrightarrow B$$

can be constructed from:

- an object $b : B$;
- a diagram $F : J \to B$;
- a natural family of cross arrows $b \to Fj$.

That is precisely the data of a cone from b to F. Reversing the two parts, a functor

$$J \star \mathbf{1} \longrightarrow B$$

can be constructed from F, an object c, and arrows $Fj \to c$: the data of a cocone.

This is an interface-level construction supplied by the recursor. It does not yet assert an equivalence between a full mapping category out of the join and a category of cones. Such an equivalence would require an eta or uniqueness principle for join maps and coherent action on transformations between them.

The connection to weights is now visible. A conical cone uses the terminal weight, and the join internalizes a terminal profunctor as its family of left-to-right cross arrows. Replacing that terminal profunctor by an arbitrary $P : A \nrightarrow B$ leads to the notion of a collage.

17.9 Join As A Directed Higher-Inductive Pattern

The join has the shape of a directed higher-inductive specification:

- two strata of objects, introduced through A and B;
- a family of directed arrow constructors from every left object to every right object;
- a recursor into an arbitrary target;
- beta observations on the two strata and the cross family.

This comparison is architectural, not a claim that `Join_cat` was generated by a general directed-HIT compiler. The primitive join is one selected signature. Its cross constructor is already internally natural, which is the category-level analogue of giving all boundary coherence with the constructor.

WalkingEnd and join illustrate two different levels of the same programme. WalkingEnd has an opaque point and endomorphism together with a contextual dependent eliminator strong enough for encode-decode. Join has two category-shaped introductions and a nondependent recursor, but no dependent eliminator or computation of all its hom-categories. Thus join supplies a promising stress test for a future directed-HIT schema without borrowing properties proved only for WalkingEnd.

17.10 The Collage Comparison

For a profunctor

$$P : A \nrightarrow B,$$

its expected collage $\mathrm{Coll}(P)$ contains A and B as two parts and uses $P(a, b)$ as the category of arrows from the left object a to the right object b. Its universal mapping property has the schematic form

$$\mathrm{Fun}\big(\mathrm{Coll}(P), E\big) \simeq \sum_{F:A\to E} \sum_{G:B\to E} \mathrm{ProfCell}\big(P, \mathrm{Hom}_E(F-, G-)\big).$$

Taking $P = \mathbf{1}_{A,B}$ gives exactly the *input shape* of the active join recursor. This is strong design evidence for reading $A \star B$ as the prospective collage of the terminal profunctor.

It is not yet a proof of that reading. A checked collage theorem would need:

1. an object decomposition into the left and right strata;
2. left-left and right-right hom comparisons recovering A and B;
3. a left-right hom comparison recovering P;
4. a right-left hom description, normally initial in the free collage;
5. composition and higher action compatible with those four regions;
6. full faithfulness of the inclusions;
7. an equivalence of mapping categories, including uniqueness and higher transformations;
8. a dependent eliminator with beta behavior on the cross cells.

The active primitive supplies the inclusions, one terminal cross cell, and the forward recursor only. It exposes none of the required hom decompositions. Likewise, one expects an opposite comparison

$$(A \star B)^{\mathrm{op}} \simeq B^{\mathrm{op}} \star A^{\mathrm{op}},$$

but no such comparison is currently selected.

> **Formal status — research boundary.** Evidence `JOIN-COLLAGE-BOUNDARY`. Side task `FTTX-S13` owns the proposed collage semantics and dependent elimination. The current chapter claims only that the checked recursor has the terminal-collage input shape.

17.11 From Universal Cuts To Directed Geometry

Weighted limits and colimits began with a mapping problem: a profunctor of cones or cocones is represented by a hom. Join begins with the same data from the other side. Instead of representing a terminal family of cross arrows inside a target, it freely presents a domain from which such data can be interpreted by recursion.

The two constructions therefore complement one another:

weighted representation	directed join recursion
classifies maps into or out of a vertex	internalizes a family of cross arrows
beta/eta from a profunctor comparison	beta from introduction/recursion
arbitrary weight	currently the terminal profunctor
right/left adjoint preservation	prospective collage and dependent elimination

This is the globally coherent role of the active universal-construction interfaces. Tensor, residuals, weighted representation, duality, and join are not isolated features. Each controls a different kind of cut, and each leaves enough categorical action visible for a future elaboration layer to compile surface mathematics into the same computational core.

18. Presheaves And Sieves

An object rarely reveals its geometry all at once. We learn about it by probing it from other objects, by changing the probe, and by asking which properties survive that change. For an object U of a category $\mathcal{K}$, the probes are simply the arrows

$$p : V \longrightarrow U.$$

The domain V is a stage of observation. A further arrow $q : W \to V$ refines the observation, and the composite $p \circ q : W \to U$ is the same probe viewed at the finer stage. This elementary picture contains the beginning of presheaf semantics, the definition of a sieve, and eventually the local-to-global language of schemes.

The decisive shift is from asking for *the* region where a property holds to recording *every probe along which it holds*. The latter collection is automatically adapted to change of stage. It is a sieve. An open subobject may represent that sieve, but representation is an additional theorem, not part of the initial definition.

This chapter develops that distinction in three steps. A presheaf gives a coherent field of views. A higher sieve may retain a category of witnesses at each probe. An ordinary sieve remembers only a proposition: whether the probe belongs. The recurring example is the invertibility sieve $D_U(s)$ of a section s over U.

18.1 A Coherent Field Of Views

Let $\mathcal{K}$ be a category. A set-valued presheaf is a functor

$$X : \mathcal{K}^{\mathrm{op}} \longrightarrow \mathsf{Set}.$$

Thus every object U has a set $X(U)$ of observations, and every arrow $p : V \to U$ has a restriction map

$$p^* = X(p) : X(U) \longrightarrow X(V).$$

Restriction along an identity does nothing, and restriction along a composite is successive restriction:

$$(\mathrm{id}_U)^* = \mathrm{id}_{X(U)}, \qquad (p \circ q)^* = q^* \circ p^*.$$

These equations say more than “data vary with U.” They say exactly how one view becomes another. A section at a large stage can be inspected at every smaller stage, and two routes to the same refined view agree.

Two familiar examples give the variance its geometric meaning. On a topological space, one may assign to every open U the continuous real-valued functions on U. An inclusion $V \subseteq U$ restricts a function on U to one on V. On the opposite of a category of commutative rings, the functor of points of a ring R assigns to a test ring S the maps $R \to S$. A map of test rings changes the point by composition. In both cases, the presheaf is not merely a table indexed by objects. Its restriction maps are the mathematical content that lets observations move.

The examples also warn against reading "smaller stage" as literal spatial inclusion. A probe can be an open subset, an étale map, a ring map viewed in the opposite category, or some other morphism selected by the geometry. The categorical definition uses only arrows. Spatial language becomes justified when a later representation theorem supplies it.

Functorial type theory keeps the same idea but permits each $X(U)$ to be a category rather than merely a set:

$$X : \mathcal{K}^{\mathrm{op}} \longrightarrow \mathsf{Cat}.$$

Now a stage may contain objects, arrows between observations, and higher coherence inherited from the ambient categorical structure. Restriction is a functor. It transports both observations and their comparisons. Presheaf maps are natural family maps, so they too act coherently at every stage.

There is also a useful change-of-base operation. Given $F : \mathcal{A} \to \mathcal{B}$ and a presheaf X on $\mathcal{B}$, precomposition produces a presheaf on $\mathcal{A}$:

$$F^{*}X := X \circ F^{\mathrm{op}}.$$

The active construction realizes this whole operation by reusing the existing pullback of directed Cat-valued families. It does not invent a second calculus for presheaves. That economy will matter repeatedly: local geometry should inherit functoriality from the categorical structure already present.

18.2 The Representable Field Of Probes

Every object U supplies a canonical presheaf

$$yU := \mathrm{Hom}_{\mathcal{K}}(-, U).$$

At a stage V, its objects are precisely the probes $p : V \to U$. If $q : W \to V$, restriction sends p to $p \circ q$. Nothing has been added to the category: the presheaf merely organizes all arrows into U as one coherent field.

This is the contravariant Yoneda construction from Chapter 13, now read geometrically. There it served as a universal coordinate system. Here its elements are stages of observation. The two readings are the same mathematics: a map out of a representable is controlled by what happens at the identity probe $\mathrm{id}_U : U \to U$.

The total category of the family yU has objects (V, p) with $p : V \to U$. For restriction it is convenient to orient its arrows in the direction in which data are reindexed. Taking the opposite recovers the conventional slice category $\mathcal{K}/U$. Keeping both orientations visible prevents a common mistake: a refinement of a probe and a map in the displayed family carry the same information, but variance determines which way the corresponding arrow points.

A **higher sieve** on U is a Cat-valued coefficient system over this restriction-oriented category of probes. Equivalently, it may be read as a Cat-valued presheaf on the conventional slice. At each $p : V \to U$ it assigns a category $S(p)$, and a refinement $q : W \to V$ induces a functor

$$S(p) \longrightarrow S(p \circ q).$$

The category $S(p)$ can retain choices of witnesses and arrows between them. This is why "higher sieve" does not yet mean ordinary sieve. The maximal higher sieve assigns the terminal coefficient category to every probe. If the base probe $V \to U$ is changed, the entire coefficient system pulls back by postcomposing probes, and the maximal higher sieve remains maximal.

> **Formal status — checked.** Evidence `PSH-YONEDA-HIGHER-SIEVE`. The active construction has a visible category of Cat-valued presheaves, restriction along a functor, the Yoneda presheaf with value $\mathrm{Hom}_{\mathcal{K}}(V, U)$, the restriction-oriented arrow total and opposite slice, and a higher-sieve classifier whose maximal object is stable under pullback. The slice and higher-sieve presentations compare through a shared family representation; they are not declared to be definitionally identical.

18.3 From Witnesses To Membership

An ordinary sieve is what remains when each coefficient category answers only a yes-or-no question. Constructively, "yes or no" does not mean that a Boolean decision has been chosen. It means that the space of answers is a proposition: if it is inhabited, all of its inhabitants agree.

Classically, a sieve R on U is a collection of arrows into U such that

$$p \in R, q : W \longrightarrow V \quad \Longrightarrow \quad p \circ q \in R.$$

The presheaf formulation replaces the collection by a subterminal value at every probe. Write $R(p)$ for the coefficient at $p : V \to U$. Its object classifier is the proposition

$$p \in R.$$

The action of the underlying higher sieve carries a witness of $p \in R$ to a witness of $p \circ q \in R$. Closure under refinement is therefore not an extra law written beside the data. It is the functorial action of the data.

Why require a *subterminal category* rather than merely proposition-valued objects? Because a category with a unique object can still have many directed endomorphisms. Ordinary membership is intended to retain no such hidden motion. The active condition combines proposition-valued objects with exact groupoidality, leaving the categorical analogues of the empty and terminal possibilities. The evidence that a coefficient is subterminal is itself a proposition, so it does not create competing ways for one arrow to belong.

Two examples orient the definition.

1. The maximal sieve contains every arrow into U. Its membership proposition is always inhabited.
2. If a monomorphism $j : W \to U$ is regarded as a subobject, it determines the sieve of arrows $p : V \to U$ that factor through j. Monicity makes the factorization proposition-valued. Refining a factorization gives another factorization automatically.

The second example is important but not exhaustive. A sieve need not be represented by one monomorphism. It can be a genuinely distributed local question whose answer is stable under refinement without having a single object that names all affirmative probes.

18.4 Pulling Back A Local Question

Suppose R is a sieve on U and $p : V \to U$. To ask the same question over V, test every $q : W \to V$ after postcomposition with p:

$$q \in p^*R \quad :\Longleftrightarrow \quad p \circ q \in R.$$

This defines a sieve p^*R on V. Indeed, if q belongs and $r : Z \to W$, then

$$p \circ (q \circ r) = (p \circ q) \circ r$$

belongs by closure of R. More conceptually, the higher-sieve action already postcomposes every probe with p. Pointwise subterminality is preserved by selecting the old witness at that postcomposed probe. Ordinary pullback is therefore inherited structure, not a separately engineered operation.

Theorem 18.1 (pullback of ordinary sieves). For every sieve R on U and arrow $p : V \to U$, there is an ordinary sieve p^*R on V. Its underlying higher sieve is the higher pullback of the underlying higher sieve of R, and membership at $q : W \to V$ is old membership at $p \circ q : W \to U$.

> **Formal status — checked.** Evidence `ORDINARY-SIEVE-PULLBACK`. The active construction supplies the pullback sieve, reuses the whole higher-sieve action, preserves subterminal evidence, and exposes the membership computation. Because an ordinary sieve retains proof fields, identity pullback is not claimed to reconstruct the entire package by raw definitional reduction; this does not change the membership formula above.

Pullback is the reason sieves are the right language for locality. A property stated only at U can be accidental. A sieve records in advance how the property appears after every change of stage. A topology will later decide which such stable questions count as *covering* questions, but the variance has already been settled.

Represented sieves make this calculation visible as ordinary base change. Suppose the sieve R is represented by a monomorphism $j : A \to U$, and suppose the pullback square

$$\begin{array}{ccc} A \times_U V & \longrightarrow & A \\ \downarrow & & \downarrow j \\ V & \xrightarrow{p} & U \end{array}$$

exists. A probe $q : W \to V$ belongs to p^*R exactly when $p \circ q$ factors through j. By the universal property of the pullback, this is exactly when q factors through $A \times_U V \to V$. Hence p^*R is represented by the base-changed monomorphism.

This is the bridge between sieve pullback and the usual restriction of an open. The sieve calculation requires only postcomposition and membership. The spatial calculation additionally requires a representing monomorphism and the relevant pullback. When those hypotheses are available, the two descriptions agree; when they are not, the sieve calculation still makes sense.

18.5 Invertibility Before Opens

Let $\mathcal{O}$ be a presheaf of commutative rings on $\mathcal{K}$, let U be a stage, and let $s \in \mathcal{O}(U)$ be a section. For a probe $p : V \to U$, restriction produces

$$p^*s \in \mathcal{O}(V).$$

Define the **invertibility sieve** of s by

$$D_U(s)(p) \;:=\; \text{“}p^*s \text{ is a unit in } \mathcal{O}(V)\text{.”}$$

This is a sieve because ring homomorphisms preserve units. If p^*s has an inverse and $q : W \to V$, applying the restriction homomorphism along q gives an inverse for $(p \circ q)^*s$. Thus invertibility propagates toward finer probes.

> **Formal status — checked.** Evidence `COMM-RING-INVERTIBILITY-SIEVE`. For a selected commutative-ring-valued presheaf and section, the active construction supplies an ordinary sieve, and membership at $p : V \to U$ computes to unit evidence for the restricted section p^*s.

The notation $D_U(s)$ is familiar from algebraic geometry, but the order of ideas matters. We have not first constructed an open object and then asked which probes land in it. We first have the stable family of all invertibility probes. The question whether that family is represented by an open object is asked afterward.

This reverses a habitual abbreviation. In settings where basic opens are already known to exist, one says “the open on which s is invertible.” The sieve formulation separates two claims hidden in that phrase:

1. invertibility is stable under change of stage, so it defines $D_U(s)$; and
2. the sieve $D_U(s)$ is represented by a suitable open object over U.

The first statement is formal and functorial. The second depends on the geometry of the site. In affine geometry, localization will supply the representing object for a basic invertibility sieve. On a more general site, there may be no single representative, while the sieve itself remains perfectly meaningful.

This point also clarifies the relation with Max Zeuner's constructive account of algebraic geometry. In the locally ringed lattices of Zeuner, the invertibility support of a section is presented as the largest compact open below U on which that section is invertible. In a posetal site, such a largest open represents the sieve $D_U(s)$: a probe lies in the sieve exactly when it factors through that open. Conversely, a compact open representing $D_U(s)$ has the required largest-property interpretation.

The sieve-centered formulation is therefore a generalization, not a rejection, of the compact-open one. It asks the useful comparison question:

> When is the invertibility sieve $D_U(s)$ representable by a compact open?

On a coherent or qcqs presentation, representability can recover the compact geometry emphasized by Zeuner. On an arbitrary site, the sieve remains the primary object even when that recovery is unavailable. In a higher setting, one may go further and retain a category of invertibility witnesses before subterminality is imposed.

The posetal case makes the comparison exact. Regard a lattice of opens below U as a category, with one arrow $V \to W$ when $V \le W$. If an open $A \le U$ represents $D_U(s)$, then for every $V \le U$,

$$s|_V \text{ is invertible} \quad \iff \quad V \le A.$$

Taking $V = A$ shows that $s|_A$ is invertible. Taking any V on which s is invertible shows $V \le A$. Thus A is the largest invertibility open. Conversely, if a largest such A exists, invertibility is preserved by restriction, so precisely the opens below A belong to $D_U(s)$. The largest open represents the sieve. Compactness is a further finiteness property of that representative, important in coherent algebraic geometry but not needed to define membership.

The distinction separates existence from use. One can calculate with $D_U(s)$, pull it back, and ask whether it covers before finding a representing open. Once a compact representative is constructed, the same sieve acquires the economical lattice presentation. The two layers reinforce rather than compete with one another.

> **Formal status — mathematical development.** The comparison with Zeuner's compact-open support is an attributed reformulation of the representation problem, principally adapted from his definition of locally ringed lattices and his functor-of-points development. The active emdash result in this chapter is the ordinary invertibility sieve. No theorem identifying the general em-dash site-relative scheme interface with Zeuner's qcqs schemes is claimed here.

18.6 Categorical Semantics As Executable Mathematics

Presheaves, slices, and sieves are often introduced as an external semantics for some other formal language. That is not the only way to make them computational. In the emdash architecture, the categories and functors just described are themselves expressed in an inner functorial type theory. They are ordinary mathematical objects of that theory: a presheaf is a functor, a slice is a category, and pullback is functorial reindexing.

The inner theory is hosted in an outer dependent logical framework. Lambdapi, or the bounded TypeScript emdash Core, supplies explicit binders, checking, rewrite computation, comparison, and unification. Consequently, categorical semantics can be internal enough to calculate with while remaining recognizably the traditional semantics of presheaves and sites.

This is a claim about relative internality, not about replacing every modal or internal language. A modal type theory may provide elegant syntax for local reasoning. Here the chosen route is to make the categorical objects themselves executable and then state locality directly in them. The advantage for the present development is transparency: the probe, its refinement, the sieve pullback, and later the matching family remain visible mathematical data.

Nor does executability settle the metatheory. Local computations accepted by the outer framework do not by themselves prove global normalization, confluence, consistency, or semantic soundness for the combined rewrite system. The categorical constructions and their checked observations are the evidence used here; the larger metatheorems retain their separate boundary.

18.7 From Sieves To Covers

A sieve answers a local question. It does not yet say that the affirmative probes are sufficient to recover information on U. For that, one must select the sieves that count as covers and demand three kinds of stability: the maximal sieve covers, a covering sieve remains covering after pullback, and covers may be refined locally.

Those laws turn a category into a site. They also prepare a more delicate question. Given compatible observations on every probe in a covering sieve, when do they determine one observation on U? That is descent, and its objects are matching families and their amalgamations.

The next step in the local-to-global spiral therefore does not abandon the sieve for an open. Chapter 19 asks which sieves cover, and what it means for a presheaf to solve every covering question. Geometry will emerge from the answers, but the questions already have their correct functorial shape.

19. Sites, Covers, And Descent

A sieve is a question stable under refinement. It does not yet say that its affirmative probes see enough of the object. The maximal sieve certainly does: it contains every probe. A smaller sieve may also be sufficient, but that is new structure. One must decide which local views are jointly adequate for recovering information at their common target.

A **Grothendieck topology** makes that decision. It selects covering sieves in a way compatible with identities, change of stage, and local refinement. A category equipped with such a topology is a **site**. The word "topology" is appropriate even when the objects are not open subsets of a space. Its role is to identify which families of probes count as local descriptions.

Once a cover has been selected, a presheaf faces a test. Compatible data on the probes of the cover form a matching family. Does that family come from one global datum, and is the global datum determined by it? This is descent. The chapter separates three layers that are often compressed into one phrase:

1. a family or sieve proposed as a cover;
2. a topology generated by chosen proposals; and
3. the locality of a presheaf with respect to the resulting covers.

Sheafification is a fourth layer. It constructs a local object from an arbitrary presheaf and therefore belongs to the next chapter.

19.1 Covering Families And Covering Sieves

Suppose a family of arrows has common codomain U:

$$u_i : U_i \longrightarrow U.$$

It generates a sieve by admitting every probe that factors through one of the u_i. Refinement closure then admits all further composites automatically. The family is a useful presentation of local pieces; the generated sieve is the complete local question determined by that presentation.

More explicitly, a probe $p : V \to U$ belongs when there are an index i and an arrow $h : V \to U_i$ such that

$$p = u_i \circ h.$$

Ordinary sieve membership remembers this existential claim only as a proposition. A computational presentation may separately retain the index, the factorization, or algebraic evidence that the family has the desired covering property. Keeping those layers separate prevents two opposite errors: discarding useful witnesses too early, and making the topology depend on accidental choices of witnesses.

Different families can generate the same sieve. Repeating a member changes the list but not the question. Replacing one member by a family that covers it may refine the presentation without changing its eventual force. A sieve also makes arbitrary change of stage immediate: pull it back as in Chapter 18, rather than choosing a new list and proving afresh that the list behaves correctly.

This does not make cover families dispensable. In constructive algebraic geometry, a finite unimodular family or a pair of affine charts can carry valuable computational witnesses. The point is to distinguish the witness from the invariant it presents. One may retain the particular generators for calculation while allowing the topology to speak in terms of the sieve they force to cover.

For an ordinary topological space, a family of open inclusions $U_i \hookrightarrow U$ is covering when their union is U. The associated sieve contains every open $V \hookrightarrow U$ that lies inside some U_i, and then all smaller opens as well. In affine geometry, a finite family of basic opens will later be presented by ring elements satisfying an algebraic cover certificate. Again, the certificate explains why the generated sieve covers; it is not identical to the sieve.

Notice that generation alone says nothing about sufficiency. Any family of arrows generates a sieve. Only the topology declares whether that sieve covers. The separation is what allows the same category of probes to carry different geometries.

At the opposite extreme lies the maximal sieve $\top_U$. Every arrow into U belongs to it. It is generated by the identity probe $\mathrm{id}_U : U \to U$, since every $p : V \to U$ factors through the identity. Any reasonable notion of locality must regard this as a cover: observing all of U at once is sufficient to observe U.

19.2 The Three Laws Of A Site

Let $J(U, R)$ be the proposition that the sieve R covers U. To make J a Grothendieck topology, require three laws.

Maximality. The maximal sieve covers:

$$J(U, \top_U).$$

Pullback stability. If R covers U and $p : V \to U$, then the pulled-back sieve covers V:

$$J(U, R) \quad \Longrightarrow \quad J(V, p^* R).$$

This is the formal expression of “covers remain covers after changing the stage of observation.” If R is represented by an open $A \to U$, then under the hypotheses of Chapter 18 the new cover is represented by $A \times_U V \to V$.

Local character. Suppose R covers U. Let S be another sieve on U. If, for every $p : V \to U$ belonging to R, the pullback $p^* S$ covers V, then S covers U:

$$J(U,R) \quad \text{and} \quad \prod_{p \in R} J(V, p^* S) \quad \Longrightarrow \quad J(U,S).$$

Local character is the transitivity of coverage. The sieve R says that its probes see all of U. Each of those probes says that S sees all of its domain. Together they say that S sees all of U. The law avoids choosing a single flattened family and consequently works equally well for finite, infinite, or witness-rich presentations.

In family language, suppose the $u_i : U_i \to U$ cover and, for every i, the arrows $v_{ij} : V_{ij} \to U_i$ cover U_i. The composites $u_i \circ v_{ij} : V_{ij} \to U$ should cover U. Let R be the sieve generated by the u_i and S the sieve generated by the composites. The pullback of S to U_i contains the locally covering v_{ij}, while R covers U. Local character is the invariant sieve statement behind this flattening argument. It also covers situations in which there is no preferred single index set of composites.

The three laws are deliberately asymmetric in purpose. Maximality supplies a unit, pullback stability transports a cover, and local character composes local sufficiency. None says that membership in a cover is decidable. Cover evidence remains proposition-valued.

Nor does a Grothendieck topology add new objects or arrows to $\mathcal{K}$. It changes which existing sieves are treated as sufficient. Two topologies on the same category can therefore encode different notions of locality. A covering sieve need not be represented by an open subobject; and when it is represented, the representing open is geometry derived from the site rather than a replacement for the cover predicate. This is the same sieve-before-open discipline that governed $D_U(s)$ in Chapter 18.

There is always a degenerate but useful model: declare every sieve covering. Then all three laws have their unique truth witness. This **chaotic topology** is usually too coarse for geometry, but it proves that the direct definition is inhabited on every category and provides an upper bound for generated topologies.

The opposite extreme declares only maximal sieves covering. Mathematically, every presheaf satisfies the resulting sheaf condition because restriction to the maximal local question loses no data. Enlarging a topology adds covering questions and therefore removes presheaves that fail them. The chaotic topology asks every sieve to be sufficient and is correspondingly severe. These extremes make the variance of the order clear: more covers mean fewer sheaves, even though the topology itself is larger as a cover predicate.

> **Formal status — checked.** Evidence `GROTH-TOPOLOGY-SIEVE-LAWS`. The active topology package contains a proposition-valued coverage together with maximality, pullback stability, and local character over ordinary sieves. The maximal sieve and the chaotic topology have direct checked models. This layer neither assumes a global classifier of all sieves nor constructs sheafification.

19.3 Generating The Least Topology

In practice one rarely declares every covering sieve independently. One gives basic covers and closes them under the topology laws. The phrase “the topology generated by these covers” should include two statements:

1. every proposed cover is covering; and
2. no additional covers are accepted except those forced by the three laws.

The active construction makes both statements precise without choosing a syntax of derivation trees. Let

$$G(U, R)$$

be the type of witnesses that R is one of the proposed generating sieves on U. This type need not be a proposition. It may remember which finite family, algebraic certificate, or chart presentation produced the same underlying sieve.

A Grothendieck topology T **accepts** G when every witness in $G(U, R)$ produces evidence that R covers in T. Now define R to cover in the generated topology when it covers in *every* topology accepting G:

$$J_G(U, R) := \prod_{T : \mathrm{GrothTopology}(\mathcal{K})} \big(T \text{ accepts } G\big) \longrightarrow T(U, R).$$

This is the intersection of all acceptable topologies. Intersections inherit the three topology laws pointwise. Every generator belongs because every topology under consideration accepts it. And if T accepts the generators, then J_G is below T: a J_G-cover is, by definition, a cover in T.

The order here is an order of consequences. Write $J \leq T$ when every J-cover is a T-cover. A smaller topology makes fewer local families covering and therefore imposes fewer sheaf conditions. The generated topology is the least element among those that accept the proposals. The chaotic topology accepts every proposal, so the class being intersected is never empty.

Witness-rich generation and proposition-valued coverhood now play distinct roles. Several inhabitants of $G(U, R)$ may explain the same proposed cover in different ways. Acceptance must handle each explanation. Once R belongs to J_G, however, its coverhood is a proposition: downstream sheaf reasoning cannot branch on which derivation happened to be supplied. The construction retains evidence where computation may need it and erases choice where geometry should be invariant.

Theorem 19.1 (least generated topology). Every type-valued family of generating sieves determines a Grothendieck topology J_G. The topology accepts every retained generator and is contained in every Grothendieck topology that accepts them.

> **Formal status — checked.** Evidence `GENERATED-GROTH-TOPOLOGY`. The generated cover predicate is proved proposition-valued, its three topology laws are constructed, generator inclusion computes, and leastness is a whole pointwise comparison of topologies. The construction is

> impredicative: it provides no inductive derivation syntax, induction principle for generation steps, coverhood normalizer, or decision procedure.

The boundary in that note is mathematically significant. An inductive presentation can explain *how* a cover was derived. The intersection presentation proves exactly *which universal property* the generated topology has. For the book's current purpose, leastness is the invariant needed by later geometry. A derivation calculus may still be valuable for automation, but it would be another presentation of the same intended closure, not the definition of a site.

There is no circularity in the universal characterization. One first knows what a Grothendieck topology is from the three laws. One then ranges over all such structures that accept the generators and intersects their cover predicates. The result is certified to satisfy the same laws. This resembles defining a generated algebraic congruence as the intersection of all congruences containing the proposed relations: closure is characterized by leastness before any normal-form algorithm is chosen.

19.4 A Sieve As A Domain Of Local Data

Let R be a sieve on U. The representable presheaf yU contains every probe into U. The sieve selects some of those probes, together with their refinements. To turn that selection into a domain on which a presheaf can be evaluated, form the extension

$$\widehat{R}(V) := \sum_{p : V \to U} R(p).$$

An object of $\widehat{R}(V)$ is a probe $p : V \to U$ together with evidence that it belongs to R. Forgetting the evidence gives a whole presheaf map

$$i_R : \widehat{R} \longrightarrow yU.$$

For a Cat-valued presheaf X, define the category of sections over U in representable form as

$$\mathrm{Sect}_X(U) := \mathrm{Hom}_{\mathrm{Psh}(\mathcal{K})}(yU, X).$$

The ordinary Yoneda lemma identifies this with $X(U)$. Retaining the Hom form has an advantage here: it expresses the local-to-global map without assuming the full Cat-valued Yoneda equivalence as an active theorem.

The category of matching families on R is

$$\mathrm{Match}_X(R) := \mathrm{Hom}_{\mathrm{Psh}(\mathcal{K})}(\widehat{R}, X).$$

Concretely, a matching family assigns an observation in $X(V)$ to every member $p : V \to U$ of the sieve, compatibly with every refinement and with the arrows retained by the Cat-valued coefficients. Compatibility is not a list of equations added afterward. It is the naturality of one presheaf map.

For a cover presented by arrows $u_i : U_i \to U$, this recovers the usual picture. A matching family begins with local elements

$$x_i \in X(U_i).$$

If the pullbacks $U_i \times_U U_j$ exist, the two restrictions of x_i and x_j to the overlap must agree. But pairwise overlap equations are only a presentation of the deeper condition. A sieve also contains longer refinements, morphisms between probes, and covers in categories without the chosen pullbacks. A map $\widehat{R} \to X$ packages compatibility with all of them at once.

In the Cat-valued case, the word “agree” may itself have categorical content. A matching object can carry coherent comparison arrows, and a map between matching objects must respect them. This is why the Hom target above is a category rather than merely a set of families.

Precomposition with i_R restricts a global section to its matching family:

$$\rho_{R,X} : \mathrm{Sect}_X(U) \longrightarrow \mathrm{Match}_X(R), \qquad \rho_{R,X}(x) = x \circ i_R.$$

This formula is the categorical core of descent. It names both the data to be glued and the direction in which global information becomes local.

19.5 Locality And The Sheaf Condition

The presheaf X is **local at** R when the restriction functor $\rho_{R,X}$ is an equivalence. Every matching family then has an amalgamation, and that amalgamation is unique at the appropriate categorical level.

For a set-valued presheaf, the statement specializes to the familiar pair:

- compatible local elements have a global amalgamation; and
- two global elements with the same local restrictions are equal.

The second clause is often called **separatedness**, while the first is the existence part of gluing. Their conjunction says that restriction is a bijection. For Cat-valued presheaves, a categorical equivalence replaces that bijection: it controls objects, maps, and their inverse laws together. Merely asking for essential surjectivity would give amalgamations without the full uniqueness and functoriality required here.

For a Cat-valued presheaf, the equivalence also retains maps between local families and the higher coherence carried by the Hom categories. Replacing the equivalence by a mere objectwise existence statement would lose this structure.

Given a topology J, say that X is **topology-local** when it is local at every J-covering sieve. This is the sheaf condition in the form used by the current categorical development:

$$\prod_{U}\prod_{R}\prod_{c:J(U,R)}\mathrm{IsEquiv}(\rho_{R,X}).$$

The cover witness c indexes the condition but does not become computational data: coverhood is a proposition. What matters computationally is the fixed forward map $\rho_{R,X}$ and its selected inverse behavior.

A topology for which every representable presheaf is local is called **subcanonical**. This is a property of the topology, not part of the definition of a site. It says that the original category embeds into its sheaf semantics without changing the representable probes. The Zariski coverage used later is intended to have this geometric behavior, but no arbitrary Grothendieck topology is subcanonical merely because its three laws hold.

> **Formal status — checked.** Evidence `SIEVE-MATCHING-LOCALITY`. The active construction extends an ordinary sieve to a whole presheaf, includes it in the representable, defines matching and section Hom categories, and makes restriction exact precomposition by that inclusion. Locality at one sieve and simultaneous locality over a topology are active fixed-forward equivalence predicates. No reflector or identification with a separate rigid sheaf category follows at this layer.

This formulation makes the categorical-semantics point particularly direct. There is no need to hide the site behind an abstract modal operator before locality can compute. The actual sieve extension, representable, Hom categories, and restriction functor are objects of the inner functorial type theory. The outer logical framework checks and reduces their selected observations. An internal modal language could summarize this behavior, but the categorical semantics already supplies an executable statement of descent.

19.6 Varying The Cover Question

A single cover is not enough for sheafification. Covers change when their base object is pulled back, and a construction that glues only after all choices have been externalized will carry a growing burden of naturality equations.

It is therefore useful to package an **eligible cover question** as

$$q = (U, R, c),$$

where R is a sieve on U and c says that R covers. These questions themselves form a category. For a fixed presheaf X, matching categories and section categories then vary as Cat-valued families over the whole question category, and restriction becomes one displayed functor between them.

An arrow between questions includes a change of base $p : V \to U$ and the pulled-back covering question over V. The topology's stability law supplies its eligibility. On local data, the arrow acts by reindexing along the canonical map from the pulled-back sieve extension to the original one. Thus the same base change that transported membership in Chapter 18 now transports the entire descent problem.

The gain is conceptual as well as formal. Pulling back a cover question moves its sieve extension, representable, matching data, sections, and restriction map together. Naturality belongs to the single varying construction. It is not reintroduced as a separate square for every cover chosen later.

The active development realizes this whole varying layer and checks that, at a literal question, the displayed restriction is exactly the precomposition map $\rho_{R,X}$ above. It deliberately stops before supplying glue. A matching family is a well-formed local answer; it is not yet an amalgamation.

19.7 The Construction Still Missing

A sheaf is a presheaf that already solves every covering question. A sheafification must start with an arbitrary presheaf P and construct a new one aP together with a unit

$$\eta_P : P \longrightarrow aP$$

that is universal among maps from P to local presheaves. Merely restating the sheaf condition does not build aP. Nor does choosing an inverse to $\rho_{R,X}$ for an already-local X explain how to add missing amalgamations coherently.

Universality asks for more than a local output. For every topology-local X, precomposition with the unit should give an equivalence

$$\mathrm{Hom}(aP, X) \simeq \mathrm{Hom}(P, X).$$

Thus a map out of P into a local target extends across the completion, and the extension is unique at the whole Hom-category level. This property is what makes sheafification a reflector rather than one arbitrary repair of local data.

The direct construction pursued next treats covers as questions whose solutions may be freely adjoined. It needs a return constructor for old data, a whole glue operation for matching families, a silent law saying that gluing the restriction of an existing section changes nothing, and a recursor expressing the universal property. The varying-cover architecture of this chapter is what lets those constructors be stated once over all eligible questions.

Thus the progression is exact:

$$\text{sieve} \longrightarrow \text{covering sieve} \longrightarrow \text{matching family} \longrightarrow \text{amalgamation} \longrightarrow \text{sheafification}.$$

The first three stages are now in view. The next chapter constructs the last two for the selected Cat-valued setting and then asks for their whole universal property.

20. Sheafification By Cover Completion

A sheaf knows how to answer every covering question. An arbitrary presheaf may answer some and fail others. Sheafification is the passage from the latter to the former, but that phrase conceals two different demands. We must add the missing global sections, and we must add no information beyond what every sheaf is already forced to accept.

The first demand is constructive. Given a matching family on a covering sieve, adjoin an amalgamation. Repeat, because the newly adjoined data may itself occur in another matching family. The second demand is universal. A map from the original presheaf to a sheaf must extend uniquely across the completion. Together they characterize a reflector from presheaves to sheaves.

This chapter carries out that program directly in categorical semantics. The site, covering sieves, representables, matching categories, and restriction functors of Chapters 18 and 19 remain visible. They are not first encoded into a modal object language. The construction instead gives a categorical higher-inductive signature with three memorable operations:

$$\mathrm{return}, \qquad \mathrm{glue}, \qquad \mathrm{silent}.$$

Return preserves old data. Glue supplies answers to covering questions. Silent says that asking a question whose answer is already known has no observable effect. A recursor then proves that this is the free such completion, and the free completion assembles into the desired Cat-valued sheafification functor.

20.1 From A Condition To A Construction

Fix a category $\mathcal{K}$, a Grothendieck topology J, and a Cat-valued presheaf P. Write $a_J P$, or simply aP, for the presheaf to be completed. There must first be a whole presheaf map

$$\eta_P : P \longrightarrow aP.$$

This is **return**. It does not assert that P was already local. It embeds the observations we began with among the observations generated by the completion.

Now take an eligible covering question $q = (U, R, c)$. Recall the categories

$$\mathrm{Match}_{aP}(R) = \mathrm{Hom}(\widehat{R}, aP), \qquad \mathrm{Sect}_{aP}(U) = \mathrm{Hom}(yU, aP).$$

The second constructor is a functor

$$\mathrm{glue}_q : \mathrm{Match}_{aP}(R) \longrightarrow \mathrm{Sect}_{aP}(U).$$

It turns compatible local data into a global section. Notice the recursion: the matching family already takes values in aP, not merely in P. A newly glued section can therefore participate in a later matching family. This is why one application of a nonrecursive repair operation would not express the

truction being made here.

The functorial type is equally important. In a set-valued account, glue may look like a function between sets. With Cat-valued coefficients, matching families have arrows between them, and amalgamation must transport those arrows coherently. The constructor acts on the entire matching category. It does not merely select an object-level amalgamation and leave its behavior on maps to be reconstructed later.

Finally let

$$\rho_q : \mathrm{Sect}_{aP}(U) \longrightarrow \mathrm{Match}_{aP}(R)$$

be restriction. The **silent** constructor is the path

$$\mathrm{glue}_q \circ \rho_q = \mathrm{id}_{\mathrm{Sect}_{aP}(U)} \,. \tag{20.1}$$

If a global section is restricted to a cover and then glued again, nothing changes. The cover was consulted, but its answer was ignored because the global section was already present.

This return/glue/silent pattern is conceptually adapted from Pierre-Marie Pédrot's computational account of free sheaves in *Pursuing Shtuck*. Pédrot describes the last equation as the erasure of a silent transition: a branching computation that does not depend on its returned witness should be indistinguishable from the unbranched computation. Here the pattern is placed directly over actual covering-sieve questions. Its branches are whole matching maps $\widehat{R} \to aP$, and its output is a whole section $yU \to aP$.

For a concrete picture, suppose a cover of U is presented by two probes

$$u_0 : U_0 \longrightarrow U, \qquad u_1 : U_1 \longrightarrow U.$$

A matching family contains local observations x_0 and x_1 whose restrictions agree wherever the two probes meet, together with compatibility under every further refinement in the generated sieve. Glue adjoins a section x over U with those local observations. If x_0 or x_1 was itself produced by gluing a finer cover, the recursive constructor accepts it without flattening the history into a chosen list of basic pieces. If the entire matching family came by restricting an old x, silent identifies the newly adjoined amalgamation with x.

That last clause is what prevents completion from accumulating duplicate global answers. Without silent, one could glue the same restricted section again and again, building formally distinct trees that encode no new local information. With an indiscriminate equation saying that every two gluings are equal, on the other hand, one would destroy genuine distinctions between matching families. Equation (20.1) is narrow: it removes precisely the detour that starts from an existing global section, passes through its restrictions, and returns by glue.

Classical accounts often construct associated sheaves by a plus operation on matching families and then iterate that operation. Such a presentation is valuable, especially when quotient objects and their exactness properties are already available. The direct completion takes a different route. It exp-

oses the free generators and their necessary path at the outset, so recursion can observe return and glue computationally. No comparison theorem with the classical plus construction is claimed here; the point is that the universal property can be reached without choosing equivalence-class representatives as the operational presentation.

The signature should not be mistaken for an algorithm that searches the site for covers. A glue constructor is available after an eligible question and its cover evidence have been supplied. Coverhood itself may be undecidable, and the generated topology of Chapter 19 provides a universal predicate, not an enumeration. Completion says coherently what to do with every admitted question; it does not promise to discover those questions by normalization.

Nor does the construction privilege the basic generators from which a topology may have arisen. Once the least topology has been formed, a derived cover is as eligible as a generating one. This is essential for locality: pullbacks and composites of covers enter the proof even when they were not in the original presentation. The witness-rich layer may still help a program produce cover evidence, but the free object depends on the invariant covering-sieve predicate.

The terminology “higher-inductive” refers to the shape of this presentation. There is a point-like return constructor, a recursive glue constructor, and a path constructor imposing the silent equation. It does not mean that the entire surrounding theory has been replaced by ordinary HoTT syntax. The objects remain categories and presheaves in functorial type theory, while the outer logical framework supplies the primitive signature, rewrite behavior, and checked equality evidence.

20.2 One Glue Operation Over All Questions

Writing glue_q separately for each cover is useful on paper, but it hides the hardest coherence. A cover can be pulled back along a probe $p : V \to U$. The matching family moves to the pulled-back sieve, the proposed amalgamation moves to V, and the two orders of those operations must agree.

Chapter 19 packaged all eligible questions into a category $\mathcal{Q}_J$. Over it, matching categories and section categories form two Cat-valued families,

$$\mathrm{Match}_{aP}, \mathrm{Sect}_{aP} : \mathcal{Q}_J \longrightarrow \mathbf{Cat}.$$

The collection of glue operations is retained as one displayed functor

$$\mathrm{glue}_{\mathrm{all}} : \mathrm{Match}_{aP} \longrightarrow \mathrm{Sect}_{aP} \quad \text{over } \mathcal{Q}_J.$$

Its component at q is glue_q. Its action along arrows of $\mathcal{Q}_J$ is precisely the compatibility of glue with change of cover question. For the canonical pullback of q along p, naturality gives the square summarized by

$$\mathrm{Sect}[p] \circ \mathrm{glue}_q = \mathrm{glue}_{p^* q} \circ \mathrm{Match}[p]. \tag{20.2}$$

Likewise, silent is not stored as an unrelated equation for every U, R, and cover witness. It is one path of displayed endofunctors:

$$\mathrm{glue}_{\mathrm{all}} \circ \rho_{\mathrm{all}} = \mathrm{id}_{\mathrm{Sect}_{aP}} . \tag{20.3}$$

Equation (20.1) is its component at a literal cover. The one whole path also controls arrows between sections and reindexing between cover questions. This is a mathematical economy: coherence is owned at the level where the varying construction lives, rather than repeated as a growing list of component squares.

> **Formal status — checked categorical-HIT boundary.** Evidence `DIRECT-COVER-COMPLETION-HIT`. For every site and Cat-valued presheaf, the active signature provides the completion presheaf, whole unit, whole cover-indexed glue functor, and whole silent path. Pullback compatibility is obtained from displayed-functor naturality, and the result is packaged as an internal direct-cover sheaf structure. The completion and path constructors are primitive at this boundary; locality and the reflector are consequences proved in the following stages, not fields silently included here.

The silent law has an intentional direction. It says that gluing the restriction of a section recovers the section. It does not initially say that restricting the glue of a matching family recovers the family. Supplying both directions as constructors would make locality true by declaration. Supplying only (20.3) leaves room for the geometry of sieves and pullback to prove the opposite composite.

Nor is silent installed as a runtime simplification that erases every visible glue term. It is internal equality evidence between whole functors. Return and recursive glue have selected computational rules under the recursor, while the higher silent coherence remains a path. This distinction avoids choosing one side of every sheaf equation as a universal normal form.

20.3 Why The Completion Is Local

To prove that aP is a sheaf, fix a covering sieve R on U. Equation (20.1) already gives

$$\mathrm{glue}_R \circ \rho_R = \mathrm{id}_{\mathrm{Sect}_{aP}(U)} .$$

The missing law is

$$\rho_R \circ \mathrm{glue}_R = \mathrm{id}_{\mathrm{Match}_{aP}(R)} . \tag{20.4}$$

Take a matching family $m : \widehat{R} \to aP$ and inspect it at a member $p : V \to U$ of R. Because R is a sieve, every refinement of p still belongs to R. Consequently the pullback p^*R is maximal on V: its identity belongs, and then so does every probe into V.

The glue operation is natural under this pullback. Thus the restriction to V of the global section $\mathrm{glue}_R(m)$ agrees with gluing the matching data transported to p^*R. But a matching family on the maximal sieve is already the restriction of the section visible at V. The silent law on that pulled-back question removes the redundant glue. At the retained member p, the result is exactly the original component of m.

This calculation works at every member of the sieve and respects every arrow between members. The pointwise paths therefore assemble into a whole transformation

$$\rho_R \circ \mathrm{glue}_R \Longrightarrow \mathrm{id}_{\mathrm{Match}_{aP}(R)},$$

and the strict pointwise-to-whole principle closes it to (20.4). No new naturality square is assumed at this stage. The necessary compatibility came from the one displayed glue functor in the categorical-HIT signature.

Theorem 20.1 (locality of cover completion). For every Cat-valued presheaf P on the site $(\mathcal{K}, J)$, the direct cover completion aP is local at every J-covering sieve. Hence aP is a Cat-valued sheaf.

> **Formal status — checked.** Evidence `DIRECT-COVER-COMPLETION-LOCALITY`. The active derivation constructs the restriction-after-glue path from cover membership, canonical pullback, whole glue naturality, and silent; combines it with the primitive glue-after-restriction path; and produces the existing two-sided fixed-forward locality interface at every eligible question and over the whole topology. It assumes neither generic functor extensionality nor a category-of-elements retraction.

This proof explains why the chosen constructor set is not merely mnemonic. Return begins the free object. Glue supplies a candidate inverse to restriction. Silent gives one inverse law. Sieve stability and change of stage force the other. The sheaf condition emerges from the interaction of the constructors with the site, not from attaching the word “sheaf” to the result.

Existence and uniqueness are therefore not two unrelated postulates. Glue is the existence mechanism. The two inverse laws say that its output is exactly the global datum classified by the matching family and that an already-global datum is unchanged. In the Cat-valued setting those laws also control arrows between families, so “unique” means unique with the categorical coherence visible to the Hom categories, rather than merely unique after forgetting all maps. That coherence is part of the theorem, not expository shorthand.

20.4 The Recursor

Locality shows that aP lands in the right class of objects. Freeness asks whether it lands there in the right way.

First consider a Cat-valued presheaf Y equipped with its own coherent glue and silent operations. Given a seed map

$$f : P \longrightarrow Y,$$

the recursor produces

$$\mathrm{rec}_Y(f) : aP \longrightarrow Y.$$

It is determined by three clauses. On returned data it extends the seed:

$$\mathrm{rec}_Y(f) \circ \eta_P = f. \tag{20.5}$$

On a recursive glue, it first maps the local family into Y and then uses Y's glue. On the silent path, those two ways of passing through glue agree with the silent path already carried by Y. The first two clauses describe objects and recursive computation; the third is the higher coherence that makes the result a map of the complete algebraic structures.

A topology-local presheaf has canonical such structure. Its restriction functor is an equivalence, so its chosen inverse supplies glue, and one inverse law supplies silent. Thus every sheaf Y is a legitimate target of the recursor without asking a reader to choose fresh gluing operations cover by cover.

The objectwise statement "every seed extends" is still weaker than a reflective universal property. Maps between seed maps must extend too, and the extension must be inverse to restriction on the whole Hom category. For a local target Y, precomposition with the unit is a functor

$$\eta_P^* : \mathrm{Hom}(aP, Y) \longrightarrow \mathrm{Hom}(P, Y), \qquad h \longmapsto h \circ \eta_P.$$

Recursion varies functorially in the seed and gives a functor in the opposite direction. Equation (20.5) is the beta law

$$\eta_P^* \circ \mathrm{rec}_Y = \mathrm{id}_{\mathrm{Hom}(P,Y)}\,.$$

The categorical-HIT uniqueness law gives the eta law

$$\mathrm{rec}_Y \circ \eta_P^* = \mathrm{id}_{\mathrm{Hom}(aP,Y)}\,.$$

These are equalities of whole functors between Hom categories. They include the action on arrows between presheaf maps, not only a bijection between object-level maps.

Theorem 20.2 (free local completion). If Y is topology-local, then precomposition with η_P is an omega-equivalence

$$\mathrm{Hom}(aP, Y) \simeq \mathrm{Hom}(P, Y). \tag{20.6}$$

> **Formal status — checked.** Evidence `DIRECT-COVER-COMPLETION-UNIVERSALITY`. The recursor extends a whole seed map, computes on return and recursive glue, and carries explicit coherence for the silent path. It varies functorially in the seed. At a topology-local target, whole beta and eta laws exhibit unit precomposition as an omega-equivalence of complete Hom categories. The eta law is scoped to local targets; no uniqueness theorem is asserted for an arbitrary, independently chosen one-sided glue algebra.

Equation (20.6) is the point at which completion becomes sheafification. Many objects might be made local by adding arbitrary data. Only a free local object has this mapping property. The unit remembers how the input enters, and every map to a local target factors through it in the uniquely coherent way.

20.5 From The Universal Property To A Reflector

Let $\mathrm{Sh}_{\mathbf{Cat}}(\mathcal{K}, J)$ denote the category whose objects are Cat-valued presheaves equipped with topology-locality evidence and whose maps are their underlying presheaf maps. There is an inclusion

$$i : \mathrm{Sh}_{\mathbf{Cat}}(\mathcal{K}, J) \longrightarrow \mathrm{Psh}_{\mathbf{Cat}}(\mathcal{K}).$$

Theorem 20.1 lets $P \mapsto aP$ land in the sheaf category. The recursor makes this assignment functorial on presheaf maps: given $f : P \to Q$, compose it with η_Q and extend the resulting seed $P \to aQ$ across aP. Thus there is a functor

$$a : \mathrm{Psh}_{\mathbf{Cat}}(\mathcal{K}) \longrightarrow \mathrm{Sh}_{\mathbf{Cat}}(\mathcal{K}, J).$$

The Hom equivalence (20.6), now read with $Y = iS$ for a sheaf S, is exactly the adjunction

$$a \dashv i.$$

Its unit at P is the return map η_P. Its counit at a sheaf S is the recursor from the identity seed on the underlying presheaf:

$$\varepsilon_S : a(iS) \longrightarrow S.$$

The return beta law shows

$$\varepsilon_S \circ \eta_{iS} = \mathrm{id}_{iS},$$

and the local-target uniqueness law gives the other cancellation. Hence the counit is an equivalence. Sheafification does not alter a sheaf except up to the native categorical equivalence appropriate to this development.

This also explains the familiar idempotent flavor of sheafification. Applying a to an arbitrary P produces a local object. Applying a again therefore meets an object already in the reflective subcategory, and the counit compares $a(aP)$ back to aP by an equivalence. The statement is not that the two presheaves collapse by raw syntax. Reflective idempotence is expressed by the adjunction and its invertible counit, at the same categorical strength as the rest of the theory.

Theorem 20.3 (Cat-valued sheafification reflector). For every site $(\mathcal{K}, J)$, direct cover completion defines a left adjoint to the inclusion of topology-local Cat-valued presheaves. The counit on every sheaf is an omega-equivalence, so this adjunction is reflective.

> **Formal status — checked.** Evidence `CAT-VALUED-SHEAFIFICATION-REFLECTOR`. The active construction realizes the Cat-valued sheaf facade as topology-local presheaves, constructs the inclusion and completion functors, supplies their whole adjunction, reduces its unit to return and its

> counit to recursion from the identity seed, and proves the two counit cancellations and reflector capability. This is a fixed-site Cat-valued result. It does not yet supply arbitrary coefficient categories, a commutative-ring lift, left exactness, or base-change comparison.

20.6 Categorical Semantics As An Executable Language

There are two legitimate ways to speak about sheafification in type theory. One may design an internal language with a modality whose semantics is a sheaf reflector. Or one may express the categorical semantics itself in a formal language rich enough to compute with categories, presheaves, sieves, and universal properties. This book follows the second route.

The distinction is about where the formalization lives, not about whether it is internal. Within functorial type theory, $\widehat{R}$, yU, matching categories, section categories, glue, and the adjunction are genuine objects and arrows. They are manipulated from inside the categorical theory. That theory in turn lives inside an outer logical framework—Lambdapi in the active oracle, and increasingly the explicit emdash Core checked by TypeScript. The outer layer supplies binders, conversion, rewrite rules, unification, and proof checking. Selected categorical observations can therefore reduce and be compared without inventing a second modal surface language first.

The division of labor can be read from one glued section. Its matching family and section are inner categorical objects. The fact that glue is natural in a change of covering question is inner functorial structure. The rule saying how a recursor acts on return or on recursive glue belongs to the outer computation theory. The equality witnessing silent is again an inner path, whose formation and use are certified by the outer checker. Neither level is dispensable, but neither needs to masquerade as the other.

This is **relative internality**. The sheaf construction is internal to the categorical semantics and executable relative to the surrounding logical framework. It is neither an informal metatheoretic diagram nor a claim that all categorical equalities are definitional. Return has a narrow computation rule under recursion; recursive glue has another; silent remains first-class equality evidence; the Hom equivalence packages explicit inverse data.

An internal modal language may still be desirable. It can hide repeated site parameters, support concise local reasoning, or make sheaf semantics available to a different class of programs. What the direct construction shows is that such a language is not a prerequisite for computational sheafification. The ordinary site can remain ordinary, the sieves can remain actual sieves, and the categorical universal property can itself be checked.

This viewpoint also clarifies the comparison with Pédrot. His return/glue/silent pattern reveals the computational content that a purely reflective statement can conceal. Emdash preserves that pattern but moves its indices outward to the category of covering questions and its values upward to categories. The result is not a transcription of his type theory. It is a categorical realization of the same free-construction idea, with whole naturality and whole Hom universality made explicit.

20.7 What Has And Has Not Been Completed

At this point the word *sheafification* is justified for Cat-valued presheaves on a fixed site. The result is local, functorial, left adjoint to inclusion, and reflective. None of those adjectives should be silently moved to a different coefficient category.

In particular, a commutative-ring-valued presheaf carries operations that must survive completion and continue to satisfy their equations. A reflector on underlying Cat-valued presheaves does not by itself construct those lifted operations. Nor has the present theorem established left exactness, preservation of finite limits, compatibility with change of site, or a general comparison with classical plus-construction sheafification. Those are separate mathematical obligations.

There is also no claim here about sheafifying every universe of coefficients uniformly. The value category has been fixed to $\mathbf{Cat}$, and the sheaf facade has been realized concretely as a presheaf paired with locality evidence. Moving to sets, groupoids with a different equality policy, commutative rings, or a universe of small categories requires a corresponding lifting theorem. The fixed-site theorem is substantial precisely because it keeps that quantification honest.

The next chapter therefore steps sideways before moving further into geometry. It develops commutative algebra by universal property: products, localizations, and the evidence that a ring map sends a chosen element to a unit. That language will let the invertibility sieve $D_U(s)$ of Chapter 18 control affine charts without pretending that Cat-valued sheafification has already manufactured a structure sheaf of rings.

The local-to-global spiral now has its completion step:

$$\text{probe} \longrightarrow \text{sieve} \longrightarrow \text{cover} \longrightarrow \text{matching family} \longrightarrow \text{free glue} \longrightarrow \text{reflective sheaf}.$$

What comes next is the algebra that turns these local questions into affine geometry.

21. Commutative Algebra By Universal Property

Algebraic geometry begins with rings, but it rarely cares how a ring was manufactured. A polynomial algebra may be presented by finite expressions; a localization may be presented by fractions; a quotient may be presented by equivalence classes. These descriptions are indispensable for hand calculation. They are not, however, the invariant meaning of the objects they describe. Change the representation and the same algebra survives.

This matters acutely in a computational foundation. A convenient syntax can make examples reduce, yet it can also make every later construction depend on accidental choices of normal form. Conversely, a universal property can be stated so weakly that it says merely that a map exists, leaving no usable factor and no uniqueness with which to compare two constructions. The middle course taken here is to make the universal property itself data of the theory. The space of admissible factors is not merely inhabited: it is contractible. It therefore has a selected center and a path from every competitor to that center.

That principle gives a representation-free but still computational account of the algebra needed by the geometry ahead. Commutative rings have set-valued carriers and structured maps. Finite unit-ideal certificates carry the algebraic content of basic-open covers. Polynomial algebras are free extensions, and localizations are initial ways of making a chosen element invertible. Unit, zero, and idempotent localizations then show that the interface is not empty formalism. Finally, uniqueness alone constructs the comparison between localization at a product and localization in two stages.

21.1 Rings As Structured Set-Carriers

A commutative ring R consists first of a set $|R|$, with operations

$$0_R, 1_R : |R|, \qquad +_R, \cdot_R : |R| \times |R| \longrightarrow |R|, \qquad -_R : |R| \longrightarrow |R|,$$

and then the usual associativity, commutativity, unit, inverse, and distributivity laws. Calling the carrier a set is not decorative. It says that equality proofs between ring elements carry no higher ambiguity. The algebraic laws are consequently properties of the selected operations rather than new layers of structure that can vary above a fixed equality.

The definition does **not** require $0_R \neq 1_R$. This convention retains the zero ring, whose carrier has one element and whose two distinguished constants coincide. Excluding it would make some later universal statements awkward or false. Localization at zero, for example, naturally lands in the zero ring: forcing zero to be a unit forces every element to coincide.

A map $h : R \to S$ is more than a function of carriers. It comes with the five preservation laws

$$h(0) = 0, \quad h(1) = 1, \quad h(x + y) = h(x) + h(y), \quad h(-x) = -h(x), \quad h(xy) = h(x)h(y).$$

Because the target carrier is a set, this preservation evidence is proposition-valued. Two structured maps are equal once their carrier functions agree pointwise. Identities and composites again preserve all five operations, so commutative rings and their structured maps form a one-category, written $\mathbf{CRing}$.

This choice of morphism is part of the mathematics. A bare function between carriers forgets the equations that make substitution legitimate. A structured map carries those equations once, after which every derived finite sum, product, unit witness, and factorization can be transported through it. Later base-change arguments therefore do not reopen the ring laws term by term. They use the fact that the arrow already lives in $\mathbf{CRing}$.

Sethood plays a second role here. A structured map contains both a function and proofs that it respects the operations. If those proofs carried independent higher data, pointwise equality of the functions would not settle equality of the complete maps. Since equality in the target ring is proposition-valued, preservation proofs create no such ambiguity. This is why the ordinary-looking extensionality principle is available without flattening the ambient functorial type theory.

There is a useful restraint here. Extensionality for maps is not a global principle saying that any two ring packages with equivalent carriers are equal. No structure identity principle for arbitrary rings is being smuggled in. The objects remain chosen carrier-operation-law packages; the homs have the extensionality required to calculate with them.

> **Formal status — checked.** Evidence `COMM-RING-STRUCTURED-CATEGORY`. The active algebra packages a set-valued carrier, operations, and ring laws; admits the zero ring; packages operation-preserving carrier functions as structured maps; proves their extensionality; and assembles them into the one-category $\mathbf{CRing}$. No inequality $0 \neq 1$, global equality of ring packages, or general structure identity theorem is assumed.

Several small models keep this abstraction honest. The zero ring computes on the one-point carrier. The two-element ring $\mathbb{F}_2$ computes on the booleans, with exclusive-or as addition and conjunction as multiplication. If R and S are rings, their cartesian carrier $|R| \times |S|$ has componentwise operations and hence a ring structure $R \times S$. A pair of maps induces a componentwise map between such products, and these maps obey whole identity and composition paths.

This last construction should not be overread. We have constructed a product ring and functorial action on paired maps. We have not yet selected projections and proved the entire categorical product universal property inside $\mathbf{CRing}$. The componentwise model is exactly what the later split-idempotent calculation needs, and no stronger theorem is required for that calculation.

21.2 Finite Certificates For Covering

The first geometric-looking datum is still entirely algebraic. Let $f_1, \ldots, f_n$ be elements of a ring R. They are **unimodular** when one retains coefficients $a_1, \ldots, a_n$ and an equality

$$a_1 f_1 + \cdots + a_n f_n = 1. \tag{21.1}$$

The coefficients are important. The bare proposition that the f_i generate the unit ideal forgets how that fact was witnessed. Equation (21.1), by contrast, is finite input that can be transported, inspected, and used in a construction. The family together with its coefficients and equality is a **unimodular presentation**.

For two generators the picture is especially sharp. From $af + bg = 1$, any map out of R that makes both f and g vanish would also make 1 vanish. Unless the target has collapsed to the zero ring, that is impossible. Geometrically, no nontrivial affine point can lie outside both regions of invertibility at once. The equation is already the finite algebraic shadow of a covering statement, even though the regions and their topology remain to be constructed.

Every ring map $h : R \to S$ transports such a presentation. Preservation of finite sums and products turns (21.1) into

$$h(a_1)h(f_1) + \cdots + h(a_n)h(f_n) = 1$$

in S. Thus a certificate does not merely remain true after base change; its chosen witnesses move pointwise with the generators. The singleton family $[1]$ has the canonical certificate $1 \cdot 1 = 1$, and a two-element helper packages the familiar equation $af + bg = 1$.

Why call this cover data? In ordinary affine geometry, (21.1) says that the basic opens $D(f_i)$ cover the whole spectrum. But that geometric conclusion uses meanings that have not yet been introduced in this spiral: a spectrum, basic opens, a topology, and the relationship between unit-ideal generation and coverage. The present layer records precisely the algebraic premise from which those notions will be built. It does not call a finite family a cover by fiat.

There is a second reason to retain the presentation rather than immediately truncate it to a proposition. Different coefficient families may witness the same unit-ideal equation. The classifier of presentations is set-valued, not claimed proposition-valued. Later invariant constructions may forget that choice; earlier computational constructions are allowed to consume it. This separation between witness-rich input and invariant output is the same pattern used for generated topologies in Chapter 19.

> **Formal status — checked algebraic boundary.** Evidence `FINITE-UNIMODULAR-COVER-DATA`. Finite sums and dot products compute on visible families, structured ring maps preserve them, and a finite Zariski presentation retains generators, coefficients, and their unit-ideal law. Such presentations are stable under structured base change and include the singleton $[1]$ and binary $af + bg = 1$ cases. At this layer they are not yet basic opens, covering sieves, localization families, or a Grothendieck topology.

21.3 Free Variables Without A Syntax Of Polynomials

Fix a ring R and a set or groupoid X of variable names. A polynomial algebra on X should contain a base map

$$\iota : R \longrightarrow P$$

and a valuation $v : X \to |P|$. Its meaning is determined by what happens when the variables are interpreted elsewhere. Given a ring S, a base map $h : R \to S$, and a valuation $u : X \to |S|$, consider structured maps $k : P \to S$ satisfying

$$k \circ \iota = h, \qquad k(v(x)) = u(x) \quad (x : X). \tag{21.2}$$

Both equations are retained pointwise, and the complete factor consists of k together with those agreements. The universal property says that the classifier of such factors is contractible for every S, h, and u. There is therefore one coherently selected extension, and every rival extension is equal to it as a structured map with its agreement evidence.

Contractibility is stronger than the phrase “there exists a unique map” when that phrase is read externally. It gives a center of the complete factor classifier and, internally, a path from every other inhabitant to that center. The center can be projected whenever an actual extension is needed; the contraction can be invoked whenever two independently constructed extensions must agree. Because the classifier retains the equations in (21.2), uniqueness does not forget that the comparison lies over R and has the prescribed values on variables.

This is the familiar freeness of $R[X]$, but it does not select a representation of its elements. There is no list of monomials, finitely supported coefficient function, inductive expression grammar, quotient by the ring laws, or preferred normalization order. Those are possible models of the universal property, not fields of the property itself.

The omission is not hostility to syntax. A concrete evaluator might sensibly represent a polynomial by normalized coefficient data, and a parser might let a reader write $x^2 + 2x + 1$. What matters is the direction of dependence. Such a representation should prove that it satisfies (21.2); the geometry should then consume the universal property. The later theory is insulated from whether one implementation uses Horner forms, sparse monomials, or an external computer-algebra package.

For a single variable t, this says that a map out of a supplied $R[t]$ is determined by two observations: what it does to coefficients and where it sends t. Familiar evaluation is recovered by choosing the target value of t. For many variables the same sentence is indexed by X, with no need to choose an ordering at the universal boundary. The interface states exactly what symbolic substitution is meant to accomplish while declining to legislate how symbols are stored.

There is already a closed sanity check. When X is empty, there is no variable data to choose. The ring R itself, with the identity base map, satisfies the polynomial universal property: every $h : R \to S$ is its own unique extension. This case exercises the complete factor classifier and its contractibility, not merely the formation of a record.

Theorem 21.1 (universal polynomial extension). A supplied polynomial algebra package on (R, X) classifies extensions of every base map and valuation by a contractible factor space. For the empty variable classifier, the identity extension on R supplies a checked model.

> **Formal status — checked interface and closed model.** Evidence `COMM-RING-POLYNOMIAL-UNIVERSALITY`. The active universal property retains base and variable agreements and proves the complete extension space contractible. The empty-variable identity model is checked. No construction of a polynomial algebra for every R and X, concrete monomial syntax, quotient presentation, normalization theorem, runtime rule, or package uniqueness theorem is claimed.

21.4 Making One Element Invertible

Let $f \in R$. A localization of R at f begins with a ring L and a map

$$\ell : R \longrightarrow L$$

for which $\ell(f)$ is a unit. Unit evidence is explicit: it consists of an inverse y and an equality $\ell(f)y = 1$. Commutativity proves that the inverse is unique, and sethood of the carrier proves that the entire evidence is a proposition. Asking for a unit therefore does not introduce a meaningful choice of inverse into the geometry.

The uniqueness calculation is elementary but instructive. If y and z are both inverses to x, then

$$y = y \cdot 1 = y(xz) = (yx)z = (xy)z = 1 \cdot z = z.$$

Associativity and commutativity do the algebraic work; sethood then says that the displayed equality has no further choices. The predicate "x is invertible" may consequently be used as the fibre of an ordinary sieve, as in Chapter 18, rather than as a higher coefficient carrying distinct inverse witnesses.

The universal property considers any map $h : R \to S$ for which $h(f)$ is a unit. A factor is a structured map $k : L \to S$ together with the pointwise triangle

$$k(\ell(x)) = h(x) \qquad (x \in R). \tag{21.3}$$

The localization property says that this factor space is contractible. We may write the selected target suggestively as $R[1/f]$, while remembering that the notation names the role of L, not a fraction grammar inside its carrier. Then (21.3) is the invariant content of the usual substitution

$$\frac{x}{f^n} \longmapsto h(x)h(f)^{-n}.$$

The displayed fraction explains the classical formula; it is not used to define the map. Contractibility supplies the map and all comparisons between maps satisfying the same triangle without choosing numerators, denominators, or exponents.

Admissibility belongs to the target map h. The localization package does not choose, for every ring in the universe, whether the image of f is a unit. Rather, when a caller supplies unit evidence, the universal property returns the unique factor. This keeps constructive content visible: testing invertibility

may be undecidable, but using an explicit proof of invertibility is computationally straightforward.

This stronger uniqueness is what makes the interface computationally useful. From a contractible factor space one projects a center, hence an actual map $R[1/f] \to S$. Given another construction with the same universal property, one obtains maps in both directions. Their composites and the identities are competitors in suitable factor spaces, so uniqueness produces the inverse laws. The universal property is thus a source of programs and equations, not an after-the-fact slogan attached to an opaque object.

It also separates existence from characterization. A **localization package** contains a chosen L, a chosen structure map, its unit evidence, and the contractible factorization theorem. The general interface says what any such package does. It does not yet construct a package for every pair (R, f). That remaining existence problem can be solved by a fraction model, by a quotient, by a suitable higher-inductive construction, or by importing a certified algebra library, without changing the consumers of localization.

> **Formal status — checked interface.** Evidence `COMM-RING-LOCALIZATION-UNIVERSALITY`. Unit evidence is proposition-valued, and a supplied localization package inverts the chosen element and gives a contractible space of structured factors through every admissible target map. The factor retains a whole ring map and its pointwise triangle. No general existence theorem for arbitrary (R, f), fraction or power representation, quotient syntax, or equality of arbitrary localization packages is asserted.

21.5 Three Localizations One Can See

The abstract interface has concrete edges where no fraction construction is needed.

First suppose f is already a unit in R. The identity map $R \to R$ is a localization at f. Every admissible map $h : R \to S$ factors through the identity by h itself, and extensionality makes that factor unique. In particular, every ring has a canonical identity localization at 1.

At the opposite extreme, localize at 0. If $h : R \to S$ sends zero to a unit, then 0_S is invertible. Since a ring map preserves zero, one obtains $0_S = 1_S$, and hence every element of S equals zero. The target is contractible as a carrier. It follows that the unique map from R to the zero ring has the localization property at 0. The zero ring is not a nuisance case patched into the theory; it is the correct universal endpoint.

The third case is more revealing. Let $e \in R$ be idempotent, so $e^2 = e$. Consider the fixed-image carrier

$$eR = \{x \in R \mid ex = x\}.$$

It is closed under the inherited additive operations and multiplication. Its zero is 0_R, while its multiplicative unit is e. Scaling defines a ring map

$$R \longrightarrow eR, \qquad x \longmapsto ex. \tag{21.4}$$

The image of e under (21.4) is the unit of eR, and any map that makes e invertible factors contractibly through this fixed image. Thus eR is a localization at e, constructed without a quotient and without fractions.

The factor has a simple formula. If $h : R \to S$ makes the idempotent e invertible, then idempotence of $h(e)$ and cancellation by its inverse force $h(e) = 1$. On an element x fixed by multiplication by e, the factor sends x to $h(x)$. Conversely, the original element r reaches the fixed image as er, and then

$$h(er) = h(e)h(r) = h(r),$$

which is the required triangle. The fixed-point equation retained in the carrier supplies exactly the coherence needed for this formula to define a structured map.

Take now $R = \mathbb{F}_2 \times \mathbb{F}_2$ and $e = (1, 0)$. Componentwise multiplication makes e idempotent, yet boolean discrimination proves $e \neq (0, 0)$ and $e \neq (1, 1)$. Its fixed image consists of the first component with the second forced to zero. This is a closed, genuinely non-endpoint localization: neither the identity localization nor the zero localization is being disguised by notation.

> **Formal status — checked models.** Evidence `COMM-RING-LOCALIZATION-MODELS`. The identity ring localizes at any supplied unit and canonically at 1; the zero ring localizes every ring at 0; and the fixed image eR localizes at an idempotent e. The product ring $\mathbb{F}_2 \times \mathbb{F}_2$ supplies a checked idempotent $(1, 0)$ distinct from both endpoints, so the fixed-image construction has a concrete nondegenerate instance. These models do not amount to arbitrary localization existence.

21.6 Localizing Once Or Twice

Universal properties earn their keep when two descriptions must be compared. Choose a localization of R at f, and then localize its target at the image of g. Also choose a localization of R at the product fg. In customary notation the two targets are

$$R[1/f][1/g] \qquad \text{and} \qquad R[1/(fg)].$$

No fraction calculation is needed to compare them. In the iterated target, the images of both f and g are units, so their product is a unit. The universal property of $R[1/(fg)]$ therefore gives a forward map

$$\Phi : R[1/(fg)] \longrightarrow R[1/f][1/g].$$

Conversely, if fg is invertible in a commutative ring, then both f and g are invertible: an inverse to f is $g(fg)^{-1}$, and symmetrically for g. The product localization consequently admits first a factor through $R[1/f]$ and then a factor through the localization at the image of g. This gives

$$\Psi : R[1/f][1/g] \longrightarrow R[1/(fg)].$$

This elementary implication is the hinge of the comparison. If w is an inverse to fg, then

$$f(gw) = (fg)w = 1, \qquad g(fw) = (fg)w = 1.$$

It converts one unit witness into the two witnesses demanded by the staged universal properties. No appeal to prime ideals, open subsets, or a spectrum is involved; the overlap theorem is already present in commutative algebra.

The two composites are not reduced by a hidden fraction normalizer. Instead, $\Psi\Phi$ and the identity are factors of the same map through the product localization, so contractibility identifies them. The other direction needs one additional step: uniqueness at the first localization aligns the maps on the intermediate ring, after which uniqueness at the second localization identifies $\Phi\Psi$ with the identity. Both results are equalities of whole structured ring maps.

Theorem 21.2 (product and iterated localization). For any supplied localizations in the preceding configuration, the canonical comparison maps satisfy both whole cancellation laws and exhibit an omega-equivalence in $\mathbf{CRing}$,

$$R[1/(fg)] \simeq R[1/f][1/g]. \tag{21.5}$$

> **Formal status — checked.** Evidence `COMM-RING-ITERATED-LOCALIZATION-EQUIV`. The active comparison constructs canonical forward and reverse structured maps from the supplied universal properties. Contractibility proves their left and right whole-map laws and packages the selected forward map as an omega-equivalence in $\mathbf{CRing}$. Equation (21.5) does not identify the carrier packages by raw equality, provide fraction computation, or choose either localization globally.

This proof displays the intended style of computation. A representation-first development might multiply fractions and cancel powers until both composites normalize. Here the observable computation is composition of structured maps, and the universal property closes the comparison. Concrete models remain free to normalize fractions internally; nothing downstream is allowed to depend on that choice.

21.7 From Localization To The Sieve $D(f)$

Localization answers a transformational question: what is the universal ring under R in which f has become invertible? The sieve of Chapter 18 answers a relational question: along which probes is the image of f already invertible? These are two faces of the same algebraic event.

Given a ring map $u : R \to S$, membership in the sieve $D_R(f)$ is unit evidence for $u(f)$. Whenever a localization $R \to R[1/f]$ has been supplied, the factorization theorem turns that membership into a structured map

$$R[1/f] \longrightarrow S$$

over R, and contractibility makes all choices of such a factor coherently unique. Conversely, any map over R carries the selected inverse of the localized image of f to an inverse of $u(f)$. The localization therefore represents the question posed by the invertibility sieve when a representing object is available.

The order of ideas is deliberate. The sieve $D_R(f)$ exists from the invertibility predicate alone; it need not wait for a chosen fraction object. A supplied localization then represents that sieve on affine points. Thus the geometry can be organized around **invertibility's sieve**, while localization supplies a computational chart rather than defining openness by decree.

The next chapter makes this bridge precise. It constructs the affine functor of points, reads $D(f)$ as an ordinary sieve on an affine, and shows how unimodular families become finite basic-open covers. The algebra developed here will reappear there not as an internal manual of ring operations, but as the universal language in which affine geometry recognizes its charts.

22. Affine Geometry And The Sieve $D(f)$

An affine scheme is often introduced as a set of prime ideals equipped with a topology and a sheaf. That description is powerful, but it begins at the end of several constructions. Constructively, even the set of prime ideals can be difficult to use: the familiar argument that a proper ideal lies in a prime ideal invokes a choice principle that one may have deliberately declined. More importantly for the present book, a point-set spectrum hides the functorial action that makes geometry computational.

There is another beginning. Instead of asking first which ideal-valued points a ring has, ask how the ring can be mapped into every test ring. Instead of declaring a subset on which an element is nonzero, ask along which of those maps its image becomes invertible. The answers are already organized by composition. They form a functor of points and, inside it, an ordinary sieve

$$D_R(f) = \{h : R \longrightarrow S \mid h(f) \text{ is invertible in } S\}.$$

This sieve is the geometric form of localization. A chosen localization $R \to R[1/f]$ represents its points: maps out of $R[1/f]$ are exactly maps out of R along which f is a unit. Products encode intersections, $D_R(fg) = D_R(f) \cap D_R(g)$, and finite unit-ideal presentations generate the Zariski topology on the big affine slice. Only after those constructions are in place do we ask for a structure sheaf and its locality.

The order matters. The sieve exists before it is represented by one chart; the topology exists before a sheafification for commutative-ring values has been constructed; and the computing coordinate presheaf exists before it is equipped with supplied sheaf and locality witnesses. Affine geometry becomes executable without pretending that each of its classical existence theorems has already been rebuilt.

22.1 Affines As Questions Of Points

Let $\mathbf{Aff} = \mathbf{CRing}^{\mathrm{op}}$. A ring map $h : R \to S$ is read geometrically in the opposite direction,

$$\mathrm{Sp}(S) \longrightarrow \mathrm{Sp}(R).$$

The notation $\mathrm{Sp}(R)$ is deliberately functorial. At a test ring S, define

$$\mathrm{Sp}(R)(S) = \mathrm{Hom}_{\mathbf{CRing}}(R, S). \tag{22.1}$$

If $\alpha : S \to T$, composition sends a point $h : R \to S$ to $\alpha \circ h : R \to T$. Thus (22.1) is not a disconnected family of sets. It is the representable presheaf on $\mathbf{Aff}$, with all change-of-test-ring maps supplied by Yoneda.

This definition changes the meaning of “point” in a useful way. A point is not required to land in a field, an algebraic closure, or a two-valued truth object. Every ring S is an admissible stage of observation. Nilpotents, idempotents, infinitesimal extensions, and degenerate rings are visible when the

chosen tests can detect them. A classical prime ideal may later be recovered through a suitable field-like test, but it does not monopolize the notion of observation.

Generalized points can distinguish phenomena that field-valued points erase. For example, a nilpotent element must map to zero in every reduced field, but it may remain visible under a map to a ring with nilpotents. Likewise, an idempotent can reveal a decomposition through product test rings even when a single ordinary point sees only one side. The functor of points does not force one preferred class of tests to carry the entire geometry. It records the response at all stages and lets later hypotheses specify which stages are sufficient for a particular theorem.

There is a useful logical economy in this approach. To compare two candidate affine constructions, one may compare how maps into every test ring are classified rather than inspect a chosen point-set representation. Conversely, an explicit test ring can refute an overstrong identification by exhibiting a point seen on one side and not the other. The functor is therefore both an extensional language and an experimental instrument. The representation theorem below uses the second role directly: it constructs and compares actual maps at an arbitrary supplied test ring.

The functor of points also remembers direction for free. Suppose $R \to S \to T$ is a commuting triangle of ring maps. The corresponding geometric arrows compose in the reverse order, and evaluating $\mathrm{Sp}(R)$ simply composes the ring maps. No separate substitution operation has to be appended to the set of points, and no proof is needed that this operation respects identity and composition: those laws are the ordinary Yoneda action already developed in Chapters 13 and 18.

One should not confuse this representable with the completed geometric object traditionally denoted $\mathrm{Spec}(R)$. At this stage we have a functor of affine tests. We have not yet selected a Zariski topology, a structure sheaf, a local-ring condition, or a comparison with a prime-ideal spectrum. The notation records the intended geometry while leaving each additional layer visible.

22.2 Invertibility Is A Sieve

Fix $f \in R$. For every test ring S, let

$$D_R(f)(S) = \sum_{h : R \to S} \mathsf{Unit}_S(h(f)), \tag{22.2}$$

where $\mathsf{Unit}_S(x)$ is the proposition that x has a multiplicative inverse. An element of (22.2) is therefore a point h together with actual evidence that the image of f is invertible.

The construction is stable under refinement of the test. If $\alpha : S \to T$, a unit witness for $h(f)$ is carried by α to a unit witness for $\alpha(h(f)) = (\alpha \circ h)(f)$. Hence every further probe of a $D_R(f)$-point is again a $D_R(f)$-point. In the geometric category $\mathbf{Aff}$, this downward closure says precisely that $D_R(f)$ is a sieve on $\mathrm{Sp}(R)$.

Unit evidence is proposition-valued, so the sieve is ordinary rather than a higher sieve with a nontrivial category of witnesses. The selected inverse is retained during a calculation, but any two inverse witnesses agree. The predicate can therefore be used as a subterminal fibre of the representable without erasing computational access to the inverse when it is needed.

This definition avoids two premature decisions. It does not ask whether invertibility is decidable, and it does not ask whether all the successful probes factor through one previously chosen open object. A caller may supply a unit witness even when no decision procedure exists. The resulting collection of successful probes is meaningful even when no single object represents it.

Invertibility is stronger than nonvanishing. Over a field the two conditions coincide away from zero, which can make the distinction disappear in a point-set picture. Over a general test ring an element may be nonzero without being a unit. The basic open $D_R(f)$ therefore does not classify probes on which f merely survives; it classifies probes on which f has become reversibly usable. That is precisely the condition needed for a map out of a localization.

Two endpoint examples calibrate the definition. Every ring map preserves one, and one has a canonical inverse, so $D_R(1)$ is the maximal sieve. By contrast, membership in $D_R(0)$ forces zero to be a unit in the test ring and hence forces that ring to collapse. The latter sieve is not literally empty when the zero ring is admitted, but it is geometrically empty relative to nondegenerate tests. The convention from Chapter 21 makes this boundary exact rather than exceptional.

In a posetal presentation of geometry, a sieve represented by an open $V \leq U$ consists of precisely the probes that factor through V. Max Zeuner's constructive development organizes affine geometry through the Zariski lattice of compact opens and, in a locally ringed lattice, assigns a section its largest compact-open invertibility support. From the present viewpoint, that compact open is a particularly economical representative of the sieve $D_U(s)$ when such a representative exists. The sieve is primary on a general site; the compact open is the coherent or qcqs form in which all of its probes can be summarized by one lattice element.

This chapter is structurally adapted from the Zariski-lattice, coverage, compact-open, and functor-of-points development in Zeuner. The change of viewpoint is substantial: rather than define invertibility's locus first as a compact open, we define the ordinary sieve of all invertibility probes and ask for representability afterward. Nothing in that change invalidates the compact-open account in its intended scope.

> **Formal status — mathematical development and attribution boundary.** The comparison with compact opens is an attributed reformulation of Zeuner's presentation. The active construction at this point is the ordinary sieve (22.2). No general theorem that $D_U(s)$ is compactly representable, no comparison with Zeuner's qcqs schemes, and no point-set spectrum theorem is claimed.

22.3 Localization Represents The Basic Open

Now suppose a localization has been selected,

$$\iota_f : R \longrightarrow R[1/f].$$

The notation again names a universal role rather than a fraction syntax. By definition, $\iota_f(f)$ is a unit, and every map $h : R \to S$ for which $h(f)$ is a unit factors contractibly through ι_f.

There are two immediate constructions at every test ring S. A map $k : R[1/f] \to S$ gives

$$\Phi_S(k) = \big(k \circ \iota_f, k(\iota_f(f)) \text{ is a unit}\big) \in D_R(f)(S). \tag{22.3}$$

Conversely, a point $(h, u) \in D_R(f)(S)$ is exactly an admissible target for the localization property. The center of its contractible factor space gives a structured map

$$\Psi_S(h, u) : R[1/f] \longrightarrow S \quad \text{with} \quad \Psi_S(h, u) \circ \iota_f = h. \tag{22.4}$$

The inverse laws require no calculation with fractions. Starting with k, the map k itself is a competitor in the factor space used to select $\Psi_S\Phi_S(k)$. Contractibility identifies the selected factor with k as a whole structured map. Starting with (h, u), the factor triangle identifies the map component of $\Phi_S\Psi_S(h, u)$ with h; proposition-valued unit evidence then completes the equality of the dependent pairs.

Theorem 22.1 (pointwise representation of the basic open). For every test ring S and every supplied localization of R at f, the maps (22.3) and (22.4) form an explicit equivalence

$$\mathrm{Hom}_{\mathbf{CRing}}(R[1/f], S) \simeq D_R(f)(S). \tag{22.5}$$

> **Formal status — checked, pointwise.** Evidence `AFFINE-BASIC-OPEN-POINT-REPRESENTATION`. Both maps and both inverse laws in (22.5) are constructed from the localization universal property, whole ring-map extensionality, and proposition-valued unit evidence. The equivalence is available for each test ring. Both sides retain their functorial action through existing owners, but the active result does not package these components as a whole natural equivalence or equality of presheaves.

That last qualification is not a defect in the mathematical idea. Formula (22.5) is exactly the component expected from representability, and its maps are defined canonically from composition and factorization. A complete presheaf-level equivalence would additionally assemble the components as internal transformations and prove their whole inverse laws in the relevant functor category. The current theorem records the strength that has actually been checked rather than replacing that missing assembly by an external assertion of naturality.

Representability also separates invariance from choice. Two constructions of a ring called $R[1/f]$ need not be definitionally equal as carrier packages. Each nevertheless classifies the same factorization problem, so their universal properties construct comparison maps and the relevant inverse laws. Geometry may consequently use "the chart $D(f)$" without requiring every implementation to share one fraction representation or normal form. What is invariant is the question asked by the sieve; a selected localization is a computational coordinate presentation of it.

This reverses a common expository dependency. If one defines $D(f)$ to be the spectrum of a fraction ring, openness is tied immediately to the chosen construction of fractions. If one begins with (22.2), closure under refinement follows from preservation of units alone. The universal property then proves that a fraction model, quotient model, idempotent fixed-image model, or any other certified localization presents the same geometric question. Representation becomes a theorem with executable maps.

The case $f = 1$ recovers the whole affine: the identity localization at an already invertible element represents the maximal invertibility question. The case $f = 0$ reaches the opposite boundary. Localization at zero is the zero ring, so maps out of it represent precisely those test maps under which zero has become a unit—that is, the degenerate stages where the target ring has collapsed. These endpoints arise from the same universal statement as every other basic open.

22.4 Multiplication Computes Intersection

The elementary algebra of units already knows how basic opens intersect. For a fixed point $h : R \to S$, if $h(fg)$ is invertible with inverse w, then

$$h(f)\big(h(g)w\big) = 1, \qquad h(g)\big(h(f)w\big) = 1.$$

Thus $h(f)$ and $h(g)$ are both units. Conversely, the product of two units is a unit. Since all three unit predicates are propositions, these operations give an equivalence between unit evidence for $h(fg)$ and paired unit evidence for $h(f)$ and $h(g)$.

Holding the underlying point h fixed and summing over all points yields

$$D_R(fg)(S) \simeq \sum_{h:R\to S} \big(\mathsf{Unit}_S(h(f)) \times \mathsf{Unit}_S(h(g))\big). \tag{22.6}$$

The right side is the explicit pointwise intersection of $D_R(f)$ and $D_R(g)$: one point of the whole affine carrying membership in both sieves. No equality of chosen localization packages is needed. If a localization at fg is supplied, Theorem 22.1 represents the left side, and composition with (22.6) represents the intersection by maps out of $R[1/(fg)]$.

This is the geometric face of Theorem 21.2. The algebraic equivalence

$$R[1/(fg)] \simeq R[1/f][1/g]$$

says that entering $D(f)$ and then $D(g)$ has the same coordinate ring as entering their product open at once. In the functor-of-points picture, equation (22.6) says that the two procedures admit the same tests. The first statement compares coordinate rings by whole structured maps; the second compares membership types at every test ring. Together they explain why multiplication is the algebra of intersection.

> **Formal status — checked, pointwise.** Evidence `AFFINE-BASIC-OPEN-INTERSECTION`. The active maps give the equivalence (22.6), and a supplied localization at fg gives an executable two-step representation of its right side with both component inverse laws. This is not yet a whole-presheaf intersection theorem, an external naturality family, a topology, or an appeal to univalence.

The formula is already the meet law of the Zariski lattice. Zeuner writes the standard compact-open support as $D(fg) = D(f) \wedge D(g)$. Here the same law is seen one probe at a time before a compact-open classifier is assumed. When a compact open represents each sieve, pointwise intersection descends to the lattice meet. When no such representative is known, equation (22.6) still calculates the intersection of the sieves themselves.

Multiplication governs finite meets, but covering uses addition as well. A certificate $a_1 f_1 + \cdots + a_n f_n = 1$ says that the chosen basic regions are jointly sufficient. The robust geometric form of that sufficiency is obtained by declaring their arrows to generate a cover and then closing under the Grothendieck laws. The equation supplies finite algebraic evidence; the topology below supplies invariant coverhood. This keeps the meet calculation (22.6) distinct from the join-like operation of generating a covering sieve.

22.5 The Big Affine Slice And Its Coordinates

To pass from one affine to its charts, fix R and consider every ring map $h : R \to S$. Geometrically it is a chart

$$\mathrm{Sp}(S) \longrightarrow \mathrm{Sp}(R).$$

These charts form the **big affine slice over** $\mathrm{Sp}(R)$. A morphism from the chart $R \to T$ to the chart $R \to S$ is a ring map $\beta : S \to T$ whose triangle over R commutes. Its geometric direction is

$$\mathrm{Sp}(T) \longrightarrow \mathrm{Sp}(S),$$

and it carries the whole structured restriction map $S \to T$ that coordinates must follow.

There is consequently a tautological commutative-ring-valued coordinate presheaf $\mathcal{O}_{\mathrm{coord}}$. It sends the chart $R \to S$ to S and the geometric arrow represented by $\beta : S \to T$ to the same structured ring map β. At the whole chart $R \to R$ its value is R. At the basic-open chart $R \to R[1/f]$ its value is the chosen localization ring. Nothing is defined only on objects: the restriction maps and their identity and composition laws are part of the existing whole functorial structure.

Theorem 22.1 now receives its geometric reading. The chart

$$\mathrm{Sp}(R[1/f]) \longrightarrow \mathrm{Sp}(R)$$

has, at every test ring S, exactly the points of the sieve $D_R(f)$. The active theorem proves this statement componentwise; the big slice supplies the chart object and its coordinate restriction. Representation is therefore not used to define the sieve, but once a localization is selected it produces the expected geometric chart.

The overlap comparison from Chapter 21 also lifts into the slice. The two whole ring maps between $R[1/(fg)]$ and $R[1/f][1/g]$ become chart arrows in opposite geometric directions. Applying $\mathcal{O}_{\mathrm{coord}}$ to those arrows computes back to the same structured comparison maps, and their already-proved cancellation laws give an equivalence of the coordinate rings. No new overlap equation is copied into the chart record; it is inherited from the algebra that owns it.

Why use the big slice rather than immediately restrict to the small category of basic opens of $\mathrm{Sp}(R)$? The big slice accepts every R-algebra as a chart. Base change, comparison maps, degenerate targets, and future affine realizations can therefore be expressed without first proving that each object belongs to a chosen basis. A small site is often more economical for compactness or cohomological arguments, but equivalence between the two presentations is a theorem. Starting big keeps the functor-of-points semantics literal and postpones that theorem rather than assuming it.

The price is controlled redundancy. Many big-slice objects describe regions already covered by smaller basic charts, and equivalent localizations may appear as distinct packages. The coordinate presheaf handles this without quotienting the objects: it follows every whole ring map, while universal properties supply comparisons where needed. A later basis theorem may show that suitable redundancies are invisible to local data. They are not erased at the computational boundary.

> **Formal status — checked.** Evidence `AFFINE-BIG-SLICE-COORDINATES`. The conventional big affine slice, the whole coordinate presheaf, literal charts, localization charts, commuting chart arrows, and the two product/iterated-localization overlap directions are active. Coordinate restriction along a chart arrow computes to its supplied whole ring map. The big slice is not identified with a small site of opens and does not by itself provide a topology, sheaf, locally ringed space, or complete scheme.

22.6 Finite Families Generate The Big Zariski Topology

The algebraic certificate from Chapter 21 becomes geometric on every chart. Let $h : R \to S$ be an object of the big affine slice, and choose a finite family $f_1, \ldots, f_n \in S$ together with coefficients satisfying

$$\sum_i a_i f_i = 1. \tag{22.7}$$

For each f_i, also choose a universal-property localization of S at f_i. It determines a whole chart arrow

$$\mathrm{Sp}(S[1/f_i]) \longrightarrow \mathrm{Sp}(S) \longrightarrow \mathrm{Sp}(R). \tag{22.8}$$

Equation (22.7) is the finite certificate that these basic charts cover. To turn that statement into a topology without discarding its computational input, first call a sieve on the chart a **generator** when it contains every arrow in one selected family (22.8). The generators remember the family, coefficients, localization packages, and literal containments. They are witness-rich data, not merely truth values.

Now intersect all Grothendieck topologies on the big affine slice that accept those generators. The result is a lawful topology, denoted $J_{\mathrm{Zar}}^{\mathrm{big}}(R)$. Every selected finite family covers in it by construction, and it is least in the precise sense that

$$J_{\mathrm{Zar}}^{\mathrm{big}}(R) \leq J$$

for every other Grothendieck topology J accepting the same generators. The presentation witnesses remain available at the generating boundary, while the statement that a sieve covers is proposition-valued. This is the same useful separation encountered in Chapter 19: rich evidence enters the generator; invariant coverhood leaves it.

Theorem 22.2 (the generated big Zariski topology). The selected finite unimodular localization families on all charts of the big affine slice generate a Grothendieck topology with computing generator inclusion and the leastness property above.

> **Formal status — checked.** Evidence `AFFINE-BIG-ZARISKI-TOPOLOGY`. Each chosen localization is a whole internal chart arrow whose coordinate restriction is the localization map. The generator retains finite containment, and the generic generated-topology construction supplies a lawful least topology. No global choice of localization packages, cover-derivation syntax, coverhood decision procedure, subcanonicity theorem, sheafification, or comparison with the small Zariski site is asserted.

This construction parallels the finite Zariski coverage in Zeuner's functorial account, but the bookkeeping is arranged around sieves. A finite family presents probes that must cover; Grothendieck closure then adds all maximal, pulled-back, and locally implied covers. In a coherent setting the same information may be summarized by joins in the Zariski lattice. The big site keeps the probes and their restriction maps explicit, which is exactly what the later computational structure presheaf consumes.

The Grothendieck closure is doing real work. A displayed family such as (22.8) is only one cover presentation on one chart. Pulling a covering sieve back along a chart arrow must again cover, and a sieve covered locally by covering pullbacks must cover globally. Stability and local character come from the generated topology, not from repeatedly manipulating the coefficients in (22.7). When an explicit mapped family is desired, structured ring maps transport the unimodular coefficients, while target localization packages are supplied rather than chosen globally. The topology can therefore be invariant even though computational presentations retain choices.

Leastness is equally important. One could declare every sieve to cover and satisfy the Grothendieck axioms vacuously. The chaotic topology is a useful feasibility model, but it forgets Zariski geometry. Intersecting all accepting topologies includes exactly the consequences forced by the selected finite

families and the topology laws. The construction is impredicative rather than an inductive syntax of derivations, so it proves the universal property of generated coverhood without claiming a normalizer or decision procedure.

A small but concrete example comes from $R = \mathbb{F}_2 \times \mathbb{F}_2$. The complementary idempotents $e = (1, 0)$ and $1 - e = (0, 1)$ satisfy $e + (1 - e) = 1$. Their fixed-image localizations therefore give two selected basic charts in a finite Zariski cover. Since $e(1 - e) = 0$, their algebraic intersection is represented by the zero localization. The corresponding overlap coordinate ring computes to the zero ring. Even this degenerate overlap is informative: the two charts are disjoint pieces of the product affine, and that fact is witnessed by restriction maps rather than inferred from an external picture of points.

22.7 The Structure Sheaf Is A Separate Commitment

The coordinate presheaf $\mathcal{O}_{\mathrm{coord}}$ computes, but computation alone does not prove descent. To call it a structure sheaf on the generated big site, one needs commutative-ring-valued sheaf semantics and a comparison identifying the selected sheaf with these coordinates.

The Cat-valued reflector constructed in Chapter 20 does not automatically supply that result. Lifting it to commutative-ring values would require the ring operations and laws to survive the completion coherently, and the active development has not claimed such a lift. The affine interface therefore makes the missing theorem an explicit input. An **affine structure-sheaf presentation** first supplies a reflective sheaf theory for commutative-ring-valued presheaves on the exact topology $J^{\mathrm{big}}_{\mathrm{Zar}}(R)$. It then selects a sheaf object $\mathcal{O}$ and a whole computational isomorphism from its included presheaf to $\mathcal{O}_{\mathrm{coord}}$.

The comparison is whole functorial data, not a list of unrelated object-by-object ring isomorphisms. Its component at a chart compares the selected structure sheaf with the chart ring, while its internal transformation action retains compatibility with every restriction. This is strong enough for downstream computation, but it is supplied evidence. The package does not construct the reflector or prove from first principles that the coordinate presheaf satisfies descent.

The whole comparison matters whenever a section is transported before it is inspected. An objectwise list of ring isomorphisms could identify $\mathcal{O}(U)$ with the coordinate ring of each chart and still fail to commute with restriction. A transformation in the presheaf category carries that compatibility internally. Its cancellation laws let a calculation move to the transparent coordinate presheaf, compute there, and return to the selected sheaf without adding a new naturality square at every use.

> **Formal status — checked, assumption-explicit.** Evidence `AFFINE-STRUCTURE-SHEAF-PRESENTATION`. Given a reflective commutative-ring-valued sheafification capability, a sheaf object, and a whole computational isomorphism to the coordinate presheaf, the active presentation constructs the corresponding reflective ringed big site and exposes the comparison at every chart. Neither the capability nor the comparison is constructed by this affine layer.

There is a second locality condition that should not be confused with sheaf descent. Let U be a chart, let $s \in \mathcal{O}_{\mathrm{coord}}(U)$, and choose a localization of the chart ring at s. Restricting a localization element along every member of the sieve $D_U(s)$ gives a coherent matching family. Cartier locality says that this restriction functor is an equivalence: every coherent family on the invertibility sieve glues uniquely to an element of the localized ring.

The active affine locality capability supplies this whole equivalence for every chart, section, and supplied localization package. Its inverse is a whole glue functor, and both composite-functor laws are retained. Yet $D_U(s)$ need not cover U. Cartier locality describes the coordinate ring of a basic-open region; ordinary sheaf descent describes reconstruction from a covering sieve. A stalk-local-ring theorem is different again. Keeping these three statements separate prevents the word “local” from doing more work than the mathematics.

This distinction mirrors the sieve-first viewpoint. Sheaf descent starts with a sieve certified to cover U and asks whether compatible data on that cover have a unique global amalgamation. Cartier locality starts with an arbitrary section s and studies the possibly noncovering region where s is invertible. Its global object is not a section over all of U but an element of the localized coordinate ring. A local-ring condition, finally, controls how unit evidence can be found locally from algebraic alternatives such as the invertibility of a sum. The three interfaces cooperate in scheme theory, but none is a synonym for either of the others.

22.8 A Thin Computational Affine Presentation

Once the preceding commitments have been named, the affine package itself can be small. For a ring R, an affine-scheme presentation consists of

$$\begin{aligned}\mathsf{AffPres}(R) = \big(&\text{structure-sheaf presentation on } J^{\mathrm{big}}_{\mathrm{Zar}}(R),\\ &\text{coordinate localization locality}\big).\end{aligned} \tag{22.9}$$

The ring R already determines the big affine slice, its coordinate presheaf, the whole chart, and the generated topology. The first entry of (22.9) relates a supplied reflective structure sheaf to those computing coordinates. The second supplies the whole restriction-and-glue equivalence on every basic invertibility sieve. There is no benefit in copying chart actions, overlap maps, or coherence equations into a larger record: they are already inherited from the functors and universal properties that own them.

> **Formal status — checked, assumption-explicit.** Evidence `AFFINE-THIN-SCHEME-PRESENTATION`. The active affine presentation pairs the supplied whole structure-sheaf presentation with supplied whole coordinate localization locality. It projects the exact generated big-Zariski ringed site, the whole coordinate comparison, and locality at each selected chart and localization. The product ring $\mathbb{F}_2 \times \mathbb{F}_2$ supplies a closed reviewer in which the two capabilities remain explicit inputs while the complementary-idempotent cover, chart rings, restriction maps, and zero overlap compute.

The smallness of (22.9) is evidence of internal organization, not of missing coherence being ignored. The topology owns pullback stability and local character. The coordinate presheaf owns restriction. Localization owns basic charts, and the whole locality equivalence owns glue over $D(s)$. An affine presentation selects only the capabilities that cannot yet be derived. Duplicating their component operations would create new obligations to prove that the copies agree with these established owners.

Calling (22.9) an affine scheme is therefore a qualified statement. It is a computational presentation whose assumptions are visible. The active work does not construct commutative-ring-valued sheafification, prove coordinate locality, compare the big site with the small Zariski site, construct stalks, establish a stalk-local-ring theorem, or build a representation-independent category of affine schemes. Nor does it prove Zeuner's comparison between locally ringed lattices and functorial qcqs schemes.

What it does construct is already a coherent geometric chain:

$$\text{ring map} \longrightarrow \text{affine probe} \longrightarrow D(f) \longrightarrow R[1/f] \longrightarrow \text{basic chart} \longrightarrow J_{\mathrm{Zar}}^{\mathrm{big}}.$$

Every arrow in that chain carries computation. Composition changes probes; unit preservation restricts the sieve; localization selects factors; multiplication computes intersections; the coordinate presheaf computes chart restriction; and generated topology turns finite algebraic certificates into coverhood. The supplied sheaf and locality capabilities begin exactly where the constructed chain ends.

The next chapter starts from the complementary direction. Instead of fixing a ring and generating its affine world, it assumes a global ringed object and a covering sieve, selects affine charts inside that cover, and asks which restrictions and overlaps can be inherited from the global object. That global-first viewpoint is the bridge from one affine presentation to site-relative schemes.

23. Schemes From Covering Charts

An atlas is persuasive because its pieces are familiar. If a space can be covered by affine charts, one is tempted to say that the scheme is simply the charts plus instructions for gluing them. But this description suppresses an important choice of direction. We may begin with charts and try to construct a global object, or we may begin with a global object and recognize some of its regions as affine. The two directions ask for different theorems.

This chapter follows the second, **global-first** direction. A global object X is already present in a ringed site, with one structure presheaf $\mathcal{O}$ and one covering sieve $\mathcal{R}$ on X. Two selected members $u_0 : U_0 \to X$ and $u_1 : U_1 \to X$ will generate that sieve constructively. Each chart is then compared, as a whole functorial restriction, with affine coordinates. Local-ring behaviour is imposed on the actual slice over X. The result is a binary, site-relative computational scheme presentation.

Starting globally pays a coherence dividend. Restriction maps already belong to $\mathcal{O}$; their identity and composition laws are already functor laws. An intersection, once selected as an actual product in the slice over X, inherits its two projections and both maps on rings. There is no reason to copy those maps into an atlas record and then ask whether the copies agree. The global object is the common source from which they are computed.

The price is equally clear. This chapter does not construct X from an abstract atlas. Its slice sites, affine-basis comparisons, and one selected chart intersection are assumption-explicit. “Scheme” here means a scheme presentation relative to those supplied categorical semantics, not yet a representation-independent category identified with every classical or functorial definition of schemes.

23.1 Two Directions Through An Atlas

Suppose first that two affine objects U_0 and U_1 have been given, along with a candidate overlap U_{01}, maps to the charts, and a transition between coordinate descriptions. The atlas-first problem is to construct an object X for which the diagram is effective: U_0 and U_1 should cover X, U_{01} should be their intersection, and functions agreeing on the overlap should glue uniquely. Even for two charts, that is a colimit and descent theorem. For more charts one must also control triple overlaps and cocycles. Merely storing the expected diagrams does not prove that a global object exists.

Now reverse the situation. Let X already be an object of a category $\mathcal{K}$, and let $u_i : U_i \to X$ be actual arrows. Their intersection, if the needed product exists in the slice $\mathcal{K}/X$, is no longer an invented compatibility object. It is the categorical object

$$U_{01} = U_0 \times_X U_1. \tag{23.1}$$

If a single presheaf $\mathcal{O}$ is already defined on $\mathcal{K}$, then its values $\mathcal{O}(U_i)$ and $\mathcal{O}(U_{01})$ and its two restriction maps are forced by (23.1). Repeated restriction agrees because $\mathcal{O}$ is a functor. Global-first geometry therefore shifts the hard question. We no longer ask whether chart data can be glued to

create X; we ask whether selected regions of X really are affine and whether they cover in the chosen topology.

Max Zeuner develops two constructive architectures for qcqs schemes: finite affine covers of locally ringed lattices and affine compact-open covers of functors of points, followed by a comparison theorem between them. The finite-cover rhythm is an important model for the present exposition, but the emdash construction changes both its starting point and its classifier of locality. The global object and its ringed categorical semantics are supplied first; coverhood is carried by an ordinary sieve; and the locus where a section is invertible is the sieve $D_U(s)$ of all successful probes. A compact open can represent that sieve in a coherent setting, but such a representability theorem is not assumed merely in order to speak about the sieve.

The distinction is not a contest between definitions. A global-first package is useful when a semantic object has already been built and one wants a computational account of its charts. An atlas-first theorem is indispensable when the charts are the input from which the object must be created. Zeuner's comparison explains why two mature theories of qcqs schemes agree. Here the more modest task is to identify exactly how far one can travel on the first route with the current owners.

> **Attribution and comparison boundary.** This chapter comparatively adapts the finite affine-cover and functor-of-points architecture of Zeuner, especially Sections 3.3, 4.2, 5.1, and 5.3. It does not import the theorem that affine charts glue to a qcqs scheme, the compact open classifier, or the equivalence between functorial and locally ringed-lattice schemes. The site-relative global-first presentation below is the active emdash result.

23.2 One Covering Sieve, Two Generators

Let $\mathcal{A}$ be a reflective commutative-ringed site with base category $\mathcal{K}$. Its included structure presheaf will be written

$$\mathcal{O} : \mathcal{K}^{\mathrm{op}} \longrightarrow \mathbf{CRing}.$$

Choose an object $X \in \mathcal{K}$, an ordinary sieve $\mathcal{R}$ on X, and evidence that $\mathcal{R}$ covers X in the selected Grothendieck topology. This is already global geometric data. Every arrow $q : V \to X$ in $\mathcal{R}$ is a region of X, and every further restriction of q remains in $\mathcal{R}$. Grothendieck stability also says that pulling $\mathcal{R}$ back along any arrow into X produces a covering sieve on its domain.

A selected chart is initially nothing more than a selected member of this sieve. Thus an arrow $u : U \to X$ becomes a chart when accompanied by $u \in \mathcal{R}$. The name does not make U affine. It records that u is one of the regions accepted by the global cover; affineness requires the separate comparison developed in Section 23.5.

Select two such members $u_0 : U_0 \to X$ and $u_1 : U_1 \to X$. Their membership alone does not say that they cover. A sieve can contain two arrows while also containing regions that factor through neither one. The computational generation condition supplies the missing direction: for every $q : V \to X$ in $\mathcal{R}$, it returns a Boolean side b, a map $h : V \to U_b$, and the triangle

$$q = u_b \circ h. \tag{23.2}$$

In dependent notation, the content is

$$\prod_{q:V\to X} (q \in \mathcal{R}) \longrightarrow \sum_{b:\mathbf{2}} \sum_{h:V\to U_b} (q = u_b h). \tag{23.3}$$

Equation (23.3) is stronger computationally than the mere assertion that some chart contains every covered region. Given a membership witness, one can execute the selection, inspect which chart was chosen, and use the actual factor map. No propositional truncation hides the branch. Conversely, because both u_0 and u_1 are themselves members and a sieve is closed under precomposition, every arrow factoring through either chart belongs to $\mathcal{R}$. The retained sieve is therefore exactly generated, in this witness-rich sense, by the two selected arrows.

There are two cautions. First, (23.3) does not construct a second sieve: it explains the already-selected covering sieve $\mathcal{R}$. Its coverhood comes from the global package, not from the unsupported observation that two arrows have been named. Second, a general member q need not itself be affine. It is a refinement of one affine generator once the relevant Boolean branch is computed. Affineness belongs to the generator's whole realization and does not automatically descend to every arbitrary arrow without another theorem.

> **Formal status — checked.** Evidence `GLOBAL-RINGED-COVER-BINARY-GENERATION`. The global package retains the reflective ringed site, object, ordinary covering sieve, structure presheaf, and Grothendieck-stable pullbacks. Binary generation computes a selected chart, factor map, and triangle for every sieve member. The active result does not infer affineness from sieve membership, construct an atlas-first object, or turn arbitrary refinements into affine charts.

The choice to retain a sieve rather than just a two-element family has a further advantage. Pulling the cover back along a region does not require a new ad hoc list of chart fragments. The sieve pullback contains exactly the arrows whose composites lie in $\mathcal{R}$, and Grothendieck stability makes it covering. When an explicit generator is needed, (23.3) still computes a factor through U_0 or U_1. Invariant coverhood and executable chart selection coexist without being collapsed into the same representation.

23.3 Local Rings Without Stalks

A sheaf of rings is not automatically a sheaf of local rings. Descent says that compatible sections glue; locality says, roughly, that algebraic alternatives can be resolved after passing to a cover. Classically the latter is often phrased by requiring every stalk to be a local ring. A computational site can express the needed alternatives directly, before stalks have been constructed.

Let T be a Grothendieck topology on a category and let $\mathcal{O}$ be a commutative-ring-valued presheaf. For $s \in \mathcal{O}(U)$, recall the ordinary sieve $D_U(s)$ of arrows $q : V \to U$ along which $s|_q$ is invertible. Two support laws are automatic in the ordinary algebraic account: one is invertible everywhere, and a product becomes invertible precisely where both factors do. In sieve notation these give the expected equations

$$\begin{aligned} D_U(1) &= \top, \\ D_U(st) &= D_U(s) \cap D_U(t). \end{aligned} \tag{23.4}$$

Equation (23.4) is mathematical orientation here; the local-ring interface below packages the two nonautomatic topology laws, not new whole sieve equalities for one and products.

The local-ring content lies in the two remaining directions. If zero becomes invertible at U, then U is locally void: the literal empty sieve must cover U. And if $s + t$ is invertible, there must be a covering sieve $\mathcal{S}$ such that every $q : V \to U$ in $\mathcal{S}$ comes with a selected alternative

$$s|_q \text{ is invertible} \quad \text{or} \quad t|_q \text{ is invertible}. \tag{23.5}$$

The disjunction in (23.5) is a Boolean-indexed dependent pair. It remembers which summand is usable and retains its unit evidence. Thus the condition is the executable Kripke--Joyal form of

$$D_U(0) = \bot, \qquad D_U(s + t) \leq D_U(s) \vee D_U(t). \tag{23.6}$$

No raw union of sieves has to be constructed for (23.6). The right side is presented by a chosen cover subordinate to the two alternatives, and a branch is requested only after an actual member of that cover is supplied. Nor is the choice erased into a mere proposition. This is exactly the kind of local information a later calculation can consume: restrict to a cover member, inspect the side, and use the selected inverse.

The formulation is categorical semantics, not an auxiliary modal object language. Objects, arrows, sieves, restrictions, unit witnesses, covers, and branches all live inside the functorial theory represented in the outer logical framework. Lambdapi's conversion and unification rules execute the transparent projections and functor actions; the same interfaces can be targeted by an explicit TypeScript core. Computation therefore does not require replacing the site by a primitive modality. It comes from making the categorical data sufficiently internal and functorial.

> **Formal status — checked, topology-local.** Evidence `TOPOLOGY-LOCAL-RING-CERTIFICATE`. The presentation executes empty-cover nontriviality and coverwise Boolean splitting of an invertible sum. It is a site-level local-ring certificate for a whole commutative-ring presheaf. It does not construct stalks, prove equivalence with stalk-local rings, form a raw sieve join, decide unit evidence, or identify the chosen topology with the classical Zariski topology.

This condition must be attached to the correct object. Our global structure presheaf lives on $\mathcal{K}$, whereas the local geometry of X lives on the slice $\mathcal{K}/X$. The next section constructs the presheaf restriction to that whole slice and makes its sheaf boundary explicit. Only then can (23.5) be read as local-ring behaviour of the global object X rather than of an unrelated family of rings.

23.4 Restricting The Whole Structure

For any $U \in \mathcal{K}$, the conventional slice $\mathcal{K}/U$ has a domain functor

$$\mathrm{dom}_U : \mathcal{K}/U \longrightarrow \mathcal{K}, \qquad (q : V \to U) \longmapsto V. \tag{23.7}$$

Precomposition with its opposite constructs the ambient restriction

$$\mathcal{O}|_U = \mathcal{O} \circ \mathrm{dom}_U^{\mathrm{op}} : (\mathcal{K}/U)^{\mathrm{op}} \longrightarrow \mathbf{CRing}. \tag{23.8}$$

At a slice object $q : V \to U$, formula (23.8) computes to $\mathcal{O}(V)$. At a triangle over U, it computes the corresponding restriction homomorphism. Identity, composition, and naturality are inherited from ordinary functor composition. Thus the whole presheaf needed on a chart slice is constructed, not copied object by object.

What (23.8) does **not** construct is a topology and sheaf theory on $\mathcal{K}/U$. The active interface asks for a reflective commutative-ringed site on that actual slice and a whole isomorphism

$$\iota_U \mathcal{O}_U \;\cong\; \mathcal{O}|_U \tag{23.9}$$

in the commutative-ring presheaf category. Here $\mathcal{O}_U$ is the selected structure sheaf of the supplied slice site and ι_U includes it as a presheaf. The topology, reflector, and sheaf object on the left of (23.9) are hypotheses. The right side and its restriction action are the computing ambient object.

The word “whole” in (23.9) matters. A separate ring isomorphism at every slice object would not ensure compatibility with restriction. A functor-level isomorphism carries that compatibility internally and supplies inverse laws in the presheaf category. One may compute using the transparent right side, then pass back to the selected sheaf semantics without introducing a new naturality square for each calculation.

For the whole object X, the topology-local ring certificate of Section 23.3 is attached to the computing presheaf $\mathcal{O}|_X$ using the topology of the supplied slice site. The resulting local presentation combines supplied reflective semantics on $\mathcal{K}/X$, a whole bridge to ambient restriction, and executable local-ring forcing on that bridge's target. It does not claim that the slice topology was induced from the ambient topology or that a general sheaf-pullback theorem has been proved.

23.5 When A Selected Region Is Affine

Return to one selected cover member $u : U \to X$. To call it affine, it is not enough to attach a ring name R to U. We must compare the actual global structure restricted over U with the affine coordinates developed in Chapter 22.

Begin with the supplied reflective slice presentation (23.9). Choose a ring R and a thin affine-scheme presentation for R. Its big affine slice has the generated Zariski topology and coordinate presheaf $\mathcal{O}_{\mathrm{coord},R}$. Next choose a whole basis functor

$$i : \mathbf{Aff}/\operatorname{Sp}(R) \longrightarrow \mathcal{K}/U. \tag{23.10}$$

The direction of (23.10) says that affine coordinate probes are realized as actual regions over U. It is accompanied by two supplied comparisons. The first is a sheaf-basis equivalence along i: restriction relates the selected sheaf categories on the affine basis and on the actual slice. This is an equivalence of sheaf semantics along the displayed functor, not an assertion that the two base categories are themselves equivalent. The second is a direct whole isomorphism from the ambient restriction to the included presheaf of the selected affine presentation. That affine presentation already retains its own whole coordinate comparison. Together they form the chain

$$i^*(\mathcal{O}|_U) \;\cong\; \iota_R \mathcal{O}_R^{\mathrm{aff}} \;\cong\; \mathcal{O}_{\mathrm{coord},R}. \tag{23.11}$$

Equation (23.11) is the computational heart of the affine label. On every affine probe, the structure inherited from the global object agrees with the coordinate ring; on every arrow between probes, the agreement respects the restriction homomorphism. Composing the supplied first bridge with the affine scheme's retained coordinate bridge derives the displayed ambient-to-coordinate comparison. Nothing has to be postulated separately for individual chart arrows.

An **affine realization** of u retains exactly these inputs: the supplied reflective site on $\mathcal{K}/U$, the coordinate ring R, its thin affine presentation, the basis functor (23.10), and the whole basis realization. The region u is now affine in the precise, site-relative sense that its actual restricted structure is realized by affine coordinates along the selected basis.

> **Formal status — checked, assumption-explicit.** Evidence `WHOLE-SLICE-AFFINE-REALIZATION`. Whole ambient restriction is constructed by precomposition with the slice-domain functor. The reflective slice, sheaf-basis equivalence, affine scheme presentation, basis functor, and direct ambient-to-affine-underlying comparison are supplied; composition with the affine presentation's retained bridge derives a whole coordinate isomorphism. No induced slice topology, raw base-category equivalence, arbitrary basis theorem, or general transport of local exactness is claimed.

Applying this package to both u_0 and u_1, and adjoining the generation witness (23.3), gives a binary affine-cover presentation. It retains two actual regions of the one global object, two coordinate rings, and two whole affine realizations. For an arbitrary $q \in \mathcal{R}$, the Boolean generator computes which affine chart it refines and exposes that chart's already-owned realization and coordinate ring. It still does not declare q itself affine.

23.6 The Site-Relative Scheme Total

The pieces can now be gathered without enlarging their claims. A binary site-relative scheme presentation consists of the following dependent data:

$$\mathsf{Scheme}^{(2)}_{\mathcal{K}} = \sum_{(\mathcal{A},X,\mathcal{R})} \big(\text{whole-slice local presentation of } X,\ u_0, u_1 \in \mathcal{R},\ \text{constructive generation as in } (23.3),\ \text{whole affine realizations of } u_0, u_1\big). \tag{23.12}$$

The first summation variable includes the global reflective ringed site, distinguished object, covering sieve, and proof of coverhood. The dependent certificate then adds local-ring forcing on the actual slice and the binary affine atlas. Because each later field is indexed by the object selected before it, malformed combinations are not representable: an affine realization cannot silently refer to a different chart, site, or global object.

Theorem 23.1 (global-first binary scheme presentation). Given the data in (23.12), there is one transparent site-relative scheme total that retains the global ringed object exactly once and exposes its structure presheaf, whole object, covering sieve, local-ring certificate, two selected generators, and two whole affine realizations through their existing owners.

> **Formal status — checked, site-relative.** Evidence `BINARY-SITE-RELATIVE-SCHEME`. The constructor and all named observations compute through the dependent total. The global structure presheaf is inherited rather than stored again; the local and atlas halves remain their exact existing packages. No overlap, transition, cocycle, gluing field, scheme morphism, effectivity theorem, or representation-independent category of schemes is added.

The absence of overlap and cocycle fields is deliberate. Suppose a section is restricted from X to U_0, from U_0 to an overlap region, and perhaps farther to another refinement. These maps are all values of the same contravariant functor $\mathcal{O}$. Their agreement with direct restriction is the functor's composition law. Adding parallel restriction maps to (23.12) would create a second source of truth and an obligation to compare it with the first.

The theorem is nevertheless conditional in meaningful ways. It does not construct the global object, the reflective slice sites, or the affine-basis equivalences. It packages them at endpoints where every downstream action can compute. Nor does it say that all finite atlases reduce to two charts. The binary case is the active vertical slice: rich enough to expose genuine covering, local forcing, affineness, refinement, and intersection, but narrow enough that the supplied boundary remains visible.

The qualifier **site-relative** prevents a more subtle overstatement. The chosen topology decides which sieves cover and the supplied basis semantics decides what counts as an affine chart. Another site may present equivalent geometry, but (23.12) alone does not construct that equivalence. In particular, it does not identify this presentation with Zeuner's compact-open qcqs schemes, with classical locally ringed spaces, or with all local functors on the big Zariski site.

23.7 The Intersection Belongs To The Global Object

Although an overlap should not be copied into the scheme total, a consumer may still need an explicit one. For the two slice objects $u_0, u_1 \in \mathcal{K}/X$, supply a selected binary product with its whole universal property,

$$u_{01} = u_0 \times u_1 \quad \text{in } \mathcal{K}/X. \tag{23.13}$$

Its domain in $\mathcal{K}$ is an actual object U_{01} equipped with arrows $p_i : U_{01} \to U_i$ whose composites to X agree. Applying the whole domain functor to the product projections derives these arrows; no base-level projections are supplied separately.

The global structure presheaf then computes three rings and two homomorphisms:

$$\mathcal{O}(U_0) \xrightarrow{p_0^*} \mathcal{O}(U_{01}) \xleftarrow{p_1^*} \mathcal{O}(U_1). \tag{23.14}$$

The variance in (23.14) is the familiar geometric one: an arrow from the intersection to a chart restricts functions from the chart to the intersection. Every term is evaluated from an existing whole owner. The overlap ring is not a new coordinate choice, and the homomorphisms are not transition fields. They are the value and arrow action of $\mathcal{O}$ on the selected categorical intersection.

> **Formal status — checked after a supplied product.** Evidence `ACTUAL-BINARY-CHART-OVERLAP`. A selected binary product in the conventional slice retains its whole universal property. Its two slice projections, underlying arrows, overlap ring, and both restriction homomorphisms are derived. The active owner does not construct arbitrary pullbacks, prove every pair of charts has an intersection, identify the overlap as affine or as a localization, or add an atlas-gluing theorem.

This is the exact sense in which coherence is inherited. If another selected restriction $W \to U_{01}$ is introduced, the maps from the chart rings to $\mathcal{O}(W)$ agree with the composites through $\mathcal{O}(U_{01})$ because presheaf action respects composition. If triple intersections are later supplied, the same principle handles their restriction diagrams. Existence of the needed limits remains a separate categorical hypothesis; compatibility of the resulting restriction maps does not.

One should also resist a plausible but invalid inference: because U_0 and U_1 have affine realizations, U_{01} has not thereby been proved affine. In familiar schemes the intersection of two affine opens is often quasi-affine and, under additional separatedness or basic-open hypotheses, admits more precise affine presentations. None of those theorems is smuggled into (23.13). Chapter 24 supplies a much narrower Laurent-coordinate adapter for the selected overlap used in the projective-line presentation.

23.8 What Has Been Built, And What Has Not

The global-first construction has now crossed a genuine threshold. It starts with one ringed object and proves that a selected binary covering sieve is generated by two regions whose whole restrictions have affine coordinate realizations. It attaches an executable local-ring condition to the actual slice

over the object. It packages those data in one dependent total. And, when a product is supplied in that slice, it computes the chart intersection, its ring, and both restriction maps from the global presheaf.

The resulting chain is worth reading from left to right:

$$\begin{aligned} \text{global ringed object} &\longrightarrow \text{covering sieve} \\ &\longrightarrow \text{two constructive generators} \\ &\longrightarrow \text{whole affine realizations} \\ &\longrightarrow \text{site-relative scheme total} \\ &\longrightarrow \text{inherited intersection.} \end{aligned} \tag{23.15}$$

At each arrow, either a new hypothesis is named or a new construction is performed. The global object, reflective slice semantics, affine presentations, basis equivalences, local-ring certificate, and selected slice product are supplied. Sieve pullbacks, whole ambient restrictions, generator refinements, coordinate comparisons, total projections, overlap arrows, and ring restrictions are derived. This ledger is mathematical content: it says which theorems a future construction must prove before an input can disappear.

Several larger results remain outside (23.15). There is no constructor taking two abstract affine schemes and transition data to their glued global object. There is no effectivity or independence-of-atlas theorem, no arbitrary finite atlas interface, no category of scheme morphisms, and no proof that changing the selected site preserves the notion. There is no comparison with a prime-spectrum locally ringed space, with Zeuner's locally ringed lattices, or with functorial qcqs schemes. There is also no general theorem representing every invertibility sieve by a compact open.

Those absences locate the work rather than diminish it. Classical definitions often bundle the results of several deep comparison theorems into one word. The site-relative presentation instead exposes a computational semantic boundary: once the global object and its honest affine comparisons are available, restrictions and overlaps should be inherited, not restated. Conversely, when only charts are available, one still owes a gluing theorem.

The next chapter tests this boundary on the first non-affine object one wants to draw: the projective line. Two affine-line charts should meet where their coordinates are invertible, and the transition should send one coordinate to the inverse of the other. The current library can package that Laurent calculation on an inherited actual overlap once the global projective-line presentation has been supplied. It does not yet construct that global object from the two charts, nor does it build Proj or general projective space. That precise mixture of calculation and boundary is where the global-first method is most revealing.

24. The Projective Line And The Boundary Of Construction

The projective line is the first scheme whose picture seems to demand gluing. One affine coordinate sees every finite point and misses infinity; a second coordinate sees infinity and misses a different point. Where both coordinates are visible, each is the inverse of the other. The whole geometry is contained in that short sentence, but its formal meaning divides into several tasks that are easy to conflate.

One may construct the two affine lines, construct their principal regions, identify those regions by inversion, and then prove that the identification is effective. That is the atlas-first route. Or one may begin with a global object already carrying two affine-line charts, take their actual intersection, and check that its two inherited coordinate descriptions are Laurent coordinates. That is the global-first route. The present development reaches the second route exactly. It does not silently acquire the first.

This distinction makes the projective line an unusually revealing example. The coordinate calculation is not deferred: it is constructed from polynomial and localization universal properties. The common region is not an invented overlap: it is an actual selected intersection of charts in the global slice. Yet the global object itself remains supplied. The calculation is complete at its stated boundary, and the boundary says precisely what a later construction must add.

24.1 Two Affine Views Of One Line

Let A be a commutative ring. The familiar projective-line atlas has two charts

$$U_0 \simeq \operatorname{Spec} A[t], \qquad U_1 \simeq \operatorname{Spec} A[u]. \tag{24.1}$$

In homogeneous coordinates $[x_0 : x_1]$, the first coordinate is $t = x_1/x_0$ where x_0 is invertible, and the second is $u = x_0/x_1$ where x_1 is invertible. Both descriptions apply precisely where neither homogeneous coordinate vanishes. Thus the expected intersection has the two coordinate presentations

$$U_{01} \simeq D(t) \subseteq U_0, \qquad U_{01} \simeq D(u) \subseteq U_1, \qquad u = t^{-1}. \tag{24.2}$$

In ordinary algebraic notation its ring is therefore written

$$A[t, t^{-1}] \simeq A[u, u^{-1}]. \tag{24.3}$$

These formulas are a specification, not yet a construction. The symbols $D(t)$ and $D(u)$ can name open subspaces only after one knows what represents the corresponding invertibility conditions. The isomorphism in (24.3) can be written down informally, but a computational account should explain why its maps exist and which equations they satisfy. Finally, even perfect overlap data do not by themselves produce the union $U_0 \cup U_1$.

The finite-cover rhythm here follows the constructive scheme architecture developed by Zeuner: global schemes are recognized through finite affine covers, and open or functorial presentations are compared only by an explicit theorem. The present chapter borrows that rhythm and its careful separation of presentation from comparison. Its projective-line and Laurent constructions are not taken from Zeuner's thesis, and no part of Zeuner's qcqs comparison theorem is claimed below.

> **Status of (24.1)--(24.3).** These are the standard mathematical model for the chapter. The checked artifact has no closed term denoting $\mathbf{P}^1_A$, no general `Proj`, and no polynomial or Laurent expression syntax whose normal forms are the displayed rings. Its owners instead work with rings, structured maps, and their universal properties directly.

24.2 Laurent Maps Without Fractions

Begin with two supplied one-variable polynomial algebras over A. Write them conventionally as $A[t]$ and $A[u]$, although what is retained formally is a base map, a distinguished variable, and the universal property of a one-variable polynomial algebra. Select a localization of each algebra at its distinguished variable:

$$\iota_t : A[t] \longrightarrow L_t, \qquad \iota_u : A[u] \longrightarrow L_u. \tag{24.4}$$

The element $\iota_u(u)$ is a unit in L_u. Its chosen inverse therefore defines a valuation of the one-variable family in L_u. Polynomial universality selects the unique structured map

$$\varphi_{tu} : A[t] \longrightarrow L_u, \qquad \varphi_{tu}(t) = \iota_u(u)^{-1}, \tag{24.5}$$

with the prescribed action on A. The image of t is again a unit. The localization universal property now extends (24.5) to a whole ring map

$$\Phi_{tu} : L_t \longrightarrow L_u. \tag{24.6}$$

Reversing the two polynomial and localization presentations constructs $\Phi_{ut} : L_u \to L_t$. Nothing in this construction parses a Laurent polynomial, reduces a fraction, or chooses representatives. The inverse coordinate is selected as an element of the target ring; polynomial universality constructs the first map; localization universality constructs the extension. The factor triangle records, for every element of $A[t]$, that the extension after ι_t agrees with φ_{tu}.

This order matters. If one began with a formula on fractions, one would still have to prove that it respects the localization relation and the ring operations. The universal property performs exactly that proof while also choosing the map. The formula $t \mapsto u^{-1}$ is not discarded; it appears as the named coordinate equation (24.5), now attached to a whole structured map.

Theorem 24.1 (canonical Laurent transition). For two supplied one-variable polynomial algebras over A and supplied localizations at their coordinates, there is a canonical whole map from the first localization to the second sending the first coordinate to the inverse of the second. Reversing the inputs gives the opposite orientation, and each map carries its whole localization-factor agreement.

> **Formal status — checked.** Evidence `LAURENT-TRANSITIONS-BY-UNIVERSALITY`. The construction is transparent and rule-free. It supplies neither a global projective object nor a theorem that the two maps in (24.6) are inverse for arbitrary polynomial and localization presentations.

24.3 One Overlap Ring, Not Two Isomorphic Copies

The generic construction still has two target rings, L_t and L_u. A global scheme supplies something stronger. Let U_0 and U_1 be its selected charts, let U_{01} be their selected actual intersection, and evaluate the single global structure presheaf:

$$R_0 = \mathcal{O}(U_0), \qquad R_1 = \mathcal{O}(U_1), \qquad L = \mathcal{O}(U_{01}). \tag{24.7}$$

Contravariance gives the two inherited restrictions

$$\rho_0 : R_0 \longrightarrow L, \qquad \rho_1 : R_1 \longrightarrow L. \tag{24.8}$$

Suppose now that R_0 and R_1 are supplied as one-variable polynomial algebras over the same base ring A, with coordinates t and u, and that the literal maps ρ_0 and ρ_1 are supplied as their localizations at those coordinates. The Laurent construction no longer produces maps between two disconnected candidates for the overlap. Both of its endpoints reduce to the exact ring L. It constructs two endomorphisms

$$\Theta_{tu} : L \longrightarrow L, \qquad \Theta_{ut} : L \longrightarrow L, \tag{24.9}$$

each expressing one coordinate in terms of the inverse of the other.

The final comparison is deliberately assumption-explicit. A Laurent overlap presentation retains whole paths

$$\Theta_{tu} = \mathrm{id}_L, \qquad \Theta_{ut} = \mathrm{id}_L\,. \tag{24.10}$$

The maps in (24.9) are constructed; the paths in (24.10) are supplied. This is the exact honest boundary. Polynomial and localization universality tell us how to extend prescribed coordinate values, but arbitrary supplied presentations of two maps into one ring need not automatically assert that the resulting endomorphisms are its identity. Retaining the whole paths says that these two coordinate descriptions really are the two Laurent views of the same inherited ring.

Theorem 24.2 (literal common-overlap coordinates). Two literal maps into one ring, each presented as a one-variable polynomial chart followed by localization at its coordinate, determine canonical coordinate-inversion endomorphisms of that ring. A supplied Laurent overlap presentation identifies both endomorphisms wholly with its identity.

> **Formal status — checked and assumption-explicit.** Evidence `LAURENT-COMMON-OVERLAP`. The identity paths are whole paths of structured maps, not elementwise equations gathered into an external compatibility square. No claim is made that every pair of localization presentations admits those paths.

24.4 The Sieve Beneath The Principal Region

The notation $D(t)$ in (24.2) is best read through the principle developed in Chapters 18 and 22. Before it is an open, invertibility is an ordinary sieve. On the chart U_0, the sieve $D_{U_0}(t)$ asks of a probe $v : V \to U_0$ whether the restricted coordinate $t|_V$ is a unit. It is defined whether or not the site possesses a chosen open object representing that question.

Localization is the algebraic representation theorem. A map out of $A[t, t^{-1}]$ is the same data as a map out of $A[t]$ for which t becomes invertible. Thus the selected restriction $\rho_0 : R_0 \to L$, when supplied as localization at t, gives the overlap ring the universal property expected of the region $D(t)$. The second restriction gives the same actual ring the universal property expected of $D(u)$.

This is the projective-line instance of the sieve-first insight:

$$\begin{aligned} \text{invertibility question} &\longrightarrow \text{ordinary sieve} \\ &\longrightarrow \text{represented principal region.} \end{aligned} \tag{24.11}$$

The first arrow is definitional; the second is a theorem or supplied presentation. In the present package, the actual geometric intersection is already selected, and its two ring restrictions are presented as the relevant localizations. The Laurent identities then say that the two coordinate answers agree on that one region.

There is a useful restraint here. The package does not prove a general site-level equality between every selected chart intersection and an abstract sieve classifier. It connects the actual overlap to the principal-open story at the literal coordinate-ring and restriction-map endpoints. Chapter 22's pointwise representability theorem explains why those localization endpoints have the intended invertibility meaning. A whole comparison between arbitrary site presentations would be additional geometry.

24.5 Laurent Coordinates On The Actual Scheme Overlap

Return to a binary site-relative scheme S. Its global object and structure presheaf are already retained once. Its two chart realizations are already affine. After a selected product in the slice supplies the actual intersection Ω, the ring L and both maps (24.8) are derived by evaluating that single presheaf. The Laurent adapter adds only a common base ring A and a coordinate presentation of those exact endpoints:

$$\mathsf{Laurent}(S, \Omega) = \sum_{A:\mathsf{CommRing}} \mathsf{LaurentOverlap}\big(A, R_0, R_1, L, \rho_0, \rho_1\big). \tag{24.12}$$

The dependency in (24.12) is the point. One cannot quietly substitute an isomorphic overlap ring, replace a restriction map, or attach the coordinates to a different pair of charts. The types refer to the rings and maps already computed from S and Ω. Conversely, the adapter does not pretend to disc-

over a common base ring or polynomial structures automatically. Those are the mathematical coordinate data being supplied.

Theorem 24.3 (actual-overlap Laurent adapter). Given a supplied binary site-relative scheme and its selected actual chart intersection, a common base ring and Laurent coordinate presentation can be attached directly to the literal chart rings, overlap ring, and inherited restriction maps, without duplicating any of those global owners.

> **Formal status — checked.** Evidence `ACTUAL-SCHEME-LAURENT-OVERLAP`. This is a thin dependent adapter. It does not add a transition or cocycle field to the general scheme presentation, construct the overlap, infer Laurent coordinates from arbitrary charts, or glue a global scheme from them.

24.6 The Supplied Projective-Line Total

All the pieces can now be named as one object. For an ambient category $\mathcal{K}$, define the supplied projective-line presentation by the dependent total

$$\mathsf{PLine}_{\mathrm{sup}}(\mathcal{K}) = \sum_{S:\mathsf{Scheme}^{(2)}_{\mathcal{K}}} \left(\sum_{\Omega:\mathsf{Overlap}(S)} \mathsf{Laurent}(S,\Omega) \right). \tag{24.13}$$

The first component is the already-global site-relative scheme of Chapter 23. It owns the ringed object, local-ring certificate, covering sieve, two constructively generating charts, and their whole affine realizations. The second component selects their actual intersection. The third says that the two actual structure-ring restrictions present one-variable polynomial charts over a common base and satisfy the Laurent identities on the shared ring.

Nothing else is required because nothing else is free-floating. The overlap projections belong to the selected product. The ring restrictions belong to the structure presheaf. Their functoriality belongs to that presheaf as well. The coordinate-transition maps are constructed by the universal properties in Section 24.2. The identity comparisons are retained by the Laurent package. Adding another transition map or cocycle field would create a second account of data that the global object already owns.

Theorem 24.4 (supplied projective-line capability). From an element of (24.13), the global scheme, actual overlap, common base ring, and exact Laurent package are recovered by projection. Its coordinate-inversion endomorphisms are the internally constructed maps of Theorem 24.2 and carry the supplied whole identity paths.

> **Formal status — checked, supplied, and conditional.** Evidence `SUPPLIED-P1`. The total and its observations compute by dependent-pair projection. It neither constructs a closed global object nor proves projectivity or non-affineness.

$$\begin{aligned} \text{supplied global scheme} &\longrightarrow \text{actual chart intersection} \\ &\longrightarrow \text{two localization presentations} \\ &\longrightarrow \text{Laurent coordinate identities.} \end{aligned} \tag{24.14}$$

Read from top to bottom, every arrow in (24.14) has a different logical force. The first passage uses a selected limit in the slice. The second adds algebraic presentations to maps already inherited from the global object. The third constructs transition maps and retains their whole comparison with identity. None of them reverses the chain and constructs the global scheme from the local data.

24.7 What A Construction Of The Line Would Require

There are two natural ways to cross the remaining boundary. The first is atlas-first. Construct $\operatorname{Spec} A[t]$ and $\operatorname{Spec} A[u]$, represent the two invertibility sieves by their localizations, construct the inversion comparison, and prove that the diagram glues effectively. The result must carry a structure sheaf whose restrictions recover the two affine sheaves, a local-ring certificate, and a proof that the selected charts cover. One must then compare the constructed object, wholly, with the supplied presentation (24.13). Merely packaging two charts and an isomorphism cannot replace this effectivity theorem.

The second route is graded. Give $A[x_0, x_1]$ its standard grading and form

$$\mathbf{P}^1_A = \operatorname{Proj} A[x_0, x_1]. \tag{24.15}$$

Here ordinary localization is only an intermediate step. For a homogeneous element x_i, one localizes the graded ring and then takes its degree-zero part. The two standard regions should compute as

$$\begin{aligned} D_+(x_0) &\simeq \operatorname{Spec} A[x_1/x_0], \\ D_+(x_1) &\simeq \operatorname{Spec} A[x_0/x_1]. \end{aligned} \tag{24.16}$$

Their common region is obtained by inverting the displayed ratio, returning exactly the Laurent calculation of this chapter. A complete `Proj` construction therefore needs graded commutative rings, homogeneous localization, a degree-zero functor, the irrelevant ideal or its covering condition, a structure sheaf, and the relevant local-ring and descent theorems. None of these objects is hidden inside the ungraded Laurent owner.

The same route explains projective n-space:

$$\mathbf{P}^n_A = \operatorname{Proj} A[x_0, \dots, x_n]. \tag{24.17}$$

Its $n + 1$ standard charts have coordinates x_j/x_i for $j \neq i$; on intersections, ratios invert and compose. The binary projective-line package is therefore the smallest nontrivial test of the coordinate machinery, not an implementation of the graded theory in disguise. A future construction should instantiate the existing site-relative and Laurent boundaries and prove a whole comparison with them, rather than introduce a competing notion of scheme.

That comparison must recover, chart by chart and on the actual intersections, the same structure-presheaf restrictions and Laurent identities. Until it does, (24.17) describes the destination rather than a second implementation.

> **Formal status — mathematical development and research boundary.** Equations (24.15)--(24.17) describe the standard graded construction and the intended next theorem. No active emdash owner defines graded rings, homogeneous localization, degree zero, an irrelevant ideal, `Proj`, or general projective space. No non-affineness theorem is claimed, even for the supplied line.

24.8 The Boundary Is Part Of The Mathematics

The achievement of this chapter is not that a familiar object has been renamed as a dependent record. It is that the local coordinate calculation takes place on the actual inherited overlap of one global structure presheaf. The two restriction maps are not copied. The Laurent transitions are not postulated as unstructured functions. The equation $u = t^{-1}$ is realized by polynomial and localization universality and compared wholly on one literal ring.

At the same time, existence remains visible. The global object, its cover, its affine realizations, the selected intersection, and the Laurent identity paths are supplied at the points where the current theory needs them. This prevents a computational presentation from being mistaken for an atlas-effectivity theorem. It also turns future work into a precise mathematical program: build the missing global object, then discharge the assumptions of the existing consumer.

The larger methodological lesson reaches beyond projective geometry. Actual presheaves, sieves, sites, rings, and categorical limits can be computationally internal because they live inside a functorial type theory whose equations are checked by an outer logical framework. One need not replace them with an abstract modal object language in order to compute. But internal computation does not abolish mathematical hypotheses. It makes their location observable.

The third spiral of the book therefore ends where a good construction should end: not with a vague promise that gluing will work, and not with a denial that gluing matters, but with a coordinate theorem on an honest overlap and a clear account of the global theorem still owed.

Appendix A. Notation

The book follows `reports/REPORT_EMDASH_V3_2_CANONICAL_SURFACE_SYNTAX_2026-06-05.md`. This appendix records the compact notation used in the mathematical line of the book. It is a reading guide, not a proposal to make every glyph parser syntax.

Book notation	Reading	Current implementation witness
$a \to_C b$	an arrow of C from a to b	`Hom C a b`
$F : A \vdash B$	a functor from A to B	`Functor A B`
$E : K \vdash \mathsf{Cat}$	a Cat-valued directed family	`Catd K`
$E[f]$	functorial action of a family on a base arrow	`catd_transport_func`
H_x	the based hom-category $\mathrm{Hom}_W(*, x)$	`Hom_cat WalkingEnd_cat walking_base x`
$W, *, \ell$	WalkingEnd, its base, and its directed generator	`WalkingEnd_cat`, `walking_base`, `walking_loop`
Code	the Nat-valued directed family over W	`walking_Code_catd`
$\mathsf{encode}_x(p)$	apply $\mathsf{Code}[p]$ to zero	`walking_encode`
ℓ^n or $\mathsf{power}(n)$	the nth generator-prefix power	`walking_power`
$\mathsf{decode}_x(c)$	the object action of the contextual decoder	`walking_directed_decode_funcd`
ν_p	the directed normalization cell $p \to \mathsf{decode}_x(\mathsf{encode}_x(p))$	`walking_directed_normalization_cell`
$\simeq$	an explicitly stated equivalence interface	in Theorem 8.1, `walking_hom_nat_type_equiv`
$F[f]$	functor action on an arrow	`fapp1_fapp0` and its iterable `fapp1_func` owner
$u_*(g)$	postcomposition by u, namely $u \circ g$	`hom_postcomp_fapp0`
$u^*(h)$	precomposition by u, namely $h \circ u$	`hom_precomp_along_fapp0`
$\eta[f]$	off-diagonal action of a transfor on $f : x \to y$	`tapp1_fapp0`
$\chi^{\Phi}_{(p,u)}$	displayed transport-comparison component	`fdapp1_int_cell`
$P : A \nrightarrow B$	a Cat-valued profunctor on $A^{\mathrm{op}} \times B$	`Prof A B`
U_A	the unit hom profunctor	`Unit_prof A`
$P \otimes_B Q$	selected fixed-middle profunctor tensor	`Prof_tensor P Q`
$F \dashv G$	adjunction data with selected triangle cuts	`Adjunction F G`
$\mathrm{Cone}_W(F)$	the weighted-cone profunctor	`WeightedCone_prof F W`
$\mathrm{IsWeightedLimit}(F, W, L)$	a chosen representation of weighted cones	`IsWeightedLimit_cov_comp F W L`
$\mathrm{Cocone}_W(F)$	the opposite-dual weighted-cocone profunctor	`WeightedCocone_prof F W`

$A \star B$	directed join with left-to-right cross arrows	`Join_cat A B`
$X : \mathcal{K}^{\mathrm{op}} \to \mathsf{Cat}$	a Cat-valued presheaf on $\mathcal{K}$	`Psh K`
yU	the representable presheaf $\mathrm{Hom}_{\mathcal{K}}(-, U)$	`yoneda_psh K U`
$p^* R$	pullback of the sieve R by postcomposing its probes with p	`sieve_pullback K V U p R`
$D_U(s)$	the sieve of probes along which the restricted section is invertible	`comm_ring_psh_invertibility_sieve K O U s`
$J(U, R)$	the proposition that R covers U in a Grothendieck topology	`groth_topology_covers K J U R`
$\widehat{R}$	the whole presheaf extension of a sieve into its representable	`ordinary_sieve_extension_psh K U R`
$\rho_{R,X}$	restriction from sections over U to matching families on R	`ordinary_sieve_local_precomp K U R X`
$a_J P$ or aP	direct Cat-valued cover completion of P on the fixed site $(\mathcal{K}, J)$	`DirectCoverCompletionPsh K J P`
$\eta_P : P \to aP$	return/unit of direct cover completion	`direct_cover_completion_unit K J P`
glue_q	whole amalgamation functor for one eligible cover question	`direct_cover_completion_glue_func K J P U R covers`
$\mathbf{CRing}$	the one-category of set-carrier commutative rings and structured maps	`CommRing_cat`
$\sum_i a_i f_i = 1$	a finite unimodular certificate for the generators (f_i)	`CommRingUnimodularPresentation R n generators`
$R[X]$	a supplied free commutative R-algebra on the variable classifier X	`CommRingPolynomialAlgebra R X`
$R[1/f]$	the target of a supplied universal-property localization at f	`comm_ring_localization_target R f localization`
$R[1/(fg)] \simeq R[1/f][1/g]$	whole product/iterated-localization comparison	`comm_ring_iterated_localization_comparison_omega_equiv`
$\mathrm{Sp}(R)(S)$	affine S-points, namely structured maps $R \to S$	`AffineSpecPoint R S`
$D_R(f)(S)$	affine S-points at which the image of f is a unit	`AffineSpecBasicOpenPoint R f S`
$\mathcal{O}_{\mathrm{coord}}$	the whole coordinate-ring presheaf on the big affine slice over $\mathrm{Sp}(R)$	`affine_spec_coordinate_psh R`
$J^{\mathrm{big}}_{\mathrm{Zar}}(R)$	the least topology generated by selected finite unimodular localization charts	`affine_spec_big_zariski_topology R`

$\mathcal{O}\|_U$	the whole ambient structure presheaf restricted along the domain functor of $\mathcal{K}/U$	`reflective_comm_ringed_site_slice_ambient_psh K A U`
$\mathsf{Scheme}^{(2)}_{\mathcal{K}}$	a binary site-relative scheme presentation over the selected categorical semantics	`BinarySiteRelativeSchemePresentation K`
$U_0 \times_X U_1$	a selected actual product of two chart objects in the conventional slice $\mathcal{K}/X$	`BinarySchemeChartOverlapPresentation K S`
$A[t,t^{-1}] \to A[u,u^{-1}]$	the canonical coordinate-inversion map between supplied one-variable localizations	`comm_ring_laurent_transition_map A P Q LP LQ`
$\mathsf{Laurent}(S,\Omega)$	a common base and Laurent presentation on the literal rings and restrictions of the actual overlap Ω	`BinarySchemeLaurentOverlapPresentation K S overlap`
$\mathsf{PLine}_{\mathrm{sup}}(\mathcal{K})$	an already-global binary scheme, its actual overlap, and its Laurent coordinate package	`SuppliedProjectiveLinePresentation K`

The bounded executable text bridge uses four intrinsic categorical lambda modes:

```
λ^f  x : A. ...
λ^n  k : K. ...
λ^fd a : E. ...
λ^nd k : K. ...
```

The superscript belongs to the lambda: it specifies ordinary functorial, natural/indexed, displayed-functorial, or displayed-natural variation. The classifier annotation after the variable may be omitted when an expected classifier supplies it bidirectionally, but the binder mode itself is not inferred from that annotation. Thus the book's mathematical telescope declaration $k :^{n} K$ corresponds to binding with `λ^n k : K. ...`; the two notations have the same mode reading without being character-for-character surface syntax. Ordinary object binding in the outer logical framework uses its ordinary dependent lambda rather than a categorical `^o` mode.

Composition is written in diagrammatic reading order as $g \circ f$: first f, then g. For the concrete model BNat, this agrees with the implemented convention $g \circ f = g + f$, where Nat addition recurses in its left argument.

The two star actions are deliberately variance-separated. If $g : w \to x$ and $u : x \to y$, then $u_*(g) : w \to y$. If $u : x \to y$ and $h : y \to z$, then $u^*(h) : x \to z$. Thus the formula $f^*(g) = g \circ f$ names **precomposition** and belongs to `hom_precomp_along_*`; it is not the postcomposition owner with a typographic variation.

These formulas are categorical, not specifically functor-categorical. Their general reading takes w, x, y, z to be objects and the displayed arrows to be arrows of an arbitrary ambient category K. The specialization $K = \mathsf{Cat}$ makes those objects categories and those arrows functors; that is the currently checked product/projection instance used in Chapter 9.

$\mathsf{Path}(A)$ denotes the equality-path category on a carrier A. It must not be confused with a directed hom-category. In particular, the objects of H_x are arrows of W, whereas arrows of H_x are directed 2-cells between those arrows.

> **Formal status — mathematical development with a checked executable subset.** The four categorical binder spellings, neutral application, and selected structural forms are implemented only for the reviewed profiles. The rest of the appendix remains mathematical notation, not an assertion that the complete book grammar is parsed.

Appendix B. Emdash Evidence

The prose cites stable evidence IDs rather than brittle line numbers. The register behind this appendix names a claim's status, its active owner, and the diagnostic or reviewer surface that exercises it. During assembly, the table below is generated directly from `book/evidence.json`; editing the table here is therefore neither necessary nor possible.

An **owner** is the declaration that supplies the interface used by the book. A **reviewer/check** is a separate executable occurrence or deliberately searched check phrase. “Formal consequence” rows name checked premises but also say that the displayed theorem has not yet been packaged. “Research boundary” rows deliberately have no purported proof owner.

Here **MathOps** means the discipline of keeping formal owners, executable reviews, exposition, and published artifacts connected without confusing their authority. The chain is intentionally one-way:

```
owner -> reviewer -> register -> prose -> artifact
```

Later stages may detect drift or preserve provenance, but they do not become new proof authorities. In particular, a reproducible PDF certifies the book artifact, not the mathematics printed inside it.

> **Formal status — checked.** This appendix describes traceability; the accompanying emdash artifact remains the proof authority. The evidence checker validates every row before this generated view is assembled.

Evidence	Status	Claim	Owners	Reviewer/check evidence
`TT-EQUALITY-INDUCTION`	checked	The equality-local foundation has reflexivity, right-based dependent equality induction, nondependent path action, and dependent path action.	`eq_refl emdash3_2.lp` `ind_eqr emdash3_2.lp` `eq_ap emdash3_2.lp` `eq_apd emdash3_2.lp`	`eq_apd examples/path_category.lp`
`TT-ELEMENTARY-INDUCTION`	checked	Empty, Unit, Bool, and Nat have decoded classifiers, and the nontrivial elementary inductive carriers have dependent eliminators with constructor computation.	`Empty_grpd emdash3_2.lp` `Unit_grpd emdash3_2.lp` `Bool_grpd emdash3_2.lp` `Nat_grpd emdash3_2.lp` `empty_elim emdash3_2.lp` `bool_elim emdash3_2.lp` `nat_elim emdash3_2.lp`	`nat_elim emdash3_2_checks.lp`
`TT-SIGMA-PI-PATHS`	checked	The groupoid layer has dependent-pair and dependent-function classifiers, observational Sigma paths, and a checked happly/funext equivalence for Pi paths.	`sigma_Fst emdash3_2.lp` `sigma_Snd emdash3_2.lp` `PiHapply emdash3_2.lp` `PiFunext emdash3_2.lp` `pi_happly_type_equiv emdash3_2.lp`	`PiFunext examples/pi_funext.lp`
`CAT-ITERATED-HOMS`	checked	A category has an object classifier and category-valued homs, so higher cells are represented by iterating Hom.	`Cat emdash3_2.lp` `Obj emdash3_2.lp` `Hom emdash3_2.lp`	`Hom emdash3_2_checks.lp`
`CAT-PATH-CATEGORY`	checked	Path(A) internalizes equality as a groupoidal category, and an ordinary function induces an iterable functor between path categories.	`Path_cat emdash3_2.lp` `Path_cat_func emdash3_2.lp` `path_map_func emdash3_2.lp`	`path_map_func examples/path_category.lp`

CAT-FUNCTOR-CALCULUS	checked	Ordinary functors expose object and iterated-hom action, with generic identity and composition computation rather than constructor-specific functor laws.	Functor emdash3_2.lp fapp0 emdash3_2.lp fapp1_func emdash3_2.lp fapp1_fapp0 emdash3_2.lp id_func emdash3_2.lp comp_cat_fapp0 emdash3_2.lp	fapp1_fapp0 emdash3_2_checks.lp
CAT-PRODUCT-CALCULUS	checked	Binary product categories have projection functors and a stable two-sided product-map constructor whose object, arrow, and projection observations compute componentwise.	Product_cat emdash3_2.lp Product_projL_func emdash3_2.lp Product_map_func emdash3_2.lp	text product/projection cut used by the book emdash3_2_checks.lp text General two-sided product maps used by pro-functor reindexing emdash3_2_checks.lp
CUT-PRODUCT-PROJECTION	formal-consequence	In the Cat-specialized case, for h:A0→A1, k:B0→B1, and g:A1→C, the readable equation pi1^*(g) composed with h×k equals pi1'*^(g composed with h); its owner-aligned projection and nested-precomposition forms are checked, while the literal raw projection-composite equality is not packaged.	hom_precomp_along_fapp0 emdash3_2.lp Product_projL_func emdash3_2.lp Product_map_func emdash3_2.lp comp_assoc emdash3_2.lp	text product/projection cut used by the book emdash3_2_checks.lp
CAT-HOM-CUTS	checked	Represented-hom postcomposition and precomposition have distinct stable full and capped owners with identity, consecutive-action, adjacent-cut, and proof-time ordinary-composition comparisons.	hom_postcomp_func emdash3_2.lp hom_postcomp_fapp0 emdash3_2.lp hom_precomp_along_func emdash3_2.lp hom_precomp_along_fapp0 emdash3_2.lp	text Hom-action functoriality joins for postcomposition emdash3_2_checks.lp text Hom-action functoriality joins for precomposition emdash3_2_checks.lp

`CAT-TRANSFOR-CALCULUS`	checked	Transformations form the next hom between functors and expose point components together with higher naturality action.	`Transf` `emdash3_2.lp` `tapp0_fapp0` `emdash3_2.lp` `tapp1_fapp0` `emdash3_2.lp`	`tapp1_fapp0` `emdash3_2_checks.lp`
`CAT-DIRECTED-FAMILIES`	checked	A Cat-valued directed family has fibres and functorial reindexing, while a family morphism has fibre functors and directed comparison cells.	`Catd` `emdash3_2.lp` `Fibre_cat` `emdash3_2.lp` `catd_transport_func` `emdash3_2.lp` `Functord` `emdash3_2.lp` `fdapp1_int_cell` `emdash3_2.lp`	`fdapp1_int_cell` `examples/dependent_hom_laxity.lp`
`CAT-SIGMA-PI`	checked	Directed families have a Sigma total category, canonical transport arrows, and a Pi section category with coherent evaluation.	`Sigma_cat` `emdash3_2.lp` `sigma_transport_arrow` `emdash3_2.lp` `Pi_cat` `emdash3_2.lp` `piapp0` `emdash3_2.lp`	`sigma_transport_arrow` `emdash3_2_checks.lp`

CAT-FIBREWISE-CONTEXT	checked	Fixed-base fibrewise products of displayed families have displayed projections and pairing whose fibre, base-arrow, internalized-cell, higher, and projection-after-pairing observations compute componentwise; swap and diagonal are derived from those owners.	Product_projL_funcd emdash3_2.lp Product_projR_funcd emdash3_2.lp Product_pair_funcd emdash3_2.lp	Product_projL_funcd_higher_check emdash3_2_checks.lp Product_projR_funcd_higher_check emdash3_2_checks.lp Product_pair_funcd_higher_check emdash3_2_checks.lp text The canonical internalized cell of displayed pairing is componentwise. emdash3_2_checks.lp text Whole displayed universal-property betas. emdash3_2_checks.lp
CAT-BASE-CHANGE-TOTALIZATION	checked	Pullback totalization sends a base-changed dependent pair (a,u) to (F[a],u) and sends its total arrow by the functorial base action while retaining the fibre component.	sigma_pullback_total_func emdash3_2.lp	text Pullback totalization exposes exactly its base-changed dependent pair. emdash3_2_checks.lp

`CAT-DISPLAYED-EVALUATION`	checked	Constant-domain displayed evaluation projects fibrewise to ordinary functor evaluation and retains generic base/higher action, while displayed terminal weakening projects to the ordinary terminal functor.	`Eval_funcd` `emdash3_2.lp` `Terminal_funcd` `emdash3_2.lp`	text `The stable displayed evaluator projects to ordinary product evaluation.` `emdash3_2_checks.lp` `Displayed_eval_higher_check` `emdash3_2_checks.lp` text `Displayed terminal weakening projects to the ordinary unique functor.` `emdash3_2_checks.lp`
`CAT-REPRESENTABLE`	checked	The fixed-source representable family has fibre Hom(x,y), and its source action is precomposition.	`Rep_catd` `emdash3_2.lp` `Rep_transport_func` `emdash3_2.lp`	`Rep_catd` `examples/path_induction_transitivity.lp`
`LOGIC-TRUNCATION-PREDICATE`	checked	Contractibility and recursive truncation evidence define the proposition, set, and groupoid levels as properties of existing classifiers.	`IsContr` `emdash3_2.lp` `IsTruncGrpd` `emdash3_2.lp` `IsPropGrpd` `emdash3_2.lp` `IsSetGrpd` `emdash3_2.lp`	`IsTruncGrpd` `examples/truncation_monotonicity.lp`
`LOGIC-NAT-SETHOOD`	checked	Unit and Empty are proposition-valued and Nat is set-valued by explicit internal evidence.	`unit_is_prop` `emdash3_2_nat_arithmetic.lp` `empty_is_prop` `emdash3_2_nat_arithmetic.lp` `nat_is_set` `emdash3_2_nat_arithmetic.lp`	`nat_is_set` `examples/walking_endomorphism_nat_prerequisites.lp`
`LOGIC-TRUNCATION-EVIDENCE-PROP`	checked	For every truncation level and classifier, the classifier of truncation evidence is proposition-valued.	`is_trunc_grpd_evidence_is_prop` `emdash3_2.lp`	`is_trunc_grpd_evidence_is_prop` `examples/truncation_evidence_property.lp`

EQUIV-TYPE	checked	TypeEquiv packages a forward map with contractible homotopy fibres and exposes a selected inverse with both inverse paths.	HFiber emdash3_2.lp IsEquivMap emdash3_2.lp TypeEquiv emdash3_2.lp type_equiv_from emdash3_2.lp type_equiv_left emdash3_2.lp type_equiv_right emdash3_2.lp	TypeEquiv examples/type_equiv_algebra.lp
EQUIV-TYPE-ALGEBRA	checked	Type equivalences have reflexivity, symmetry, and categorical-order composition with checked forward-map computation.	type_equiv_refl emdash3_2.lp type_equiv_sym emdash3_2.lp type_equiv_comp emdash3_2.lp	type_equiv_comp examples/type_equiv_algebra.lp
UNIV-GROUPOID	checked	The groupoid universe has a decoder-oriented univalence equivalence between universe paths and TypeEquiv, with named round trips and transport agreement.	grpd_univalence_type_equiv emdash3_2.lp grpd_equiv_path emdash3_2.lp grpd_equiv_path_idtoequiv emdash3_2.lp idtoequiv_grpd_equiv_path emdash3_2.lp	grpd_univalence_from_decoder examples/grpd_univalence_decoder.lp
UNIV-TRUNCATED	checked	For packaged n-truncated classifiers, package equality is equivalent to TypeEquiv of the retained carriers, with named encode/decode round trips.	trunc_grpd_univalence_type_equiv emdash3_2.lp trunc_grpd_idtoequiv emdash3_2.lp trunc_grpd_equiv_path emdash3_2.lp	trunc_grpd_univalence_type_equiv examples/truncation_universe_univalence.lp

EQUIV-OMEGA	checked	OmegaEquivAlong carries equality-valued inverse-arrow data, OmegaEquiv packages a selected arrow, and an object path has a transparent computational equivalence package.	OmegaEquivAlong emdash3_2.lp OmegaEquiv emdash3_2.lp object_path_equiv emdash3_2.lp	object_path_equiv examples/equality_valued_omega_equivalence.lp
EQUIV-EVIDENCE-PROP	checked	For every fixed arrow, its equality-valued OmegaEquivAlong evidence is proposition-valued without a finite-dimensional hypothesis.	omega_equiv_along_evidence_is_prop emdash3_2_eq1_evidence_property.lp	omega_equiv_along_evidence_is_prop examples/equality_valued_omega_equivalence_evidence_property.lp
EQUIV-HOM-ACTION	checked	Equality-valued equivalence evidence has a one-way next-hom action, and coherent groupoidality makes directed fibre transport an equivalence.	omega_equiv_along_fapp1 emdash3_2_eq1_hom_action.lp groupoidal_fibre_transport_equiv emdash3_2_eq1_hom_action.lp	omega_equiv_along_fapp1 examples/equality_valued_omega_equivalence_hom_action.lp
EQUIV-ORDINARY-ISO-LIFT	checked	Ordinary categorical isomorphism evidence has a one-way lift to the native equality-valued OmegaEquiv package.	IsoEvidence emdash3_2.lp iso_evidence_omega_equiv emdash3_2.lp	iso_evidence_omega_equiv emdash3_2_checks.lp
UNIV-FULL-OBJECT-ISO	research-boundary	A full native equivalence between arbitrary categorical object equality and ordinary isomorphism evidence is not part of the selected API.	—	—
IND-PATHOUT	checked	PathOut_Z(x) is the Sigma total of the fixed-source representable, with a reflexive object and a canonical rho arrow to every outgoing arrow.	PathOut_cat emdash3_2.lp pathout_refl_obj emdash3_2.lp pathout_refl_arrow emdash3_2.lp	pathout_refl_arrow emdash3_2_checks.lp
IND-ARROW	checked	Fixed-source arrow induction extends data at the reflexive outgoing arrow to a section, and the theorem is internalized functorially as fixed-source and varying-source interfaces.	path_ind_sec emdash3_2.lp PathInd_func emdash3_2.lp PathInd_transfd emdash3_2.lp	path_ind_sec examples/path_induction_transitivity.lp

`IND-COMPOSITION`	checked	Applying arrow induction to the represented composition motive yields a functor whose object action computes to categorical composition.	`CompMotive_cat` `d` `emdash3_2.lp` `path_comp_sec` `emdash3_2.lp` `path_comp_fun` `c` `emdash3_2.lp`	`path_comp_fun` `c` `examples/path_` `induction_tran` `sitivity.lp`
`IND-PATH-COMPARISON`	checked	For a literal path-category base, structured directed transport agrees propositionally with primitive right-based equality induction.	`path_cat_struc` `tured_transpor` `t_agrees_ind_e` `qr` `emdash3_2.lp`	`path_cat_struc` `tured_transpor` `t_agrees_ind_e` `qr` `examples/group` `oidal_structur` `ed_j_eq1.lp`
`IND-GENERAL-INITIALITY`	research-boundary	A general theorem identifying all selected categorical induction interfaces with homotopy-initial algebras has not been packaged.	—	—
`DHIT-DERIVED-ELIMINATORS`	checked	The contextual WalkingEnd eliminator specializes to a dependent section and an ordinary recursor with base and generator observations.	`walking_end_in` `d_sec` `emdash3_2_walk` `ing_end_hit.lp` `walking_end_re` `c_func` `emdash3_2_walk` `ing_end_hit.lp` `walking_end_re` `c_beta_base` `emdash3_2_walk` `ing_end_hit.lp` `walking_end_re` `c_beta_loop_or` `dinary` `emdash3_2_walk` `ing_end_hit.lp`	`walking_end_re` `c_func` `examples/walki` `ng_endomorphis` `m_hit.lp`
`DHIT-GENERAL-SCHEMA`	research-boundary	The selected WalkingEnd interface does not yet constitute a reusable schema for arbitrary directed higher-inductive categories, pushouts, or cell complexes.	—	—

TRUNC-CLOSURE	checked	Recursive truncation evidence is monotone and has checked dependent Pi and same-level dependent Sigma closure operations.	is_trunc_grpd_succ emdash3_2.lp is_trunc_pi emdash3_2.lp is_trunc_sigma emdash3_2.lp	is_trunc_pi examples/truncation_pi_closure.lp is_trunc_sigma examples/truncation_sigma_closure.lp
TRUNC-RETRACT	checked	Every recursive truncation level is closed under explicit retractions.	is_trunc_retract emdash3_2_eq1_evidence_property.lp	is_trunc_retract emdash3_2_checks.lp
CAT-DIMENSION	checked	IsNCat defines finite directed height by hom recursion, and every such category has the recursively predicted object-truncation evidence.	IsNCat emdash3_2.lp cat_dim_trunc_level emdash3_2.lp ncat_obj_trunc emdash3_2_eq1_evidence_property.lp	IsNCat examples/directed_dimension.lp ncat_obj_trunc examples/equality_valued_omega_equivalence_evidence_property.lp
TRUNC-REFLECTOR	research-boundary	The active truncation layer supplies properties and packaged truncated universes, not a general truncation reflector or arbitrary truncation HIT.	—	—
WE-SIGNATURE	checked	WalkingEnd has an opaque category, base, directed loop, and explicit one-dimensional evidence.	WalkingEnd_cat emdash3_2_walking_end_hit.lp walking_base emdash3_2_walking_end_hit.lp walking_loop emdash3_2_walking_end_hit.lp walking_end_is_one_cat emdash3_2_walking_end_hit.lp	walking_end_is_one_cat examples/walking_endomorphism_hit.lp
WE-ONE-DIMENSIONAL	checked	Every WalkingEnd hom-category is discrete, without WalkingEnd itself becoming discrete.	walking_end_hom_discrete emdash3_2_walking_end_hit.lp	walking_end_hom_discrete emdash3_2_checks.lp

`WE-CONTEXTUAL-ELIMINATOR`	checked	The contextual WalkingEnd eliminator has base and loop computation at the selected displayed observers.	`walking_end_ind_funcd` `emdash3_2_walking_end_hit.lp` `walking_end_ind_funcd_beta_base` `emdash3_2_walking_end_hit.lp` `walking_end_ind_funcd_beta_loop` `emdash3_2_walking_end_hit.lp`	`walking_end_ind_funcd` `examples/walking_endomorphism_hit.lp`
`WE-BNAT-MODEL`	checked	BNat is a separate one-object Nat-monoid model and receives a functor from WalkingEnd.	`BNat_cat` `emdash3_2_walking_end_hit.lp` `bnat_comp_nat_add` `emdash3_2_walking_end_hit.lp` `walking_bnat_model_func` `emdash3_2_walking_end_hit.lp`	`walking_bnat_model_func` `examples/walking_endomorphism_hit.lp`
`WE-CODE`	checked	Code sends the walking base to Path(Nat) and the walking loop to the Nat successor functor.	`walking_Code_catd` `emdash3_2_walking_end_hit.lp` `walking_Code_beta_base` `emdash3_2_walking_end_hit.lp` `walking_Code_beta_loop` `emdash3_2_walking_end_hit.lp`	`walking_Code_catd` `examples/walking_endomorphism_hit.lp`
`WE-ENCODE`	checked	Encode acts with Code on a based arrow and evaluates at zero.	`walking_encode` `emdash3_2_walking_end_hit.lp`	`walking_encode` `examples/walking_endomorphism_hit.lp`
`WE-POWER`	checked	Natural powers map zero to identity and successor to generator-prefix composition.	`walking_power` `emdash3_2_walking_end_hit.lp` `walking_power_func` `emdash3_2_walking_end_hit.lp`	`walking_power` `examples/walking_endomorphism_hit.lp`

WE-SPIRAL	checked	The selected explicit-kappa directed spiral supplies the loop coherence for powers.	walking_power_spiral emdash3_2_walking_end_hit.lp walking_power_spiral_cell emdash3_2_walking_end_hit.lp	walking_power_spiral_cell examples/walking_endomorphism_hit.lp
WE-CONTEXTUAL-DECODER	checked	The contextual decoder maps Code to the based representable and computes to powers at the base.	walking_directed_decode_funcd emdash3_2_walking_end_hit.lp walking_directed_decode_beta_base emdash3_2_walking_end_hit.lp	walking_directed_decode_funcd examples/walking_endomorphism_hit.lp
WE-NORMALIZATION-CELL	checked	Every based arrow has a directed normalization cell toward decode(encode(p)).	walking_directed_normalization_cell emdash3_2_walking_end_hit.lp	walking_directed_normalization_cell examples/walking_endomorphism_hit.lp
WE-NORMALIZATION-PATH	checked	Hom-discreteness converts the directed normalization cell into equality.	walking_directed_normalization_path emdash3_2_walking_end_hit.lp	walking_directed_normalization_path emdash3_2_checks.lp
WE-POWER-ENCODE	checked	For a based endomorphism p, power(encode(p)) equals p.	walking_power_encode_roundtrip emdash3_2_walking_end_hit.lp	walking_power_encode_roundtrip examples/walking_endomorphism_hit.lp
WE-ENCODE-POWER	checked	For every natural n, encode(power(n)) equals n.	walking_encode_power_roundtrip emdash3_2_walking_end_hit.lp	walking_encode_power_roundtrip examples/walking_endomorphism_hit.lp
WE-HOM-NAT-CARRIER	checked	The underlying based-endomorphism carrier is equivalent to Nat.	walking_hom_nat_type_equiv emdash3_2_walking_end_hit.lp walking_hom_nat_omega_equiv emdash3_2_walking_end_hit.lp	walking_hom_nat_type_equiv examples/walking_endomorphism_hit.lp

WE-HOM-SETHOOD	checked	The based-endomorphism carrier is a set both from dimension and from the Nat equivalence.	walking_end_hom_is_set_from_dimension emdash3_2_walking_end_hit.lp walking_end_hom_is_set_from_equiv emdash3_2_walking_end_hit.lp	walking_end_hom_is_set_from_dimension examples/walking_endomorphism_hit.lp
WE-LOOP-NOT-IDENTITY	checked	The walking generator is not equal to the identity.	walking_loop_not_identity emdash3_2_walking_end_hit.lp	walking_loop_not_identity emdash3_2_checks.lp
WE-LOOP-NO-RIGHT-INVERSE	checked	The walking generator has no right inverse.	walking_loop_no_right_inverse emdash3_2_walking_end_hit.lp	walking_loop_no_right_inverse emdash3_2_checks.lp
WE-LOOP-NONINVERTIBLE	checked	The walking generator carries no native omega-equivalence evidence.	walking_loop_not_omega_equiv emdash3_2_walking_end_hit.lp	walking_loop_not_omega_equiv examples/walking_endomorphism_hit.lp
WE-ENCODE-PREFIX	checked	Encoding a generator-prefixed based endomorphism produces the successor of its code.	walking_encode_loop_prefix_path emdash3_2_walking_end_hit.lp	walking_encode_loop_prefix_path examples/walking_endomorphism_hit.lp
WE-STRUCTURED-ENCODER	checked	The based encoder is also packaged as a functor from the based hom-category to Path(Nat).	walking_encode_func emdash3_2_walking_end_hit.lp	walking_encode_func examples/walking_endomorphism_hit.lp
WE-COMPOSITION-ADDITION	formal-consequence	Compatibility of based composition with Nat addition follows from the checked power recursion, inverse laws, and ordinary category/Nat induction, but is not packaged as a monoid equivalence.	walking_encode_loop_prefix_path emdash3_2_walking_end_hit.lp walking_power_encode_roundtrip emdash3_2_walking_end_hit.lp walking_encode_power_roundtrip emdash3_2_walking_end_hit.lp	—

`WE-FULL-CATEGORICAL-COMPARISON`	research-boundary	A reverse BNat functor, full hom-category equivalence, and functor-category initiality require additional reusable infrastructure.	—	—
`WE-GROUP-COMPLETION`	research-boundary	Group completion toward BInt or the circle comparison has not been implemented.	—	—
`TRANSF-POINT-OFFDIAGONAL`	checked	An ordinary transfor has point components and an iterable off-diagonal hom action from F(x) to G(y) along every source arrow x to y.	`Transf_cat` `emdash3_2.lp` `tapp0_fapp0` `emdash3_2.lp` `tapp1_func` `emdash3_2.lp` `tapp1_fapp0` `emdash3_2.lp`	`tapp1_at_transf` `emdash3_2_checks.lp`
`TRANSF-STRICT-NATURALITY`	checked	Postcomposition and precomposition adjacent to an ordinary transfor's off-diagonal action reduce to the action on the corresponding composite source arrow.	text `Full strict naturality for ordinary transfors` `emdash3_2.lp`	text `Full strict naturality: post/left accumulation before capping` `emdash3_2_checks.lp`
`TRANSF-HORIZONTAL-CALCULUS`	checked	The product-composition action supplies an iterable horizontal composite of a pair of ordinary transfors, with checked point, full off-diagonal, and capped off-diagonal projections.	`comp_prod_fapp1_fapp0` `emdash3_2.lp`	`comp_prod_fapp1_fapp0` `emdash3_2_checks.lp`
`TRANSFD-FIBRE-COMPONENTS`	checked	A natural family transformation between displayed functors has a transformation in every fibre and a point component at every fibre object, with identity and vertical composition inherited from the generic transfor calculus.	`Transfd_cat` `emdash3_2.lp` `Fibre_transf` `emdash3_2.lp` `Fibre_transf_app` `emdash3_2.lp`	`Fibre_transf_app` `emdash3_2_checks.lp`
`FUNCTORD-DISPLAYED-LAXITY`	checked	For a natural family morphism and a base arrow, the internal displayed hom action supplies a directed component from target transport after the source fibre functor to the target fibre functor after source transport.	`functord_transport_lhs_func` `emdash3_2.lp` `functord_transport_rhs_func` `emdash3_2.lp` `fdapp1_int_cell` `emdash3_2.lp`	`fdapp1_int_cell` `examples/dependent_hom_laxity.lp`

FUNCTORD-SIGMA-ACTION	checked	The Sigma-total map induced by a natural family morphism sends a total arrow to the same base arrow paired with the capped internal displayed hom action in the fibre.	sigma_map_func emdash3_2.lp fdapp1_int_hom_fapp0 emdash3_2.lp	sigma_map_transf examples/sigma_total.lp
FUNCTORD-WHOLE-LAXITY	research-boundary	A standalone whole-transfor laxity facade between the two transport composites is intentionally deferred; the active owner is the internal displayed hom action and its component projections.	—	—
PROF-CATEGORY	checked	A Cat-valued profunctor from A to B is a directed family on A opposite times B, and vertical profunctor maps are natural family morphisms with the existing identity and composition calculus.	Prof_base emdash3_2.lp Prof_cat emdash3_2.lp ProfMap emdash3_2.lp	text Cat-valued profunctor facade and representable hom action emdash3_2_checks.lp
PROF-REPRESENTABLE	checked	The unit profunctor evaluates to the ambient hom, and reindexing it along endpoint functors gives the binary representable with action by precomposition and postcomposition.	Unit_prof emdash3_2.lp Hom_prof_along emdash3_2.lp Hom_fapp0 emdash3_2.lp	Hom_prof_along emdash3_2_checks.lp
PROF-REINDEXING	checked	Profunctors reindex functorially in both endpoints by pullback along an opposite-times-covariant product map, including identity, nesting, and representable accumulation laws.	Prof_reindex emdash3_2.lp Prof_reindex_func emdash3_2.lp	text Stable profunctor reindexing and its representable fold emdash3_2_checks.lp
PROF-REPRESENTATION-PACKAGE	checked	Representability of a profunctor is expressible as ordinary isomorphism to a conjoint, and a representation package retains a chosen representing functor with that evidence.	Conjoint_prof emdash3_2.lp IsRepresentedBy_iso emdash3_2.lp Representation_iso emdash3_2.lp	Representation_iso emdash3_2_checks.lp

`PROF-COMPARISON-BETA-ETA`	checked	A computational profunctor comparison exposes mutually inverse push and pull operations on every incoming profunctor map, with runtime beta and eta reductions and functorial reindexing and composition.	`ProfComparison` `emdash3_2.lp` `prof_comparison_push` `emdash3_2.lp` `prof_comparison_pull` `emdash3_2.lp`	text `Computational profunctor comparisons and weighted representability` `emdash3_2_checks.lp` `prof_comparison_pull` `emdash3_2_checks.lp`
`PROF-SHAPED-CELLS`	checked	A profunctor cell over endpoint functors is a natural family morphism into a reindexed target; unit-source specialization and narrow cell application provide shaped elements and their action.	`Prof_transf_cat` `emdash3_2.lp` `Prof_hom` `emdash3_2.lp` `Prof_cell_apply` `emdash3_2.lp`	text `Shaped profunctor cells and elements` `emdash3_2_checks.lp`
`PROF-TENSOR`	checked	The selected profunctor tensor is an opaque fixed-middle composite with exposed-endpoint reindexing and a fixed-endpoint bifunctorial map action.	`Prof_tensor` `emdash3_2.lp` `Prof_tensor_func` `emdash3_2.lp` `Prof_tensor_hom_hom` `emdash3_2.lp`	text `Primitive profunctor tensor and exposed-endpoint reindexing` `emdash3_2_checks.lp` `Prof_tensor` `examples/profunctor_weighted_limits.lp`
`PROF-CLOSED-CALCULUS`	checked	The two selected profunctor residuals expose fixed-endpoint evaluation and lambda operations inverse by runtime beta and eta, without asserting semantic end formulas for their opaque objects.	`Prof_imply_cov` `emdash3_2.lp` `Prof_eval_cov_map` `emdash3_2.lp` `Prof_lambda_cov_map` `emdash3_2.lp` `Prof_imply_con` `emdash3_2.lp` `Prof_eval_con_map` `emdash3_2.lp` `Prof_lambda_con_map` `emdash3_2.lp`	`Prof_lambda_cov_map` `emdash3_2_checks.lp` `Prof_lambda_con_map` `emdash3_2_checks.lp` `Prof_lambda_cov_map` `examples/profunctor_weighted_limits.lp`

`PROF-COYONEDA`	checked	Tensoring on either side with the unit profunctor has a natural co-Yoneda map to the original profunctor, with fixed shaped-element beta and naturality-fusion computations.	`Prof_coyoneda_cov_transf emdash3_2.lp` `Prof_coyoneda_con_transf emdash3_2.lp` `Prof_coyoneda_cov_map emdash3_2.lp` `Prof_coyoneda_con_map emdash3_2.lp`	`Prof_coyoneda_cov_transf emdash3_2_checks.lp` `Prof_coyoneda_con_transf emdash3_2_checks.lp`
`PROF-GENERAL-COEND`	research-boundary	The selected opaque tensor and co-Yoneda computations do not yet provide a general coend or coinserter construction, tensor associativity package, or a bicategory of profunctors.	—	—
`YONEDA-FULLY-FAITHFUL`	research-boundary	The active representable and co-Yoneda interfaces do not yet package a general fully faithful Yoneda embedding or the full Yoneda equivalence for arbitrary Cat-valued presheaves.	—	—
`UCAT-IDENTITY-ISOMORPHISM`	mathematical-development	In the ordinary set-valued-hom specialization, a category is a precategory whose identity-to-isomorphism map is an equivalence; its object type is consequently a 1-type.	—	—
`UCAT-FUNCTOR-CATEGORY`	mathematical-development	An ordinary natural transformation is invertible exactly when its components are, and for a precategory A and univalent category B the functor precategory from A to B is univalent, with functor identity corresponding to natural isomorphism.	—	—

`UCAT-ADJOINT-REPRESENTABILITY`	mathematical-development	In ordinary univalent 1-category theory, right adjoints are equivalently coherent representations of the contravariant hom functors obtained by fixing the target of the left adjoint.	—	—
`UCAT-ADJOINT-UNIQUENESS`	mathematical-development	For an ordinary functor whose source is a univalent category, right-adjoint structures are unique in the appropriate identity sense and existence of a right adjoint is a mere proposition.	—	—
`UCAT-EQUIVALENCE-CRITERIA`	mathematical-development	Ordinary adjoint equivalences correspond to fully faithful and split essentially surjective functors, and between univalent categories mere essential surjectivity suffices.	—	—
`UCAT-CATEGORY-IDENTITY`	mathematical-development	Identity of ordinary precategories corresponds to isomorphism of precategories, and for univalent categories this agrees with categorical equivalence; the resulting type of categories is a 2-type.	—	—
`UCAT-YONEDA`	mathematical-development	The ordinary set-valued Yoneda evaluation map is an equivalence, making the Yoneda embedding fully faithful and representing objects unique up to isomorphism, hence up to identity in a univalent category.	—	—
`UCAT-STRICT-CATEGORY`	mathematical-development	An ordinary HoTT strict category is a precategory whose object type is a set; it need not be univalent, and an ordinary univalent category is strict exactly when it is gaunt.	—	—

`UCAT-DAGGER-CATEGORY`	mathematical-development	An ordinary dagger precategory has a chosen identity-on-objects contravariant involution; unitary arrows use the dagger as inverse, and dagger univalence identifies object identity with unitary isomorphism.	—	—
`UCAT-STRUCTURE-IDENTITY`	mathematical-development	For a standard proposition-valued notion of structure over an ordinary univalent category, the precategory of structured objects and structure-preserving arrows is itself univalent.	—	—
`UCAT-REZK-COMPLETION`	mathematical-development	Every ordinary precategory admits a fully faithful and essentially surjective map into a univalent category, constructible by a Yoneda image or a 1-truncated HIT; univalent categories are exactly the targets that invert all such weak equivalences by precomposition.	—	—
`NATIVE-DAGGER-INTERFACE`	research-boundary	The active opposite operation does not provide a chosen identity-on-objects involutive self-duality, coherent higher dagger action, a unitary classifier, or native dagger univalence.	—	—
`NATIVE-STRUCTURE-IDENTITY`	research-boundary	A generic native structure identity theorem requires a directed structure signature, a higher structure-preserving equivalence classifier, qualified base univalence, and coherent off-diagonal and next-hom action.	—	—
`NATIVE-REZK-COMPLETION`	research-boundary	No selected native saturation predicate, Rezk completion object, unit weak equivalence, or iterable higher mapping property is currently implemented.	—	—

ADJ-TRIANGLE-CUTS	checked	An indexed adjunction exposes stable unit and counit observations whose two component-level triangle cuts reduce to the corresponding functorial composites.	Adjunction emdash3_2.lp unit_adj_transf emdash3_2.lp counit_adj_transf emdash3_2.lp	unit_adj_transf examples/adjunction_triangles.lp text Indexed adjunction triangles, opposite, and operation-trust boundary emdash3_2_checks.lp
ADJ-HOM-PROF-COMPARISON	checked	An indexed adjunction supplies a reindexable profunctor comparison between Hom_B(LM,F) and Hom_A(M,RF), whose push and pull inherit the generic comparison beta and eta laws.	Adjunction_hom_prof_comparison emdash3_2.lp Adjunction_hom_prof_comparison_along emdash3_2.lp	Adjunction_hom_prof_comparison_along emdash3_2_checks.lp
WEIGHTED-LIMIT-REPRESENTABILITY	checked	A parameterized weighted-cone profunctor is formed by the selected covariant residual, and its computational representation by a functor exposes inverse push and pull operations on every reindexed incoming profunctor map.	WeightedCone_prof emdash3_2.lp IsWeightedLimit_cov_iso emdash3_2.lp IsWeightedLimit_cov_comp emdash3_2.lp weighted_limit_cov_push emdash3_2.lp weighted_limit_cov_pull emdash3_2.lp	text Computational profunctor comparisons and weighted representability emdash3_2_checks.lp weighted_limit_cov_pull emdash3_2_checks.lp weighted_limit_cov_pull examples/profunctor_weighted_limits.lp
WEIGHTED-LIMIT-SPECIALIZATIONS	formal-consequence	Terminal weights and conjoint weights inhabit the selected weighted-limit classifier, so the general right-adjoint preservation construction specializes to the corresponding conical-limit and right-Kan interfaces; this does not assert their missing semantic end identifications.	Terminal_prof emdash3_2.lp Conjoint_prof emdash3_2.lp IsWeightedLimit_cov_comp emdash3_2.lp right_adjoint_preserves_weighted_limit_cov_comp emdash3_2.lp	text Weighted limit and colimit specialization typing emdash3_2_checks.lp

`WEIGHTED-END-KAN-SEMANTICS`	mathematical-development	With semantic ends and coends, terminal weights recover ordinary cone and cocone categories while conjoint and companion weights recover the standard pointwise right and left Kan extension formulas.	—	—
`DEPENDENT-ADJUNCTIONS`	research-boundary	A general dependent Sigma-change-of-base-Pi adjunction chain requires base-arrow action, off-diagonal and next-hom coherence, and Beck-Chevalley comparisons beyond the current Sigma and Pi family interfaces.	—	—
`OP-DUALITY`	checked	Opposite category, functor, transfor, and adjunction operations expose the selected involutive and variance-reversing computations used by the duality arguments.	`Op_cat emdash3_2.lp` `Op_func emdash3_2.lp` `Op_transf emdash3_2.lp` `Op_adjunction emdash3_2.lp`	`Op_adjunction emdash3_2_checks.lp`
`WEIGHTED-LIMIT-PRESERVATION`	checked	The selected profunctor comparison certifying a weighted limit is transported by an indexed right adjoint to a weighted-limit comparison for the composed diagram and cone point.	`IsWeightedLimit_cov_comp emdash3_2.lp` `right_adjoint_preserves_weighted_limit_cov_comp emdash3_2.lp`	`right_adjoint_preserves_weighted_limit_cov_comp emdash3_2_checks.lp` `right_adjoint_preserves_weighted_limit_cov_comp examples/profunctor_weighted_limits.lp`
`WEIGHTED-COLIMIT-PRESERVATION`	checked	Weighted colimits are presented through opposite weighted limits, and the selected left-adjoint preservation certificate is derived by applying right-adjoint preservation to the opposite adjunction.	`WeightedColimit_con emdash3_2.lp` `left_adjoint_preserves_weighted_colimit_con emdash3_2.lp`	`left_adjoint_preserves_weighted_colimit_con emdash3_2_checks.lp`

`WEIGHTED-COLIMIT-SPECIALIZATIONS`	formal-consequence	Terminal weights and companion weights inhabit the selected opposite-defined weighted-colimit classifier, so left-adjoint preservation specializes to the corresponding conical-colimit and left-Kan interfaces without supplying semantic coend identifications.	`Terminal_prof` `emdash3_2.lp` `Companion_prof` `emdash3_2.lp` `WeightedColimit_con` `emdash3_2.lp` `left_adjoint_preserves_weighted_colimit_con` `emdash3_2.lp`	text `Weighted limit and colimit specialization typing` `emdash3_2_checks.lp`
`JOIN-RECURSOR`	checked	The primitive directed join has two inclusion functors, an internally natural cross cell, and a nondependent recursor with beta computation on both inclusions and the cross-cell datum.	`Join_cat` `emdash3_2.lp` `join_cross_transf` `emdash3_2.lp` `join_elim_func` `emdash3_2.lp` `join_elim_cross_transf` `emdash3_2.lp`	`join_elim_cross_transf` `emdash3_2_checks.lp` `join_elim_cross_transf` `examples/directed_join.lp`
`JOIN-COLLAGE-BOUNDARY`	research-boundary	The join recursor has the input shape of the collage of the terminal profunctor, but no object or hom decomposition, mapping-category equivalence, opposite comparison, or dependent collage eliminator is active.	—	—
`FORMAL-KERNEL-PRESENTATION`	checked	The active v3.2 modules expose categories, iterated homs, functors, transfors, and directed families through explicit classifiers and full or capped application owners, with executable assertions checking representative typing and computation.	`Cat` `emdash3_2.lp` `Hom_cat` `emdash3_2.lp` `fapp1_func` `emdash3_2.lp` `tapp1_func` `emdash3_2.lp` `Catd` `emdash3_2.lp`	`fapp1_fapp0` `emdash3_2_checks.lp` `tapp1_fapp0` `emdash3_2_checks.lp`

`FORMAL-ELABORATION-BOUNDARY`	research-boundary	The renewed TypeScript product implements a bounded direct-TypeScript and categorical-text path through scoped contextual elaboration, backend-neutral explicit Core, and a generic checker/evaluator, together with bounded outer-LF adjunction and dependent-structure declarations, a client-side reviewer, and optional Lambdapi conformance. A complete compiler for the book's canonical surface, arbitrary displayed coherence, a general record or inductive facility, and whole-library transfer are not claimed.	—	—
`FORMAL-METATHEORY-BOUNDARY`	research-boundary	Local source acceptance, diagnostics, warning inventories, and model examples do not establish global confluence, strong normalization, canonicity, decidability, consistency, or semantic soundness for the whole emdash rewrite and unification theory.	—	—

`GROTH-TOPOLOGY-SIEVE-LAWS`	checked	An ordinary-sieve Grothendieck topology packages proposition-valued coverhood with maximality, pullback stability, and local character; the maximal sieve and the topology in which every sieve covers have checked direct models.	`GrothTopology` `emdash3_2_sites.lp` `groth_topology_maximal` `emdash3_2_sites.lp` `groth_topology_pullback` `emdash3_2_sites.lp` `groth_topology_local_character` `emdash3_2_sites.lp` `chaotic_groth_topology` `emdash3_2_sites.lp`	`GrothTopology` `examples/grothendieck_topology.lp` `groth_topology_local_character` `examples/grothendieck_topology.lp`
`GENERATED-GROTH-TOPOLOGY`	checked	Every type-valued family of generating ordinary sieves determines an internally constructed least Grothendieck topology that accepts each retained generator and lies below every topology accepting them.	`generated_sieve_cover_is_prop` `emdash3_2_generated_topologies.lp` `generated_groth_topology` `emdash3_2_generated_topologies.lp` `generated_groth_topology_accepts_generators` `emdash3_2_generated_topologies.lp` `generated_groth_topology_least` `emdash3_2_generated_topologies.lp`	`generated_groth_topology` `examples/generated_grothendieck_topologies.lp` `generated_groth_topology_least` `examples/generated_grothendieck_topologies.lp`

SIEVE-MATCHING-LOCALITY	checked	An ordinary sieve extends to a whole presheaf with a whole inclusion into Yoneda; precomposition defines the section-to-matching restriction functor, locality is its fixed-forward Hom equivalence, and matching, section, and restriction families vary internally over eligible cover questions.	ordinary_sieve_extension_inclusion emdash3_2_sieve_extensions.lp ordinary_sieve_local_precomp emdash3_2_sieve_extensions.lp PshLocalAtOrdinarySieve emdash3_2_sieve_extensions.lp IsTopologyLocalPsh emdash3_2_sieve_extensions.lp DirectCoverQuestionMatching_catd emdash3_2_direct_cover_question_families.lp DirectCoverQuestionSection_catd emdash3_2_direct_cover_question_families.lp direct_cover_question_restriction_funcd emdash3_2_direct_cover_question_families.lp	ordinary_sieve_local_precomp examples/sieve_extensions.lp DirectCoverQuestionMatching_catd examples/direct_cover_question_families.lp direct_cover_question_restriction_funcd examples/direct_cover_question_families.lp

`PSH-YONEDA-HIGHER-SIEVE`	checked	Cat-valued presheaves reuse whole directed-family restriction; the Yoneda presheaf evaluates to the represented hom category; the conventional slice is the opposite restriction total; and the maximal Cat-valued higher sieve is stable under pullback.	`Psh_cat` `emdash3_2_presheaves.lp` `Psh_pullback_func` `emdash3_2_presheaves.lp` `yoneda_psh_func` `emdash3_2_presheaves.lp` `Slice_cat` `emdash3_2_presheaves.lp` `HigherSieveClassifier` `emdash3_2_presheaves.lp` `maximal_higher_sieve` `emdash3_2_presheaves.lp`	`Psh_pullback_func` `examples/presheaf_facade.lp` `HigherSieveClassifier` `examples/higher_sieve_classifier.lp`
`ORDINARY-SIEVE-PULLBACK`	checked	An ordinary sieve is a Cat-valued higher sieve with pointwise subterminal evidence; pullback reuses the higher-sieve action, preserves that evidence, and computes membership at a probe as old membership at its postcomposition image.	`Sieve` `emdash3_2_sieves.lp` `ordinary_sieve_pullback_evidence` `emdash3_2_sieves.lp` `sieve_pullback` `emdash3_2_sieves.lp` `SieveMembership` `emdash3_2_sites.lp` `sieve_pullback_membership` `emdash3_2_sites.lp`	`sieve_pullback` `examples/ordinary_sieves.lp` `SieveMembership` `examples/grothendieck_topology.lp`

COMM-RING-INVERTIBILITY-SIEVE	checked	For a commutative-ring-valued presheaf and a section over U, the invertibility construction produces an ordinary sieve whose membership at a probe computes to unit evidence for the restricted section.	CommRingPshInvertibleAlong emdash3_2_commutative_algebra_presheaves.lp comm_ring_psh_invertibility_sieve emdash3_2_commutative_algebra_presheaves.lp	comm_ring_psh_invertibility_sieve examples/commutative_ring_presheaf_invertibility.lp CommRingPshInvertibleAlong examples/commutative_ring_presheaf_invertibility.lp
DIRECT-COVER-COMPLETION-HIT	checked	For every site and Cat-valued presheaf, the direct cover-completion categorical-HIT boundary provides a whole unit, one cover-question-indexed glue functor, and one whole silent path, with pullback compatibility inherited from displayed functoriality and a packaged internal direct-cover sheaf structure.	DirectCoverSheafStructure emdash3_2_direct_cover_internal_sheaves.lp direct_cover_sheaf_structure_glue_funcd emdash3_2_direct_cover_internal_sheaves.lp direct_cover_sheaf_structure_silent_funcd emdash3_2_direct_cover_internal_sheaves.lp DirectCoverCompletionPsh emdash3_2_direct_cover_completion_hit.lp direct_cover_completion_unit emdash3_2_direct_cover_completion_hit.lp direct_cover_completion_glue_funcd emdash3_2_direct_cover_completion_hit.lp direct_cover_completion_silent_funcd emdash3_2_direct_cover_completion_hit.lp	DirectCoverSheafStructure examples/direct_cover_internal_sheaves.lp direct_cover_completion_glue_funcd examples/direct_cover_completion_hit.lp direct_cover_completion_silent_funcd examples/direct_cover_completion_hit.lp

`DIRECT-COVER-COMPLETION-LOCALITY`	checked	Canonical cover pullback, retained-member calculation, whole glue naturality, and silent derive restriction after glue as the second inverse law; the direct cover completion is consequently local at every eligible question and over the whole topology.	`direct_cover_completion_restriction_glue_path` `emdash3_2_direct_cover_completion_locality.lp` `direct_cover_completion_local_at_question` `emdash3_2_direct_cover_completion_locality.lp` `direct_cover_completion_is_topology_local` `emdash3_2_direct_cover_completion_locality.lp`	`direct_cover_completion_restriction_glue_path` `emdash3_2_checks.lp` `direct_cover_completion_is_topology_local` `emdash3_2_checks.lp`

DIRECT-COVER-COMPLETION-UNIVERSALITY	checked	The completion recursor extends a whole seed map with return, glue, and silent coherence; it varies functorially in the seed, and at a topology-local target its whole beta and eta laws make unit precomposition an omega-equivalence of complete Hom categories.	direct_cover_completion_rec emdash3_2_direct_cover_completion_eliminator.lp direct_cover_completion_rec_beta_unit emdash3_2_direct_cover_completion_eliminator.lp direct_cover_completion_rec_beta_glue emdash3_2_direct_cover_completion_eliminator.lp direct_cover_completion_rec_beta_silent emdash3_2_direct_cover_completion_eliminator.lp direct_cover_completion_rec_func emdash3_2_direct_cover_completion_universality.lp direct_cover_completion_rec_eta_local_func emdash3_2_direct_cover_completion_universality.lp direct_cover_completion_hom_omega emdash3_2_direct_cover_completion_universality.lp	direct_cover_completion_rec_beta_glue examples/direct_cover_completion_eliminator.lp direct_cover_completion_rec_eta_local_func emdash3_2_checks.lp direct_cover_completion_hom_omega emdash3_2_checks.lp

`CAT-VALUED-SHEAFIFICATION-REFLECTOR`	checked	At Cat-valued coefficients, direct cover completion forms a functor into topology-local presheaves left adjoint to inclusion; its unit is return, its counit is local recursion from the identity seed, and the two counit cancellations make the adjunction reflective and instantiate the sheafification capability.	`CatValuedSheafData` `emdash3_2_direct_cover_sheafification.lp` `cat_valued_sheaf_include_psh_func` `emdash3_2_direct_cover_sheafification.lp` `direct_cover_sheafification_func` `emdash3_2_direct_cover_sheafification.lp` `direct_cover_sheafification_adjunction` `emdash3_2_direct_cover_sheafification.lp` `direct_cover_sheafification_reflector` `emdash3_2_direct_cover_sheafification.lp` `direct_cover_sheafification_capability` `emdash3_2_direct_cover_sheafification.lp`	`direct_cover_sheafification_func` `emdash3_2_checks.lp` `direct_cover_sheafification_reflector_at` `emdash3_2_checks.lp` `direct_cover_sheafification_capability` `emdash3_2_checks.lp`

`COMM-RING-STRUCTURED-CATEGORY`	checked	Commutative rings have set-valued carriers and retained operations and laws, including the zero ring; operation-preserving carrier maps are extensional structured homs and form the one-category CommRing_cat, while componentwise products and the Boolean-carrier F2 ring supply closed models without a claimed categorical-product universal property.	`comm_ring_carrier_is_set` `emdash3_2_commutative_algebra.lp` `zero_comm_ring` `emdash3_2_commutative_algebra.lp` `CommRingHom` `emdash3_2_commutative_algebra_category.lp` `comm_ring_hom_ext` `emdash3_2_commutative_algebra_category.lp` `CommRing_cat` `emdash3_2_commutative_algebra_category.lp` `comm_ring_cat_is_one_cat` `emdash3_2_commutative_algebra_category.lp` `comm_ring_product` `emdash3_2_commutative_algebra_product.lp` `f2_comm_ring` `emdash3_2_commutative_algebra_f2.lp`	`zero_comm_ring` `examples/commutative_ring_objects.lp` `CommRing_cat` `examples/commutative_ring_morphisms.lp` `f2_comm_ring` `examples/commutative_ring_split_idempotent_localization.lp`

`FINITE-UNIMODULAR-COVER-DATA`	checked	Finite sums and dot products compute on finite-family constructors and are preserved by structured ring maps; unimodular presentations retain coefficients witnessing that the generators span one, assemble into set-valued finite Zariski presentations, transport under ring maps, and include singleton and binary constructors.	`comm_ring_finite_sum` `emdash3_2_commutative_algebra_finite.lp` `comm_ring_finite_dot` `emdash3_2_commutative_algebra_finite.lp` `CommRingUnimodularPresentation` `emdash3_2_commutative_algebra_finite.lp` `comm_ring_unimodular_map` `emdash3_2_commutative_algebra_finite.lp` `CommRingZariskiCoverPresentation` `emdash3_2_commutative_algebra_finite.lp` `comm_ring_zariski_cover_map` `emdash3_2_commutative_algebra_finite.lp` `comm_ring_binary_zariski_cover` `emdash3_2_commutative_algebra_finite.lp`	`CommRingUnimodularPresentation` `examples/commutative_ring_finite_covers.lp` `comm_ring_unit_zariski_cover` `examples/commutative_ring_finite_covers.lp` `comm_ring_binary_zariski_cover` `examples/commutative_ring_finite_covers.lp`

`COMM-RING-POLYNOMIAL-UNIVERSALITY`	checked	A supplied commutative-ring polynomial algebra retains a base map and variables and makes the structured extension space contractible for every target base map and valuation; the identity ring is a complete checked model for the empty variable classifier.	`CommRingPolynomialFactor` `emdash3_2_commutative_algebra_polynomial.lp` `IsCommRingPolynomialAlgebra` `emdash3_2_commutative_algebra_polynomial.lp` `comm_ring_polynomial_factorization_is_contr` `emdash3_2_commutative_algebra_polynomial.lp` `CommRingPolynomialAlgebra` `emdash3_2_commutative_algebra_polynomial.lp`	`empty_is_polynomial` `examples/commutative_ring_polynomial_algebra.lp` `empty_polynomial` `examples/commutative_ring_polynomial_algebra.lp` `comm_ring_polynomial_factorization_is_contr` `examples/commutative_ring_polynomial_algebra.lp`

`COMM-RING-LOCALIZATION-UNIVERSALITY`	checked	Unit evidence in a commutative ring is proposition-valued, and a supplied localization at one element retains an inverting structure map and a contractible classifier of whole structured factors with their pointwise triangles through every admissible target map.	`CommRingUnitEvidence` `emdash3_2_commutative_algebra_localization.lp` `comm_ring_unit_evidence_is_prop` `emdash3_2_commutative_algebra_localization.lp` `CommRingLocalizationFactor` `emdash3_2_commutative_algebra_localization.lp` `IsCommRingLocalizationAt` `emdash3_2_commutative_algebra_localization.lp` `comm_ring_localization_factorization_is_contr` `emdash3_2_commutative_algebra_localization.lp` `CommRingLocalizationAt` `emdash3_2_commutative_algebra_localization.lp`	`zero_is_localization_at_point` `examples/commutative_ring_localization.lp` `zero_localization_factor_is_contr` `examples/commutative_ring_localization.lp`

COMM-RING-LOCALIZATION-MODELS	checked	Identity is a localization at an already invertible element, the zero ring is a localization at zero, and the fixed image of an idempotent is a quotient-free localization at that idempotent; the split element (1,0) in F2 times F2 gives a closed idempotent distinct from zero and one.	comm_ring_unit_is_identity_localization emdash3_2_commutative_algebra_localization_unit.lp comm_ring_identity_localization_at_one emdash3_2_commutative_algebra_localization_unit.lp comm_ring_zero_is_zero_localization emdash3_2_commutative_algebra_localization_zero.lp comm_ring_zero_localization emdash3_2_commutative_algebra_localization_zero.lp comm_ring_idempotent_image_is_localization emdash3_2_commutative_algebra_localization_idempotent.lp comm_ring_idempotent_image_localization emdash3_2_commutative_algebra_localization_idempotent.lp f2_split_idempotent_not_zero emdash3_2_commutative_algebra_localization_split.lp f2_split_idempotent_not_one emdash3_2_commutative_algebra_localization_split.lp f2_split_idempotent_localization	

			emdash3_2_commutative_algebra_localization_split.lp	comm_ring_identity_localization_at_one examples/commutative_ring_unit_localization.lp comm_ring_zero_localization examples/commutative_ring_zero_localization.lp comm_ring_idempotent_image_localization examples/commutative_ring_idempotent_localization.lp f2_split_idempotent_not_zero examples/commutative_ring_split_idempotent_localization.lp

COMM-RING-ITERATED-LOCALIZATION-EQUIV	checked	For supplied localizations at f, at the image of g, and at fg, the universal properties construct canonical product-to-iterated and iterated-to-product comparison maps whose two whole structured cancellation laws exhibit the selected forward map as an omega-equivalence in CommRing_cat.	CommRingIteratedLocaliza-tionComparison emdash3_2_commutative_algebra_localization_comparison.lp comm_ring_iterated_localization_comparison emdash3_2_commutative_algebra_localization_comparison.lp comm_ring_iterated_localization_comparison_left_law emdash3_2_commutative_algebra_localization_overlap.lp comm_ring_iterated_localization_comparison_right_law emdash3_2_commutative_algebra_localization_overlap.lp comm_ring_iterated_localization_comparison_omega_equiv_along emdash3_2_commutative_algebra_localization_overlap.lp comm_ring_iterated_localization_comparison_omega_equiv emdash3_2_commutative_algebra_localization_overlap.lp	comm_ring_iterated_localization_comparison_left_law examples/commutative_ring_localization_overlap.lp comm_ring_iterated_localization_comparison_right_law examples/commutative_ring_localization_overlap.lp comm_ring_iterated_localization_comparison_omega_equiv examples/commutative_ring_localization_overlap.lp

AFFINE-BASIC-OPEN-POINT-REPRESENTATION	checked	The affine Yoneda presheaf has S-points CommRingHom(R,S), its semantic basic open D(f) is the ordinary invertibility sieve, and every supplied localization R to R[1/f] gives, at each test ring S, an explicit TypeEquiv from whole structured maps R[1/f] to S to D(f)-points, with both inverse laws derived from localization contractibility and proposition-valued unit evidence.	affine_spec_functor_of_points emdash3_2_commutative_algebra_affine_points.lp affine_spec_basic_open_sieve emdash3_2_commutative_algebra_affine_points.lp affine_spec_basic_open_point_left_law emdash3_2_commutative_algebra_affine_points.lp affine_spec_basic_open_point_right_law emdash3_2_commutative_algebra_affine_points.lp affine_spec_basic_open_point_type_equiv emdash3_2_commutative_algebra_affine_points.lp	AffineSpecPoint examples/commutative_ring_affine_points.lp affine_spec_basic_open_point_left_law examples/commutative_ring_affine_points.lp affine_spec_basic_open_point_right_law examples/commutative_ring_affine_points.lp affine_spec_basic_open_point_type_equiv examples/commutative_ring_affine_points.lp

`AFFINE-BASIC-OPEN-INTERSECTION`	checked	At each test ring S, unit evidence for h(fg) is equivalent to paired unit evidence for h(f) and h(g), yielding an explicit pointwise TypeEquiv from D(fg)(S) to the same-map intersection of D(f)(S) and D(g)(S); a supplied localization at fg represents that intersection through executable maps with both component laws.	`affine_spec_basic_open_product_unit_type_equiv` `emdash3_2_commutative_algebra_affine_intersections.lp` `affine_spec_basic_open_product_point_type_equiv` `emdash3_2_commutative_algebra_affine_intersections.lp` `affine_spec_basic_open_intersection_representation` `emdash3_2_commutative_algebra_affine_intersections.lp` `affine_spec_basic_open_intersection_representation_left` `emdash3_2_commutative_algebra_affine_intersections.lp` `affine_spec_basic_open_intersection_representation_right` `emdash3_2_commutative_algebra_affine_intersections.lp`	`affine_spec_basic_open_product_point_type_equiv` `examples/commutative_ring_affine_intersections.lp` `affine_spec_basic_open_intersection_representation_left` `examples/commutative_ring_affine_intersections.lp` `affine_spec_basic_open_intersection_representation_right` `examples/commutative_ring_affine_intersections.lp`

AFFINE-BIG-SLICE-COORDINATES	checked	The conventional big affine slice over Sp(R) has objects R-algebras and geometric arrows given by commuting structured triangles; its whole CommRing-valued coordinate presheaf evaluates a chart at its ring and restriction at the supplied structured map, includes selected localization charts, and internalizes both product/iterated-localization overlap directions with the existing whole coordinate equivalence.	AffineSpecBigSlice_cat emdash3_2_commutative_algebra_affine_spec.lp affine_spec_coordinate_psh emdash3_2_commutative_algebra_affine_spec.lp affine_spec_chart_arrow emdash3_2_commutative_algebra_affine_spec.lp affine_spec_overlap_forward_chart_arrow emdash3_2_commutative_algebra_affine_spec.lp affine_spec_overlap_reverse_chart_arrow emdash3_2_commutative_algebra_affine_spec.lp affine_spec_overlap_coordinate_omega_equiv emdash3_2_commutative_algebra_affine_spec.lp	affine_spec_coordinate_psh examples/commutative_ring_affine_spec.lp affine_spec_chart_arrow examples/commutative_ring_affine_spec.lp affine_spec_overlap_coordinate_omega_equiv examples/commutative_ring_affine_spec.lp

AFFINE-BIG-ZARISKI-TOPOLOGY	checked	At every chart R to S, each supplied localization in a selected finite unimodular family lifts to a whole chart arrow whose coordinate restriction computes to the localization map; literal finite containment forms witness-rich generators, and their generic intersection constructs the lawful least big-affine Zariski topology with generator coverhood and leastness.	affine_spec_chart_localization_arrow emdash3_2_commutative_algebra_affine_zariski.lp AffineSpecBigZariskiGenera-tors emdash3_2_commutative_algebra_affine_zariski.lp affine_spec_big_zariski_topology emdash3_2_commutative_algebra_affine_zariski.lp affine_spec_big_zariski_topology_covers emdash3_2_commutative_algebra_affine_zariski.lp affine_spec_big_zariski_topology_least emdash3_2_commutative_algebra_affine_zariski.lp	affine_spec_chart_localization_restriction_review examples/commutative_ring_affine_zariski.lp affine_spec_big_zariski_topology_covers examples/commutative_ring_affine_zariski.lp affine_spec_big_zariski_topology_least examples/commutative_ring_affine_zariski.lp

AFFINE-STRUCTURE-SHEAF-PRESENTATION	checked	Given a supplied reflective CommRing-valued sheafification capability on the exact generated big-affine Zariski topology, a selected sheaf object, and a whole DefIso from its included presheaf to the computing coordinate presheaf, the affine structure-sheaf presentation determines a reflective commutative-ringed site and retains the whole coordinate comparison and its chart components.	AffineStructureSheafPresentation emdash3_2_commutative_algebra_affine_ringed_sites.lp affine_structure_sheaf_ringed_site emdash3_2_commutative_algebra_affine_ringed_sites.lp affine_structure_sheaf_coordinate_defiso emdash3_2_commutative_algebra_affine_ringed_sites.lp affine_structure_sheaf_to_coordinate_at emdash3_2_commutative_algebra_affine_ringed_sites.lp	affine_structure_sheaf_ringed_site examples/commutative_ring_affine_ringed_sites.lp affine_structure_sheaf_coordinate_defiso examples/commutative_ring_affine_ringed_sites.lp affine_structure_sheaf_to_coordinate_at examples/commutative_ring_affine_ringed_sites.lp

`AFFINE-THIN-SCHEME-PRESENTATION`	checked	The thin affine-scheme presentation pairs a supplied whole reflective structure-sheaf presentation with supplied whole coordinate-localization locality; it inherits the exact generated big-Zariski ringed site, a whole coordinate DefIso, and fixed-forward localization matching equivalences, while the F2 times F2 reviewer keeps both capabilities explicit and computes its complementary-idempotent cover, chart rings, restrictions, and zero overlap.	`AffineCoordinateLocalizationLocality` `emdash3_2_commutative_algebra_affine_locality.lp` `affine_coordinate_localization_locality_at` `emdash3_2_commutative_algebra_affine_locality.lp` `AffineSchemePresentation` `emdash3_2_commutative_algebra_affine_schemes.lp` `affine_scheme_ringed_site` `emdash3_2_commutative_algebra_affine_schemes.lp` `affine_scheme_coordinate_deFiso` `emdash3_2_commutative_algebra_affine_schemes.lp` `affine_scheme_locality_at` `emdash3_2_commutative_algebra_affine_schemes.lp`	`affine_coordinate_localization_locality_at` `examples/commutative_ring_affine_locality.lp` `f2_split_affine_scheme` `examples/commutative_ring_affine_schemes.lp` `affine_scheme_coordinate_deFiso` `examples/commutative_ring_affine_schemes.lp` `affine_scheme_locality_at` `examples/commutative_ring_affine_schemes.lp`

GLOBAL-RINGED-COVER-BINARY-GENERATION	checked	A global reflective CommRinged cover retains one site, whole object, ordinary covering sieve, and whole structure presheaf; Grothendieck stability derives every pullback cover, while witness-rich binary generation computes, for each retained sieve member, a Boolean-selected chart factorization and triangle, and the affine refinement route exposes the selected generator without asserting the arbitrary member affine.	ReflectiveCommRingedSpaceCover emdash3_2_commutative_algebra_ringed_space_covers.lp reflective_comm_ringed_space_cover_underlying_psh emdash3_2_commutative_algebra_ringed_space_covers.lp reflective_comm_ringed_space_cover_pullback_covers emdash3_2_commutative_algebra_ringed_space_covers.lp BinarySelectedCoverGeneration emdash3_2_commutative_algebra_binary_covers.lp binary_selected_cover_generation_at emdash3_2_commutative_algebra_binary_covers.lp binary_affine_cover_refinement_at emdash3_2_commutative_algebra_affine_cover_refinements.lp	reflective_comm_ringed_space_cover_pullback_covers examples/commutative_ring_ringed_space_covers.lp BinarySelectedCoverGeneration examples/commutative_ring_binary_covers.lp binary_selected_cover_generation_at examples/commutative_ring_binary_covers.lp binary_affine_cover_refinement_at examples/commutative_ring_affine_cover_refinements.lp

`WHOLE-SLICE-AFFINE-REALIZATION`	checked	Precomposition with the whole slice-domain functor constructs the ambient CommRing presheaf on K/U; a supplied reflective slice retains a whole DefIso to that target, and supplied sheaf-basis semantics plus an ambient-to-affine-underlying bridge and the affine presentation's retained coordinate bridge derive one whole ambient-to-coordinate DefIso for a selected cover chart.	`slice_domain_func` `emdash3_2_commutative_algebra_ringed_space_restrictions.lp` `reflective_comm_ringed_site_slice_ambient_psh` `emdash3_2_commutative_algebra_ringed_space_restrictions.lp` `SuppliedReflectiveComm-RingedSlicePresentation` `emdash3_2_commutative_algebra_ringed_space_restrictions.lp` `supplied_reflective_comm_ringed_slice_ambient_defiso` `emdash3_2_commutative_algebra_ringed_space_restrictions.lp` `SuppliedSheafBasisEquiva-lenceAlong` `emdash3_2_site_basis.lp` `AffineBasisRealizationAlong` `emdash3_2_commutative_algebra_affine_basis.lp` `affine_basis_realization_coordinate_defiso` `emdash3_2_commutative_algebra_affine_basis.lp` `AffineCoverChartRealization` `emdash3_2_comm`	

			a_affine_cover_charts.lp affine_cover_chart_coordinate_defiso emdash3_2_commutative_algebra_affine_cover_charts.lp	supplied_reflective_comm_ringed_slice_ambient_defiso examples/commutative_ring_ringed_space_restrictions.lp affine_basis_realization_coordinate_defiso examples/commutative_ring_affine_basis.lp AffineCoverChartRealization examples/commutative_ring_affine_cover_charts.lp affine_cover_chart_coordinate_defiso examples/commutative_ring_affine_cover_charts.lp

TOPOLOGY-LOCAL-RING-CERTIFICATE	checked	A topology-local CommRing presheaf presentation makes a zero-unit stage empty-covering and turns every invertible sum into a selected covering sieve whose members compute a Boolean branch with unit evidence for one restricted summand; the whole-object package attaches that capability to the computing ambient presheaf on the supplied actual slice K/X.	CommRingPshTopologyLocal-RingPresentation emdash3_2_commutative_algebra_local_ringed_sites.lp comm_ring_psh_topology_local_ring_empty_covers emdash3_2_commutative_algebra_local_ringed_sites.lp comm_ring_psh_topology_local_ring_split emdash3_2_commutative_algebra_local_ringed_sites.lp ReflectiveCommRingedWholeObjectLocalPre-sentation emdash3_2_commutative_algebra_locally_ringed_space_presentations.lp reflective_comm_ringed_whole_object_local_ambient_defiso emdash3_2_commutative_algebra_locally_ringed_space_presentations.lp reflective_comm_ringed_whole_object_local_ring emdash3_2_commutative_algebra_locally_ringed_space_presentations.lp	comm_ring_psh_topology_local_ring_nontriviality examples/commutative_ring_local_ringed_sites.lp comm_ring_psh_topology_local_ring_split examples/commutative_ring_local_ringed_sites.lp ReflectiveCommRingedWholeObjectLocalPre-sentation examples/commutative_ring_locally_ringed_space_presentations.lp reflective_comm_ringed_whole_object_local_ambient_defiso examples/commutative_ring_locally_ringed_space_presentations.lp

BINARY-SITE-RELATIVE-SCHEME	checked	A BinarySiteRelativeSchemePresentation totals one existing global reflective CommRinged cover with its whole-object topology-local certificate and a constructively generated binary affine cover; the global structure presheaf, covering sieve, local package, selected charts, and both whole affine realizations remain exact existing owners rather than duplicated overlap, cocycle, transition, or gluing fields.	BinaryAffineCoverPresentation emdash3_2_commutative_algebra_affine_cover_presentations.lp BinaryLocallyRingedAffineCoverPresentation emdash3_2_commutative_algebra_locally_ringed_space_presentations.lp BinarySiteRelativeSchemePresentation emdash3_2_commutative_algebra_site_relative_schemes.lp binary_site_relative_scheme_underlying_psh emdash3_2_commutative_algebra_site_relative_schemes.lp binary_site_relative_scheme_local emdash3_2_commutative_algebra_site_relative_schemes.lp binary_site_relative_scheme_atlas emdash3_2_commutative_algebra_site_relative_schemes.lp binary_site_relative_scheme_realization0 emdash3_2_commutative_algebra_site_relative_schemes.lp binary_site_relative_scheme_realization1 emdash3_2_comm	

			a_site_relative_schemes.lp	binary_affine_cover_generation examples/commutative_ring_affine_cover_presentations.lp binary_locally_ringed_affine_cover_atlas examples/commutative_ring_locally_ringed_space_presentations.lp BinarySiteRelativeSchemePre-sentation examples/commutative_ring_site_relative_schemes.lp binary_site_relative_scheme_underlying_psh examples/commutative_ring_site_relative_schemes.lp binary_site_relative_scheme_realization0 examples/commutative_ring_site_relative_schemes.lp binary_site_relative_scheme_realization1 examples/commutative_ring_site_relative_schemes.lp

ACTUAL-BINARY-CHART-OVERLAP	checked	Given a selected binary product of the two chart objects in the conventional slice K/X, its whole universal property derives both slice projections and their base arrows; evaluating the single global structure presheaf derives the overlap ring and both restriction homomorphisms, without adding an overlap field to the scheme total or constructing arbitrary pullbacks.	BinaryProductPresentation emdash3_2_finite_limits.lp BinarySchemeChartOverlapPresentation emdash3_2_commutative_algebra_scheme_chart_overlaps.lp binary_scheme_chart_overlap_to_chart0 emdash3_2_commutative_algebra_scheme_chart_overlaps.lp binary_scheme_chart_overlap_to_chart1 emdash3_2_commutative_algebra_scheme_chart_overlaps.lp binary_scheme_chart_overlap_domain emdash3_2_commutative_algebra_scheme_chart_overlaps.lp binary_scheme_chart_overlap_ring emdash3_2_commutative_algebra_scheme_chart_overlaps.lp binary_scheme_chart_overlap_restriction0 emdash3_2_commutative_algebra_scheme_chart_overlaps.lp binary_scheme_chart_overlap_restriction1 emdash3_2_commutative_algebra_scheme_chart_overlaps.lp	BinarySchemeChartOverlapPresentation examples/commutative_ring_scheme_chart_overlaps.lp binary_scheme_chart_overlap_to_chart0 examples/commutative_ring_scheme_chart_overlaps.lp binary_scheme_chart_overlap_to_chart1 examples/commutative_ring_scheme_chart_overlaps.lp binary_scheme_chart_overlap_ring examples/commutative_ring_scheme_chart_overlaps.lp binary_scheme_chart_overlap_restriction0 examples/commutative_ring_scheme_chart_overlaps.lp binary_scheme_chart_overlap_restriction1 examples/commutative_ring_scheme_chart_overlaps.lp

LAURENT-TRANSITIONS-BY-UNIVERSALITY	checked	For supplied one-variable polynomial algebras over A and selected localizations at their coordinates, polynomial universality constructs A[t] to A[u,1/u] with t sent to the chosen inverse of u, localization universality extends it to A[t,1/t] to A[u,1/u], reversing the inputs constructs the opposite orientation, and the retained factor agreement is a whole pointwise triangle.	CommRingLaurentLocalization emdash3_2_commutative_algebra_laurent.lp comm_ring_laurent_polynomial_coordinate_path emdash3_2_commutative_algebra_laurent.lp comm_ring_laurent_transition_map emdash3_2_commutative_algebra_laurent.lp comm_ring_laurent_transition_agreement emdash3_2_commutative_algebra_laurent.lp	comm_ring_laurent_polynomial_coordinate_path examples/commutative_ring_laurent.lp comm_ring_laurent_transition_map examples/commutative_ring_laurent.lp comm_ring_laurent_transition_agreement examples/commutative_ring_laurent.lp

LAURENT-COMMON-OVERLAP	checked	Given two literal restriction maps R to L and T to L, each supplied simultaneously as a one-variable polynomial chart over one base ring and a localization at its coordinate, the Laurent owner constructs both coordinate-inversion endomorphisms of that exact L; a Laurent overlap presentation then supplies whole paths identifying both constructed endomorphisms with the identity, rather than a disconnected overlap isomorphism or componentwise square.	CommRingOneVariableLocaliza-tionPresenta-tion emdash3_2_commutative_algebra_laurent.lp comm_ring_laurent_common_overlap_transition emdash3_2_commutative_algebra_laurent.lp CommRingLaurentOverlapPre-sentation emdash3_2_commutative_algebra_laurent.lp comm_ring_laurent_overlap_forward_identity emdash3_2_commutative_algebra_laurent.lp comm_ring_laurent_overlap_reverse_identity emdash3_2_commutative_algebra_laurent.lp	CommRingOneVariableLocaliza-tionPresenta-tion examples/commutative_ring_laurent.lp comm_ring_laurent_common_overlap_transition examples/commutative_ring_laurent.lp CommRingLaurentOverlapPre-sentation examples/commutative_ring_laurent.lp comm_ring_laurent_overlap_forward_identity examples/commutative_ring_laurent.lp comm_ring_laurent_overlap_reverse_identity examples/commutative_ring_laurent.lp
ACTUAL-SCHEME-LAURENT-OVERLAP	checked	For a supplied binary site-relative scheme and selected actual chart intersection, the thin Laurent adapter adds one common base ring and retains a generic Laurent overlap presentation at the literal two chart structure rings, inherited overlap ring, and already-derived restriction homomorphisms; it neither duplicates those global owners nor constructs the coordinate presentation from no data.	BinarySchemeLaurentOverlapP-resentation emdash3_2_commutative_algebra_scheme_laurent_overlaps.lp binary_scheme_laurent_overlap_base_ring emdash3_2_commutative_algebra_scheme_laurent_overlaps.lp binary_scheme_laurent_overlap_coordinates emdash3_2_commutative_algebra_scheme_laurent_overlaps.lp	binary_scheme_laurent_overlap_base_ring examples/commutative_ring_scheme_laurent_overlaps.lp binary_scheme_laurent_overlap_coordinates examples/commutative_ring_scheme_laurent_overlaps.lp

SUPPLIED-P1	checked	A SuppliedProjectiveLinePresentation is the transparent dependent total of one already-global binary site-relative scheme, its selected actual chart intersection, and a Laurent-coordinate presentation on the literal inherited restriction maps; its scheme, overlap, base ring, and coordinate package compute by projection, while no closed projective object, Proj construction, gluing theorem, projectivity proof, or non-affineness proof is produced.	SuppliedProjectiveLinePresentation emdash3_2_commutative_algebra_projective_line.lp supplied_projective_line_scheme emdash3_2_commutative_algebra_projective_line.lp supplied_projective_line_overlap emdash3_2_commutative_algebra_projective_line.lp supplied_projective_line_base_ring emdash3_2_commutative_algebra_projective_line.lp supplied_projective_line_coordinates emdash3_2_commutative_algebra_projective_line.lp	supplied_projective_line_scheme examples/commutative_ring_projective_line.lp supplied_projective_line_overlap examples/commutative_ring_projective_line.lp supplied_projective_line_base_ring examples/commutative_ring_projective_line.lp supplied_projective_line_coordinates examples/commutative_ring_projective_line.lp
EH-COMMUTATIVITY	checked	Two 2-endomorphisms of an identity 1-cell commute in the selected Eckmann-Hilton slice.	EH_comm emdash3_2.lp	text Eckmann-Hilton specialization emdash3_2_checks.lp

Appendix C. From The Circle To The Walking Endomorphism

The proof of Theorem 8.1 is inspired by the encode-decode calculation of the loop space of the circle in the *Homotopy Type Theory* book. This appendix records the correspondence so that the analogy can guide the reader without smuggling groupoidal assumptions into the directed theorem.

The source reference is revision `578b85cc8d586b1677ec4335148adeb443057d24` of the HoTT Book repository. The detailed, machine-readable adaptation ledger is `book/references/third-party-sources.json`.

C.1 The Rhetorical Spine

The HoTT calculation proceeds through four questions:

1. What are the evident powers of the loop, and how might a loop be assigned a winding number?
2. What does the classical free/universal-cover picture predict?
3. How can the cover be represented internally as a family?
4. How do encode and decode become inverse after the endpoint is generalized?

Chapter 8 retains that sequence. It changes the mathematical answer at every point where the circle proof uses reversibility.

Circle calculation	Walking-endomorphism calculation
A base point and an identity loop in `S¹`	A base object and a directed arrow `ell` in `W`
Loop carrier `base = base`	Based hom carrier `Hom_W(*,*)`
Integer-indexed positive and negative powers	Natural-number-indexed forward powers
A cover with integer fibre	A Cat-valued code with `Path(Nat)` base fibre
Successor equivalence on integers	Successor functor on naturals
A universe path obtained through univalence	A directed endofunctor accepted directly by recursion into `Cat`
Transport zero along an identity path	Apply the Code functor to zero along a directed arrow
Decode by dependent circle induction	Decode by the contextual displayed eliminator
Hard inverse by generalized path induction	Directed normalization cell, then hom-discreteness
Easy inverse by integer/circle induction	Easy inverse by Nat induction
`Omega(S¹) ≃ Int`	`Hom_W(*,*) ≃ Nat` as carriers
Group structure and inverse loops	Free-monoid direction and a noninvertible generator

The table is a dependency map, not a dictionary identifying the objects.

C.2 Where Univalence Moves

In the circle proof, a family $\mathsf{Code} : S^1 \to \mathsf{Type}$ must map the loop of `S¹` to a loop in the universe. Successor on the integers is an equivalence, and univalence converts that equivalence into the needed universe identity. Its inverse supplies predecessor action.

In the directed proof,

$$\mathsf{Code} : W \longrightarrow \mathsf{Cat}$$

is specified by a category and an endofunctor. Natural-number successor is therefore accepted without being an equivalence. No universe identity and no predecessor are needed.

This does not make univalence irrelevant to functorial type theory. It clarifies its jurisdiction. Equality, transport, equivalence, and univalent universes govern groupoidal identification. A directed family may additionally act along arrows that are not identities and whose action is not invertible. The title of this book refers to a foundation containing both layers.

> **Formal status — mathematical development.** The comparison of the roles of univalence is explanatory. The individual emdash interfaces cited by Chapter 8 are checked; a general metatheorem comparing all directed families with HoTT fibrations is not claimed.

C.3 The Generalized Endpoint

Both proofs teach the same strategic lesson in different formal languages: when a statement about a fixed loop cannot be inducted on, vary its endpoint.

For the circle, one considers all `x:S¹` and paths `base=x`. For `W`, one considers all objects `x` and based arrows $* \to x$. The target family is the representable

$$x \longmapsto \mathrm{Hom}_W(*, x).$$

The crucial difference is what generalized elimination produces. Groupoidal path induction produces equality. The contextual directed eliminator first produces coherent arrow action, and at an arbitrary based arrow it yields a directed 2-cell

$$p \longrightarrow \mathsf{decode}_x(\mathsf{encode}_x(p)).$$

Only the separate one-dimensionality premise converts that cell into equality. Thus the directed proof factors the groupoidal conclusion into two conceptually independent steps: normalization and dimension.

C.4 What Was Not Transferred

The adaptation removes, rather than renames, the following ingredients:

- inverse paths and negative powers;
- predecessor action on the code;

- the claim that successor is an equivalence;
- a universe path produced from successor;
- cancellation in a group;
- equality as the first output of the hard inverse;
- the conclusion that the result is already a group isomorphism.

It also adds ingredients absent from the circle proof in this form:

- a separate `BNat` model whose definitional structure is not imposed on the HIT;
- a displayed contextual algebra comparing postcomposition with successor;
- a coherent directed spiral;
- a dimension witness used explicitly at the equality boundary;
- negative results proving the generator has no right inverse or omega-equivalence evidence.

These changes explain why a textual search-and-replace from `S¹/Int` to `W/Nat` would be mathematically misleading.

C.5 Stronger Comparisons

The carrier equivalence suggests two future constructions.

First, composition/addition compatibility should package the comparison as a monoid isomorphism. Chapter 8 records this as a formal consequence of the checked recursion and inverse laws, but the library does not yet expose the package.

Second, group completion should freely invert the generator. Only after that construction is available should one compare the result with an integer one-object category or the circle's loop object. Such a comparison must state whether it concerns carriers, monoids, categories, or a universal property.

> **Formal status — research boundary.** A reverse `BNat` functor, full categorical initiality, group completion, and the precise `BInt`/circle bridge remain future work.

C.6 Attribution Method

The subsection order and proof strategy are structural adaptations from the pinned HoTT sources. The prose in this edition is newly written. Each target, source file, source label, and adaptation type is recorded before the adaptation is committed. The book's CC BY-SA 3.0 license and credits preserve the upstream attribution and ShareAlike requirements.

Appendix D. Glossary And Concept Index

This appendix fixes the vocabulary used by the expanded development edition. Each entry points to the place where the idea is constructed or used, rather than to a page number that would change with paper size and typography.

D.1 Glossary

Affine chart realization. A selected region $U \to X$ together with a supplied reflective presentation of the actual slice $\mathcal{K}/U$, a coordinate ring and thin affine presentation, a whole affine-basis functor, and a whole comparison from ambient restriction to affine coordinates. The label is site-relative and is not inferred from cover membership alone. See Chapter 23.

Affine scheme, computational presentation. A base ring together with a supplied reflective structure-sheaf presentation on its generated big Zariski site and supplied whole localization locality for the coordinate presheaf. The current interface is assumption-explicit and does not construct sheafification, stalks, or a representation-independent category of affine schemes. See Chapter 22.

Binary site-relative scheme presentation. One supplied global reflective ringed object with topology-local ring behaviour and a covering sieve constructively generated by two whole affine chart realizations. The global structure presheaf is retained once, so restrictions and selected overlaps are inherited rather than duplicated as atlas fields. See Chapter 23.

Adjunction. Functors $F : A \to B$ and $G : B \to A$ equipped either with a unit and counit satisfying the two triangle laws or with an equivalent natural hom comparison. In the active calculus, the triangles are universal cuts with selected computational owners. See Chapter 12.

Arrow induction. Extension of data at the reflexive outgoing arrow to a section over $\mathsf{PathOut}$. Unlike equality induction, its base category may contain noninvertible arrows. See Chapter 5.

Basic open. Primarily, the ordinary sieve $D_R(f)$ of maps $R \to S$ that make f invertible. A supplied localization $R \to R[1/f]$ represents its points at every test ring; a compact open is a further representation when available. See Chapters 18 and 22.

Based hom-category. For a selected base object $*$ and endpoint x, the category $H_x = \mathrm{Hom}_W(*, x)$. Its objects are based 1-arrows and its arrows are based 2-cells. See Chapter 8.

BNat. The separate one-object category with $\mathbb{N}$-valued hom, zero identity, and addition composition. It is a concrete model of the walking signature, not the definition of WalkingEnd. See §8.1.2.

Canonical mathematical surface. The readable notation in which the book states categorical judgments and rule schemas. It maps to stable kernel owners and has a bounded executable subset, but the full notation is broader than the implemented text grammar. See Appendix G.5.

Carrier equivalence. An equivalence between underlying classifiers, here $\mathrm{Hom}_W(*,*) \simeq \mathbb{N}$. It does not by itself package preservation of composition or an equivalence of ambient categories. See Theorem 8.1.

Cat-valued directed family. A functor $E : K \to \mathsf{Cat}$. It assigns a category $E[k]$ to each base object and a transport functor $E[p]$ to each directed base arrow. See Chapter 2.

Categorical height. The recursive `IsNCat` condition: dimension zero is discreteness, and successor dimension asks every hom-category to have the preceding dimension. See Chapter 7.

Category, native. An object of `Cat`, with iterable category-valued homs. It is not definitionally an ordinary HoTT precategory. See Chapters 2 and 10.

Commutative ring. A set-valued carrier with zero, one, addition, negation, multiplication, and the usual commutative-ring laws. The zero ring is retained; structured maps preserve all five operations and form $\mathbf{CRing}$. See Chapter 21.

Code. The Cat-valued family over WalkingEnd whose base fibre is $\mathsf{Path}(\mathbb{N})$ and whose generator action is successor. See §8.1.3.

Contractible factor space. The classifier of structure-preserving maps and their required triangles, equipped with a center and a path from every competitor to it. It turns a universal property into both a selected factor and coherent uniqueness. See Chapter 21.

Co-Yoneda cut. Elimination of a representable leg from a profunctor composite. The checked theorem is a shaped, fixed-middle beta/fusion law; a general coend theorem remains separate. See Chapter 13.

Cover. An ordinary sieve selected as locally sufficient by a Grothendieck topology. A family of arrows can present or generate a covering sieve while retaining separate computational witnesses. See Chapter 19.

Cover generation, binary and witness-rich. For every member $q : V \to X$ of one retained covering sieve, an executable Boolean branch, factor map $V \to U_b$, and triangle $q = u_b h$ through one of two selected members. It explains the retained sieve without asserting that every member is itself affine. See Chapter 23.

Contextual eliminator. An eliminator that constructs a displayed functor between two varying families from base-fibre data and coherent constructor cells. Sections and ordinary recursors are special cases. See Chapter 6.

Cut elimination. Controlled normalization at the semantic owner of an arrow, family, structural, or universal cut. It does not mean installing unrestricted associativity as a global rewrite. See Chapter 9.

Dagger structure. A chosen contravariant involution on one category, identity on objects in the ordinary presentation and coherent with retained higher action in a prospective native presentation. It is not merely the operation of taking an arbitrary opposite category. See Chapter 14.

Directed higher-inductive type/category. A presentation with object, directed-arrow, and possibly higher-cell constructors whose arrow generators are not silently inverted. The current book has one selected implementation, WalkingEnd, not a general schema. See Chapter 6.

Directed normalization cell. The cell $p \to \mathsf{decode}(\mathsf{encode}(p))$ constructed by the contextual decoder before hom-discreteness extracts equality. See §8.1.4.

Discrete category. A category whose hom structure agrees with equality of objects at the selected interface. In a one-dimensional category every hom-category is discrete; this does not make every ambient 1-arrow invertible. See §7.6.

Duality. Either a proof method that transports a theorem through the opposite construction or additional structure comparing a category with an opposite. It never licenses an unannounced variance reversal. See Chapters 14 and 17.

Elaborator. The implemented bounded TypeScript layer that interprets directly constructed or parsed categorical surface terms against expected classifiers, selects stable owners, and emits backend-neutral explicit Core. It fails closed when a required coherent construction is absent. It is not a compiler for the whole book surface or a second mathematical kernel. See Appendix G.5.

Evidence status. One of checked, formal consequence, mathematical development, or research boundary. The status describes the relation between prose and the active artifact. See How to Read and Appendix B.

Explicit emdash Core. The backend-neutral representation produced after elaboration has selected logical and categorical owners and made their arguments explicit. The generic TypeScript LF checks and reduces it; optional deterministic Lambdapi emission is a conformance route, not its definition. See Appendix G.5.

Formal presentation. The layered account relating the canonical mathematical surface, bounded contextual elaboration and outer-LF declaration conveniences, backend-neutral explicit Core, the generic TypeScript LF, the active Lambdapi authority, and separately stated semantic models. The categorical kernel comes first; it is not post-hoc semantics for an unspecified traditional syntax. See Appendix G.

Functor. A map with object and iterated-hom action. Generic functoriality, not constructor-specific laws, owns identity and composition preservation. See Chapter 2.

Generated topology. The least Grothendieck topology accepting a selected type-valued family of generating sieves. The active presentation is the intersection of all accepting topologies, not an inductive derivation syntax. See Chapter 19.

Global-first scheme architecture. An approach that begins with one already-existing global ringed object and recognizes selected regions as covering affine charts. Restriction and overlap coherence are inherited from the global presheaf; constructing the object from abstract chart data remains a separate gluing theorem. See Chapter 23.

Group completion. A future construction freely adjoining inverse motion to the walking directed generator. It is the proper route from Nat powers toward integers or a circle comparison. See §8.1.5.

Hom action. The functorial action induced on a hom-category. Emdash keeps covariant postcomposition, contravariant precomposition, and simultaneous two-endpoint action as distinct computational owners. See Chapters 2, 9, and 13.

Higher sieve. A Cat-valued coefficient system on the restriction-oriented category of probes into a fixed object; equivalently, a Cat-valued presheaf on the conventional slice. Its values may retain witnesses and arrows between them; it is not automatically an ordinary sieve. See Chapter 18.

Invertibility sieve. For a section s over U, the sieve $D_U(s)$ of all probes $p : V \to U$ for which $p^* s$ is a unit. An open may represent this sieve, but representability is a further claim. See Chapter 18.

Join. The selected directed category generated by left and right embeddings together with cross arrows from the left side to the right side. Its recursor and three beta observations are checked; a general collage mapping property and dependent eliminator are not. See Chapter 17.

Kan extension. A universal extension along a functor. This edition expresses right and left Kan interfaces as conjoint- and companion-weighted limits and colimits; identifying those interfaces with the full standard pointwise semantics remains mathematical development. See Chapters 16 and 17.

Lower-star action. Postcomposition: if $g : w \to x$ and $u : x \to y$, then $u_*(g) = u \circ g : w \to y$. Its active owners are `hom_postcomp_func` and `hom_postcomp_fapp0`. See §9.2.

Localization at an element. A structured map $R \to R[1/f]$ that makes f invertible and has a contractible factor space through every map in which the image of f is already invertible. The notation names a universal role, not a required fraction representation. See Chapter 21.

Laurent overlap. Two literal chart restrictions into one actual overlap ring, each supplied as localization of a one-variable polynomial algebra over one common base, together with whole paths identifying both internally constructed coordinate-inversion endomorphisms with the overlap identity. The transition maps are constructed; the identity paths are supplied. See Chapter 24.

Matching family. A whole presheaf map $\widehat{R} \to X$ assigning local data to every member of a sieve R, compatibly with refinement. A global section restricts to a matching family by precomposition. See Chapter 19.

Natural transformation. In the ordinary specialization, a pointwise family $\alpha_x : F(x) \to G(x)$ satisfying a naturality equation. A native transfor retains an off-diagonal action and iterates to higher hom levels, so the two notions are related but not definitionally identical. See Chapter 11.

Off-diagonal transfor action. If $\eta : F \Rightarrow G$ and $f : x \to y$, then $\eta[f] : F(x) \to G(y)$. Adjacent functor actions accumulate into this term by strict naturality. See Chapter 9.

Omega-equivalence. The native recursive equality-valued equivalence interface for categorical cells. It is distinct from a bare carrier `TypeEquiv` and from ordinary isomorphism evidence. See Chapter 4.

Opposite category. The active arrow-reversing construction $C \mapsto C^{\mathrm{op}}$. Opposite duality exchanges selected limit and colimit interfaces while preserving a visible variance ledger. See Chapter 14.

Ordinary sieve. A refinement-closed, proposition-valued family of probes into one object. In the active categorical presentation it is a higher sieve whose every coefficient category is subterminal. See Chapter 18.

Outer-LF declaration convenience. A typed host operation that validates selected higher-level input and expands it into ordinary dependent-LF declarations and rules before explicit Core is checked. The current adjunction and bounded dependent-structure forms add no trusted term node or new categorical semantics. See Appendix G.5.

Path category. $\mathsf{Path}(A)$, the equality-local groupoidal category on a classifier A. It embeds ordinary identity reasoning into the directed calculus without identifying every directed hom with equality. See Chapter 2.

PathOut. The outgoing-arrow category $\sum_{y:Z} \mathrm{Hom}_Z(x, y)$ at a fixed source x. Its canonical arrow from (x, id_x) to (y, p) drives arrow induction. See Chapter 5.

Precategory, ordinary. A classifier of objects with set-valued homs, identities, composition, and category laws. It is used as a readable one-categorical specialization of the native iterated-hom architecture. See Chapter 10.

Polynomial algebra. A free commutative R-algebra on a variable classifier X, characterized by contractible structured extension spaces for every base map and valuation. The universal interface does not select a monomial representation. See Chapter 21.

Proj. The standard construction of a projective scheme from a graded ring, using homogeneous localization and degree-zero parts on standard regions. It is mathematical development and a research boundary in this edition; the active artifact has no graded `Proj` owner. See Chapter 24.

Projective-line presentation, supplied. One already-global binary site-relative scheme, its selected actual chart intersection, and a Laurent coordinate presentation on the literal inherited restriction maps. It is an end-to-end computational capability, not a construction of $\mathbf{P}^1$, a projectivity

or non-affineness proof, or a substitute for `Proj`. See Chapter 24.

Presheaf. A contravariant functor $X : \mathcal{K}^{\mathrm{op}} \to \mathsf{Set}$ or, in the active higher presentation, to Cat. It assigns observations to stages and functorial restriction to probes. See Chapters 13 and 18.

Profunctor. A Cat-valued functor $A^{\mathrm{op}} \times B \to \mathsf{Cat}$, contravariant in its first endpoint and covariant in its second. See Chapter 13.

Representable. A family or profunctor obtained from an ambient hom. Its action is composition, which makes it the computational bridge between universal properties and cut elimination. See Chapters 5 and 13.

Reflector. A left adjoint to the inclusion of a full subcategory whose counit on objects already in that subcategory is an equivalence. Direct cover completion is a reflector from Cat-valued presheaves to topology-local ones at the fixed site. See Chapter 20.

Rezk completion. A completion intended to turn the selected weak equivalences into equivalences as seen by saturated targets. The book gives ordinary Yoneda-image and higher-inductive constructions and a prospective native specification; no native implementation is claimed. See Chapter 15.

Runtime rewrite. A directed reduction selecting an intended normal form. It is distinct from proof-time unification and internal propositional equality. See Appendix E.

Saturation. The property that the chosen identity-to-equivalence map is itself an equivalence, or the result of freely enforcing that property by a completion. Saturation is not finite categorical height. See Chapter 15.

Sheaf. A presheaf local at every covering sieve: restriction from global sections to matching families is an equivalence. This condition on an existing presheaf is distinct from a sheafification construction. See Chapters 19 and 20.

Sheafification. A reflective free-local completion of a presheaf. In the active Cat-valued construction, return preserves old data, whole glue adjoins amalgamations over eligible cover questions, silent removes redundant restrict-and-glue detours, and the recursor gives the whole Hom universal property. See Chapter 20.

Site. A category equipped with a Grothendieck topology, whose covering sieves satisfy maximality, pullback stability, and local character. A site need not be a poset of open subsets. See Chapter 19.

Topology-local ring presentation. A commutative-ring presheaf capability making a zero-unit stage empty-covering and splitting an invertible sum over a selected cover into an executable branch where one restricted summand is a unit. It is a direct site-level forcing condition, not a constructed stalk or a theorem comparing with stalk-local rings. See Chapter 23.

Strict category, ordinary HoTT sense. A precategory whose object classifier is a set. This is object-level truncation and must not be confused with strict transfor computation or strict higher associativity. See Chapter 14.

Strict transfor. A native transfor for which the selected two-sided naturality cuts compute through the global `tapp*` calculus. The adjective does not say that every coherence law in its ambient category is judgmental. See Chapters 9 and 14.

Structure identity principle. A theorem identifying equality of structured objects with an appropriate structure-preserving equivalence when the displayed notion of structure is univalent or standard. The ordinary theorem is developed mathematically; a generic native package is a research boundary. See Chapter 15.

Transfor. An arrow in a functor category, generalizing a natural transformation through off-diagonal and higher hom action. See Chapters 9 and 11.

Truncation evidence. A recursive property of an existing classifier: at the base level it is contractibility, and at successor levels it truncates all identity classifiers one step lower. It is not a truncation reflector. See Chapter 7.

Unimodular presentation. A finite family (f_i) together with coefficients (a_i) and a retained equation $\sum_i a_i f_i = 1$. It is the witness-rich algebraic input for a finite basic-open cover, not by itself a covering sieve or topology. See Chapter 21.

Univalence. An interface relating identity of classifiers or packages to an appropriate equivalence. This edition has checked groupoid and restricted truncated-universe interfaces but does not claim one universal directed univalence axiom. See Chapter 4.

Unitary arrow. In a dagger category, an arrow whose dagger is an inverse. The ordinary theory is developed in Chapter 14; a native unitary classifier awaits the selected dagger interface and coherent higher action. See Chapter 14.

Upper-star action. Precomposition: if $u : x \to y$ and $h : y \to z$, then $u^*(h) = h \circ u : x \to z$. Its active owners are `hom_precomp_along_func` and `hom_precomp_along_fapp0`. The action is contravariant in u. See §9.2.

Weighted colimit. A representation of a weighted-cocone profunctor. The selected interface and left-adjoint preservation theorem are checked through opposite duality; full coend semantics is not assumed. See Chapter 17.

Weighted limit. A representation of a weighted-cone profunctor, with beta and eta supplied by a chosen profunctor comparison. The selected interface and right-adjoint preservation theorem are checked. See Chapter 16.

WalkingEnd. The opaque one-dimensional directed HIT/category with a base object and a directed generating endomorphism. See Chapters 6 and 8.

Yoneda principle. Natural maps out of a representable are determined by their value at the identity. The ordinary equivalence is developed by encode-decode, while the active native theorem is the shaped co-Yoneda cut; full Cat-valued Yoneda remains a named boundary. See Chapter 13.

D.2 Index strategy

This edition uses the linked concept index as its stable index. Terms are curated rather than extracted from raw identifier frequency; synonyms point to one canonical entry, and every destination is an explicit HTML anchor checked by the source gate.

Page numbers are deliberately absent from the source because they depend on the renderer, paper size, and font metrics. A later release tool may resolve these anchors to PDF page labels, but the anchor remains the authority. New index entries should be added when a concept is defined or changes status, not for every occurrence of its implementation name.

Appendix E. Computation And Normalization

The prose uses equations freely, but the implementation distinguishes three ways expressions can agree. That distinction is essential whenever a theorem depends on a particular normal form.

E.1 Three Forms Of Agreement

Runtime reduction selects a computational direction. A rewrite

```
left  ↪  right
```

makes `right` the intended normal form when the left pattern is observed. Constructor beta rules, functor projection rules, and selected cut-elimination rules live here.

Proof-time comparison helps elaboration recognize two typed expressions without making either one compute to the other. Emdash uses narrow `unif_rule` declarations for this purpose. A typed reflexivity proof is the relevant diagnostic; mere conversion testing does not exercise the same interface.

Internal equality is mathematical data in a classifier `x=y`. It can be transported, acted on, inverted, or used in an equivalence proof. A propositional theorem may compare two stable runtime presentations without changing either presentation's reduction behavior.

The book's symbol `=` normally denotes internal equality or an ordinary mathematical equality justified by a formal-status note. It never implies that the source expressions are definitionally identical.

E.2 Semantic Owners

A computational operation should have one owner. Generic functoriality is owned by the `fapp*` calculus; generic naturality by `tapp*`; displayed hom action by `fdapp*` and `tdapp*`; Sigma and Pi expose their own structural projections. Readable aliases route through these owners instead of copying their semantic bodies.

This prevents two kinds of drift:

- competing rewrites can no longer silently choose incompatible normal forms;
- a theorem at the next hom level retains the functor or transfor object it needs for further iteration.

The WalkingEnd development illustrates the policy. The contextual eliminator owns the constructor-specific base and generator observations. It does not restate generic preservation of identity or composition. The decoder's normalization cell is the displayed hom-action of one constructed functor; it is not a custom recursion rule for every arbitrary based arrow.

E.3 Direction And Variance In Normal Forms

Covariant postcomposition and contravariant precomposition have different runtime owners. Their mathematical comparison through opposites is available at proof time, but forcing both into one rewrite direction would erase the variance used by `PathOut` and profunctor action.

Similarly, an identity may appear as an ambient categorical identity, a functor identity, a displayed identity, or a specialized projection. These forms are joined only where a typed consumer requires it. Broad eta-style rewrites are avoided because unification is experimental and because a functor-level normal form may be needed to act on the next cell.

E.4 How A Checked Prose Claim Is Reviewed

For a code-facing claim, the review path is:

1. identify the mathematical interface and its direction;
2. locate the active owner declaration with lexical or type-aware search;
3. identify an independent regression or reviewer example;
4. decide whether the observation is runtime, proof-time, or propositional;
5. add or update the evidence-register entry;
6. cite the evidence identifier beside the prose claim;
7. run the evidence, assembly, source, and browser-render checks.

Changes to rewrite or unification behavior require the stronger repository workflow: an owner-position probe, bounded typecheck, warning comparison when relevant, focused assertions, and full CI before handoff. Book prose does not authorize changing kernel normal forms merely to make an explanation shorter.

E.5 What Has Not Been Proved Metatheoretically

The passing executable checks establish the selected interfaces and regression observations. They do not by themselves prove global confluence, strong normalization, canonicity, consistency of every future extension, or soundness with respect to a complete weak omega-categorical model.

Those are research-level metatheorems. A future semantics chapter should state its fragment, model, and computation theorem explicitly. Until then, the book uses “computes” locally for named checked reductions and uses a formal-status note for every stronger reading.

The current development SOP remains the operational authority for rule design and validation. This appendix explains the mathematical reading needed by a book reader; it is not a replacement for that SOP.

Appendix F. Implementation Status And Research Directions

This appendix summarizes the boundary of the expanded development edition. The generated evidence register remains the detailed claim-by-claim authority.

F.1 Status Matrix

Area	Checked nucleus used by the book	Explicit boundary
Equality-local type theory	Equality induction, path action, Sigma/Pi path interfaces, elementary inductives	No claim of a complete standalone HoTT implementation
Directed categories	Iterated homs, identities, composition, functors, transfors, opposites, products	No complete weak omega-category metatheory or model theorem
Directed families	Fibres, transport, family morphisms, Sigma totals, Pi sections, displayed hom action, fibrewise products, pull-back totalization, displayed evaluation, and finite canonical sibling/Sigma telescopes	Arbitrary dependency or variance graphs, unrestricted mixed introduction/evaluation, and exchange across genuine dependency remain open
Cut and transfor calculus	Lower-star postcomposition, upper-star precomposition, off-diagonal `tapp1`, horizontal composition, selected universal beta/eta cuts	No unrestricted runtime associativity rewrite or claim that all higher coherence is judgmental
Equivalence and univalence	`TypeEquiv`, groupoid univalence, truncated-universe univalence, native recursive omega-equivalence facade and one-way hom action	No full general object-equality/ordinary-isomorphism equivalence for arbitrary categories
Induction	Nat and equality induction, fixed/varying-source `PathOut` induction, composition benchmark	No general equivalence with homotopy-initial categorical algebras
Directed HITs	One opaque WalkingEnd signature, contextual eliminator, section and recursor specializations	No general directed-HIT signature compiler or arbitrary cell-complex schema
Truncation and height	Recursive truncation properties and closure, evidence-property, finite `IsNCat` object truncation	No general truncation reflector or arbitrary truncation HIT
WalkingEnd calculation	Code, encode, power, spiral, contextual decoder, normalization cell/path, two inverse laws, carrier equivalence and noninvertibility results	No packaged monoid isomorphism, reverse `BNat` functor, full hom-category equivalence, or initiality theorem
Higher groupoidal shadow	Selected Eckmann–Hilton commutativity slice	No claim that all directed structure is groupoidal
Ordinary categorical specialization	Precategories, univalent categories, strict categories, functors, natural transformations, and ordinary Yoneda developed over the native vocabulary	These readable one-categorical theorems are mathematical development, not definitions of native `Cat`
Adjunctions and equivalences	Triangle cuts and hom-profunctor comparison; one-way lift from ordinary isomorphism to native evidence	No checked native fully-faithful/essentially-surjective characterization or general adjointification package
Yoneda and profunctors	Cat-valued profunctors, endpoint reindexing, representables, shaped cells, fixed-middle tensor, co-Yoneda beta/fusion	No general coend semantics, tensor associativity package, full Cat-valued Yoneda equivalence, or profunctor bicategory

Presheaves and sieves	Cat-valued presheaves, Yoneda and slices, higher sieves, ordinary pointwise-subterminal sieves, pullback membership, and commutative-ring invertibility sieves	No global ordinary-sieve classifier, automatic representation by one open, topology, descent, or sheafification follows from this layer
Sites and descent	Ordinary-sieve Grothendieck topology laws, chaotic model, internally generated least topology, whole sieve extensions, matching and section Hom families, and topology-locality	No inductive cover derivations, coverhood decision procedure, automatic subcanonicity, sheafification reflector, or identification with a separate rigid sheaf facade follows from locality alone
Direct cover sheafification	Cat-valued categorical-HIT completion with whole return/glue/silent data, derived topology-locality, recursor, whole Hom universality, adjunction, and reflective counit	Fixed-site and Cat-valued only; no arbitrary coefficients, commutative-ring lift, left exactness, site base-change theorem, or classical plus-construction comparison
Commutative algebra	Set-carrier rings and structured maps, finite unimodular presentations, polynomial and localization universal-property interfaces, selected unit/zero/idempotent models, and whole iterated/product-localization equivalence	No arbitrary polynomial/localization existence, monomial or fraction representation, categorical product theorem, global ring-package identity, or affine geometry follows from this layer alone
Affine geometry	Yoneda functor of points; ordinary basic-open sieve; pointwise localization representation and multiplicative intersection; big affine slice, coordinate presheaf, and least generated Zariski topology; assumption-explicit reflective structure sheaf, localization locality, and thin affine presentation	No whole natural basic-open equivalence, global localization choice, CommRing-valued sheafification construction, small-site comparison, subcanonicity, stalk-local theorem, qcqs comparison, or representation-independent category of affine schemes
Site-relative schemes	One global reflective ringed object and covering sieve; witness-rich binary generation; whole actual-slice restriction; supplied affine-basis realizations; topology-local ring forcing; dependent binary scheme total; selected actual overlap with derived ring restrictions	Binary and relative to the supplied site; no atlas-first gluing, induced slice topology, arbitrary pullback construction, overlap-affineness theorem, scheme-morphism category, compact-open/classical comparison, or representation-independent scheme theorem
Supplied projective-line boundary	Universal-property Laurent transition maps; literal common-overlap identity package; thin adapter to actual inherited chart restrictions; dependent total of one already-global scheme, its actual overlap, and Laurent coordinates	The global object and Laurent identity paths remain supplied; no atlas-first gluing, projectivity or non-affineness proof, graded ring, homogeneous localization, degree-zero construction, `Proj`, or general projective space
Opposite, duality, and dagger	Opposite category action and selected opposite-duality comparisons	Dagger, unitary structure, and dagger univalence are mathematical development pending a native involutive interface
Structure identity and saturation	Truncation/evidence-property footholds and ordinary-isomorphism lift	Generic native structure identity and Rezk completion, including their higher universal properties, are research boundaries
Weighted limits and Kan interfaces	Weighted representability, beta/eta comparison, right-adjoint preservation, terminal/conjoint specializations	Standard end formulas, pointwise Kan semantics, existence, and general dependent adjunctions are not globally packaged

Weighted colimits and join	Opposite-dual colimit preservation, terminal/companion specializations, primitive join recursor and three beta observations	General coend semantics and join-as-collage mapping, hom-decomposition, opposite, and dependent-elimination theorems remain open
Formal presentation	Checked categorical owners; a bounded TypeScript outer LF, explicit Core, contextual elaborator, checker/runtime, reviewed text subset, adjunction/structure declaration conveniences, and client-side reviewer	No compiler for the complete book surface, arbitrary displayed coherence, general record/inductive facility, or whole-library transfer; readable notation is not a second kernel
Metatheory and models	Bounded typechecking, subject-reduction checks performed by Lambdapi, focused diagnostics, and the concrete BNat model	No global confluence, normalization, canonicity, decidability, consistency, or semantic-soundness theorem for the full combined calculus
Production artifact	Manifest assembly, provenance/evidence checks, local assets, bounded browser validation, and deterministic PDF export	External mathematical peer review and a non-draft public edition remain future release work

F.2 Near-Term Formal Strengthening

The most direct strengthening of Theorem 8.1 is to package composition and addition compatibility. Its proof can use the checked power recursion, Nat addition associativity, and both carrier inverse laws. The desired result is a monoid-level comparison with an explicit orientation matching `BNat`.

Next comes a reverse functor from `BNat` to WalkingEnd and a comparison with the existing model functor. This requires reusable action-to-functor and functor-extensionality infrastructure; it should not be simulated by making the opaque hom definitionally Nat-valued.

Full initiality is a further layer. It asks for a category of endomorphism algebras, structured maps, and coherent higher transfors, followed by an appropriate contractibility or equivalence theorem.

F.3 Foundational Extensions

A reusable directed-HIT schema should generate contextual elimination and constructor computation from typed object, arrow, and higher-cell boundaries. Its validation must include rewrite overlap and subject-reduction behavior, not only a semantic signature.

A truncation reflector should construct a universal truncated target rather than merely certify an existing classifier. Directed categorical truncation would additionally need to specify which lower arrows and compositions are preserved.

The univalence programme should continue to separate carrier equivalence, ordinary categorical isomorphism, and native equality-valued recursive equivalence. A full theorem relating object equality and ordinary isomorphism must be proved at the intended categorical level rather than recovered through retired compatibility aliases.

F.4 Categorical Extensions

The representable/profunctor and identity layers suggest four staged projects:

1. package a fully faithful Yoneda embedding with mapping-category equivalences and higher naturality;
2. construct or model Cat-valued coends/coinserters and relate the opaque tensor to their universal property;
3. assemble associators, unitors, and horizontal cell composition into a coherent profunctor bicategory or suitable omega-categorical analogue.
4. design generic structure identity and Rezk completion interfaces only after the intended native equivalence and higher mapping properties are fixed.

Weighted limits, colimits, adjunctions, duality, and joins now enter the expanded chapter sequence through the triangle reductions, right-adjoint weighted-limit preservation, its opposite-dual colimit theorem, and the join recursor. The checked interfaces are the theorem spine; neighboring Kan, end/coend, dagger, collage, and dependent-elimination theory remains explicitly status-labeled rather than presented as a feature catalogue.

F.5 Semantics And Proof-Assistant Engineering

The largest research objective is a semantics and metatheory for a precisely stated fragment: typing, substitution, subject reduction, normalization or a weaker operational theorem, and interpretation in a suitable strict/lax omega-categorical model. The current executable artifact is evidence for specific interfaces, not a substitute for that theorem.

The renewed TypeScript product now elaborates a bounded direct-TypeScript and categorical-text surface into backend-neutral explicit Core, then checks and reduces that Core with a small dependent logical framework. Its contextual categorical layer covers reviewed ordinary, natural, displayed-functorial, and displayed-natural binders. Within the canonical sibling/Sigma normal form it supports finite dependency depth and sibling groups; qualified finite Hom-category recursion and finite rigid indexed-section chains are also executable. An optional deterministic Lambdapi path remains a conformance oracle. It is not a production dependency, and the active Lambdapi development remains the mathematical authority.

The same outer LF has two bounded authoring conveniences. One declares an adjunction from already typed rectangular data, or from a counit and whole hom transpose, while retaining proof-time rather than runtime agreement with the stable observations. The other declares an unparameterized, nonrecursive, single-constructor dependent structure with named projections and projection beta rules. Both expand to ordinary declarations; neither adds a trusted Core form, categorical owner, general record eta, eliminator, recursion, or positivity principle.

This is a real executable bridge, visible in the client-side integrated reviewer, but not completion of the canonical mathematical surface. Arbitrary dependency and variance graphs, coherence outside the qualified grammar, a compiler for the whole book notation, a general record or inductive facility,

and systematic transfer of the remaining library are still engineering boundaries. The older TypeScript prototype remains historical feasibility evidence; its stale category-specific layer is neither an authority nor the architecture of the renewed product.

Ordinary DevOps makes checks, assembly, and release repeatable. The project's MathOps discipline additionally separates mathematical owners, independent reviewers, generated evidence and health views, authored sources, and deterministic release artifacts. That separation makes drift and provenance auditable. It does not convert a passing build, warning inventory, browser run, or reproducible PDF into a confluence, normalization, consistency, or soundness theorem.

F.6 Reading Claims Across Editions

The edition version in `book/book.json` identifies the source snapshot policy for generated artifacts. A later edition may promote a research boundary only when the evidence register names active owners and independent checks, or when the claim is explicitly reclassified as mathematical development with stated prerequisites. Dated reports preserve why a boundary was chosen; they do not override the current code.

Appendix G. Formal Presentation Of Functorial Type Theory

The mathematical chapters have used rules from the beginning: an identity path eliminates by reflexivity, a functor acts on an arrow, a transfor absorbs a naturality cut, and a representing comparison eliminates a universal map. This appendix puts those rules in one formal architecture. It follows the discipline of Appendix A of the HoTT Book—contexts and judgments first, then rule families, extensions, and metatheory—but changes the order of explanation at one decisive point.

In functorial type theory, category theory is not added later as a model of a previously specified, traditional term calculus. The explicit categorical calculus is already the computational core. Its objects include categories, iterated homs, functors, transfors, and directed families; its reductions express functoriality, naturality, induction, and universal properties. The active Lambdapi v3.2 development authors and checks that calculus and remains the mathematical authority.

The canonical mathematical surface used by this book is deliberately more spacious than any one executable grammar. A renewed TypeScript product now implements a bounded route from readable categorical binders to explicit emdash Core and a small dependent logical framework. That route makes a reviewed fragment directly usable; it does not become a second categorical kernel or define the mathematics retroactively.

The architecture therefore distinguishes these roles before its two executable paths meet in the operational diagram that follows:

Layer or role	Responsibility	Present status
canonical mathematical surface	the notation and rule presentation used by this book	active for prose, comments, and examples; not a parser grammar
scoped contextual elaboration	recursively interprets reviewed categorical variables, binders, neutral applications, and structural forms against typed expectations	active for the bounded direct-TypeScript and text profiles
typed outer-LF declarations	validate selected higher-level declarations and expand them into ordinary LF declarations and rules	active for adjunction assumptions and one bounded dependent-structure form; no new trusted Core node
backend-neutral explicit emdash Core	records the selected logical and categorical owners without committing to one runtime backend	active TypeScript intermediate representation
generic TypeScript dependent LF	checks Core terms, performs conversion and bounded reduction, and runs the reviewed proof-time rules	active for the recorded product boundary
active Lambdapi v3.2 kernel	authors the categorical declarations, computation, and proof-time comparisons used as mathematical authority	active and checked in the cited modules; also the conformance oracle
external semantic models	interpret a stated kernel fragment in mathematical categories or other structures	separate mathematical work; available only in selected examples

```
 canonical mathematical surface (broader than implemented text)
  -> reviewed direct TypeScript / text expressions
  -> scoped contextual elaboration
  -> explicit Core terms ------------------------------------------+
                                                                   |
typed host declarations                                            |
  -> deterministic expansion                                       |
  -> ordinary LF declarations and rules --------------------------+
                                                                   |
                                                                   v
generic TypeScript LF checker / conversion / bounded runtime
  -> optional deterministic Lambdapi emission / conformance

active authored Lambdapi v3.2 kernel = mathematical authority
external models                    = separate mathematical work
```

The text adapter is not the checker, an authoring macro is not a new term former, the TypeScript checker is not the active mathematical authority, and the implemented text subset is not the whole canonical surface. External interpretation is separate again. Keeping these roles distinct lets us say exactly which claims are checked computation, which are executable presentation, which are mathematical exposition, and which remain research.

G.1 Judgments, Contexts, And Classifiers

A formal presentation begins with judgments made in contexts. We use Γ for a finite ordered list of declarations. The basic external judgments have the schematic forms

$$\Gamma \ \mathsf{ctx}, \qquad \Gamma \vdash T : \mathsf{TYPE}, \qquad \Gamma \vdash t : T, \qquad \Gamma \vdash t \equiv u : T.$$

These are metatheoretic assertions about expressions. In particular, `t:T` is not itself an internal proposition whose inhabitant is a proof that `t` has type `T`. Lambdapi checks the judgment while reading a declaration, definition, rule, or assertion.

Contexts are ordered because later entries may depend on earlier ones:

$$x : A, \quad y : B(x), \quad z : C(x, y).$$

Substitution replaces a declared variable by a term of the required type and acts in every later entry and conclusion. Renaming, weakening, exchange when dependencies permit it, and substitution are structural operations of the ambient dependent framework. They are not constructors of an internal `Context` classifier in the active emdash kernel.

External Types And Decoded Classifiers

The kernel uses two related universes. `TYPE` is Lambdapi's ambient type level. `Grpd : TYPE` is the small groupoidal or type-like classifier universe used by emdash, and

$$\tau : \mathsf{Grpd} \longrightarrow \mathsf{TYPE}$$

decodes a classifier to the ambient type of its elements. Thus the book's readable judgment $a : A$ normally abbreviates the literal judgment `a : τ A` for `A : Grpd`.

Categories live at the ambient level:

$$\mathsf{Cat} : \mathsf{TYPE}.$$

Their object and arrow collections are internal classifiers:

$$\begin{aligned} \mathrm{Obj}(C) &: \mathsf{Grpd}, \\ \mathrm{Hom}_C(x, y) &: \mathsf{Cat}, \\ \mathrm{Hom}(C, x, y) := \mathrm{Obj}(\mathrm{Hom}_C(x, y)) &: \mathsf{Grpd}. \end{aligned}$$

The second line is the source of higher structure. If $f, g : x \to y$, then a 2-cell $\alpha : f \to g$ is an object of $\mathrm{Hom}_{\mathrm{Hom}_C(x,y)}(f, g)$; repeating the same construction exposes higher cells without changing the grammar at each dimension.

The main internal classifiers used in the book are:

Mathematical classifier	Literal owner	Decoded inhabitants
objects of C	`Obj C`	`τ (Obj C)`
arrows $x \to_C y$	`Hom C x y`	`τ (Hom C x y)`
functors $A \to B$	`Functor A B`	`τ (Functor A B)`
transfors $F \Rightarrow G$	`Transf F G`	`τ (Transf F G)`
Cat-valued families over K	`Catd K`	`τ (Catd K)`
displayed functors $E \to D$	`Functord E D`	`τ (Functord E D)`
displayed transfors $FF \Rightarrow GG$	`Transfd FF GG`	`τ (Transfd FF GG)`

This distinction prevents two common category mistakes. First, `C : Cat` is not an object of some silently assumed set of all categories; it is an ambient kernel judgment. Second, an arrow classifier is not merely a set of morphisms: it is the object classifier of another category and can therefore be iterated.

Six Ways To Compare Expressions

Several notations that resemble equality have different force.

Notation	Layer	Meaning
$t \rightsquigarrow u$ in this book; `t ↪ u` in source	runtime	a selected oriented rewrite makes u the computational form
$t \equiv u$	external conversion	the checker regards the terms as definitionally convertible using active computation
`unif_rule t ≡ u ↪ [...]`	proof-time elaboration	a unification problem is replaced by narrower subproblems; neither side is selected as a runtime normal form
$p : x =_A y$	internal mathematics	`p` inhabits an equality classifier and can be eliminated by `ind_eqr`
$A \simeq B$	internal mathematical structure	specified maps and inverse/coherence evidence, such as `TypeEquiv` or a categorical comparison
$t = u$ in status-labeled free-form prose	mathematical development	a theorem to be proved in the named future interface, not an undisclosed kernel conversion

A runtime rule does not by itself assert equality reflection. An internal path does not make its endpoints definitionally identical. A proof-time unification rule is not a path constructor, and an equivalence is not an unlabeled use of conversion. Chapters 1 and 9 rely on precisely these distinctions.

> **Formal status — checked.** Evidence `FORMAL-KERNEL-PRESENTATION` covers the active categorical classifiers, their application operations, and representative executable checks. The displayed turnstile notation is the book's metanotation; it is not a new internal judgment former.

G.2 The Mathematical Categorical Presentation

We now give a compact signature in the notation of the book. It is the human-readable presentation of the kernel, not an untyped concrete grammar. Implicit arguments are suppressed only when their recovery is forced by the displayed source and target.

Categories And Iterated Homs

The core categorical judgments are

$$\frac{}{C : \mathsf{Cat}}, \qquad \frac{C : \mathsf{Cat}}{x : \mathrm{Obj}(C)}, \qquad \frac{x, y : \mathrm{Obj}(C)}{f : \mathrm{Hom}_C(x, y)}.$$

Every object has an identity, and composable arrows have a composite:

$$\mathrm{id}_x : x \to_C x, \qquad \frac{f : x \to_C y \quad g : y \to_C z}{g \circ f : x \to_C z}.$$

The convention is always “first f, then g.” Identity and associativity are available at the intended comparison layers. The implementation does not install unrestricted reassociation as a general runtime rewrite.

Opposite categories reverse the hom endpoints:

$$\mathrm{Hom}_{C^{\mathrm{op}}}(x, y) \rightsquigarrow \mathrm{Hom}_C(y, x).$$

The path category embeds the equality-local fragment:

$$\mathrm{Obj}(\mathsf{Path}(A)) \rightsquigarrow A, \qquad \mathrm{Hom}_{\mathsf{Path}(A)}(x, y) \rightsquigarrow \mathsf{Path}(x =_A y).$$

This is a groupoidal specialization. It does not identify arbitrary directed arrows with equality paths.

Functors And Their Iterable Action

For categories A, B, the category $A \vdash B$ has functors as objects:

$$\frac{A, B : \mathsf{Cat}}{A \vdash B : \mathsf{Cat}}, \qquad \frac{}{F : \mathrm{Obj}(A \vdash B)}.$$

We write the second judgment as $F : A \to B$. A functor has object action and, for every pair $x, y : A$, a functor on the whole hom-category:

$$\begin{aligned} &F[x] : \mathrm{Obj}(B), \\ &F_{x,y} : \mathrm{Hom}_A(x, y) \longrightarrow \mathrm{Hom}_B(F[x], F[y]), \\ &F[f] := F_{x,y}[f]. \end{aligned}$$

The hom action, not only its value at one arrow, is primary. It can act again on a 2-cell between arrows, and its own hom action continues the same pattern. At the first capped level, the selected functoriality cuts are

$$F[\mathrm{id}_x] \rightsquigarrow \mathrm{id}_{F[x]}, \qquad F[g] \circ F[f] \rightsquigarrow F[g \circ f].$$

Transfors And Family Action

For parallel functors $F, G : A \to B$, the category $F \Rightarrow G$ is their first hom in the functor category. A transfor $\eta : F \Rightarrow G$ has a point component

$$\eta_x : F[x] \longrightarrow G[x]$$

and, more fundamentally, an off-diagonal hom functor

$$\eta_{x,y} : \mathrm{Hom}_A(x, y) \longrightarrow \mathrm{Hom}_B(F[x], G[y]).$$

We write $\eta[f]$ for its value at $f : x \to y$. Its two adjacent naturality cuts compute:

$$\begin{aligned} G[g] \circ \eta[f] &\rightsquigarrow \eta[g \circ f], \\ \eta[f] \circ F[h] &\rightsquigarrow \eta[f \circ h]. \end{aligned}$$

The diagonal component is the identity-arrow instance of this family action. Thus naturality is not a proposition pasted onto a bare family of arrows. It is part of an operation whose higher action remains available.

Directed Families, Totals, And Sections

A directed Cat-valued family over K is written

$$E : K \longrightarrow \mathsf{Cat}.$$

For $k : K$ it has a fibre $E[k]$, and for $p : k \to_K k'$ it has transport

$$E[p] : E[k] \longrightarrow E[k'].$$

A displayed functor $FF : E \to D$ contains fibre functors and off-diagonal comparison cells over base arrows. In book notation its classifier is

$$k :^n K \;;\; E[k] \vdash D[k].$$

Displayed transfors arise by taking the next hom in this category. The `Catd`, `Functord`, and `Transfd` facades keep these levels visible instead of flattening them into pointwise functions.

Two categorical dependent formers organize families:

$$\sum_{k:^n K} E[k] \qquad \text{and} \qquad \prod_{k:^n K} E[k].$$

The Sigma total has objects (k, u) with $u : E[k]$. An arrow consists of a base arrow $p : k \to k'$ and a fibre arrow

$$E[p](u) \longrightarrow u' \quad \text{in } E[k'].$$

The Pi category has coherent sections as objects. Its evaluation at k is a functor to $E[k]$, not merely a carrier-level application function.

Chosen Arrows And Natural Families

The two represented-hom actions expose the first Došen-style cut discipline. For $u : x \to_A y$,

$$u_*(g) = u \circ g \quad \text{and} \quad u^*(h) = h \circ u.$$

Lower star is covariant postcomposition; upper star is contravariant precomposition. With a functor H, the acting arrow is $H[u]$, so $(H[u])^*(h) = h \circ H[u]$. By contrast, $\eta[f]$ uses the whole natural family η, not one selected arrow. Structural and universal eliminators continue this progression.

The product/projection benchmark of Chapter 9 is a theorem in an arbitrary category K equipped with products. Its Cat-specialized executable probe is evidence about the current owner calculus, not a restriction of the mathematical statement to the category of categories.

Signature-To-Owner Map

The following table records the correspondence without treating readable notation as a second implementation.

Readable operation	Active owner
$x \to_C y$	`Hom_cat C x y` and `Hom C x y`
$A \vdash B$	`Functor_cat A B`
$F[x], F[f]$	`fapp0`, `fapp1_func`, `fapp1_fapp0`
$F \Rightarrow G$	`Transf_cat F G`
$\eta_x, \eta[f]$	`tapp0_fapp0`, `tapp1_func`, `tapp1_fapp0`
$E : K \to \mathsf{Cat}$ and $E[k]$	`Catd K` and `Fibre_cat E k`
displayed functors and transfors	`Functord` and `Transfd`
$\sum_k E[k], \prod_k E[k]$	`Sigma_cat E` and `Pi_cat E`
u_*, u^*	`hom_postcomp_*` and `hom_precomp_along_*`

> **Formal status — checked.** Evidence `CAT-ITERATED-HOMS`, `CAT-FUNCTOR-CALCULUS`, `TRANSF-POINT-OFFDIAGONAL`, `TRANSF-STRICT-NATURALITY`, and `CAT-DIRECTED-FAMILIES` support the displayed nucleus. The typography in this section is canonical mathematical surface notation; the next section shows literal source.

G.3 The Checked Lambdapi Presentation

The active source is a Lambdapi signature. This section gives representative literal excerpts, enough to explain how the mathematical presentation is checked without reproducing the kernel.

Declarations And Definitions

At the universe boundary the source says:

```
 constant symbol Grpd : TYPE;
injective symbol τ : Grpd → TYPE;

constant symbol Cat : TYPE;
symbol Obj : Cat → Grpd;
injective symbol Hom_cat :
  Π (A : Cat) (X_A Y_A : τ (Obj A)), Cat;
injective symbol Hom (A : Cat) (X_A Y_A : τ (Obj A)) : Grpd
≔ Obj (Hom_cat A X_A Y_A);
```

These lines exhibit three declaration policies.

- A `constant symbol` cannot receive a definition or rewrite rules. `Cat`, `Grpd`, and the WalkingEnd constructors use this literal policy.
- A plain `symbol` may be an undefined operation that later receives rules, or it may have a transparent body after `≔`.
- An `injective symbol` gives the unifier a rigid constructor-like head. The modifier is a trusted declaration choice and is used only at selected classifier and stable-owner boundaries.

Lambdapi also supports an `opaque` modifier for a defined symbol whose body must not reduce. In the book, "opaque WalkingEnd" describes the mathematical effect of its constant declarations; it does not claim that those lines use the literal `opaque symbol` spelling.

Implicit parameters appear in square brackets and explicit parameters in parentheses. Prefix `@` exposes normally implicit parameters at a use site. In rule patterns, a dollar-prefixed name such as `$A` is a pattern variable, while `_` asks typing and unification to recover a slot that is not a genuine discriminator.

Rewrite Owners

Functor application is declared at a full hom level and a capped arrow level:

```
 symbol fapp1_func : Π [A B : Cat], Π (F_AB : τ (Functor A B)),
  Π [X_A Y_A : τ (Obj A)],
  τ (Functor
    (Hom_cat A X_A Y_A)
    (Hom_cat B (fapp0 F_AB X_A) (fapp0 F_AB Y_A)));

symbol fapp1_fapp0 : Π [A B : Cat], Π (F_AB : τ (Functor A B)),
  Π [X_A Y_A : τ (Obj A)],
  Π (f : τ (Hom A X_A Y_A)),
  τ (Hom B (fapp0 F_AB X_A) (fapp0 F_AB Y_A));

rule fapp0 (fapp1_func $F_AB) $f
  ↪ fapp1_fapp0 $F_AB $f;
```

The last line is runtime computation: observing the full hom-action at one arrow exposes the capped action. The generic identity and composition rules then contract $F[\mathrm{id}]$ and $F[g] \circ F[f]$. Concrete functor constructors inherit those rules; they do not each receive private copies of ordinary functoriality.

The same ownership policy governs transfors. `tapp0_fapp0` observes a point component, while `tapp1_func` and `tapp1_fapp0` own off-diagonal action. The two strict naturality rewrites are attached to that generic action. A constructor-specific rule is justified only when it expresses extra constructor computation, not the fact that something already typed as a transfor is natural.

Proof-Time Unification

Some stable category presentations should elaborate together without one being erased at runtime. For rigid hom-category heads the source includes:

```
unif_rule Obj (Hom_cat $A $X $Y) ≡ Obj (Hom_cat $A' $X' $Y')
  ↪ [ $A ≡ $A'; $X ≡ $X'; $Y ≡ $Y' ];
```

This rule decomposes one proof-time unification problem into three. It does not rewrite an object classifier during execution, prove an internal path, or assert that `Obj` is globally injective for every category construction.

Similar narrow comparisons relate the ordinary functor-category presentation to `Catd_cat`, and the ordinary transfor presentation to `Functord_cat`.

Associativity illustrates the boundary particularly well. The two bracketings of ordinary composition are compared at proof time, and `comp_assoc` packages a propositional witness. There is no global runtime rule that continually reassociates every composite. Represented hom actions, `tapp1`, and universal comparisons instead own the specific cuts they can normalize without losing higher action.

Assertions And Negative Assertions

Executable diagnostics are source commands, not theorem prose. A small example is:

```
 assert ⊢ Nat_grpd : Grpd;
assert ⊢ zero : τ Nat_grpd;
assertnot ⊢ @eq_refl Nat_grpd zero ≡ tt;
```

The first two ask Lambdapi to accept a type and an inhabitant. The last checks that two terms are not definitionally convertible. A typed reflexivity term is used when a proof-time unification rule must be exercised; a bare conversion assertion does not test that same mechanism.

The diagnostic suite is intentionally separate from implementation owners. Permanent examples and assertions provide regression evidence, while the evidence register connects book claims to both declarations and reviewers.

Modules And Ownership

The source graph is larger than a useful reading list. The following map groups adjacent modules by mathematical responsibility; the evidence register supplies the exact owner and reviewer for each cited claim.

Module family	Formal role
`emdash3_2.lp`	categorical nucleus: classifiers, iterated homs, functors, transfors, directed families, cuts, and universal-construction interfaces
`emdash3_2_presheaves.lp`, `emdash3_2_sieves.lp`, `emdash3_2_sites.lp`	presheaves, higher and ordinary sieves, pullback, and the direct Grothendieck-topology laws
`emdash3_2_generated_topologies.lp`, `emdash3_2_sieve_extensions.lp`, `emdash3_2_site_basis.lp`, `emdash3_2_ringed_sites.lp`	least generated topology, whole matching/section families, basis comparison, and ringed-site presentations
`emdash3_2_direct_cover_*.lp`	return/glue/silent cover completion, recursion, topology-locality, whole Hom universality, and the resulting Cat-valued reflector

Module family	Formal role
`emdash3_2_commutative_algebra.lp` through the polynomial and localization modules	set-carrier rings and structured maps, finite unit-ideal data, free extension, universal localization, and whole localization comparisons without polynomial or fraction syntax
the commutative-algebra presheaf, affine-points, affine-Zariski, ringed-site, and affine-scheme modules	the invertibility sieve $D(f)$, localization representation, generated big Zariski topology, coordinate presheaf, and assumption-explicit affine presentations
the ringed-space cover, affine-chart, site-relative-scheme, and chart-overlap modules	one supplied global ringed object, constructively generated covers, whole actual-slice restrictions, affine realizations, topology-local rings, and inherited overlaps
`emdash3_2_commutative_algebra_laurent.lp`, `emdash3_2_commutative_algebra_scheme_laurent_overlaps.lp`, `emdash3_2_commutative_algebra_projective_line.lp`	universal-property coordinate inversion on one literal overlap and the supplied projective-line boundary; no graded `Proj` construction
`emdash3_2_eq1_*.lp`, `emdash3_2_nat_arithmetic.lp`, `emdash3_2_walking_end_hit.lp`	equality-valued higher action, reusable arithmetic, and the WalkingEnd encode-decode development
`emdash3_2_checks.lp` and `examples/`	executable diagnostics and independent reviewer-facing witnesses rather than mathematical owners

Imports use `require`; `open` brings imported public names into scope. The file split records dependency and evidence ownership. A conceptual chapter may use several owners, and one source family may support several chapters; neither direction is forced to mirror the table of contents.

Three source policies are essential for reading rules correctly.

1. Match a computation at its semantic owner and retain stable heads needed by later higher action.
2. Keep inferred rule slots anonymous unless a slot is a measured type, subject-reduction, or decision-tree guard.
3. Use runtime rewrites only for intended normal forms; use narrowly typed proof-time comparisons when neither side should compute to the other.

> **Formal status — checked.** Evidence `FORMAL-KERNEL-PRESENTATION` records the representative declaration, action, rule, module, and diagnostic surface described here. Successful source checking warrants these interfaces; it does not establish the global metatheorems listed in G.7.

G.4 Formation, Introduction, Elimination, And Computation

The familiar rule schema remains useful, provided we add a sixth question suited to directed mathematics.

Rule aspect	Question
formation	when is the classifier or categorical object well formed?
introduction	what data construct an inhabitant or structured object?
elimination	how may an inhabitant be observed or used?
computation	what does elimination do to introduced data?
uniqueness or universality	is an arbitrary inhabitant recovered, propositionally compared, or characterized by a mapping property?
action and coherence	how does the construction act on arrows and higher cells as its parameters vary?

The last row is not optional decoration. A pointwise formula may answer the first five questions at objects while failing to define a functor, transfor, or displayed family.

Equality Induction

For $A : \mathsf{Grpd}$ and $x, y : A$, equality formation gives $x =_A y$. Reflexivity introduces an inhabitant $\mathsf{refl}_x : x = x$. Right-based elimination fixes y, takes

$$P : \prod_{x:A} (x = y) \longrightarrow \mathsf{Grpd}, \qquad u : P(y, \mathsf{refl}_y),$$

and returns

$$\mathsf{ind\_eqr}(P, u, p) : P(x, p) \quad \text{for } p : x = y.$$

Its literal-reflexivity beta computes:

$$\mathsf{ind\_eqr}(P, u, \mathsf{refl}_y) \rightsquigarrow u.$$

Path action, dependent path action, symmetry, and transitivity are derived uses. No equality reflection, uniqueness of identity proofs, or global path-eta rule is added. At the categorical layer, `Path_cat` and `path_map_func` package equality and function action so that higher path action can be iterated.

> **Formal status — checked.** Evidence `TT-EQUALITY-INDUCTION`. Formation, reflexivity, right-based elimination, beta, `ap`, and `apd` are active. A stronger global uniqueness principle is not silently inferred from the beta rule.

Categories, Functors, And Transfors

`Cat`, `Obj`, and `Hom_cat` give formation for the categorical tower. Identity and composition are introduction operations for arrows, while iteration of `Hom_cat` eliminates an arrow into its next-cell context. The selected unit and associativity comparisons say how these introductions compose; there is no eliminator claiming that every category is freely generated by its displayed arrows.

For a functor, formation is `Functor A B`. Its elimination operations are `fapp0` and `fapp1_func`. The projection beta from full hom action to `fapp1_fapp0` and the identity/composition cuts are its generic computations. The active theory obtains inhabitants from named functor constructors and categorical operations; the book does not posit one record-style constructor whose fields may be supplied incoherently.

For a transfor, formation is `Transf F G`. Point and off-diagonal application are eliminations. Identity-boundary, composition, and the two naturality cuts are computations. The full off-diagonal functor is the action clause: it says what happens not just to an arrow f but to a higher cell between possible values of f.

This gives a useful criterion:

> A point-component formula does not define a transfor until the off-diagonal arrow action and its next-hom behavior are supplied or explicitly deferred.

Sigma Totals And Pi Sections

For $E : K \to \mathsf{Cat}$, categorical Sigma formation gives $\Sigma_K E : \mathsf{Cat}$. An object is introduced as (k, u) with $u : E[k]$. The first projection and the fibre component eliminate it. A total arrow is introduced by a pair

$$\big(p : k \to k', \alpha : E[p](u) \to_{E[k']} u'\big).$$

Projection and composition rules compute on this structure. The `sigma_intro_transf` packages the inclusions $E[k] \to \Sigma_K E$ naturally in k, so introduction also has a directed action rather than only a pair constructor.

Pi formation gives the section category $\Pi_K E$. Evaluation is packaged by `pi_eval_transf`, whose component at k is the functor

$$\mathrm{ev}_k : \Pi_K E \longrightarrow E[k].$$

The evaluation projection computes through `piapp0_func`. Constant sections and pullback of sections supply important introductions, but the current interface does not assert a general categorical Pi-eta or a fully packaged dependent adjunction. Those stronger universal laws require the base-arrow, off-diagonal, and Beck–Chevalley data described in Chapter 16.

At the groupoid layer, encoded Sigma and Pi classifiers separately provide dependent pairs, projections, pointwise path observation, and the selected `happly` / `funext` equivalence. The categorical Sigma/Pi operations above must not be flattened into those carrier-level formers: their objects vary in categories and their arrow action is part of the interface.

> **Formal status — checked nucleus.** Evidence `CAT-SIGMA-PI` and `TT-SIGMA-PI-PATHS`. The active rules cover the cited constructors, projections, evaluation, and action. General dependent adjunctions remain the research boundary recorded in Chapter 16.

The WalkingEnd Rule Package

The walking endomorphism makes every row of the schema visible:

Aspect	Selected WalkingEnd datum
formation	`WalkingEnd_cat : Cat`
object introduction	`walking_base : Obj(WalkingEnd)`
arrow introduction	`walking_loop : Hom(WalkingEnd,base,base)`
height datum	`walking_end_is_one_cat : IsNCat(1,WalkingEnd)`
contextual algebra	$R, D : W \to \mathsf{Cat}$, $u : R[*] \to D[*]$, and $\sigma : D[\ell] \circ u \Rightarrow u \circ R[\ell]$
elimination	`walking_end_ind_funcd R D u sigma : Functord R D`
base computation	the fibre component at $*$ reduces to u
loop computation	the displayed action at ℓ reduces to the supplied component of σ
uniqueness	no general initiality or contractibility theorem is currently packaged
action	the result is a displayed functor, so base-arrow and higher action are retained

The section eliminator specializes R to the terminal family. The ordinary recursor specializes both families to constants. They are derived views of the contextual eliminator, not three independent postulates. The literal constructor betas are attached to stable generic observers, with two narrow projection joins for the concrete ordinary-recursion consumers.

> **Formal status — checked.** Evidence `WE-SIGNATURE`, `WE-CONTEXTUAL-ELIMINATOR`, and `DHIT-DERIVED-ELIMINATORS`. The absence of a uniqueness theorem is part of the formal statement, not a prose omission.

Adjunction And Weighted Representability

An adjunction illustrates a rule package whose principal eliminations are observations rather than a public record constructor. For named functors $F : R \to L$ and $G : L \to R$, the classifier

$$\mathsf{Adjunction}(F, G)$$

forms the proposition-like structure accepted by the kernel. From $J : \mathsf{Adjunction}(F, G)$ one eliminates the selected unit and counit:

$$\eta : \mathrm{id}_R \Rightarrow GF, \qquad \varepsilon : FG \Rightarrow \mathrm{id}_L\,.$$

The two triangle cuts compute at off-diagonal components. Their role is exactly beta reduction: an introduction by a unit followed by elimination by a counit contracts to the underlying map, and dually.

A weighted limit is presented one level more abstractly. Given $F : J \to B$, $W : J' \nrightarrow J$, and $L : J' \to B$, formation gives the classifier

$$\mathsf{IsWeightedLimit}_{\mathrm{cov}}(F, W, L).$$

An inhabitant is a computational comparison between the weighted-cone profunctor and the representable hom profunctor. Reindexing and applying that certificate eliminates it into inverse operations

$$\mathsf{push} \quad \text{and} \quad \mathsf{pull}.$$

Their composites reduce by the generic profunctor-comparison beta and eta rules. The action clause is reindexing along every probe functor $M : I \to B$; the universal property is not restricted to one set of global elements. Adjunction mates then transport the whole comparison, which is why right-adjoint preservation is a computation on certificates rather than a fresh pointwise proof.

Existence for every diagram and uniqueness of representing objects are separate theorems. The active classifier says what data certify a chosen L; it does not postulate a global limit operator or a native univalent uniqueness package.

> **Formal status — checked.** Evidence `ADJ-TRIANGLE-CUTS` and `WEIGHTED-LIMIT-REPRESENTABILITY`.
>
> The formation/elimination/computation interface is active. Semantic end formulas, general existence, and univalent uniqueness remain separately status-labeled mathematics.

G.5 Elaboration And Canonical Surface Syntax

Readable notation is indispensable, but it need not be the foundational layer. The renewed TypeScript path compiles a reviewed subset into the explicit owners described above:

Stage	Implemented bounded profile	Retained boundary
parse	located `^f`, `^n`, `^fd`, and `^nd` binders, neutral application, selected constructors, and grouped displayed contexts	not the complete book or Lambdapi grammar
elaborate	typed expected classifiers route recursively through the existing contextual categorical program	no arbitrary pointwise-to-coherent synthesis
select owner	reviewed operation families lower to internal categorical and structural owners	no whole-library owner-acquisition claim
check and reduce	the generic TypeScript LF checks explicit Core, compares terms, and executes the bounded runtime	no global metatheory
conform	optional deterministic Lambdapi emission and a bounded oracle compare selected results with the active kernel	no production Lambdapi dependency

Elaboration may recover information; it may not invent mathematics. If a pointwise family lacks arrow action, the elaborator must report missing coherence rather than synthesize an arbitrary transfor. If lower-star and upper-star action are both type-correct, the written variance or expected type must disambiguate them.

Surface Forms And Explicit Targets

The canonical notation includes:

Surface	Explicit target
`a ->^C b`	`Hom_cat C a b`
`A ⊢ B`	`Functor_cat A B`
`F => G`	`Transf_cat F G`
`E[k]`	`Fibre_cat E k`
`A[k^-] ⊢_[k] B[k]`	`Functor_catd A B`
`Π (k :^n K), E[k]`	`Pi_cat E`
`u_*(g)`	a `hom_postcomp_*` application
`u^*(h)`	a `hom_precomp_along_*` application

For example, if $\eta : F \Rightarrow G$ and $f : x \to_A y$, the readable term $\eta[f] : F[x] \to_B G[y]$ elaborates toward the fully explicit owner `@tapp1_fapp0 A B F G x y eta f`.

The source notation does not have to expose all seven parameters, but the result must typecheck as that operation or an explicitly documented equivalent owner. Readability changes what the author writes, not what the checker trusts.

One compositional motif, four binder modes. Binder modes say how a variable is allowed to vary. The reviewed executable forms are

```
λ^f  x : A. ...
λ^n  k : K. ...
λ^fd a : E. ...
λ^nd k : K. ...
```

The mode belongs to the lambda. The classifier annotation after the variable may be omitted when the bidirectional expected classifier supplies it, but the mode is not inferred from that annotation. The mathematical telescope notation $k :^{n} K$ therefore records the same natural/indexed role as `λ^n k : K. ...` without pretending that mathematical declarations and executable binders are literally the same grammar. Ordinary object binding in the outer LF uses its ordinary dependent lambda.

The four modes are easiest to compare around one compositional motif. Let $H : A \to B$ be an ordinary functor. Let $E, D, Q : K \to \mathsf{Cat}$ be directed families, $FF : E \to D$ and $GG : D \to Q$ displayed functors, and s a coherent section of E. Finally, let $\eta : F_0 \Rightarrow F_1$ and $\theta : F_1 \Rightarrow F_2$ be displayed transfors. At successive categorical levels the same idea appears as:

Mode	Representative expression	Mathematical reading
`^f`	`λ^f x : A. H x`	an ordinary functorial variable inside one category
`^n`	`λ^n k : K. (GG k) ((FF k) (s k))`	a base variable whose result is a coherent section of Q
`^fd`	`λ^fd a : E. GG (FF a)`	an object varying in a displayed family, retaining its hidden base index
`^nd`	`λ^nd k : K. composeCells (theta k) (eta k)`	a coherent family of cells between displayed functors, one hom level higher

These are not four spellings for an ordinary lambda. The `^n` form must respect transport in the base; the `^fd` form must retain displayed object and arrow action; the `^nd` form must construct a transfor rather than a bare pointwise family. In each case the expected classifier selects a reviewed internal construction. If that construction is absent, elaboration fails instead of accepting a JavaScript callback with an external naturality promise.

Ordinary nesting already shows why recursive scope matters. Assume `A, B, C : Cat` and `E : Functor B (Functor_cat A C)`. The reviewed expression is:

```
λ^f x : A. λ^f y : B. E y x
```

This term has classifier `Functor A (Functor_cat B C)`. Neutral application first selects the action of `E` on `y` and then its action on `x`. Recursive abstraction lowers the result through the existing `exchange-functor-abstraction` owner before explicit Core is checked.

No external functoriality equation accompanies the source expression. The selected owner already carries object and arrow action, and the resulting explicit Core is checked by the same generic LF as other terms.

Dependency Levels And Independent Siblings

Displayed contexts make the distinction between dependency and independence visible. A representative mixed telescope is

```
λ^fd (a : A; b : B, c : C; d : D). fibrePair b c
```

It has dependency levels `A; B,C; D`. A semicolon advances to a family over the preceding total context, while a comma groups independent siblings over the same prefix. The middle pair lowers through the transparent fibrewise product, displayed pairing, Sigma projections, and reindexing owners. Thus `b` and `c` may be paired, weakened, contracted, or exchanged fibrewise; no exchange of `a` across a classifier depending on `a` is implied. Object and base-arrow behavior remain internal to those owners rather than being supplied as external coherence evidence.

The implemented normal form is not limited to the displayed four-variable example. It supports any finite sequence of these canonical dependency levels, with finite sibling groups at a level, for the reviewed displayed functorial and displayed-natural constructions. Separately, the category resolver can descend through any finite number of qualified Hom-category levels over its supported roots, and the indexed-section route can compose a finite rigid chain of displayed functors on a section. These depth results do not amount to arbitrary dependency or variance graphs. Exchange across a dependency edge, unrestricted mixed introduction and currying, and coherence synthesis outside the qualified grammar remain open.

Declaration Convenience Without New Mathematics

Some repetition belongs to the surrounding logical framework rather than to categorical terms. Two direct-TypeScript declaration forms remove that repetition before explicit Core is checked.

For an adjunction, `assumeAdjunction` receives already declared functors, unit, and counit. It expands to an ordinary `Adjunction(F,G)` assumption and two proof-time agreements identifying the declared transformations with the kernel's stable unit and counit observations. A second form accepts a counit and a coherent whole hom-profunctor transpose. In both cases the declaration preserves the distinction between proof-time agreement and runtime conversion: independently named maps do not silently become new reduction rules for the categorical kernel.

For a finite dependent package, `declareStructure` expands one unparameterized, nonrecursive, single-constructor structure into an opaque carrier, an injective constructor, named primitive projections, and one ordered, subject-reducing beta rule for each projection. Later field types may depend on earlier fields, which is the essential convenience for mathematical presentations. The form generates no record eta, eliminator, recursion, positivity theorem, general inductive declaration, or browser/text syntax.

Both conveniences are conservative in the practical architectural sense: their output consists of the same ordinary LF declarations and rules that could have been written explicitly. Neither adds a trusted Core node or a Lambdapi mathematical owner. The elaborator improves the act of stating a presentation; it cannot turn missing structure or coherence into a theorem.

Located Text And The Browser Reviewer

The text adapter accepts a small, located language rather than a string that is later treated as trusted code. It records source spans, parses the reviewed binders, grouped contexts, neutral whitespace application, and selected term, category, and displayed-family constructors, then delegates typing and owner selection to the same contextual program used by direct TypeScript. A failure therefore reports its parsing, resolution, or elaboration phase together with the source location. It is not a second action table or checker.

The integrated browser reviewer makes this path inspectable without a server. Its twelve editable examples span the four binder modes, the canonical sibling/Sigma context, qualified recursive Hom categories, and finite rigid section chains. The current natural-binder example is the two-step section

```
λ^n k : K. (GG k) ((FF k) (s k))
```

from the running motif above. For an accepted expression the client displays the explicit backend-neutral Core, inferred and expected classifiers, and the structural owners used in lowering. For a rejected edit it displays the source-located diagnostic. The same page can run the outer-LF/ordinary/displayed research report, retain the minimal explicit-Core playground, and open the generated book. All of this execution is client-side; Lambdapi is an optional development oracle, not a browser or production dependency.

Historical Prototype And Retained Boundary

The repository history contains an older TypeScript feasibility prototype with generic bidirectional checking, holes, unification, rewriting, and proof-state machinery. Those mechanisms informed the renewed work, but its stale category-specific layer is not an authority for v3.2. The renewed product instead targets backend-neutral explicit Core aligned with active owners and uses Lambdapi only as an optional conformance oracle.

The current path is deliberately bounded: it does not parse every notation in the book, accept arbitrary dependency or variance graphs, synthesize coherence from an unstructured pointwise function, or mechanically transfer the whole Lambdapi library. Its qualified finite-depth results are neither

one hard-coded example nor a complete surface language. They are explicit continuation boundaries, not hidden assumptions of the examples that run.

> **Formal status — research boundary.** Evidence `FORMAL-ELABORATION-BOUNDARY`. The direct-TypeScript and categorical-text paths, explicit Core, generic checker/evaluator, bounded adjunction and dependent-structure declarations, client-side reviewer, and optional conformance route are executable for the reviewed profile. A complete compiler for the canonical surface, arbitrary displayed coherence, a general record or inductive facility, and whole-library transfer are not claimed. The active Lambdapi sources remain the mathematical authority.

G.6 Directed Higher-Inductive Signatures

A directed higher-inductive signature must specify more than a list of generators. At minimum it needs:

1. object, arrow, and higher-cell constructors with typed boundaries;
2. the categories and families into which they may be interpreted;
3. recursion, dependent elimination, and any contextual elimination principles;
4. coherence data demanded by varying source and target families;
5. constructor computation at named observers;
6. action on arrows and higher cells of every varying parameter;
7. optional dimension or truncation evidence;
8. a statement of uniqueness or initiality when one has actually been proved.

The WalkingEnd signature is the selected worked instance. With $W = \mathsf{WalkingEnd}$, $* : \mathrm{Obj}(W)$, and $\ell : * \to_W *$, its contextual algebra is

$$\begin{aligned} R, D &: W \longrightarrow \mathsf{Cat}, \\ u &: R[*] \longrightarrow D[*], \\ \sigma &: D[\ell] \circ u \Longrightarrow u \circ R[\ell]. \end{aligned}$$

The eliminator returns

$$\mathsf{ind}^d(R, D, u, \sigma) : R \longrightarrow^d D.$$

At the base, its fibre functor reduces to u. At the literal generator, its displayed laxity component reduces to the supplied component of σ. These are constructor beta rules. Generic functoriality and transfor naturality own the remaining identity, composition, and ordinary naturality cuts.

The coherence cell is directed:

$$D[\ell] \circ u \Longrightarrow u \circ R[\ell].$$

It is neither an equality nor an automatically invertible path. Reversing it would define a different eliminator orientation. Supplying only its pointwise components would also be incomplete, because the transfor must act on arrows of $R[*]$ and on their higher cells.

The one-dimensional witness is separate signature data. It lets a later directed 2-cell between parallel based arrows be converted to equality; it does not make the generating 1-cell invertible. This separation is essential to the normalization proof of Chapter 8.

The ordinary section eliminator and recursor are specializations:

view	R	D
contextual	arbitrary	arbitrary
section	terminal family	arbitrary
recursor	terminal family	constant family.

This relationship is an architectural requirement for a future signature compiler: it should generate one coherent principle and derive weaker views, not postulate unrelated recursors whose computations may disagree.

What is not yet active is equally specific. There is no general language of directed cell boundaries, no compiler that generates contextual eliminators and projection-stable beta rules, no general algebra category, and no theorem that every such presentation is initial. A plausible implementation must also generate focused diagnostics for typing, subject reduction, critical pairs, and both possible projection orders.

> **Formal status — checked instance and research boundary.** Evidence `WE-SIGNATURE` and `WE-CONTEXTUAL-ELIMINATOR` support the complete selected instance. Evidence `DHIT-GENERAL-SCHEMA` records the missing general signature and compiler rather than extrapolating them from one HIT.

G.7 Basic Metatheory And Its Boundary

A successful checker run is strong evidence that the submitted declarations and rules satisfy the tool's current acceptance conditions. It is not, by itself, a proof of every global property one might want from the combined rewrite and unification theory. The current warranted statements are:

Property	What this edition may say
typing of active sources	checked by bounded Lambdapi runs over the active module graph
local subject-reduction obligations of promoted rewrite rules	checked by Lambdapi's ordinary rule acceptance; not separately formalized as one global project theorem
selected computation	witnessed by promoted rules and focused positive and negative assertions
selected proof-time comparison	exercised by typed uses or typed reflexivity checks, not inferred merely from a conversion assertion
evidence traceability	checked syntactically from book markers to active owners and reviewers
build and render reproducibility	tested by the release tooling for the recorded source snapshot
global confluence	not established for the whole emdash rewrite and unification theory
strong normalization	not established for the whole theory
global canonicity	not established; only selected constructor and normalization computations are tested
decidable conversion or type checking as an emdash metatheorem	not claimed beyond observed behavior of the current Lambdapi toolchain on the active sources
consistency and semantic soundness	require model and metatheory proofs for a precisely stated fragment; they do not follow silently from compilation

Engineering Evidence Is Not A Metatheorem

The repository records warning inventories, focused owner-position probes, critical-pair investigations, strict inferred-slot audits, and bounded full checks. These practices are indispensable. They locate overlap risks, reject ill-typed rules, compare reduction orders, and protect intended normal forms. They still sample or delegate parts of a global theorem rather than proving one in the object theory.

Warning counts are therefore diagnostics, not a numeric confluence proof. A zero count would not establish normalization, and a nonzero count does not by itself refute a deliberately joined computation. Likewise, deterministic PDF output establishes release reproducibility, not mathematical consistency.

The Role Of Models

`BNat` is a separate concrete one-object category whose endomorphisms are natural numbers under addition. The checked functor from WalkingEnd to `BNat` is meaningful model evidence for the selected generators and recursor. The encode–decode theorem then proves much more about the based hom inside the opaque source.

That model is not a soundness interpretation for all of `emdash3_2.lp`. It does not interpret every higher category, displayed family, equality principle, profunctor, or rewrite rule. A global consistency claim would require a stated syntax fragment, an interpretation of every formation and computation

rule in that fragment, and a proof that conversion is preserved.

> **Formal status — checked local model evidence.** Evidence `WE-BNAT-MODEL` supports the separate WalkingEnd-to-BNat interpretation. No global soundness theorem is inferred from it.

Adaptation Of The HoTT Formal Appendix

The four source units of the HoTT formal appendix enter this presentation as follows:

HoTT source unit	Functorial adaptation
A.1, first presentation	G.2 gives the readable categorical signature; G.5 keeps it distinct from a parser
A.2, second presentation	G.1 and G.3 state judgments and literal source; G.4 organizes representative rule families
A.3, homotopy type theory	equality and univalence remain qualified layers, while G.6 presents the selected directed WalkingEnd extension
A.4, basic metatheory	this section retains the taxonomy of properties but replaces inherited conclusions by the conservative status matrix above

The adaptation deliberately does not import the HoTT appendix's normalization or metatheoretic conclusions as results about emdash. Different rewrite owners, proof-time unification rules, opaque categorical structure, and directed higher cells require their own theorem.

> **Formal status — research boundary.** Evidence `FORMAL-METATHEORY-BOUNDARY`. The matrix states what current checks establish and gives a concrete specification for future metatheory. Global confluence, strong normalization, canonicity, decidability, consistency, and semantic soundness remain unclaimed until separately proved.

The resulting architecture is intentionally asymmetric. The categorical kernel computes without depending on a traditional front end. The bounded elaborator improves usability without changing the foundation, and future semantic models may justify larger fragments without becoming a second source language. That is the formal sense in which functorial type theory begins from categorical computation.

Bibliography

1. The Univalent Foundations Program. *Homotopy Type Theory: Univalent Foundations of Mathematics*. Institute for Advanced Study, 2013. Source repository, reviewed at revision `578b85cc8d586b1677ec4335148adeb443057d24`.

2. Saunders Mac Lane. *Categories for the Working Mathematician*, second edition. Graduate Texts in Mathematics 5. Springer, 1998; DOI 10.1007/978-1-4757-4721-8.

3. G. M. Kelly. *Basic Concepts of Enriched Category Theory*. London Mathematical Society Lecture Note Series 64. Cambridge University Press, 1982; reprinted in *Theory and Applications of Categories*, Reprints 10, 2005. TAC reprint.

4. Jean Bénabou. "Introduction to Bicategories." In *Reports of the Midwest Category Seminar*, Lecture Notes in Mathematics 47, pages 1–77. Springer, 1967. DOI 10.1007/BFb0074299.

5. GPT 5.6 Codex.

6. The Lambdapi contributors. *Lambdapi User Manual*. Current online manual. The repository copies under `docs/` are the operational references used while maintaining the accompanying formal development.

7. The emdash contributors. *emdash v3.2 Lambdapi Sources*. Accompanying computational artifact for this development edition: `emdash3_2.lp` and its one-way extension modules.

8. Kosta Došen. *Cut Elimination in Categories*. Trends in Logic 6. Kluwer Academic Publishers, Dordrecht, 1999; DOI 10.1007/978-94-017-1207-1.

9. Max Zeuner. *Univalent Foundations of Constructive Algebraic Geometry*. arXiv:2407.17362v1, 2024. arXiv record.

10. Pierre-Marie Pédrot. "Pursuing Shtuck." Preprint, 2023. HAL record.

Items 1–5 and 8–10 situate the mathematical development; items 6–7 identify the proof infrastructure and checked artifact. Citation does not by itself confer the book's formal-status label. The exact HoTT source revision and Zeuner and Pédrot versions, section maps, adaptation targets, and license metadata, together with the reference-only policy for Došen's book, are recorded in `book/references/third-party-sources.json`.

Credits And Third-Party Attribution

This development edition is prepared by the emdash contributors. Individual authorship and editorial credits will be made explicit before a public edition is released.

Homotopy Type Theory book

The organization and adapted passages of this book are inspired by:

> The Univalent Foundations Program, *Homotopy Type Theory: Univalent Foundations of Mathematics*, Institute for Advanced Study, 2013.

The reviewed source is the HoTT Book repository at revision `578b85cc8d586b1677ec4335148adeb443057d24` (2026-05-12). That work is licensed under the Creative Commons Attribution-ShareAlike 3.0 Unported License.

Any adapted material in this book is modified for a directed, category-theoretic setting. In particular, the circle/integer calculation is not reproduced as the WalkingEnd/Nat calculation: invertible paths are replaced by genuinely directed arrows, integer powers by natural powers, and the hard inverse by a directed normalization cell followed by one-dimensionality.

The exact source map and adaptation ledger live in `references/third-party-sources.json`. They were established before the corresponding prose was drafted. The Chapter 8 vertical slice records structural and conceptual adaptations from the pinned source, while Chapters 10–15 adapt all nine sections of `categories.tex` and Appendix G adapts all four parts of `formal.tex`. The resulting prose is newly written for the directed categorical setting; the ledger records the source labels, adaptation kind, and target under this attribution and ShareAlike notice.

Max Zeuner's constructive algebraic geometry

The local-to-global geometry spiral takes mathematical and expository inspiration from:

> Max Zeuner, *Univalent Foundations of Constructive Algebraic Geometry*, arXiv:2407.17362v1, 2024.

The reviewed arXiv version is licensed under the Creative Commons Attribution 4.0 International License. Chapter 18 adapts the locally ringed lattice's largest compact-open invertibility support into a comparison with the sieve $D_U(s)$ of all invertibility probes. This is a change of organizing viewpoint: the compact open remains the appropriate representative in Zeuner's coherent or qcqs setting when it exists, while the sieve is defined on a general site before representability is known. Chapter 22 structurally adapts the Zariski-lattice, coverage, compact-open, and functor-of-points narrative to this sieve-first organization: a supplied localization represents $D_R(f)$ pointwise, and the big-site topology is generated from selected finite localization charts. Neither chapter imports Zeuner's qcqs comparison theorem or general scheme theorem as an emdash result. Chapter 23 comparatively adapts the finite affine-cover architecture, but reverses the direction of construction: its global ringed

object is supplied first, two charts constructively generate one retained covering sieve, and restrictions and a selected intersection are inherited from that single object. It does not import Zeuner's gluing theorem, compact-open classifier, or equivalence between functorial and locally ringed-lattice qcqs schemes. Chapter 24 carries that finite-cover rhythm and comparison boundary into a supplied projective-line presentation, but its Laurent calculation and explicit `Proj` horizon are an emdash synthesis rather than a construction drawn from Zeuner's thesis. The source sections, targets, adaptation kinds, and mathematical changes are recorded in `references/third-party-sources.json`.

Pierre-Marie Pédrot's computational sheafification

The return/glue/silent presentation in Chapter 20 takes conceptual and structural inspiration from:

> Pierre-Marie Pédrot, “Pursuing Shtuck,” preprint, 2023.

The reviewed HAL version is licensed under the Creative Commons Attribution 4.0 International License. Pédrot presents free sheaves by a return constructor, a branching glue constructor, and an equation that erases a branch whose result is ignored, then explains the last as a silent transition. Emdash adapts that computational picture to actual varying cover questions in categorical semantics: the branches are matching objects of Cat-valued presheaves, and the checked endpoint is a whole Hom-category universal property and reflector. The book does not import the paper's internal type theory, metatheory, universe claims, or dependent-elimination results. Exact source sections and adaptation boundaries are recorded in `references/third-party-sources.json`.

Došen's cut-elimination perspective

The four-level cut calculus in Chapter 9 takes conceptual inspiration from Kosta Došen's *Cut Elimination in Categories* (Kluwer, 1999). The cited work is not licensed for textual adaptation here. It is used only as a bibliographic and conceptual reference: the exposition, notation, examples, and emdash correspondence in this book are newly written, and no passage from Došen's text is copied or closely paraphrased.

License For The Book

Except where a source or quotation is identified separately, the textual contents of the `book/` directory and the generated `print/public/emdash-book.md` artifact are licensed under the Creative Commons Attribution-ShareAlike 3.0 Unported License, abbreviated `CC BY-SA 3.0`.

You may share and adapt this material under the terms of that license, including attribution and ShareAlike. Third-party material remains subject to its identified license and attribution requirements.

This notice applies only to the book text and its generated Markdown form. It does not assign or change a license for Lambdapi sources, renderer code, reports, or other repository content outside that book artifact.